Application Security Verification Standard

Version 5.0.0



May 2025

ف‌ه‌رس‌ت

ص‌ف‌ح‌ه اص‌ل‌ی 7
درب‌اره ای‌ن اس‌ت‌ان‌دارد 7
ک‌پ‌ی‌رای‌ت و م‌ج‌وز 7
س‌رپ‌رس‌ت‌ان پ‌روژه 7
گ‌روه ک‌اری 7
س‌ای‌ر ه‌م‌ک‌اران اص‌ل‌ی 7
م‌ش‌ارک‌ت‌ک‌ن‌ن‌د‌گ‌ان و ب‌ازب‌ی‌ن‌ه‌ای د‌ی‌گ‌ر 8

پ‌ی‌ش‌گ‌ف‌ت‌ار **8**
م‌ق‌دم‌ه 8
اص‌ول ک‌ل‌ی‌د‌ی ن‌س‌خ‌ه ۵.۰ 8
چ‌ش‌م‌ان‌داز آ‌ی‌ن‌ده 9

ASVS چ‌ی‌س‌ت؟ **9**
دام‌ن‌ه ASVS 9
Application 10
ام‌ن‌ی‌ت 10
ت‌أ‌ی‌ی‌د 10
اس‌ت‌ان‌دارد 10
ال‌زام (Requirement) 11
ت‌ص‌م‌ی‌م‌ات ام‌ن‌ی‌ت‌ی م‌س‌ت‌ن‌د‌ش‌ده 11
س‌ط‌وح ت‌أ‌ی‌ی‌د ام‌ن‌ی‌ت ب‌رن‌ام‌ه‌ه‌ای ک‌ارب‌ردی 12
ارزی‌اب‌ی س‌ط‌وح 12
س‌ط‌ح ۱ 13
س‌ط‌ح ۲ 13
س‌ط‌ح ۳ 13
ک‌دام س‌ط‌ح را ان‌ت‌خ‌اب ک‌ن‌ی‌م 14
چ‌گ‌ون‌ه از ASVS اس‌ت‌ف‌اده ک‌ن‌ی‌م 14
س‌اخ‌ت‌ار ASVS 14
Release strategy 14
ان‌ع‌ط‌اف‌پ‌ذ‌ی‌ری در اس‌ت‌ف‌اده از ASVS 15
ن‌ح‌وه ارج‌اع ب‌ه ال‌زام‌ات ASVS 15
ف‌ورک ک‌ردن ASVS (Forking) 16
م‌وارد اس‌ت‌ف‌اده از ASVS 16
ب‌ه‌ع‌ن‌وان راه‌ن‌م‌ای د‌ق‌ی‌ق م‌ع‌م‌اری ام‌ن‌ی‌ت‌ی 16
ب‌ه‌ع‌ن‌وان م‌رج‌ع ت‌خ‌ص‌ص‌ی ب‌رای ب‌رن‌ام‌ه‌ن‌وی‌س‌ی ام‌ن 16
ب‌ه‌ع‌ن‌وان راه‌ن‌م‌ای آ‌زم‌ون‌ه‌ای واح‌د و ی‌ک‌پ‌ارچ‌ه‌س‌ازی خ‌ود ک‌ار 16
ب‌رای آ‌م‌وزش ت‌وس‌ع‌ه ام‌ن 17
ب‌ه‌ع‌ن‌وان چ‌ارچ‌وب‌ی ب‌رای راه‌ن‌م‌ای‌ی در خ‌ری‌د ن‌رم‌اف‌زار ام‌ن 17
ک‌ارب‌رد ع‌م‌ل‌ی ASVS 17

ارزیابی و صدور گواهی **17**
دیدگاه OWASP در مورد گواهی‌نامه‌ها و نشان‌های اعتماد ASVS 17
نحوه تأیید انطباق با ASVS 18
گزارش‌گیری ارزیابی 18
دامنه ارزیابی 18
روش‌های ارزیابی 18

تغییرات نسبت به نسخه 4.x **20**
مقدمه 20
فلسفه الزامات 20
دامنه و تمرکز. 20
تمرکز بر اهداف امنیتی به‌جای سازوکارها 20
تصمیمات امنیتی مستند 20
تغییرات ساختاری و فصل‌های جدید 21
حذف نگاشت مستقیم با استانداردهای دیگر 22
کاهش وابستگی به دستورالعمل‌های هویت دیجیتال NIST 22
فاصله گرفتن از نگاشت CWE 22
بازنگری در تعریف سطوح (Levels) 22
ورود ساده‌تر به سطح اول 22
افسانه قابلیت آزمون‌پذیری 23
فقط ریسک مطرح نیست 23

V1 Encoding and Sanitization **23**
هدف 23
V1.1 ساختار Encoding and Sanitization 24
V1.2 پیشگیری از تزریق 24
V1.3 پاکسازی (Sanitization) 26
V1.4 حافظه، رشته و کد بدون مدیریت (Memory, String, and Unmanaged Code) 27
V1.5 سریال‌زدایی ایمن (Safe Deserialization) 28
References 29

V2 Validation and Business Logic **29**
هدف 29
V2.1 مستندات اعتبارسنجی و منطق کسب‌وکار) (Validation and Business Logic Documentation 30
V2.2 اعتبارسنجی ورودی (Input Validation) 30
V2.3 امنیت منطق کسب‌وکار (Business Logic Security) 31
V2.4 مقابله با خودکارسازی (Anti-automation) 32
References 33

V3 امنیت رابط کاربری وب (Web Frontend Security) **33**
هدف کنترلی 33
V3.1 مستندسازی امنیت رابط کاربری وب) (Web Frontend Security Documentation 33

34 . . . . . V3.2 تفسیر ناخواسته محتوا (Unintended Content Interpretation)
34 . . . . . V3.3 پیکربندی کوکی (Cookie Setup)
35 . . . . . V3.4 هدرهای مکانیزمهای امنیتی مرورگر (Browser Security Mechanism Headers)
37 . . . . . V3.5 تفکیک مبدا توسط مرورگر (Browser Origin Separation)
38 . . . . . V3.6 یکپارچگی منابع خارجی (External Resource Integrity)
39 . . . . . V3.7 سایر ملاحظات امنیتی مرورگر (Other Browser Security Considerations)
39 . . . . . References

40 V4 API and Web Service
40 . . . . . هدف
40 . . . . . V4.1 Generic Web Service Security
41 . . . . . V4.2 HTTP Message Structure Validation
42 . . . . . V4.3 GraphQL
42 . . . . . V4.4 WebSocket
43 . . . . . References

43 V5 File Handling
43 . . . . . هدف
44 . . . . . V5.1 File Handling Documentation
44 . . . . . V5.2 File Upload and Content
45 . . . . . V5.3 File Storage
45 . . . . . V5.4 File Download
46 . . . . . References

46 V6 Authentication
46 . . . . . Control Objective
47 . . . . . V6.1 Authentication Documentation
47 . . . . . V6.2 Password Security
48 . . . . . V6.3 General Authentication Security
50 . . . . . V6.4 Authentication Factor Lifecycle and Recovery
51 . . . . . V6.5 General Multi-factor authentication requirements
52 . . . . . V6.6 Out-of-Band authentication mechanisms
53 . . . . . V6.7 Cryptographic authentication mechanism
54 . . . . . V6.8 Authentication with an Identity Provider
55 . . . . . References

55 V7 Session Management
55 . . . . . هدف
56 . . . . . V7.1 Session Management Documentation
57 . . . . . V7.2 Fundamental Session Management Security
57 . . . . . V7.3 Session Timeout
58 . . . . . V7.4 Session Termination

59 . . . . . V7.5 Defenses Against Session Abuse
59 . . . . . V7.6 Federated Re-authentication
60 . . . . . References

60 V8 Authorization
60 . . . . . هدف
60 . . . . . V8.1 Authorization Documentation
61 . . . . . V8.2 General Authorization Design
62 . . . . . V8.3 Operation Level Authorization
62 . . . . . V8.4 Other Authorization Considerations
63 . . . . . References

63 V9 Self-contained Tokens
63 . . . . . هدف
63 . . . . . V9.1 Token source and integrity
64 . . . . . V9.2 Token content
65 . . . . . References

65 V10 OAuth and OIDC
65 . . . . . هدف
67 . . . . . V10.1 Generic OAuth and OIDC Security
68 . . . . . V10.2 OAuth Client
69 . . . . . V10.3 OAuth Resource Server
70 . . . . . V10.4 OAuth Authorization Server
72 . . . . . V10.5 OIDC Client
73 . . . . . V10.6 OpenID Provider
73 . . . . . V10.7 Consent Management
74 . . . . . References

75 V11 Cryptography
75 . . . . . هدف
75 . . . . . V11.1 Cryptographic Inventory and Documentation
76 . . . . . V11.2 Secure Cryptography Implementation
77 . . . . . V11.3 Encryption Algorithms
78 . . . . . V11.4 Hashing and Hash-based Functions
79 . . . . . V11.5 Random Values
79 . . . . . V11.6 Public Key Cryptography
80 . . . . . V11.7 In-Use Data Cryptography
80 . . . . . References

81 V12 Secure Communication
81 . . . . . هدف
81 . . . . . V12.1 General TLS Security Guidance

82 . . . . . V12.2 HTTPS Communication with External Facing Services
82 . . . . . V12.3 General Service to Service Communication Security
83 . . . . . References

83 V13 Configuration
83 . . . . . هدف
83 . . . . . V13.1 Configuration Documentation
84 . . . . . V13.2 Backend Communication Configuration
85 . . . . . V13.3 Secret Management
86 . . . . . V13.4 Unintended Information Leakage
87 . . . . . References

87 V14 Data Protection
87 . . . . . هدف
87 . . . . . V14.1 Data Protection Documentation
88 . . . . . V14.2 General Data Protection
89 . . . . . V14.3 Client-side Data Protection
90 . . . . . References

90 V15 Secure Coding and Architecture
90 . . . . . Control Objective
90 . . . . . V15.1 Secure Coding and Architecture Documentation
91 . . . . . V15.2 Security Architecture and Dependencies
92 . . . . . V15.3 Defensive Coding
93 . . . . . V15.4 Safe Concurrency
94 . . . . . References

94 V16 Security Logging and Error Handling
94 . . . . . هدف
95 . . . . . V16.1 Security Logging Documentation
95 . . . . . V16.2 General Logging
96 . . . . . V16.3 Security Events
97 . . . . . V16.4 Log Protection
97 . . . . . V16.5 Error Handling
98 . . . . . References

98 V17 WebRTC
98 . . . . . هدف
99 . . . . . V17.1 TURN Server
100 . . . . . V17.2 Media
101 . . . . . V17.3 Signaling
102 . . . . . References

102 Appendix A: Glossary

108 Appendix B: References
109 . . . . . OWASP Core Projects
109 . . . . . OWASP Cheat Sheet Series project
109 . . . . . Mobile Security Related Projects
109 . . . . . OWASP Internet of Things related projects
109 . . . . . OWASP Serverless projects
109 . . . . . Others

110 Appendix C: Cryptography Standards
110 . . . . . Cryptographic Inventory and Documentation
110 . . . . . Equivalent Strengths of Cryptographic Parameters
111 . . . . . Random Values
112 . . . . . Cipher Algorithms
113 . . . . . AES Cipher Modes
114 . . . . . Key Wrapping
114 . . . . . Authenticated Encryption
115 . . . . . Hash Functions
115 . . . . . Hash Functions for General Use Cases
116 . . . . . Hash Functions for Password Storage
117 . . . . . Key Derivation Functions (KDFs)
117 . . . . . General Key Derivation Functions
118 . . . . . Password-based Key Derivation Functions
118 . . . . . Key Exchange Mechanisms
118 . . . . . KEX Schemes
119 . . . . . Diffie-Hellman groups
120 . . . . . Message Authentication Codes (MAC)
121 . . . . . Digital Signatures
121 . . . . . Post-Quantum Encryption Standards

121 Appendix D: Recommendations
121 . . . . . Introduction
122 . . . . . Recommended, in-scope mechanisms
122 . . . . . Software Security principles
123 . . . . . Software Security processes

123 Appendix E - Contributors

صفحه اصلی

درباره این استاندارد

استاندارد تأیید امنیت نرم‌افزار (ASVS)، فهرستی از الزامات امنیتی برای نرم‌افزاره‌است که مهم‌اران، توسعه‌دهندگان، آزمون‌گران، متخصصان امنیت، فروشندگان ابزار و مصرف‌کنندگان می‌توانند از آن برای تعریف، ساخت، آزمون و تأیید امنیت برنامه‌های نرم‌افزاری استفاده کنند.

کپی‌رایت و مجوز

Version 5.0.0, May 2025



شکل 1: license

Copyright © 2008-2025 The OWASP Foundation.

این سند تحت مجوز Creative Commons Attribution-ShareAlike 4.0 International License منتشر شده است.

در هرگونه بازنشر یا توزیع، باید شرایط این مجوز را به‌روشنی به دیگران اعلام نمایید.

سرپرستان پروژه

Elar Lang	Josh C Grossman
Jim Manico	Daniel Cuthbert

گروه کاری

Tobias Ahnoff	Ralph Andalis	Ryan Armstrong	Gabriel Corona
Meghan Jacquot	Shanni Prutchi	Iman Sharafaldin	Eden Yardeni

سایر همکاران اصلی

Sjoerd Langkemper	Isaac Lewis

Mark Carney Sandro Gauci

مشارکت کنندگان و بازبینه‌ای دیگر

فهرست سایر مشارکت کنندگان در ضمیمه E درج شده است.

در صورتی که نام فردی در فهرست اعتبارهای نسخه x.5 ذکر نشده باشد، لطفاً با ثبت تی کت در گیتهاب، نسبت به شناساندن ایشان در به‌روزرسانی‌های آتی نسخه x.5 اقدام فرمایید.

استاندارد ASVS بر پایه‌ی تلاش‌های افراد درگیر در نسخه‌های ۱.۰ (۲۰۰۸) تا ۴.۰ (۲۰۱۹) بنا شده است. بسیاری از ساختارها و اقلام تأییدی که امروزه در ASVS باقی مانده‌اند، در ابتدا توسط Andrew van der Stock، Mike Boberski، Jeff Williams و Dave Wichers نوشته شده‌اند، در کنار بسیاری دیگر از مشارکت-کنندگان. از همه کسانی که در گذشته مشارکت داشته‌اند صمیمانه سپاسگزاریم. برای مشاهده فهرست کامل مشارکت کنندگان پیشین، لطفاً به هر نسخه‌ی قبلی مراجعه نمایید.

پیشگفتار

به نسخه ۵.۰ استاندارد تأییدی امنیت نرم‌افزار (ASVS) خوش آمدید.

مقدمه

استاندارد ASVS که در سال ۲۰۰۸ و از طریق همکاری یک جامعه جهانی راه‌اندازی شد، ASVS مجموعه‌ای جامع از الزامات امنیتی برای طراحی، توسعه و آزمون برنامه‌ها و خدمات وب مدرن تعریف می‌کند.

پس از انتشار نسخه ۴.۰ در سال ۲۰۱۹ و به‌روزرسانی جزئی آن (نسخه ۴.۰.۳ در سال ۲۰۲۱)، نسخه ۵.۰ یک نقطه‌ی عطف مهم به شمار می‌آید—این نسخه برای همراستایی با آخرین پیشرفت‌ها در حوزه‌ی امنیت نرم‌افزار، به‌روزرسانی و مدرن‌سازی شده است.

نسخه ۵.۰ نتیجه‌ی مشارکت گسترده سرپرستان پروژه، اعضای گروه کاری و جامعه بزرگ OWASP در راستای به‌روزرسانی و ارتقای این استاندارد مهم است.

اصول کلیدی نسخه ۵.۰

این بازنگری عمده با چند اصل کلیدی طراحی و توسعه یافته است:

- حوزه و تمرکز دقیق‌تر: این نسخه از استاندارد به گونه‌ای طراحی شده تا همراستا با ارکان اصلی موجود در نام آن باشد: برنامه کاربردی، امنیت، تأیید و استاندارد. الزامات به گونه‌ای بازنویسی شده‌اند که پیشگیری از نقص‌های امنیتی را در اولویت قرار دهند، نه الزام به پیاده‌سازی‌های فنی خاص. متن الزامات به‌صورت خودتوضیحی نگاشته شده‌اند تا چرایی وجود هر مورد را روشن کنند.

- پشتیبانی از تصمیمات مستند امنیتی: نسخه ۵.۰ الزامات جدیدی برای مستندسازی تصمیمات کلیدی امنیتی معرفی کرده است. این اقدام موجب افزایش قابلیت ردیابی شده و پیادهسازیهای متناسب با زمینه (context) را پشتیبانی میکند، به سازمانها اجازه میدهد تا موضع امنیتی خود را با نیازها و ریسکهای خاص خود تطبیق دهند.
- سطوح بهروزشده :درحالیکه مدل سهسطحی ASVS حفظ شده است، تعاریف سطحها بازطراحی شدهاند تا پذیرش آن سادهتر گردد. سطح ۱ بهعنوان گام آغازین برای پذیرش ASVS طراحی شده و لایهی نخست دفاعی را فراهم میآورد. سطح ۲ نمایانگر دیدی جامع از رویههای استاندارد امنیتی است. سطح ۳ به نیازهای پیشرفته و با تضمین بالا میپردازد.
- محتوای بازآراییشده و گستردهشدهتر: نسخه ۵.۰ شامل حدود ۳۵۰ الزام در قالب ۱۷ فصل است. فصلها بهمنظور وضوح بیشتر و سهولت استفاده، بازسازماندهی شدهاند. نگاشتی دوسویه بین نسخههای ۴.۰ و ۵.۰ نیز ارائه شده تا فرآیند مهاجرت را تسهیل نمایند.

چشمانداز آینده

همانطورکه تأمین امنیت یک نرمافزار هرگز نقطهی پایان ندارد، ASVS نیز بهعنوان یک استاندارد همواره در حال توسعه است. گرچه نسخه ۵.۰ یک انتشار عمده بهشمار میآید، اما روند توسعه ادامه خواهد داشت. این نسخه، همزمان با فراهمسازی دسترسی جامعه وسیعتری به بهبودها و افزودههای صورت-گرفته، بستری برای ارتقاهای آینده نیز فراهم میکند. از جمله این موارد، میتوان به تلاشهای جامعهمحور برای ایجاد راهنمای پیادهسازی و تأیید بر پایهی مجموعه الزامات اصلی اشاره کرد.

ASVS 5.0 بهمنظور فراهمسازی یک پایهی قابلاعتماد برای توسعهی نرمافزار امن طراحی شده است. ما از جامعه دعوت میکنیم تا این استاندارد را بپذیرند، در آن مشارکت کنند و بر پایهی آن، اقدامات بیشتری انجام دهند؛ تا بهصورت جمعی، وضعیت امنیت برنامههای نرمافزاری را ارتقا دهیم.

ASVS چیست؟

استاندارد تأیید امنیت نرمافزار (ASVS) مجموعهای از الزامات امنیتی برای برنامهها و خدمات وب تعریف میکند و منبعی ارزشمند برای تمامی افرادی است که قصد دارند برنامههای ایمن طراحی، توسعه یا نگهداری کنند یا امنیت آنها را ارزیابی نمایند.

این فصل به جنبههای اساسی استفاده از ASVS ، از جمله دامنه کاربرد، ساختار سطوح اولویت-بندیشده، و موارد اصلی استفاده از این استاندارد میپردازد.

دامنه ASVS

دامنه استاندارد ASVS از نام آن الهام گرفته شده است: برنامه کاربردی(Application)، امنیت(Security)، تأیید(Verification) و استاندارد(Standard). این دامنه مشخص میکند که چه الزامی گنجانده یا مستثنی شده است و هدف اصلی آن، شناسایی اصول امنیتیای است که باید محقق شوند. دامنه همچنین الزامات مستندسازی را نیز در بر میگیرد که پایههای برای الزامات پیادهسازی محسوب میشود.

برای مهاجمان، چیزی به نام «دامنه» معنا ندارد؛ بنابراین الزامات ASVS باید در کنار راهنماهایی برای سایر جنبه‌های چرخه عمر نرم‌افزار، مانند فرآیندهای CI/CD، میزبانی و عملیات اجرایی مورد ارزیابی قرار گیرد.

Application

ASVS، "برنامه" را به‌عنوان محصول نرم‌افزاری در حال توسعه تعریف می‌کند که کنترل‌های امنیتی باید درون آن گنجانده شود. ASVS فعالیت‌های چرخه عمر توسعه یا نحوه ساخت برنامه از طریق فرآیند CI/CD را تجویز نمی‌کند، بلکه بر نتایج امنیتی مورد انتظار درون خود محصول تمرکز دارد.

اجزایی مانند load balancer WAF، یا پراکسی‌هایی که ترافیک HTTP را پردازش، اصلاح یا اعتبارسنجی می‌کنند، ممکن است به‌منظور خاصی بخشی از برنامه تلقی شوند، زیرا برخی کنترل‌های امنیتی مستقیماً به آن‌ها وابسته هستند یا از طریق آن‌ها پیاده‌سازی می‌شوند. این اجزا باید در الزاماتی که مرتبط با Response Cache، Rate Limite، یا کنترل ارتباطات ورودی و خروجی بر اساس مبدا و مقصد در نظر گرفته شوند.

در مقابل، ASVS به‌طور کلی الزاماتی را که مستقیماً مرتبط با برنامه نیستند یا پیکربندی آن‌ها خارج از حوزه مسئولیت برنامه است، مستثنی می‌کند. به‌عنوان مثال، مسائل مرتبط با DNS معمولاً توسط تیم یا واحدی مجزا مدیریت می‌شود.

به‌طور مشابه، اگرچه برنامه مسئول نحوه دریافت ورودی و تولید خروجی است، ولی اگری که فرآیند خارجی با برنامه یا داده‌های آن تعامل داشته باشد، خارج از دامنه ASVS محسوب می‌شود. به‌عنوان نمونه، پشتیبان‌گیری از برنامه یا داده‌های آن، عموماً برعهده یک فرآیند خارجی است و در کنترل برنامه یا توسعه‌دهندگان آن قرار ندارد.

امنیت

هر الزام باید تأثیری قابل اندازه‌گیری بر امنیت داشته باشد. عدم وجود یک الزام باید منجر به ناامن‌تر شدن برنامه شود و پیاده‌سازی آن الزام باید احتمال وقوع یا تأثیر یک ریسک امنیتی را کاهش دهد.

سایر ملاحظات مانند جنبه‌های عملکردی، سبک کدنویسی یا الزامات سیاستی خارج از دامنه ASVS هستند.

تأیید

هر الزام باید قابل تأیید باشد و نتیجه تأیید باید منجر به تصمیم "موفق" یا "ناموفق" گردد.

استاندارد

ASVS مجموعه‌ای از الزامات امنیتی است که برای انطباق با استاندارد باید پیاده‌سازی شوند. به همین دلیل، الزامات به تعریف هدف امنیتی‌ای محدود می‌شوند که باید محقق گردد. اطلاعات جانبی

می‌توانند بر پایه ASVS ساخته شود یا از طریق نگاشت به آن متصل گردد.

به‌طور خاص، OWASP دارای پروژه‌های متعددی است و ASVS به‌صورت آگاهانه از تکرار محتوای سایر پروژه‌ها اجتناب کرده است. به‌عنوان مثال، اگر توسعه‌دهنده‌ای بپرسد "چگونه باید فلان الزام را در فناوری خاص خود پیاده‌سازی کنم"، این سؤال در حوزه پروژه Cheat Sheet Series قرار می‌گیرد. اگر ارزیاب بپرسد "چگونه بایدِ این الزام را در این محیط آزمون کنم"، پاسخ آن در پروژه Web Security Testing Guide یافت می‌شود

در حالی که ASVS فقط برای استفاده متخصصان امنیت طراحی نشده، انتظار می‌رود که خواننده دانش فنی کافی برای درک محتوا یا توانایی جست‌وجو درباره مفاهیم خاص را داشته باشد.

الزام (Requirement)

واژه "الزام" در ASVS دقیقاً برای توصیف آنچه باید برای تأمین امنیت انجام شود به‌کار می‌رود. ASVS فقط شامل الزامات (must) است و توصیه‌ها (should) را به‌عنوان شرط اصلی شامل نمی‌شود.

به‌عبارت دیگر، توصیه‌هایی که ممکن است فقط یکی از راه‌حل‌های ممکن یا ملاحظات سبک کنونی باشند، در تعریف "الزام" نمی‌گنجند.

الزامات ASVS برای پاسخگویی به اصول امنیتی خاص طراحی شده‌اند، بدون آنکه به روش پیاده‌سازی یا فناوری خاصی وابسته باشند، و درعین حال به گونه‌ای نوشته شده‌اند که خودبه‌خود دلیل وجودشان را توضیح دهند. این بدین معناست که الزامات مبتنی بر یک روش تأییدِ یا پیاده‌سازی خاص نیستند.

تصمیمات امنیتی مستندشده

در امنیت نرم‌افزار، طراحی و مشخص کردن سازوکارهای امنیتی به موقع منجر به پیاده‌سازی‌ای منسجم‌تر و قابل‌اعتمادتر در محصول نهایی یا ویژگی موردنظر می‌شود.

علاوه بر این، برخی الزامات به گونه‌ای هستند که پیاده‌سازی آن‌ها بسیار خاص و وابسته به زیازه‌ای هر برنامه است. برای مثال، کنترل مجوزها، اعتبارسنجی ورودی‌ها و کنترل‌های محافظتی بر روی سطوح مختلف داده‌ای حساس.

برای حل این موضوع، به‌جای بیانه‌ایی کلی مثل "تمام داده‌ها باید رمزگذاری شوند" یا تلاش برای پوشش همه موارد ممکن در قالب یک الزام، الزامات مستندسازی در نظر گرفته شده‌اند که توسعه‌دهنده را ملزم می‌سازند نحوه پیاده‌سازی و پیکربندی این نوع کنترل‌ها را به‌صورت مستند ارائه دهد. این مستندات سپس می‌توانند از نظر تناسب مورد بازبینی قرار گرفته و پیاده‌سازی واقعی با مستندات مقایسه شود تا میزان انطباق بررسی گردد.

این الزامات برای ثبت تصمیماتی طراحی شده‌اند که سازمان توسعه‌دهنده درباره نحوه پیاده‌سازی الزامات امنیتی اتخاذ کرده است.

الزامات مستندسازی همواره در بخش اول هر فصل قرار دارند (هرچند همه فصل‌ها چنین الزامی ندارند) و همواره با یک الزام پیاده‌سازی مرتبط هستند، جایی که تصمیمات مستندسازی‌شده باید عملاً پیاده‌سازی شوند. نکته اینجاست که بررسی مستندات و بررسی پیاده‌سازی واقعی، دو فعالیت مستقل هستند.

دو عامل اصلی برای گنجاندن این الزامات وجود دارد. اولین الزام این است که بسیاری از الزامات امنیتی شامل تعریف قوانین خاصی هستند. مانند انواع فایله‌ایی که مجاز به آپلود هستند، کنترله‌ای کسب و کاری مورد نیاز، یا کاراکتره‌ای مجاز برای یک فیلد خاص. این قوانین در هر برنامه متفاوت است، بنابراین ASVS نمی‌تواند به‌صورت امری آنها را تعیین کند، و حتی Cheat sheet یا پاسخ مفصل هم در این موارد کمک نخواهد کرد. بدون مستند بودن این تصمیمات، تأیید پیاده‌سازی آنها ممکن نخواهد بود.

دومین عامل این است که توسعه دهندگان نرم‌افزار در انتخاب برخی الزامات آزادی عمل داشته باشند. برای مثال، در نسخه‌های پیشین ASVS، قواعد مربوط به پایان جلسه (session timeout) بسیار امری بودند. ولی بسیاری از برنامه‌ها، به‌ویژه برنامه‌های مشتری‌محور، قواعدی آسان‌گیرانه‌تر را ترجیح می‌دهند و از کنترله‌ای جایگزین استفاده می‌کنند. به همین دلیل، الزامات مربوط به مستندسازی به‌صورت صریح اجازه‌ی چنین انعطافی را فراهم می‌کنند.

بدیهی است که انتظار نمی‌رود توسعه‌دهندگان به‌صورت فردی این تصمیمات را بگیرند و مستند کنند، بلکه این وظیفه‌ی سازمان است که تصمیم‌گیری کرده، آنها را مستند کرده و به توسعه‌دهندگان منتقل کند تا در کدنویسی لحاظ شوند.

در توسعه نرم‌افزار، همانطور که برای طراحی ویژگی‌ها و ظاهر برنامه چارچوب‌ها، راهنماها و مؤلفه‌های مشخصی وجود دارد و انتظار می‌رود که توسعه‌دهندگان از آنها پیروی کنند، در مورد امنیت هم باید همین روی کرد وجود داشته باشد. یعنی امنیت نباید چیزی باشد که هر توسعه‌دهنده بنا به سلیقه خود تصمیم بگیرد، بلکه باید از استانداردها، ابزارها و طراحی‌های مشخصی تبعیت نماید.

همچنین، نحوه مستندسازی نیز انعطاف‌پذیر است. تصمیمات امنیتی می‌توانند در قالب یک سند رسمی نوشته شده باشند که توسعه‌دهندگان به آن مراجعه می‌کنند، یا این تصمیمات می‌توانند به‌صورت کد در یک کتابخانه مشترک پیاده‌سازی شده باشند که استفاده از آن برای همه توسعه‌دهندگان الزامی است. در هر دو حالت، هدف نهایی محقق می‌شود.

سطوح تأیید امنیت برنامه‌های کاربردی

ASVS سه سطح تأیید امنیت تعریف می‌کند که با افزایش سطح، عمق و پیچیدگی الزامات امنیتی نیز افزایش می‌یابد. هدف کلی این است که سازمان‌ها از سطح اول شروع کرده و همت‌ورزین دغدغه‌های امنیتی را برطرف کنند، و سپس متناسب با نیازهای سازمان و برنامه، به سطوح بالاتر حرکت کنند. در استناد و متن الزامات، این سطوح با نمادهای L1، L2 و L3 نمایش داده می‌شوند.

هر سطح از ASVS بیانگر الزامات امنیتی‌ای است که باید در آن سطح محقق شود؛ الزامات سطوح بالاتر به‌عنوان توصیه در نظر گرفته می‌شوند.

به‌منظور جلوگیری از تکرار الزامات یا گنجاندن الزامات نامربوط در سطوح بالاتر، برخی الزامات فقط در یک سطح مشخص اعمال می‌شوند اما در سطوح بالاتر شرایط سخت‌گیرانه‌تری دارند.

ارزیابی سطوح

سطوح ASVS بر پایه‌ی اولویت‌بندی الزامات و بر اساس تجربه‌ی پیاده‌سازی و آزمون آنها تعیین شده‌اند. تمرکز اصلی بر مقابله‌ی کاهش ریسک در برابر هزینه‌ی پیاده‌سازی الزام است. عامل کلیدی دیگر، سهولت در ورود و پذیرش استاندارد است.

کاهش ریسک به میزان کاهش ریسک امنیتی در برنامه توجه دارد و عوامل سنتی مانند محرمانگی(Confidentiality)، یکپارچگی (Integrity)، دسترس‌پذیری و(Availability) همچنین لایه دفاع اولیه یا دفاع در عمق بودن الزام را در نظر می‌گیرد.

بحث‌های دقیق درباره معیارها و تصمیم‌گیری سطح‌بندی منجر به توزیع‌ی شده که در بیشتر موارد قابل اعتماد است، هرچند ممکن است برای همه موارد ۱۰۰٪ مناسب نباشد. در نتیجه، ممکن است سازمان‌ها تصمیم بگیرند برخی الزامات سطوح بالاتر را زودتر و بر اساس تحلیل ریسک خود در اولویت قرار دهند.

انواع الزامات در هر سطح را می‌توان به‌صورت زیر توصیف کرد.

سطح ۱

این سطح حداقل الزامات برای ایمن‌سازی برنامه را شامل می‌شود و نقطه شروعی حیاتی محسوب می‌شود. حدود ۲۰٪ از کل الزامات ASVS در این سطح قرار دارد. هدف این سطح، داشتن کمترین تعداد الزامات برای سهولت در ورود و پذیرش استاندارد است.

این الزامات عموماً حیاتی یا پایه‌ای هستند و اولین خط دفاعی در برابر حملات رایج محسوب می‌شوند، حملاتی که برای بهره‌برداری از آن‌ها به آسیب‌پذیری یا پیش‌شرط دیگری نیاز نیست.

همچنین برخی الزامات مثل موارد مربوط به گذرواژه‌ها در سطح ۱ اهمیت بیشتری دارند، زیرا در سطوح بالاتر استفاده از احراز هویت چندمرحله‌ای مطرح می‌شود.

سطح ۱ الزاماً از طریق آزمون نفوذ خارجی و بدون دسترسی به کد یا مستندات (مانند آزمون جعبه‌سیاه) قابل ارزیابی نیست، ولی تعداد کمتر الزامات آن فرآیند تأیید را ساده‌تر می‌کند.

سطح ۲

بیشتر برنامه‌ها باید به‌دنبال دستیابی به این سطح از امنیت باشند. حدود ۵۰٪ از الزامات ASVS در سطح ۲ هستند، و بنابراین یک برنامه باید حدود ۷۰٪ از الزامات ASVS (یعنی تمام الزامات L1 و L2) را برای انطباق با این سطح پیاده‌سازی کند.

این الزامات معمولاً به حملات کمتر رایج یا دفاع‌های پیچیده‌تر در برابر حملات رایج مرتبط هستند. ممکن است این الزامات همچنان دفاع اولیه باشند، یا برای بهره‌برداری نیاز به پیش‌شرط‌های داشته باشند.

سطح ۳

این سطح باید هدف برنامه‌هایی باشد که می‌خواهند بالاترین سطح امنیت را اثبات کنند و شامل حدود ۳۰٪ نهایی الزامات است.

الزامات این سطح عمدتاً شامل مکانیزم‌های دفاع در عمق یا کنترل‌های پیچیده و دشوار برای پیاده‌سازی هستند.

کدام سطح را انتخاب کنیم

سطوح اولویت‌بندی‌شده ASVS برای نشان دادن بلوغ امنیتی برنامه و سازمان طراحی شده‌اند. به‌جای آنکه ASVS الزام کند که یک برنامه باید در چه سطحی باشد، این مسئولیت بر عهده‌ی سازمان است که با تحلیل ریسک، و با توجه به حساسیت برنامه و انتظارات کاربران، سطح مناسب را انتخاب کند.

برای نمونه، یک استارتاپ نوپا که فقط مقدار محدودی از داده‌های حساس را جمع‌آوری می‌کند، ممکن است تصمیم بگیرد سطح ۱ را برای اهداف اولیه امنیتی انتخاب کند؛ اما یک بانک برای برنامه بانکی آنلاین خود، به‌سختی می‌تواند چیزی کمتر از سطح ۳ را به مشتریانش توجیه کند.

چگونه از ASVS استفاده کنیم

ساختار ASVS

ASVS شامل حدود ۳۵۰ الزام امنیتی است که در قالب ۱۷ فصل دسته‌بندی شده‌اند. هر فصل نیز به بخش‌های کوچک‌تر تقسیم شده است.

هدف از این تقسیم‌بندی فصل‌ها و بخش‌ها، ساده‌سازی انتخاب یا فیلتر کردن موارد مرتبط با برنامه‌ی خاص است. برای مثال، در یک API ماشین‌به‌ماشین، الزامات فصل V3 که مربوط به Frontend است، کاربردی نخواهد داشت. اگر از OAuth یا WebRTC استفاده نشده، آن فصل‌ها نیز می‌توانند نادیده گرفته شوند.

Release strategy

انتشاره‌ای ASVS از الگوی "Major.Minor.Patch" پیروی می‌کنند که این سه عدد، نوع تغییرات را نشان می‌دهند، در یک Major، عدد اول تغییر می‌کند، در Minor، عدد دوم تغییر می‌کند، و در Patch، عدد سوم تغییر می‌کند.

- Major release): بازسازمان‌دهی کامل؛ ممکن است تقریباً همه چیز از جمله شماره‌های الزامات تغییر کند. ارزیابی مجدد برای انطباق لازم است. مثال: 5.0.0 → 4.0.3
- Minor release): ممکن است الزامات جدیدی اضافه یا حذف شوند، اما شماره‌گذاری کلی حفظ می‌شود. ارزیابی مجدد لازم است، ولی ساده‌تر خواهد بود. مثال: 5.1.0 → 5.0.0
- Patch release): ممکن است الزامات تکراری یا منسوخ حذف شوند یا سخت‌گیری آن‌ها کاهش یابد، ولی برنامه‌ای که با نسخه قبلی انطباق داشته، با نسخه وصله‌شده نیز انطباق خواهد داشت.مثال: 5.0.1 → 5.0.0

نکته: آنچه در بالا آمد، صرفاً مربوط به الزامات ASVS است. تغییرات متنی در بخش‌های جانبی یا ضمائم، به‌عنوان تغییرات بنیادی در نظر گرفته نمی‌شوند.

انع‌طاف‌پذیری در است‌فاده از ASVS

برخی از موارد پی‌ش‌گ‌فت‌ه، مانند الزامات مست‌ندس‌ازی و س‌ازوک‌ار س‌طوح، ای‌ن ام‌ک‌ان را فراهم می‌کنند ک‌ه از ASVS به‌ص‌ورت انع‌طاف‌پذی‌رتر و م‌تناس‌ب با نی‌ازه‌ای ه‌ر س‌ازمان است‌فاده ش‌ود.

ع‌لاوه بر ای‌ن، قوی‌اً ت‌وص‌ی‌ه می‌ش‌ود ک‌ه س‌ازمان‌ه‌ا ی‌ک نس‌خه ش‌خص‌ی‌س‌ازی‌ش‌ده از ASVS ای‌جاد کنند ک‌ه الزامات را با ت‌وجه به وی‌ژگ‌ی‌ه‌ا و س‌طح ری‌س‌ک برنام‌ه‌ه‌ای خ‌ود ت‌نظ‌ی‌م نم‌ای‌د. با ای‌ن ح‌ال، ح‌فظ قاب‌ل‌ی‌ت ره‌گ‌ی‌ری (traceability) الزامی است؛ به‌گ‌ون‌ه‌ای ک‌ه م‌ثلاً گ‌نراندن الزام 4.1.1 در ت‌مام نس‌خه‌ه‌ا مع‌نای ی‌ک‌س‌انی داش‌ته باش‌د.

در ح‌الت ای‌ده‌آل، ه‌ر س‌ازمان ب‌ای‌د نس‌خه اخ‌ت‌ص‌اص‌ی ASVS خ‌ود را ت‌ه‌ی‌ه ک‌ند و ب‌خ‌ش‌ه‌ای نامرب‌وط مانند GraphQL، WebSockets ی‌ا SOAP (در ص‌ورت ع‌دم است‌فاده) را ح‌ذف نم‌ای‌د. چ‌ن‌ی‌ن نس‌خه‌ای ه‌مچ‌ن‌ی‌ن مح‌ل م‌ناس‌ب‌ی برای اف‌زودن راه‌نم‌ای‌ی‌ه‌ای خ‌اص س‌ازمان، مانند ک‌تاب‌خانه‌ه‌ا ی‌ا م‌نابع ت‌وص‌ی‌ه‌ش‌ده برای پ‌ی‌اده‌س‌ازی الزامات، خ‌واه‌د ب‌ود.

نح‌وه ارجاع به الزامات ASVS

ه‌ر الزام دارای ش‌ناس‌ه‌ای با ق‌الب `<chapter>.<section>.<requirement>` است ک‌ه ه‌ر ب‌خ‌ش ی‌ک ع‌دد می‌باش‌د. برای م‌ثال :`1.11.3`.

- م‌قدار `<chapter>` ن‌ش‌ان‌ده‌ن‌ده ف‌ص‌ل الزام است؛ برای ن‌م‌ون‌ه، ت‌مام الزامات `1.#.#` م‌ت‌ع‌ل‌ق به ف‌ص‌ل "رم‌زگ‌ذاری و پ‌اک‌س‌ازی" ه‌س‌ت‌ند.
- م‌قدار `<section>` ن‌ش‌ان‌ده‌ن‌ده ب‌خ‌ش درون آن ف‌ص‌ل است؛ م‌ثلاً ت‌مام الزامات `1.2.#` در ب‌خ‌ش "پ‌ی‌ش‌گ‌ی‌ری از ت‌زری‌ق" ف‌ص‌ل رم‌زگ‌ذاری و پ‌اک‌س‌ازی قرار دارند.
- م‌قدار `<requirement>` ب‌ی‌ان‌گ‌ر ش‌م‌اره الزام خ‌اص درون ف‌ص‌ل و ب‌خ‌ش است؛ م‌ثلاً `1.2.5` ک‌ه در نس‌خه 5.0.0 چ‌ن‌ی‌ن ت‌ع‌ری‌ف ش‌ده است:

> ت‌أی‌ی‌د ش‌ود ک‌ه ب‌رنام‌ه از OS Command injection م‌حاف‌ظت ک‌ند و ف‌راخ‌وان‌ی‌ه‌ای س‌ی‌س‌ت‌م‌ع‌ام‌ل از پ‌رس‌وج‌وی پ‌ارام‌ت‌ری با Encoding م‌ناس‌ب است‌فاده نم‌ای‌د.

از آن‌جا ک‌ه ش‌ناس‌ه‌ا م‌م‌ک‌ن است ب‌ی‌ن نس‌خه‌ه‌ای م‌خ‌ت‌ل‌ف ت‌غ‌ی‌ی‌ر ک‌نند، ت‌وص‌ی‌ه می‌ش‌ود ک‌ه در اس‌ناد، گ‌زارش‌ه‌ا ی‌ا اب‌زاره‌ا از ق‌الب زی‌ر است‌فاده ش‌ود: `v<version>-<chapter>.<section>.<requirement>` ک‌ه در آن version ن‌ش‌ان ده‌ن‌ده ب‌رچ‌س‌ب نس‌خه ASVS می‌باش‌د. برای م‌ثال: `v5.0.0-1.2.5` به‌ص‌ورت دق‌ی‌ق ب‌ه الزام پ‌نج‌م از ب‌خ‌ش « Injection Prevention در ف‌ص‌ل « Encoding and Sanitization در نس‌خه 5.0.0 اش‌اره دارد. (ای‌ن ق‌الب را می‌ت‌وان به‌ص‌ورت خ‌لاص‌ه‌ش‌ده ب‌ه ش‌ک‌ل `v<version>-<requirement_identifier>` نم‌ای‌ش داد.)

ن‌ک‌ت‌ه: ح‌رف v پ‌ی‌ش از ش‌م‌اره نس‌خه ب‌ای‌د ه‌مواره با ح‌روف ک‌وچ‌ک نوش‌ت‌ه ش‌ود.

اگ‌ر ش‌ناس‌ه‌ا ب‌دون ع‌ن‌ص‌ر`v<version>` است‌فاده ش‌ون‌د، ب‌ای‌د ف‌رض ش‌ود ک‌ه به آخ‌ری‌ن م‌ح‌ت‌وای ASVS اش‌اره دارند. با گ‌س‌ت‌رش و ت‌غ‌ی‌ی‌ر است‌ان‌دارد، ای‌ن ک‌ار م‌ش‌ک‌ل‌س‌از می‌ش‌ود؛ به ه‌م‌ی‌ن دل‌ی‌ل ت‌وص‌ی‌ه می‌ش‌ود نوی‌س‌ن‌دگ‌ان و ت‌وس‌ع‌ه‌ده‌ن‌دگ‌ان ه‌مواره نس‌خه را نی‌ز ذک‌ر ک‌نند.

ف‌ه‌رست الزامات ASVS در ق‌الب‌ه‌ای‌ی مانند JSON،CSV و س‌ای‌ر ف‌رمت‌ه‌ا نی‌ز برای ارجاع ی‌ا است‌فاده ب‌رنام‌ه‌نوی‌س‌ی در دس‌ت‌رس است.

فورک کردن ASVS (Forking)

سازمان‌ها می‌توانند با انتخاب یکی از سه سطح یا ایجاد نسخه فورک‌شده‌ی دامنه‌محور، از مزای‌ای ASVS به‌ره‌مند شوند. این نسخه فورک‌شده می‌تواند الزامات را بر اساس سطح ریسک برنامه تنظیم کند. این اقدام تشویق می‌شود، مشروط بر آنکه قابلیت ره‌گیری حفظ شود (یعنی مثلاً الزام 4.1.1 در همه نسخه‌ها معنای یکسانی داشته باشد).

در حالت ایده‌آل، هر سازمان باید نسخه‌ای متناسب با نیازهای خود از ASVS تهیه کند و بخش‌های غیرمرتبط (مانند GraphQL، SOAP Websockets در صورتی که مورد استفاده قرار نمی‌گیرند) را حذف نماید. ایجاد نسخه‌ی اختصاصی (forking) باید با سطح ۱ ASVS به‌عنوان پایه آغاز شود و سپس بر اساس سطح ریسک برنامه، به سطوح ۲ یا ۳ ارتقاء یابد.

موارد استفاده از ASVS

ASVS می‌تواند برای ارزیابی امنیت یک برنامه کاربردی مورد استفاده قرار گیرد. که این موضوع در فصل بعد با جزئیات بیشتری بررسی خواهد شد. با این حال، موارد استفاده‌ی دیگری نیز برای ASVS (یا نسخه‌های فورک‌شده آن) شناسایی شده‌اند.

به‌عنوان راهنمای دقیق معماری امنیتی

یکی از رایج‌ترین کاربردهایASVS، استفاده از آن به‌عنوان مرجعی برای معماران امنیتی است. منابع محدودی برای نحوه طراحی معماری ایمن برای برنامه‌های مدرن وجود دارد ASVS می‌تواند خلأ موجود را با ارائه کنترل‌های مناسب برای چالش‌های رایج، مانند الگوهای حفاظت از داده‌ها یا استراتژی‌های اعتبارسنجی ورودی، پر کند. الزامات مربوط به معماری و مستندسازی در این زمینه به‌وی‌ژه مفید خواهند بود.

به‌عنوان مرجع تخصصی برای برنامه‌نویسی امن

ASVS می‌تواند به‌عنوان مبنایی برای تهیه مرجع برنامه‌نویسی امن در طول توسعه نرم‌افزار مورد استفاده قرار گیرد و به توسعه‌دهندگان کمک کند تا در حین ساخت نرم‌افزار، امنیت را به‌طور مستمر مدنظر داشته باشند. اگرچه ASVS می‌تواند پایه‌ی این مرجع باشد، اما توصیه می‌شود سازمان-ها راهنمای اختصاصی خود را تهیه کنند—راهنمایی که شفاف، یکپارچه، و ترجیحاً با مشارکت مهندسان امنیت یا معماران امنیتی تدوین شده باشد.به‌عنوان مکمل این راهنما، به سازمان‌ها تاکید می‌شود تا در صورت امکان، سازوکارها و کتابخانه‌های امنیتی تأییدشده‌ای تهیه کنند که در راهنما معرفی شده و توسعه‌دهندگان در پروژه‌های خود از آن‌ها استفاده نمایند.

به‌عنوان راهنمای آزمون‌های واحد و یکپارچه‌سازی خودکار

ASVS به‌گونه‌ای طراحی شده که بسیار قابل آزمون باشد. برخی از الزامات ماهیتی فنی دارند، در حالی‌که برخی دیگر(مانند الزامات معماری یا مستندسازی) نیاز به بررسی مستندات یا بازبینی معماری دارند. با طراحی آزمون‌های واحد و یکپارچه‌سازی که آسیب‌پذیری‌ها و سوءاستفاده‌های خاص

مرتبط با الزامات قابل‌تأیید را بررسی و fuzz می‌کنند، می‌توان اطمینان یافت که این کنترل‌ها در هر نسخه از برنامه (build) به‌درستی عمل می‌کنند. برای نمونه، می‌توان تست‌های اضافی‌ای به مجموعه تست‌ها برای یک کنترلر ورود (login controller) افزود که پارامتر نام کاربری (username) را از نظر مواردی مثل نام‌های کاربری پیش‌فرض رایج، تشخیص وجود حساب (account enumeration)، حملات brute force، تزریق LDAP و SQL، و XSS بررسی کنند. به‌صورت مشابه، تست پارامتر رمز عبور (password) نیز باید شامل بررسی رمزهای رایج، طول رمز عبور، تزریق null byte، حذف پارامتر، XSS، و موارد دیگر باشد.

برای آموزش توسعه امن

ASVS می‌تواند برای تعریف ویژگی‌های نرم‌افزار ایمن نیز مورد استفاده قرار گیرد. بسیاری از دوره‌های "برنامه‌نویسی امن" در واقع صرفاً دوره‌هایی هستند که نکاتی جزئی در مورد کدنویسی امن به آن‌ها اضافه شده است. و این لزوماً به به‌بود مهارت توسعه‌دهندگان در نوشتن کد امن کمک نمی‌کند.در عوض، دوره‌های آموزش توسعه امن می‌توانند با محوریت ASVS و تمرکز قوی بر سازوکارهایی مثبت و کاربردی آن طراحی شوند، نه صرفاً تمرکز بر فهرست Top 10 آسیب‌پذیری‌ها یا کارهایی که نباید انجام داد. ساختار ASVS همچنین یک چارچوب منطقی برای مرور گام‌به‌گام موضوعات مختلف مرتبط با ایمن‌سازی برنامه فراهم می‌کند.

به‌عنوان چارچوبی برای راهنمایی در خرید نرم‌افزار امن

ASVS یک چارچوب عالی برای کمک به خرید نرم‌افزار امن یا خدمات توسعه اختصاصی امن است. خریدار می‌تواند به‌سادگی مشخص کند که نرم‌افزاری که قصد تهیه آن را دارد، باید مطابق با سطح X از ASVS توسعه یافته باشد و از فروشنده بخواهد اثبات کند که نرم‌افزار با آن سطح از استانداردASVS منطبق است.

کاربرد عملی ASVS

تهدیدهای مختلف، انگیزه‌های متفاوتی دارند. برخی صنایع دارای دارایی‌های اطلاعاتی و فناوری منحصربه‌فرد، و همچنین الزامات انطباق قانونی خاصِ حوزه‌ی خود هستند.

به سازمان‌ها تاکید می‌شود که با توجه به ماهیت کسب‌وکار خود، ویژگی‌های خاص ریسک‌های خود را عمیقاً بررسی کنند و بر اساس این تحلیل، سطح مناسب ASVS را که با نیازهای کسب‌وکارشان مطابقت دارد، تعیین نمایند.

ارزیابی و صدور گواهی

دیدگاه OWASP در مورد گواهی‌نامه‌ها و نشان‌های اعتماد ASVS

OWASP به‌عنوان یک سازمان غیرانتفاعی و بی‌طرف از نظر تجاری، هیچ فروشنده، ارزیاب یا نرم‌افزاری را گواهی نمی‌کند. هرگونه ادعا مبنی بر گواهی‌بودن ،ASVS ارائه نشان اعتماد یا تضمین

انطباق با ASVS توسط اشخاص ثالث، به‌ه‌یچ‌وجه تأیید رسمی OWASP نیست. از این‌رو، سازمان‌ها باید نسبت به چنین ادعاهایی با احتیاط عمل کنند.

البته سازمان‌ها می‌توانند خدمات تضمین امنیت ارائه دهند، مشروط بر آنکه ادعای دریافت گواهی رسمی از سوی OWASP نداشته باشند.

نحوه تأیید انطباق با ASVS

ASVS به‌طور عام‌دانه جزئیات چگونگی ارزیابی انطباق در سطحی مانند یک راهنمای تست را تجویز نمی‌کند، اما تأکید بر برخی نکات کلیدی اهمیت دارد.

گزارش‌گیری ارزیابی

گزارش‌های سنتی تست نفوذ تنها به ارائه "موارد استثناء" می‌پردازند و صرفاً شکست‌ها را فهرست می‌کنند. اما گزارش گواهی‌نامه ASVS باید شامل موارد دامنه ارزیابی(Scope)، خلاصه‌ای از تمام الزامات بررسی‌شده، الزامات دارای استثناء یا عدم انطباق و راهنمایی برای رفع مشکلات شناسایی‌شده. برخی الزامات ممکن است به‌دلایل فنی یا عملکردی، قابل اعمال نباشند (مثلاً مدیریت نشست در هایAPI بدون وضعیت) که در این صورت، باید در گزارش ذکر شود.

دامنه ارزیابی

سازمانی که یک نرم‌افزار را توسعه می‌دهد، معمولاً همه الزامات ASVS را پیاده‌سازی نمی‌کند، زیرا برخی الزامات ممکن است برای آن نرم‌افزار خاص نامربوط یا کم‌اهمیت باشند. بنابراین، ارزیاب باید دامنه بررسی را به‌روشنی مشخص کند؛ از جمله اینکه سازمان در پی دستیابی به کدام سطح (Level) است و کدام الزامات را شامل کرده است. این شفاف‌سازی باید بر اساس آنچه در گزارش گنجانده شده انجام گیرد، نه آنچه حذف شده است. همچنین ارزیاب باید نظر خود را در مورد دلایل مستند حذف یا پیاده‌سازی‌نشدن الزامات خاص ارائه دهد.

این موضوع به مصرف‌کننده گزارش کمک می‌کند تا زمینه ارزیابی را به‌تر درک کرده و در مورد میزان اعتماد به نرم‌افزار، تصمیم آگاهانه‌تری بگیرد.

سازمان‌هایی که گواهی صادر می‌کنند می‌توانند روش‌های مختلفی برای آزمون انتخاب کنند، اما باید این روش‌ها را در گزارش خود به‌طور شفاف ذکر کنند و به‌تر است این روش‌ها قابل تکرار باشند. بسته به نوع نرم‌افزار و نیازهای آن، ممکن است برای بررسی مواردی مانند اعتبارسنجی ورودی، از روش‌هایی مثل تست نفوذ دستی یا تحلیل کدمنبع استفاده شود.

روش‌های ارزیابی

تکنیکی‌های مختلفی برای تأیید الزامات خاص ASVS ممکن است مورد نیاز باشند. علاوه بر تست نفوذ (با استفاده از اعتبارنامه‌های مختلف برای پوشش کامل برنامه)، تأیید الزامات ASVS ممکن است به دسترسی به مستندات، کدمنبع، پیکربندی، و افراد درگیر در فرآیند توسعه نیاز داشته باشد؛ به‌ویژه برای تأیید الزامات سطح ۲ (L2) و سطح ۳ (L3) ارائه شواهد مستند و قوی از

یافت‌ه‌ها با مست‌ندسازی دقیق، از جمله اوراق کاری، تصاویر صفحه (اسکری‌نشات)، اسکری‌پت‌ها و لاگ‌های تست، یک روی‌ه استان‌دارد مح‌سوب می‌شود. اجرای صرف یک ابزار خودکار بدون انجام تست‌های دقیق، برای دریافت گواهی‌نامه کافی نیست؛ چراکه هر الزام باید به‌طور قابل تأیید بررسی و آزموده شود.

استفاده از ابزاره‌ای خودکار برای تأیید الزامات ASVS همواره موضوعی مورد توجه بوده است. بنابراین، مهم است که نکاتی در مورد تست خودکار و تست جعبه سیاه (black box testing) روشن و شفاف بیان شوند.

نقش ابزاره‌ای خودکار امنیتی زمانی که ابزاره‌ای خودکار تست امنیتی مانند DAST (تست امنیتی پویا) و SAST (تست امنیتی ایستا) به‌درستی در خط ساخت (build pipeline) پیاده‌سازی شوند، ممکن است بتوانند برخی از مشکلات امنیتی را شناسایی کنند که اصولاً نباید وجود داشته باشند. با این حال، اگر این ابزاره‌ا به‌دقت پیکربندی و تنظیم نشوند، پوشش لازم را فراهم نخواهند کرد و حجم زیاد هشداره‌ای نادرست (noise) باعث می‌شود مشکلات امنیتی واقعی قابل شناسایی و رفع نباشند.

در حالی که این ابزاره‌ا ممکن است برخی از نیازمندی‌های فنی ساده‌تر مانند کدگذاری خروجی یا پاک‌سازی (sanitization) را پوشش دهند، نکته بسیار مهم این است که نمی‌توانند بسیاری از الزامات پیچیده‌تر ASVS (استان‌دارد تأیید امنیت نرم‌افزار) را، به‌ویژه آن‌های که به منطق تجاری یا کنترل دسترسی مربوط می‌شوند، به‌طور کامل بررسی کنند.

برای نیازمندی‌های که ساده نیستند، همچنان می‌توان از خودکارسازی استفاده کرد، اما باید بررسی‌های خاص برای هر نرم‌افزار نوشته شود تا این الزامات پوشش داده شوند. این بررسی‌ها می‌توانند شبیه به تست‌های واحد (unit test) یا تست‌های یکپارچه‌سازی (integration test) باشند که ممکن است قبلاً در سازمان وجود داشته باشند. بنابراین ممکن است بتوان از زیرساخت فعلی تست خودکار برای نوشتن تست‌های مخصوص ASVS استفاده کرد. اگرچه انجام این کار در کوتاه‌مدت نیاز به سرمایه‌گذاری دارد، اما مزایای بلندمدت آن در امکان بررسی مستمر الزامات امنیتی ASVS بسیار قابل توجه خواهد بود.

به طور خلاصه قابل تست بودن از طریق خودکارسازی به این معنا نیست که می‌توان صرفاً یک ابزار آماده را اجرا کرد.

نقش تست نفوذ در نسخه 4.0، سطح L1 برای انجام تست نفوذ «جعبه سیاه» (بدون دسترسی به مستندات یا کمی به‌ینه‌سازی شده بود، اما حتی در آن نسخه نیز تأکید شده بود که این روش نباید به‌عنوان فعالیت اصلی تضمین امنیتی تلقی شود و باید از آن پره‌یز کرد.

تست بدون دسترسی به اطلاعات تکمیلی، شیوه‌ای ناکارآمد و ضعیف برای ارزیابی امنیت است، چراکه فرصت بررسی کدمنبع، شناسایی ته‌دیده‌ها، کشف کنترل‌های ناقص و انجام تست جامع‌تر در زمان کمتر را از بین می‌برد.

به‌طور جدی توصیه می‌شود که از روش‌های ترکیبی (Hybrid) برای تست نفوذ استفاده شود، یعنی تست‌های که با دسترسی کامل به مستندات، کد و توسعه‌دهندگان انجام می‌شوند؛ زیرا چنین سطحی از دسترسی برای ارزیابی دقیق بسیاری از الزامات ASVS ضروری خواهد بود.

تغییرات نسبت به نسخه 4.x

مقدمه

کاربرانی که با نسخه x.4 این استاندارد آشنایی دارند، ممکن است مرور تغییرات کلیدی معرفی‌شده در نسخه 5.0 را مفید بیابند؛ تغییراتی که شامل به‌روزرسانی‌های در محتوا، دامنه و فلسفه زیربنایی استاندارد هستند.

از میان 286 الزام موجود در نسخه ،4.0.3 تنها 11 مورد بدون تغییر باقی مانده‌اند و ۱۵ مورد صرفاً اصلاحات جزئی گرامری داشته‌اند که معنای آن‌ها را تغییر نداده است. در مجموع، 109 الزام (معادل 38%) در نسخه 5.0 دیگر به‌صورت الزامات جداگانه وجود ندارند؛ که از این میان، 50 مورد حذف شده‌اند، 28 مورد به‌عنوان موارد تکراری حذف شده‌اند و 31 مورد در الزامات دیگر ادغام شده‌اند. مابقی الزامات نیز به‌نوعی مورد بازنگری قرار گرفته‌اند. حتی الزامات تغییریافته‌ای که از لحاظ محتوایی تغییر زیادی نداشته‌اند نیز به‌دلیل بازآرایی یا بازساختاردهی، شناسه‌های متفاوتی دارند.

برای تسهیل پذیرش نسخه ،5.0 اسنادی برای نگاشت الزامات ارائه شده‌اند تا به کاربران کمک کنند نحوه تطابق الزامات نسخه x.4 با نسخه 5.0 را دنبال کنند. این نگاشت‌ها به نسخه انتشار خاصی وابسته نیستند و در صورت لزوم به‌روزرسانی یا شفاف‌سازی خواهند شد.

فلسفه الزامات

دامنه و تمرکز

در نسخه x.4 الزامات متعددی گنجانده شده بودند که با دامنه مورد نظر این استاندارد همراستا نبودند؛ این موارد حذف شده‌اند. همچنین، الزاماتی که با معیارهای دامنه نسخه 5.0 سازگار نبودند یا قابلیت ارزیابی نداشتند نیز کنار گذاشته شده‌اند.

تمرکز بر اهداف امنیتی به‌جای سازوکارها

در نسخه 4.x بسیاری از الزامات بر سازوکارهای خاصی متمرکز بودند و نه بر اهداف زیربنایی امنیتی. در نسخه ،5.0 تمرکز الزامات بر اهداف امنیتی است و تنها در مواردی که یک سازوکار خاص تنها راه‌حل عملی است، به آن اشاره شده یا به‌عنوان نمونه یا راه‌نمایی مکمل آورده شده است.

این رویکرد به رسمیت می‌شناسد که روش‌های متعددی ممکن است برای رسیدن به یک هدف امنیتی وجود داشته باشند و از تجویزهای غیرضروری که می‌توانند انعطاف‌پذیری سازمان‌ها را محدود کنند، اجتناب می‌کند.

همچنین، الزامات مرتبط با یک دغدغه امنیتی واحد، در صورت لزوم با هم ادغام شده‌اند.

تصمیمات امنیتی مستند

هرچند مفهوم تصمیمات امنیتی مستند ممکن است در نسخه 5.0 جدید به‌نظر برسد، در واقع ادامه و تکامل الزاماتی در نسخه 4.0 است که به تحلیل سی‌است‌ها و مدل‌سازی تهدید اشاره داشتند.

در گذشته، برخی الزامات به‌طور ضمنی نیازمند تحلیل برای پیاده‌سازی کنترل‌های امنیتی بودند، مانند تعیین اتصالات شبکه مجاز.

به‌منظور اطمینان از در دسترس بودن اطلاعات لازم برای اجرا و ارزیابی، این انتظارات اکنون به‌طور صریح به‌عنوان الزامات مستندسازی تعریف شده‌اند تا شفاف، قابل اجرا و قابل ارزیابی باشند.

تغییرات ساختاری و فصل‌های جدید

چند فصل در نسخه 5.0 محتوای کاملاً جدیدی را معرفی کرده‌اند:

- OAuth وOIDC: با توجه به رواج گسترده این پروتکل‌ها برای واگذاری دسترسی و ورود یکپارچه ،(SSO) الزامات اختصاصی برای این حوزه تعریف شده‌اند تا سناریوهای متنوعی که توسعه‌دهندگان مکن است با آن‌ها مواجه شوند، پوشش داده شود. این حوزه مکن است در آینده به استانداردی مستقل تبدیل شود(مشابه آنچه برای حوزه موبایل و IoT در نسخه‌های قبلی رخ داد).
- :WebRTC با توجه به رشد سریع این فناوری، ملاحظات و چالش‌های امنیتی خاص آن اکنون در بخش‌ی اختصاصی پوشش داده شده‌اند.

همچنین، تلاش شده ساختار فصل‌ها به‌گونه‌ای تنظیم شود که مجموعه‌های منسجم و مرتبطی از الزامات را در کنار هم قرار دهد.

این بازسازماندهی منجر به ایجاد فصل‌های جدیدی نیز شده است:

- توکن‌های مستقل(Tokens): Self-contained که پیش‌تر زیرمجموعه مدیریت نشست بودند، اکنون به‌عنوان یک سازوکار مستقل شناخته می‌شوند و یکی از عناصر بنیادی در ارتباطات بدون وضعیت (stateless) مانند OAuth و OIDC به‌شمار می‌روند. به‌دلیل پیامدهای امنیتی خاص این نوع توکن‌ها، در نسخه x.5 یک فصل اختصاصی به آن‌ها اختصاص داده شده و برخی الزامات جدید نیز در این زمینه معرفی شده‌اند.
- امنیت فرانت‌اند وب: با توجه به پیچیدگی فزاینده برنامه‌های تحت مرورگر و معماری‌های مبتنی بر ،API الزامات امنیتی فرانت‌اند به فصل مستقلی منتقل شده‌اند.
- کدنویسی و معماری ایمن: الزاماتی که به شیوه‌های کلی امنیت مربوط می‌شدند ولی در فصل‌های قبلی جای نمی‌گرفتند، در این بخش جمع‌آوری شده‌اند.

سایر تغییرات ساختاری در نسخه 5.0 به‌منظور شفاف‌سازی مقصود اعمال شده‌اند. برای مثال، الزامات مربوط به اعتبارسنجی ورودی به‌جای قرارگیری در کنار پاک‌سازی و کدگذاری، در کنار منطق تجاری قرار داده شده‌اند تا نقش آن‌ها در اجرای قوانین تجاری را به‌تر نشان دهند.

فصل قبلی با عنوان معماری V1 به‌طور کامل حذف شده است. بخش ابتدایی آن شامل الزاماتی بود که خارج از محدوده موضوعی قرار داشتند، در حالی که بخش‌های بعدی به فصل‌های مرتبط منتقل شده‌اند و الزامات نیز در صورت لزوم بازنگری، شفاف‌سازی و حذف موارد تکراری شده‌اند.

حذف نگاشت مستقیم با استانداردهای دیگر

نگاشت مستقیم با استانداردهای دیگر از متن اصلی استاندارد حذف شده است. هدف آن است که نگاشتی با پروژه OWASP CRE (Common Requirement Enumeration) ارائه شود که به‌نوبه خود ASVS را به طیفی از پروژه‌های OWASP و استانداردهای خارجی متصل کند.

نگاشت مستقیم با CWE و NIST دیگر حفظ نمی‌شود، که در ادامه توضیح داده شده است.

کاهش وابستگی به دستورالعمل‌های هویت دیجیتال NIST

دستورالعمل‌های هویت دیجیتال (Digital Identity Guidelines (SP 800-63 مدت‌ها به‌عنوان مرجع برای کنترل‌های احراز هویت و مجوزدهی استفاده می‌شدند. در نسخه x.4 برخی فصل‌ها با ساختار و واژگان NIST هم‌راستا بودند.

با آنکه این دستورالعمل‌ها همچنان مرجع مهمی محسوب می‌شوند، این هم‌راستایی سخت‌گیرانه چالش‌های ایجاد کرده بود؛ از جمله واژگان کمتر شناخته‌شده، تکرار الزامات مشابه، و نگاشت‌های ناقص. نسخه 5.0 از این روی کرد فاصله گرفته تا وضوح و ارتباط مؤثرتر را ارتقا دهد.

فاصله گرفتن از نگاشت CWE

Common Weakness Enumeration (CWE) طبقه‌بندی مفیدی از ضعف‌های امنیتی نرم‌افزار ارائه می‌دهد. با این حال، چالش‌هایی مانند هایCWE که فقط شامل دسته‌بندی هستند، دشواری در نگاشت دقیق الزامات به یک CWE خاص، و نگاشت‌های نامشخص در نسخه x.4 باعث شد تصمیم گرفته شود که در نسخه 5.0 نگاشت مستقیم با CWE حذف شود.

بازنگری در تعریف سطوح (Levels)

نسخه x.4 سطوح امنیتی را به‌صورت) L1"حداقلی")،) L2"استاندارد") و) L3"پیشرفته") تعریف می‌کرد و تلویحاً بیان می‌کرد که تمام برنامه‌هایی که با داده‌های حساس سروکار دارند باید حداقل به سطح L2 برسند.

نسخه 5.0 به چندین ایراد در این روی کرد پرداخته است که در پاراگراف‌های ذیل آمده است:

از لحاظ عملی برخلاف نسخه x.4 که از علامت تیک () برای نمایش سطح استفاده می‌کرد، نسخه x.5 از عدد ساده در تمام قالب‌های سند (شامل markdown، PDF، DOCX، JSON، CSV و (XML استفاده می‌کند. برای حفظ سازگاری با نسخه‌های قبلی، خروجی‌های قدیمی JSON، CSV و XML با علامت تیک نیز تولید می‌شوند.

ورود ساده‌تر به سطح اول

بازخوردها نشان دادند که تعداد بالای الزامات سطح اول 120) مورد) و این‌که این سطح به‌عنوان حداقل اما ناکافی معرفی شده بود، موجب کاهش تمایل به استفاده می‌شد. نسخه ۵.۰ تلاش کرده این مانع را با تعریف سطح L1 حول الزامات لایه اول دفاعی کاهش دهد؛ نتیجه این کار الزامات کمتر و روشن‌تر

در ای‌ن س‌طح اس‌ت. ب‌رای م‌قای‌س‌ه ع‌ددی: در ن‌س‌خ‌ه 4.0.3 ت‌ع‌داد 128 ال‌زام L1 از م‌ج‌م‌وع 278(م‌ع‌ادل (46% وج‌ود داش‌ت، اما در ن‌س‌خ‌ه 5.0.0 ای‌ن ت‌ع‌داد ب‌ه 70 ال‌زام از م‌ج‌م‌وع 345 (م‌ع‌ادل (20% ک‌اه‌ش ی‌اف‌ت‌ه اس‌ت.

اف‌س‌ان‌ه ق‌اب‌ل‌ی‌ت آزم‌ون‌پ‌ذی‌ری

در ن‌س‌خ‌ه x.4 ی ک‌ی از م‌ع‌ی‌ارهای اص‌ل‌ی ان‌ت‌خ‌اب ال‌زام‌ات س‌طح 1، ق‌اب‌ل‌ی‌ت ارزی‌اب‌ی آن‌ه‌ا از ط‌ری‌ق ت‌س‌ت ن‌ف‌وذ خ‌ارج‌ی ب‌ه‌ص‌ورت "ج‌ع‌ب‌ه س‌ی‌اه" ب‌ود. اما ای‌ن روی ک‌رد ب‌ا ه‌دف واق‌ع‌ی س‌طح 1 (ح‌داق‌ل ک‌ن‌ت‌رل‌ه‌ای ام‌ن‌ی‌ت‌ی) ه‌م‌خ‌وان‌ی ک‌ام‌ل ن‌داش‌ت. ب‌رخ‌ی ک‌ارب‌ران م‌ع‌ت‌ق‌د ب‌ودن‌د س‌طح 1 ب‌رای ت‌أم‌ی‌ن ام‌ن‌ی‌ت ک‌اف‌ی ن‌ی‌س‌ت، و ب‌رخ‌ی دی‌گ‌رآن را ب‌ی‌ش از ح‌د س‌خ‌ت ب‌رای آزم‌ون ت‌ل‌ق‌ی م‌ی‌ک‌ردن‌د.

ات‌ک‌ا ب‌ه ق‌اب‌ل‌ی‌ت آزم‌ون‌پ‌ذی‌ری ب‌ه‌ع‌ن‌وان م‌ع‌ی‌ار، روی ک‌ردی ن‌س‌ب‌ی و گ‌اه گ‌م‌راه‌ک‌ن‌ن‌ده اس‌ت؛ چ‌را ک‌ه آزم‌ون‌پ‌ذی‌ر ب‌ودن ال‌زام ل‌زوماً ب‌ه م‌ع‌ن‌ای س‌ه‌ول‌ت ی‌ا ام‌ک‌ان خ‌ودک‌ارس‌ازی آزم‌ون آن ن‌ی‌س‌ت. ه‌م‌چ‌ن‌ی‌ن ال‌زام‌ات آس‌ان‌ت‌ر ب‌رای آزم‌ون ل‌زوماً پ‌راث‌رت‌ری‌ن ی‌ا آس‌ان‌ت‌ری‌ن گ‌زی‌ن‌ه‌ا ب‌رای اج‌را ن‌ی‌س‌ت‌ن‌د.

ب‌ن‌اب‌رای‌ن در ن‌س‌خ‌ه 5.0، ت‌ص‌م‌ی‌م‌گ‌ی‌ری در م‌ورد س‌طوح ع‌م‌دتاً ب‌ر اس‌اس ک‌اه‌ش ری‌س‌ک انج‌ام ش‌ده و م‌ی‌زان ت‌لاش م‌وردن‌ی‌از ب‌رای پ‌ی‌اده‌س‌ازی ن‌ی‌ز در ن‌ظر گ‌رف‌ت‌ه ش‌ده اس‌ت.

ف‌ق‌ط ری‌س‌ک م‌ط‌رح ن‌ی‌س‌ت

اس‌ت‌ف‌اده از س‌طوح ن‌س‌خ‌ه‌ای و م‌ب‌ت‌ن‌ی ب‌ر ری‌س‌ک ک‌ه ال‌زام س‌طح خ‌اص‌ی را ب‌رای ب‌رن‌ام‌ه‌ه‌ا ت‌ج‌وی‌ز م‌ی‌ک‌ن‌ن‌د، در ع‌م‌ل ب‌ی‌ش از ح‌د خ‌ش‌ک و ان‌ع‌ط‌اف‌ن‌اپ‌ذی‌ر ب‌وده اس‌ت. در واق‌ع، اول‌وی‌ت‌ب‌ن‌دی و پ‌ی‌اده‌س‌ازی ک‌ن‌ت‌رل‌ه‌ای ام‌ن‌ی‌ت‌ی ب‌ه ع‌وام‌ل م‌ت‌ع‌ددی ب‌س‌ت‌گ‌ی دارد، از ج‌م‌ل‌ه ک‌اه‌ش ری‌س‌ک و م‌ی‌زان ت‌لاش اج‌رای‌ی.

ب‌ن‌اب‌رای‌ن ت‌وص‌ی‌ه م‌ی‌ش‌ود س‌ازم‌ان‌ه‌ا س‌طح‌ی از ASVS را ان‌ت‌خ‌اب ک‌ن‌ن‌د ک‌ه م‌ت‌ن‌اس‌ب ب‌ا ب‌ل‌وغ ام‌ن‌ی‌ت‌ی آن‌ه‌ا و پ‌ی‌ام‌ی ک‌ه م‌ی‌خ‌واه‌ن‌د ب‌ه ک‌ارب‌ران خ‌ود م‌ن‌ت‌ق‌ل ک‌ن‌ن‌د، ب‌اش‌د.

V1 Encoding and Sanitization

ه‌دف

ای‌ن ف‌ص‌ل ب‌ه ض‌ع‌ف‌ه‌ای رای‌ج ام‌ن‌ی‌ت‌ی در ب‌رن‌ام‌ه‌ه‌ای وب م‌ی‌پ‌ردازد ک‌ه ن‌اش‌ی از پ‌ردازش ن‌اام‌ن داده‌ه‌ای غ‌ی‌رق‌اب‌ل‌اع‌ت‌م‌اد اس‌ت. چ‌ن‌ی‌ن ض‌ع‌ف‌ه‌ای‌ی م‌م‌ک‌ن اس‌ت م‌ن‌ج‌ر ب‌ه آس‌ی‌ب‌پ‌ذی‌ری‌ه‌ای ف‌ن‌ی م‌خ‌ت‌ل‌ف‌ی ش‌ون‌د، در ش‌رای‌ط‌ی ک‌ه داده‌ه‌ای غ‌ی‌رق‌اب‌ل‌اع‌ت‌م‌اد، م‌ط‌اب‌ق ب‌ا دس‌ت‌ور زب‌ان م‌ف‌س‌ر م‌رب‌وط‌ه ت‌ف‌س‌ی‌ر م‌ی‌ش‌ون‌د.

در ب‌رن‌ام‌ه‌ه‌ای وب م‌درن، ه‌م‌واره ب‌ه‌ت‌ری‌ن ک‌ار اس‌ت‌ف‌اده از ه‌ایAPI ام‌ن‌ت‌ر م‌ان‌ن‌د parameterized، auto- queries escaping ی‌ا templating frameworks اس‌ت. در غ‌ی‌ر ای‌ن ص‌ورت، اج‌رای دق‌ی‌ق و م‌ح‌ت‌اط‌ان‌ه‌ی escaping encoding، ی‌ا sanitization ب‌رای ام‌ن‌ی‌ت ب‌رن‌ام‌ه ح‌ی‌ات‌ی خ‌واه‌د ب‌ود.

اع‌ت‌ب‌ارس‌ن‌ج‌ی ورودی ب‌ه‌ع‌ن‌وان م‌ک‌ان‌ی‌زم «دف‌اع در ع‌م‌ق» ع‌م‌ل م‌ی‌ک‌ن‌د و از ورود م‌ح‌ت‌وای غ‌ی‌رم‌ن‌ت‌ظ‌ره ی‌ا خ‌ط‌رن‌اک ج‌ل‌وگ‌ی‌ری م‌ی‌ک‌ن‌د. ب‌ا ای‌ن ح‌ال، از آن‌ج‌اک‌ه ه‌دف اص‌ل‌ی آن ت‌ط‌اب‌ق م‌ح‌ت‌وا ب‌ا ال‌زام‌ات ع‌م‌ل‌ک‌ردی و ت‌ج‌اری اس‌ت، ال‌زام‌ات م‌رت‌ب‌ط ب‌ا آن در ف‌ص‌ل «اع‌ت‌ب‌ارس‌ن‌ج‌ی و م‌ن‌ط‌ق ک‌س‌ب‌وک‌ار» آم‌ده اس‌ت.

V1.1 ساخت‌ار Encoding and Sanitization

در بخش‌های زیر، الزامات مختص سورس کدی‌ا مختص مفسر برای پردازش امن داده‌های ناامن به‌منظور جلوگیری از آسیب‌پذیری‌های امنیتی ارائه شده‌اند. این الزامات، تزریق‌ب انجام این پردازش و محل مناسب اجرای آن را پوشش می‌دهند. همچنین، هدف آن‌ها این است که هنگام ذخیره‌سازی داده، محتوا در حالت اولیه خود باقی بمانند و به‌صورت encoded یا escaped (مثلاً HTML encoding) ذخیره نشود تا از بروز مشکلات ناشی از رمزگذاری دوگانه (double encoding) جلوگیری شود.

#	Description	Level
1.1.1	ورودی فقط یک‌بار به فرم است‌اندارد decoded یا unescaped شود، و این تنها زمانی انجام شود که انتظار می‌رود داده واقعاً در آن قالب رمزگذاری‌شده دریافت شده باشد. این فرآیند باید قبل از هرگونه پردازش بعدی روی ورودی انجام شود، و به‌هیچ‌وجه نباید پس از اعتبارسنجی یا پاک‌سازی انجام گیرد.	2
1.1.2	برنامه فرآیند encode و escape خروجی را به‌عنوان آخرین مرحله انجام می‌دهد؛ به‌گونه‌ای که این عملیات یا دقیقاً پیش از استفاده داده در مفسر مربوطه انجام شود، یا اینکه خود مفسر به‌طور داخلی این کار را انجام دهد.	2

V1.2 پیشگیری از تزریق

encode و escape خروجی‌ها زمان دریافت ورودی‌های حساس که ممکن است منجر به تزریق شوند، برای امنیت برنامه حیاتی است. معمولاً این encode و escape خروجی ذخیره نمی‌شوند، بلکه صرفاً برای ایمن‌سازی نمایش در همان لحظه در مفسر استفاده می‌شوند. انجام زودهنگام این عملیات ممکن است باعث خرابی محتوا یا بی‌اثر شدن فرایند encoding و escaping شود.

در بسیاری از موارد، کتابخانه‌های نرم‌افزاری توابع امن یا امنتری را برای انجام خودکار این کار ارائه می‌دهند، با این حال باید اطمینان حاصل شود که برای زمینه (context) موردنظر صحیح هستند.

#	Description	Level
1.2.1	encoding خروجی برای پاسخ HTTP، سند HTML یا سند XML با زمینه‌ای که داده در آن استفاده می‌شود باید متناسب باشد؛ برای مثال، در رمزگذاری کاراکترهای مربوط به عناصر HTML، صفات HTML، توضیحات HTML، CSS یا فیلدهای هدر HTTP به‌گونه‌ای عمل شود که ساختار پیام یا سند تغییر نکند.	1
1.2.2	هنگام ساخت پویا آدرس‌های URL، داده‌های غیرقابل‌اعتماد باید متناسب با زمینه استفاده‌شان رمزگذاری شوند (مثلاً URL Encoding یا base64url برای پارامترهای کوئری یا مسیر) همچنین اطمینان حاصل شود که فقط پروتکل‌های امن در URL مجاز باشند (برای نمونه، استفاده از JavaScript: یا data: ممنوع شود)	1

#	Description	Level
1.2.3	هنگام تولید پویا محتوای JavaScript (از جمله JSON)، از encoding یا escaping خروجی استفاده شود تا از تغییر ساختار پیام یا سند جلوگیری گردد (و از حملات تزریق JavaScript یا JSON پیشگیری شود).	1
1.2.4	در انتخاب داده یا اجرای کوئری‌های پایگاه داده (مانند SQL، NoSQL HQL، ...)، از parameterized queries، ORMها یا Entity Framework استفاده شود یا به روش‌های دیگر، برنامه در برابر تزریق SQL و سایر حملات تزریقی پایگاه داده ایمن شده باشد. این موضوع هنگام نوشتن stored procedureها نیز صدق می‌کند.	1
1.2.5	برنامه در برابر تزریق دستورات سیستم‌عامل (OS command injection) محافظت شده است و در فراخوانی‌های سیستم‌عامل، از کوئری‌های پارامتری‌شده سیستم‌عامل یا Encoding خروجی دستورات سیستم‌عاملی استفاده می‌شود.	1
1.2.6	برنامه در برابر آسیب‌پذیری‌های تزریق LDAP محافظت شده است، یا این‌که کنترل‌های امنیتی مشخصی برای جلوگیری از تزریق LDAP پیاده‌سازی شده‌اند.	2
1.2.7	برنامه در برابر حملات تزریقXPath، به‌وسیله‌ی استفاده از کوئری‌های پارامتری‌شده یا کوئری‌های از پیش کامپایل‌شده محافظت شده باشد.	2
1.2.8	پردازشگرهای LaTeX به‌صورت امن پیکربندی شده‌اند (برای نمونه با عدم استفاده از گزینه‌ی (shell-escape– و از فهرست مجاز دستورات (allowlist) برای جلوگیری از حملات تزریق LaTeX استفاده شده است.	2
1.2.9	برنامه کاراکترهای ویژه در عبارات Regex را escape می‌کند (معمولاً با استفاده از ،(تا این کاراکترها به‌اشتباه به‌عنوان متاکاراکتر تفسیر نشوند.	2
1.2.10	برنامه در برابر حملات تزریق CSV و فرمول (Formula Injection) محافظت شده است. برنامه باید هنگام خروجی گرفتن فایل‌های CSV، از قوانین escape تعیین‌شده در بخش‌های ۲.۶ و ۲.۷ استاندارد RFC 4180 پیروی کند. همچنین، هنگام خروجی گرفتن به فرمت CSV یا سایر فرمت‌های صفحه‌گسترده (مانند XLS، XLSX یا ODF)، اگر کاراکترهای ویژه‌ای مانند ‘=’، ‘+’، ‘-’، ‘@’، \t (tab) و (Null character) 0 در ابتدای مقدار یک فیلد قرار داشته باشند، باید با یک تک‌کوتیشن (’) escape شوند.	3

نکته: استفاده از کوئری‌های پارامتری‌شده (Parameterized Queries) یا دادهEscapingها در SQL همیشه به‌تنهایی کافی نیست. بخش‌هایی از کوئری مثل نام جدول‌ها و ستون‌ها (از جمله نام ستون‌هایی که در دستور ORDER BY استفاده شده‌اند) را نمی‌توان با escape کردن امن کرد .اگر داده‌هایی که کاربر وارد کرده (حتی بعد از escape کردن) در این بخش‌ها استفاده شوند، ممکن است:کوئری به‌درستی اجرا نشود (خطای SQL بده)، یا باعث حمله‌ی SQL Injection شود.

V1.3 پاک‌سازی (Sanitization)

ای‌ده‌آل‌تری‌ن روش محافظت در برابر استفاده از محتوای غیرقابل اعتماد در یک بستر ناامن، استفاده از encoding یا escaping متناسب با همان بستر است؛ روشی که معنای اصلی محتوای ناامن را حفظ می‌کند اما آن را برای استفاده در همان زمینه ای‌من می‌سازد — همان‌طور که در بخش قبلی به‌تفصیل توضیح داده شد.

اما اگر این امکان وجود نداشته باشد، باید از sanitization (پاک‌سازی) استفاده شود؛ یعنی حذف کاراکترها یا بخش‌های از محتوا که ممکن است خطرناک باشند. در برخی موارد، این کار ممکن است باعث تغییر معنای ورودی شود، اما از منظر امنیتی، چاره‌ای جز این وجود ندارد.

#	Description	Level
1.3.1	تمام ورودی‌های HTML غیرقابل اعتماد (مثلاً از طریق ویرایش‌گرهای WYSIWYG یا ابزارهای مشابه)، با استفاده از یک کتابخانه یا قابلیت امن و معتبر sanitization (پاک‌سازی) شوند.	1
1.3.2	برنامه استفاده از تابع eval یا سایر قابلیت‌های اجرای پویا (dynamic) کد مانند Spring Expression Language (SpEL)، اجتناب کند. در مواردی که استفاده از این قابلیت‌ها اجتناب‌ناپذیر است، ورودی‌های کاربر که در اجرای کد گنجانده می‌شوند باید پیش از اجرا، به‌طور کامل پاک‌سازی (sanitized) شوند.	1
1.3.3	بخش‌های حائز اهمیت که احتمال وقوع حملات ناشی از ورودی‌های کاربر در آن‌ها بالاست؛ باید پیش‌اپیش عملیات Sanitization جهت اقدامات ایمنی روی داده‌ها انجام شود؛ برای مثال، تنها کاراکترهایی که برای آن محتوا ایمن محسوب می‌شوند مجاز باشند، و ورودی‌های بیش‌ازحد بلند نیز کوتاه (trim) شوند.	2
1.3.4	محتوای اسکریپت‌پذیر (scriptable) فایل‌های SVG که توسط کاربر ارائه می‌شود، مورد اعتبارسنجی یا پاک‌سازی (sanitization) قرار گرفته باشد تا فقط شامل تگ‌ها و ویژگی‌هایی (attributes) باشد که برای برنامه ایمن هستند؛ برای مثال، از درج اسکریپت‌ها و عناصر خطرناک مانند foreignObject جلوگیری شده باشد.	2
1.3.5	برنامه محتوای زبانه‌ای قابل‌نویسی یا اسکریپت‌پذیر ارائه‌شده توسط کاربر، مانند Markdown، CSS یا استایل‌شیت‌های XSL، BBCode یا موارد مشابه را پاک‌سازی (sanitize) کرده یا غیرفعال کرده باشد.	2
1.3.6	برنامه در برابر حملات جعل درخواست سمت سرور (SSRF) محافظت شده باشد؛ به این صورت که داده‌های غیرقابل اعتماد را در برابر یک لیست مجاز از پروتکل‌ها، دامنه‌ها، مسیرها و پورت‌ها اعتبارسنجی کرده و کاراکترهای بالقوه خطرناک را پیش از استفاده از داده برای فراخوانی سرویس دیگر پاک‌سازی (sanitize) می‌کند.	2

#	Description	Level
1.3.7	برنامه در برابر حملات تزریق قالب (Template Injection) محافظت شده باشد؛ به این صورت که ساخت قالب‌ها بر پایه داده‌های غیرقابل اعتماد مجاز نباشد. در صورتی که هیچ راه‌حل جایگزینی وجود نداشته باشد، هرگونه ورودی غیرقابل اعتماد که به‌صورت پویا در فرآیند ایجاد قالب وارد می‌شود، باید پیش از استفاده یا پاک‌سازی (sanitize) شود یا به‌صورت سخت‌گیرانه اعتبارسنجی (validate) گردد.	2
1.3.8	برنامه، ورودی غیرقابل اعتماد را پیش از استفاده در کوئری‌های Java Naming and Directory Interface (JNDI) به‌درستی پاک‌سازی (sanitize) می‌کند و همچنین اطمینان حاصل شود که پیکربندی JNDI به‌گونه‌ای امن انجام شده تا از بروز حملات تزریق JNDI جلوگیری شود.	2
1.3.9	برنامه، محتوا را پیش از ارسال به memcache پاک‌سازی (sanitize) می‌کند تا از حملات تزریق (injection attacks) جلوگیری شود.	2
1.3.10	رشته‌های قالب‌بندی (format strings) که ممکن است هنگام استفاده، به‌شکلی ناخواسته یا مخرب تفسیر شوند، پیش از پردازش پاک‌سازی (sanitize) شوند.	2
1.3.11	برنامه ورودی‌های کاربر را پیش از ارسال به سامانه‌های ایمیل (مانند SMTP یا IMAP) پاک‌سازی (sanitize) کند تا از حملات تزریق در SMTP یا IMAP جلوگیری شود.	2
1.3.12	عبارات منظم (Regular Expressions) عاری از اجزایی هستند که باعث بازگشت‌پذیری نمایی (exponential backtracking) می‌شوند، و همچنین ورودی‌های غیرقابل اعتماد به‌درستی پاک‌سازی (sanitize) شوند تا از حملات ReDoS (انکار سرویس با استفاده از عبارات منظم) یا Runaway Regex جلوگیری گردد.	3

V1.4 حافظه، رشته و کد بدون مدیریت (Memory, String, and Unmanaged Code)

الزامات زیر به ریسک‌های ناشی از استفاده ناایمن از حافظه می‌پردازند، که معمولاً در شرایطی مطرح هستند که برنامه از زبان‌های سیستمی یا کدهای مدیریت‌نشده (unmanaged code) استفاده می‌کند.

در برخی موارد، می‌توان با تنظیم پرچم‌های کامپایلر (compiler flags) به این هدف دست یافت؛ مانند فعال‌سازی محافظت‌ها و هشدارهای مربوط به سرریز بافر (buffer overflow)، تصادفی‌سازی پشته (stack randomization)، جلوگیری از اجرای داده‌ها (DEP) و همچنین توقف فرایند ساخت (build) در صورت شناسایی عملیات ناایمن روی اشاره‌گرها، حافظه، رشته‌های قالب‌بندی، اعداد صحیح یا رشته‌های متنی.

#	Description	Level
1.4.1	برنامه از رشته‌های ایمن از نظر مدیریت حافظه، کپی حافظه‌ای ایمن‌تر و حساب‌گری ایمن‌تر روی اشاره‌گرها استفاده می‌کند تا از سرریز پشته (stack overflow)، سرریز بافر (buffer overflow) یا سرریز حافظه پویای (heap overflow) جلوگیری کرده یا آن‌ها را شناسایی کند.	2
1.4.2	از تکنیک‌های اعتبارسنجی علامت (sign validation)، محدوده (range validation) و ورودی (input validation) استفاده شده است تا از بروز سرریز عدد صحیح (integer overflow) جلوگیری شود.	2
1.4.3	حافظه و منابعی که به‌صورت پویا تخصیص داده شده‌اند، به‌درستی آزاد (release) می‌شوند و همچنین ارجاع‌ها یا اشاره‌گرها (pointers) به حافظه آزادشده حذف یا به null تنظیم می‌شوند تا از اشاره‌گرهای معلق (dangling pointers) و آسیب‌پذیری‌های استفاده پس از آزادسازی (use-after-free) جلوگیری شود.	2

V1.5 سریال‌زدایی ایمن (Safe Deserialization)

تبدیل داده‌ها از یک قالب ذخیره‌شده یا انتقال‌یافته به یک object واقعی برنامه(فرایند deserialization) در گذشته منبع بسیاری از آسیب‌پذیری‌های تزریق کد بوده است. بنابراین، انجام این فرایند باید با دقت و به‌صورت ایمن صورت گیرد تا از بروز چنین مشکلاتی جلوگیری شود.

به‌ویژه، برخی از روش‌های deserialization طبق مستندات زبان‌های برنامه‌نویسی یا چارچوب‌ها به‌عنوان ناایمن شناخته شده‌اند و نمی‌توانند هنگام کار با داده‌های غیرقابل‌اعتماد به‌صورت ایمن استفاده شوند. برای هر مکانیزم مورداستفاده، باید بررسی دقیق و مسئولانه (due diligence) انجام شود.

#	Description	Level
1.5.1	برنامه، Parserهای XML را به‌گونه‌ای پیکربندی کرده باشد که از تنظیمات محدودکننده استفاده کنند، و ویژگی‌های ناایمنی مانند resolve کردن موجودیت‌های خارجی (external entities) غیرفعال شده باشند تا از حملات XML External Entity (XXE) جلوگیری شود.	1
1.5.2	فرآیند deserialization داده‌های غیرقابل‌اعتماد، با اجرای مدیریت ایمن ورودی‌ها همراه باشد، برای مثال، استفاده از لیست مجاز (allowlist) انواع objectها یا محدود کردن انواع تعریف‌شده توسط کلاینت برای جلوگیری از حملات مرتبط با deserialization. همچنین، مکانیزم‌های deserialization که به‌طور صریح به‌عنوان ناایمن شناخته شده‌اند، نباید با داده‌های غیرقابل‌اعتماد مورد استفاده قرار گیرند.	2

#	Description	Level
1.5.3	Parser های مختلفی که در برنامه برای یک نوع داده مشخص استفاده می‌شوند (برای مثال: Parser های JSON، XML یا URL)، رفتار تحلیلی یکسانی دارند و از مکانیزم یکسانی برای encoding کاراکترها استفاده می‌کنند، تا از بروز مشکلاتی مانند آسیب‌پذیری در سازگاری JSON (JSON Interoperability) یا تفاوت در نحوه تحلیل URI یا مسیر فایلی که ممکن است در حملات RFI یا SSRF سوءاستفاده شود، جلوگیری شود.	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- OWASP LDAP Injection Prevention Cheat Sheet
- OWASP Cross Site Scripting Prevention Cheat Sheet
- OWASP DOM Based Cross Site Scripting Prevention Cheat Sheet
- OWASP XML External Entity Prevention Cheat Sheet
- OWASP Web Security Testing Guide: Client-Side Testing
- OWASP Java Encoding Project
- DOMPurify - Client-side HTML Sanitization Library
- RFC4180 - Common Format and MIME Type for Comma-Separated Values (CSV) Files

برای اطلاعات بیشتر، به‌ویژه در مورد مشکلات deserialization یا parsing، لطفاً مراجعه کنید به:

- OWASP Deserialization Cheat Sheet
- An Exploration of JSON Interoperability Vulnerabilities
- Orange Tsai - A New Era of SSRF Exploiting URL Parser In Trending Programming Languages

V2 Validation and Business Logic

هدف

یک برنامه مورد تائید باید الزامات سطح بالا را به شرح زیر برآورده سازد:

- داده‌های دریافتی برنامه با الزامات تجاری یا عملکردی مطابقت داشته باشد.
- منطق تجاری (Business Logic) دارای جریان ترتیبی بوده، به‌صورت گام‌به‌گام پردازش شود و قابل دور زدن نباشد.
- منطق تجاری شامل محدودیت‌ها و کنترل‌های برای شناسایی و جلوگیری از حملات خودکار است؛ مانند انتقال‌های مکرر با مبالغ کم یا اضافه‌کردن میلیون‌ها دوست به‌صورت تک-تک.

• در طراحی و پیاده‌سازی جریان‌های منطق تجاری با ارزش بالا، سناریوهای سوءاستفاده و کاربران مخرب در نظر گرفته شود و تدابیری نظیر محافظت در برابر جعل هویت (Spoof-ing)، دستکاری (Tampering)، افشای اطلاعات و ارتقای سطح دسترسی (Privilege Escalation) اتخاذ شده باشد.

V2.1 مستندات اعتبارسنجی و منطق کسب‌وکار (Validation and Business Logic Documentation)

مستندات اعتبارسنجی و منطق تجاری باید به‌روشنی محدودیت‌های منطق کسب و کار، قوانین اعتبارسنجی و سازگاری مناسب بین داده‌های ترکیب شده (از نظر مفهومی و متنی) را تعریف کنند، تا مشخص باشد چه مواردی باید در برنامه پیاده‌سازی شوند.

#	Description	Level
2.1.1	قوانین اعتبارسنجی ورودی‌های برنامه (مندرج در مستندات برنامه) باید چگونگی بررسی صحت داده‌ها را مطابق با ساختار مورد انتظار مشخص نمایند. این ساختارها می‌توانند شامل فرمت‌های رایج داده مانند شماره کارت اعتباری، آدرس ایمیل، شماره تلفن یا یک قالب داده‌ای داخلی باشد.	1
2.1.2	قوانین اعتبارسنجی ورودی‌های برنامه (مندرج در مستندات برنامه) باید چگونگی بررسی سازگاری مناسب بین داده‌های ترکیب شده (از نظر مفهومی و متنی) را مشخص نمایند؛ مانند تطبیق کدپستی با نام محله.	2
2.1.3	انتظارات مربوط به محدودیت‌ها و اعتبارسنجی‌های منطق کسب و کار مستند شده باشند؛ این مستندات باید شامل محدودیت‌ها در سطح هر کاربر و محدودیت‌های عمومی در سطح برنامه باشند.	2

V2.2 اعتبارسنجی ورودی (Input Validation)

کنترل‌های مؤثر در اعتبارسنجی ورودی، انتظارات کسب و کار یا عملکردی مرتبط با نوع داده‌ای که برنامه باید دریافت کند را اعمال می‌کنند. این کار منجر به کیفیت بهتر داده‌ها و کاهش دامنه حمله (Attack Surface) می‌شود. با این حال، اعتبارسنجی ورودی جایگزین استفاده از encoding ، پارامتری‌سازی یا پاک‌سازی داده‌ها هنگام استفاده از داده در سایر بخش‌های برنامه یا هنگام نمایش درخروجی نمی‌باشد.

در این زمینه، «ورودی» می‌تواند از منابع متنوعی دریافت شود، از جمله: فیلدهای فرم ،HTML درخواست‌های ،REST پارامترهای ،URL فیلدهای هدر کوکی،HTTPها، فایل‌های ذخیره‌شده در دیسک، پایگاه‌های داده، هایAPI خارجی.

یک کنترل منطق کسب و کاری مم‌کن است بررسی کند که مقدار ورودی، عددی کمتر از 100 است.یک انتظار عملکردی مم‌کن است بررسی کند که یک عدد (که تعیین می‌کند یک حلقه چند بار اجرا شود تا پردازش بیش از حد و در نتیجه وضعیت انکار سرویس(DoS) اتفاق نیافتد) از یک آستانه‌ی خاص کمتر باشد.

با این‌که اعتبارسنجی بر پایه Schema الزام صریح ندارد، ولی این روش می‌تواند مؤثرترین راه‌کار برای پوشش کامل اعتبارسنجی در هایHTTP API یا سایر واسطه‌هایی که از JSON یا XML استفاده می‌کنند باشد.

نکات مهم درباره اعتبارسنجی بر اساس Schema:

- "نسخه منتشرشده" از مشخصات JSON Schema، Validation برای استفاده در محیط تولید مناسب تلقی می‌شود، نه لزوما پایدار(Stable). هنگام استفاده از این نوع اعتبارسنجی، باید اطمینان حاصل شود که هیچ مغایرتی با الزامات زیر وجود ندارد.
- در صورت نیاز پس از رسمی و نهایی شدن استاندارد، تمامی کتابخانه‌های مورد استفاده برای JSON Schema بایدValidation به‌صورت مستمر پایش و به‌روزرسانی شوند.
- اعتبارسنجی DTD نباید مورد استفاده قرار گیرد و ارزیابی DTD در فریم‌ورک‌ها باید غیرفعال شود، تا از حملات XXE مرتبط با DTD جلوگیری شود.

#	Description	Level
2.2.1	ورودی‌ها باید به‌منظور اعمال انتظارات کسب و کار یا عملکردی مرتبط با آن‌ها اعتبارسنجی شوند.این اعتبارسنجی باید یکی از این دو روش زیر را داشته باشد، استفاده از اعتبارسنجی مثبت (Positive Validation) مبتنی بر فهرست مجاز از مقادیر، الگوها یا بازه‌های قابل قبول و تطبیق با ساختار مورد انتظار و محدودیت‌های منطقی، بر اساس قوانینی از پیش تعریف‌شده. برای سطح 1 ،(L1) تمرکز می‌تواند بر ورودی‌هایی باشد که در تصمیم‌گیری‌های خاص تجاری یا امنیتی استفاده می‌شوند. برای سطح 2 و بالاتر، این اعتبارسنجی باید شامل تمام ورودی‌ها باشد.	1
2.2.2	برنامه به‌گونه‌ای طراحی شود که اعتبارسنجی ورودی را در لایه قابل اعتمادی از سرویس (Trusted Service Layer) اعمال کند. هرچند اعتبارسنجی سمت کلایانت (Client-side Validation) باعث به‌بود تجربه کاربری می‌شود و توصیه می‌گردد، اما نباید به‌عنوان یک کنترل امنیتی اتکا شود.	1
2.2.3	برنامه باید اطمینان حاصل نماید که تركیب داده‌های مربوطه با توجه به قوانین از پیش تعریف‌شده معقول و منطقی باشد.	2

V2.3 امنیت منطق کسب‌وکار (Business Logic Security)

این بخش به الزامات کلیدی می‌پردازد که هدف آن‌ها اطمینان از پیاده سازی صحیح منطق کسب و کاری برنامه است و برنامه در برابر حملات مربوط به کسب و کار، مصون است.

#	Description	Level
2.3.1	اطمی‌نان حاصل شود که برنامه فقط جری‌انه‌ای منطق تجاری را برای همان کاربر و طبق ترتی‌ب گام‌به‌گام از پی‌ش تع‌ی‌ی‌ن شده و مورد انتظار پردازش می‌کند و امکان رد کردن مراحل (Skipping Steps) وجود ندارد.	1
2.3.2	اطمی‌نان حاصل شود که محدودی‌ت‌ه‌ای منطق تجاری مطابق با مس‌ت‌ندات برنامه پی‌اده‌س‌ازی شده‌اند، تا از س‌وءاس‌ت‌فاده از نق‌ص‌ه‌ای منطقی جل‌وگی‌ری شود.	2
2.3.3	تراکن‌ش‌ه‌ای س‌طح منطق کس‌ب‌وکار (business logic) بای‌د به‌گون‌ه‌ای اس‌ت‌فاده شوند که یا ی‌ک عمل‌ی‌ات منطق کس‌ب‌وکار به‌طور کامل با موفق‌ی‌ت انجام شود، یا در صورت بروز خطا، به حالت صحی‌ح قبل‌ی بازگردد(rollback) شود.	2
2.3.4	در منطق کس‌ب‌وکار (business logic) بای‌د اطمی‌نان حاصل شود که منابع محدود (مانند صندلی‌ه‌ای س‌ی‌نما یا بازه‌ه‌ای زمانی تحوی‌ل)، قابل رزرو دوباره (double-booked) از طری‌ق دست‌کاری منطق برنامه نباشند.	2
2.3.5	اطمی‌نان حاصل شود که جری‌انه‌ای منطق کس‌ب و کاری با ارزش بالا نی‌ازمند تای‌ی‌د چند کاربره (Multi-user Approval) هس‌تند تا از اقدامات غی‌رمجاز یا تص‌ادفی جل‌وگی‌ری شود. ای‌ن موارد می‌توانند شامل – اما نه محدود به – انتق‌ال‌ه‌ای مالی بزرگ، تای‌ی‌د قراردادها، دست‌رسی به اطلاعات طبق‌ه‌بن‌ده ش‌ده، یا غی‌رفع‌ال‌س‌ازی‌ه‌ای ای‌منی در محی‌ط‌ه‌ای تول‌ی‌د صنع‌ت‌ی باشد.	3

V2.4 مقابله با خودکارسازی (Anti-automation)

ای‌ن بخ‌ش شامل کنت‌رله‌ای ضد خودکارسازی است تا تع‌املات انسانی‌مانند الزامی شود و از درخواس‌ت‌ه‌ای خودکار بی‌ش‌ازحد جل‌وگی‌ری گردد.

#	Description	Level
2.4.1	کنت‌رله‌ای ضد خودکارسازی (Anti-Automation) در س‌ی‌س‌تم پی‌اده‌س‌ازی شده باشند تا از س‌وءاس‌ت‌فاده‌ای مانند فراخوانی بی‌ش از حد توابع برنامه جل‌وگی‌ری شود؛ چراکه ای‌ن نوع حمل‌ات می‌توانند منجر به اس‌تخراج غی‌رمجاز داده‌ها، تول‌ی‌د داده‌ای زائ‌د، مص‌رف کامل س‌ه‌می‌ه‌ها، عبور از محدودی‌ت‌ه‌ای نرخ (Rate Limit)، ایجاد اخت‌لال در س‌رویس (DoS) یا اس‌ت‌فاده بی‌ش از حد از منابع گران‌ق‌ی‌مت شوند.	2

#	Description	Level
2.4.2	بررسی کنید که جریان‌های منطق کسب‌وکار (Business Logic Flows) به گونه‌ای طراحی شده باشند که نیازمند زمان‌بندی واقعی انسان باشند، و از ارسال سریع و غیرطبیعی تراکنش‌ها (Transaction Submissions) جلوگیری کنند.	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- OWASP Web Security Testing Guide: Input Validation Testing
- OWASP Web Security Testing Guide: Business Logic Testing
- جلوگیری از خودکارسازی (Anti-automation) را می‌توان به روش‌های مختلفی انجام داد، از جمله با استفاده از OWASP Automated Threats to Web Applications
- OWASP Input Validation Cheat Sheet
- JSON Schema

V3 امنیت رابط کاربری وب (Web Frontend Security)

هدف کنترلی

این دسته از الزامات بر روی محافظت در برابر حملات‌ی تمرکز دارد که از طریق رابط وب (Web Frontend) برنامه اجرا می‌شوند. این الزامات برای راه‌کاره‌ای از نوع ماشین-به-ماشین (Machine-to-Machine) کاربردی ندارند.

V3.1 مستندسازی امنیت رابط کاربری وب (Web Frontend Security Documentation)

این بخش ویژگی‌های امنیتی مرورگر را که باید در مستندات برنامه مشخص شوند، توضیح می‌دهد.

#	Description	Level
3.1.1	در مس‌تندات برنامه بای‌د وی‌ژگی‌های امنی‌تی مربوط به مرورگری ک‌ه برنامه را اجرا می‌ک‌ند مش‌خص ش‌ده باش‌د (نظی‌ر اس‌تف‌اده از HTTPS، پش‌تی‌بانی از HTTP Strict Transport Security (HSTS)، اجرای Content Security Policy (CSP) و س‌ای‌ر م‌کانی‌زم‌های امنی‌تی مرتب‌ط با HTTP). هم‌چنی‌ن در مس‌تندات بای‌د مش‌خص ش‌ود مرورگ‌ر در ص‌ورت ع‌دم دس‌ترس‌ی به ای‌ن وی‌ژگی‌ها چه رفت‌اری بای‌د داش‌ته باش‌د. (برای م‌ثال: نم‌ای‌ش ه‌ش‌دار به ک‌اربر ی‌ا ممانع‌ت از دس‌ترس‌ی به برنامه).	3

V3.2 تف‌سی‌ر ناخواس‌ته محت‌وا (Unintended Content Interpretation)

پردازش محت‌وا ی‌ا ع‌مل‌ک‌رد مربوط به آن در ی‌ک ش‌رای‌ط زمی‌نه‌ای نادرس‌ت می‌توان‌د منج‌ر به اج‌رای ی‌ا نم‌ای‌ش محت‌وای مخرب برای ک‌اربر ش‌ود.

#	Description	Level
3.2.1	ک‌نت‌رله‌ای امنی‌تی متناس‌ب با مرورگره‌ا در هنگ‌ام پردازش پاس‌خه‌ای HTTP به منظور جل‌وگی‌ری از پردازش محت‌وا ی‌ا ع‌مل‌ک‌رد مربوط به آن در ش‌رای‌ط زمی‌نه‌ای نادرس‌ت بای‌د ب‌ک‌ارگرفت‌ه ش‌ود. (برای م‌ثال، زمانی‌ک‌ه ی‌ک ی‌API ی‌ک فای‌ل بارگذاری‌ش‌ده توس‌ط ک‌اربر ی‌ا هر منبع دی‌گری به‌طور مس‌تقی‌م درخواس‌ت می‌ش‌ود). ک‌نت‌رله‌ای مم‌ک‌ن می‌توان‌ند ش‌امل ای‌ن م‌وارد باش‌ند: ع‌دم پردازش محت‌وا م‌گ‌ر در ص‌ورت وجود هدره‌ای متناس‌ب با درخواس‌ته‌ای HTTP (مانند Sec-Fetch-*) به منظور اطم‌ی‌ان از صحی‌ح ب‌ودن پردازش محت‌وا ی‌ا ع‌مل‌ک‌رد مربوط به . اس‌تف‌اده از عبارت sandbox در هدر Content-Security-Policy ی‌ا اس‌تف‌اده از نوع attachment در هدر . Content-Disposition	1
3.2.2	اطم‌ی‌نان حاص‌ل ش‌ود ک‌ه محت‌وای‌ی ک‌ه برای نم‌ای‌ش به‌ص‌ورت متن طراحی ش‌ده نه برای پردازش به ص‌ورت HTML ، با اس‌تف‌اده از توابع امن نظی‌ر CreateTextNode ی‌ا textContent به منظور جل‌وگی‌ری از اج‌رای ناخواس‌ته محت‌وای‌ی مانند HTML ی‌ا JavaScript پردازش ش‌ود.	1
3.2.3	برنامه هنگ‌ام اس‌تف‌اده از جاواس‌ک‌ری‌پت س‌مت ک‌اربر (Client-Side JavaScript)، با اس‌تف‌اده از اع‌لام متغ‌ی‌ره‌ا به‌ص‌ورت ص‌ری‌ح، انج‌ام بررس‌ی نوع به‌ص‌ورت س‌خت‌گی‌رانه، اجت‌ناب از ذخی‌ره‌س‌ازی متغ‌ی‌ره‌ای س‌راس‌ری بر روی ش‌یء document، و پی‌اده‌س‌ازی ای‌زول‌اس‌ی‌ون ف‌ض‌ای نام (namespace) (isolation) از وق‌وع DOM Clobbering جل‌وگی‌ری می‌ک‌ند.	3

V3.3 پی‌ک‌ربندی ک‌وک‌ی (Cookie Setup)

ای‌ن بخ‌ش الزامات مربوط به پی‌ک‌ربندی ای‌من ک‌وک‌ی‌های ح‌س‌اس را تش‌ری‌ح می‌ک‌ند تا س‌طح بالات‌ری از اطم‌ی‌نان نس‌بت به ای‌نک‌ه آن‌ها توس‌ط خود برنامه ایج‌اد ش‌ده‌اند فراهم ش‌ود و هم‌چنی‌ن

از نشت محتوای آن‌ها یا تغییر نام‌های جلوگیری گردد.

#	Description	Level
3.3.1	اطمینان حاصل شود که کوکی‌ها دارای ویژگی 'Secure' هستند. در صورتی که از پیشوند '__Host-' برای نام کوکی استفاده نمی‌شود، باید پیشوند __Secure- در نام کوکی لحاظ شده باشد.	1
3.3.2	اطمینان حاصل شود که مقدار ویژگی SameSite برای هر کوکی، متناسب با هدف آن کوکی تنظیم شده باشد تا میزان آسیب‌پذیری در برابر حملات فریب سمت کلاینت نظیر CSRF کاهش یابد.	2
3.3.3	اطمینان حاصل شود که کوکی‌ها از پیشوند __Host- در نام خود استفاده می‌کنند، مگر در مواردی که کوکی به‌طور مشخص برای اشتراک‌گذاری بین چند دامنه (Hosts) طراحی شده باشد.	2
3.3.4	اطمینان حاصل شود که اگر مقدار کوکی (مثلاً توکن نشست) نباید برای اسکریپت‌های سمت کلاینت در دسترس باشد، ویژگی HttpOnly باید برای آن کوکی فعال شده باشد، و همچنین مقدار کوکی (مثلاً همان توکن نشست) تنها از طریق هدر Set-Cookie به کلاینت ارسال شود.	2
3.3.5	اطمینان حاصل شود که حداکثر مجموع نام و مقدار کوکی از ۴۰۹۶ بایت تجاوز نکند. کوکی‌هایی که بیش از این مقدار باشند توسط مرورگر ذخیره نخواهند شد، و بنابراین در درخواست‌ها ارسال نمی‌شوند؛ موضوعی که ممکن است باعث شود عملکردهایی از برنامه که به آن کوکی وابسته‌اند، غیرفعال یا مختل شوند.	3

V3.4 هدرهای مکانیزم‌های امنیتی مرورگر (Browser Security Mechanism Headers)

این بخش مشخص می‌کند که کدام هدرهای امنیتی باید در پاسخ‌های HTTP تنظیم شوند، تا هنگام پردازش پاسخ توسط مرورگر، ویژگی‌ها و محدودیت‌های امنیتی مرورگر فعال گردند.

#	Description	Level
3.4.1	اطمینان حاصل شود که در تمامی پاسخ‌های HTTP، هدر Strict-Transport-Security وجود دارد تا سیاست HSTS (HTTP Strict Transport Security) اعمال شود. حداقل مقدار max-age باید برابر با یک سال باشد. برای برنامه‌های سطح (L2) 2 و بالاتر، این سیاست باید برای تمام زیردامنه‌ها (subdomains) نیز اعمال شود.	1

#	Description	Level
3.4.2	اطمی‌نان حاصل شود که هدرAccess-Control-Allow-Origin مربوط به ،(CORS) دارای ی ک مقدار ثابت و مشخص توسط برنامه باشد یا اگر هدر Origin در درخواست مورد استفاده قرار می گیرد باید با فهرستی از مبدأهای مورد اعتماد (Allowlist) اعتبارسنجی شود. در صورتی که استفاده از مقدار * در Access-Control-Allow-Origin تنظیم شده باشد، اطمی‌نان حاصل شود که هیچ اطلاعات حساسی در پاسخ قرار ندارد.	1
3.4.3	اطمی‌نان حاصل شود که تمامی پاسخ‌های HTTP شامل هدر Content-Security-Policy باشند تا ریسک اجرای اسکریپت‌های مخرب (مانند JavaScript) مخرب کاهش یابد. دستورهای 'none' object-src و 'none' base-uri الزامی هستند .برای برنامه‌های سطح 3 ،(L3) باید سیاست CSP به‌صورت اختصاصی برای هر پاسخ(Per-response (Policy و مبتنی بر nonce یا hash تعریف شده باشد.	2
3.4.4	اطمی‌نان حاصل شود که تمامی پاسخ‌ها شامل هدر :X-Content-Type-Options nosniff باشند، این (هدر) به مرورگرها دستور می‌دهد که برای پاسخ دریافتی از Content Sniffing یا حدس‌زدن نوع محتوا (MIME Type Guessing) استفاده نکنند و آن‌ها را ملزم می کند که نوع محتوای واقعی را دقیقاً مطابق مقدار هدر Content-Type در درخواست/پاسخ در نظر بگیرند.برای مثال، پاسخ به ی ک درخواست برای ی ک فایل استایل (CSS) تنها در صورتی پذیرفته می‌شود که مقدار Content-Type در پاسخ 'text/css' باشد.این کار همچنین ام کان استفاده از قابلیت مسدودسازی خواندن منابع از مبدا متفاوت Cross-Origin Read Blocking یا (CORB) توسط مرورگر را فراهم می‌سازد.	2
3.4.5	برنامه بایدِ (وب‌سایت یا اپلی کیشن) ی ک "Referrer Policy" (سیاست ارجاع) تنظیم کرده باشد تا از نشت اطلاعات حساس از طریق هدر HTTP به نام Referer به سرویس‌های شخص ثالث جلوگیری شود. این کار را می‌توان از طریق هدر پاسخ HTTP به نام Referrer-Policy یا با استفاده از ویژگی‌های درعناصر HTML انجام داد.اطلاعات حساس ممکن است شامل مسیر (Path) و داده‌های کوئری (Query) در آدرس URL باشد، و در مورد برنامه‌های داخلی و غیرعمومی، حتی نام میزبان (hostname) نیز می‌تواند جزو اطلاعات حساس محسوب شود.	2
3.4.6	برنامه بایدِ از frame-ancestors در هدر (CSP) Content-Security-Policy برای تمام پاسخ‌های HTTP استفاده کند،تا به‌صورت پیش‌فرض، اجازه‌ی نمایش سایت در قالب iframe داده نشود، و فقط در صورت نیاز، نمایش منابع خاص (specific resources) در iframe مجاز باشد.به این نکته توجه کنید که هدر X-Frame-Options با اینکه هنوز توسط برخی مرورگرها پشتیبانی می‌شود، اما قدیمی (obsolete) است و نباید به آن تکیه کرد.	2
3.4.7	هدر (CSP) Content-Security-Policy بایدِ شامل ی ک آدرس برای گزارش (Report violation) باشد.	3

#	Description	Level
3.4.8	تمام پاسخ‌های HTTP که باعث رندر (نمایش) یک سند می‌شوند مانند (پاسخ‌های با نوع محتوا text/html) باید شامل هدر Cross-Origin-Opener-Policy با دستور same-origin یا same-origin-allow-popups باشند (بنا به نیاز). این کار از حملات جلوگیری می‌کند که از دسترسی مشترک به اشیای Window سوءاستفاده می‌کنند، مانند حملات tabnabbing و frame counting.	3

V3.5 تفکیک مبدا توسط مرورگر (Browser Origin Separation)

وقتی یک درخواست برای انجام عملکرد حساس در سمت سرور پذیرفته می‌شود، برنامه باید اطمینان حاصل کند که این درخواست واقعاً از طرف خود برنامه یا یک منبع مورد اعتماد ارسال شده و توسط مهاجم جعل نشده است.

عملکرد حساس در این زمینه می‌تواند شامل مواردی مانند ارسال فرم‌ها برای کاربران تأییدشده یا تأییدنشده (مثلاً درخواست احراز هویت)، عملیات تغییر وضعیت (state-changing operations)، یا عملکردهایی که منابع زیادی مصرف می‌کنند (مانند خروجی گرفتن از داده‌ها) باشد.

مکانیزم‌های کلیدی برای محافظت در برابر چنین حملاتی شامل سیاست‌های امنیتی مرورگرها مانند Same Origin Policy برای جاوااسکریپت و همچنین منطق SameSite برای کوکی‌ها هستند. یکی دیگر از روش‌های رایج برای محافظت، مکانیزم CORS preflight است. این مکانیزم به‌ویژه برای اندپوینت‌هایی که قرار است از یک مبدأ متفاوت فراخوانی شوند حیاتی است، اما حتی برای اندپوینت‌هایی که نباید از مبدأهای دیگر فراخوانی شوند نیز می‌توانند به‌عنوان یک لایه محافظتی مؤثر در برابر جعل درخواست Request Forgery عمل کنند.

#	Description	Level
3.5.1	برنامه باید از مکانیزم Preflight در CORS برای جلوگیری از درخواست‌های cross-origin به عملکردهای حساس استفاده نمی‌کنند، این درخواست‌ها به گونه‌ای اعتبارسنجی شوند که فقط از مبدأ اصلی برنامه ارسال شده باشند. این اعتبارسنجی می‌تواند با استفاده از توکن‌های anti- forgery (tokens یا الزام به ارسال هدرهای HTTP اضافی انجام شود که در لیست هدرهای مجاز (CORS-safelisted) CORS قرار ندارند. این اقدام برای محافظت در برابر حملات جعل درخواست از طریق مرورگر، که با نام CSRF (Cross-Site Request Forgery) شناخته می‌شوند، ضروری است.	1
3.5.2	اگر برنامه برای جلوگیری از استفاده غیرمجاز cross-origin از عملکردهای حساس، به مکانیزم Preflight در CORS متکی است، باید امکان فراخوانی این عملکردها با درخواست‌هایی که باعث ایجاد Preflight نمی‌شوند، وجود نداشته باشد. برای تحقق این هدف ممکن است لازم باشد مقدار هدرهای Origin و Content-Type بررسی شوند یا از هدر اضافی استفاده شود که در لیست هدرهای مجاز CORS قرار ندارد.	1

#	Description	Level
3.5.3	برنامه بایـد درخواسـت‌های HTTP کـه بـه عـمل کـرده‌ای حـساس از مـتـده‌ای مـناسـب مانند ،POST ،PUT PATCH یا DELETE اسـتـفـاده مـی کـنند، و نـه از مـتـده‌ای‌ی کـه در اسـتـاندارد HTTP بـه‌عـنوان "ای‌مـن" تـعـریـف شـده‌اند، مـانند ،HEAD GET یا OPTIONS. بـه عـنوان جـای گـزیـن یـا مـکـمل، مـی‌تـوان از بـررسـی سـخـت گـی‌رانه هـدره‌ای Sec-Fetch-* اسـتـفـاده کـرد تـا اطـمی‌نان حـاصـل شـود کـه درخواسـت از مـنبـع نـامـناسـبـی مـثـل بـارگـذاری تـصـویـر، نـاوبـری صـفـحـه یـا فـراخوانی cross-origin ارسـال نـشـده اسـت.	1
3.5.4	برنامه‌های مجزا بایـد روی نـام‌های دامـنه (hostname) مـتـفـاوت مـی‌زبـانی شـده‌اند تـا از مـحـدودی‌تـه‌ای ارائـه‌شـده تـوسـط (Same-Origin Policy) بـه‌رهـمـند شـوند. ایـن شـامـل کـنـتـرل تـعـامـل اسـناد یـا اسـکـریـپـت‌های بـارگـذاری‌شـده از یـک مـبـدأ بـا مـنابـع مـبـدأ دیـگـر و اعـمـال مـحـدودیـت بـر اسـاس دامـنه بـرای کـوکـی‌ها مـی‌شـود.	2
3.5.5	برنامه بایـد پـیـام‌های دریـافـت‌شـده از طـریـق رابـط postMessage در صـورتـی کـه مـبـدأ پـیـام مـورد اعـتـمـاد نـباشـد یـا سـاخـتـار پـیـام نـامـعـتـبـر بـاشـد، نـادیـده گـرفـتـه مـی‌شـوند.	2
3.5.6	هـیـچ کـجـای بـرنـامه از قـابـلـیـت JSONP اسـتـفـاده نـشـود تـا از حـمـلات Cross-Site Script Inclusion (XSSI) جـلـوگـیـری شـود.	3
3.5.7	داده‌ای‌ی کـه نـیـاز بـه احـراز هـویـت دارنـد، در پـاسـخ‌های مـربـوط بـه مـنابـع اسـکـریـپـتـی(مانند فـایـل‌های JavaScript) نـبـایـد قـرار بـگـیـرنـد تـا از حـمـلات XSSI جـلـوگـیـری شـود.	3
3.5.8	اطـمی‌نان حـاصـل کـنـی‌د کـه مـنابـع احـراز هـویـت‌شـده (مانند تـصـاویـر، ویـدیـوها، اسـکـریـپـت‌ها و سـایـر اسـناد) فـقـط در مـواقـعـی‌ کـه هـدف آن اسـت، قـابـل بـارگـذاری یـا جـاسـازی تـوسـط کـاربـر بـاشـنـد. ایـن کـار را مـی‌تـوان بـا اعـتـبـارسـنـجـی دقـیـق هـدره‌ای HTTP مانند Sec-Fetch-* انـجـام داد تـا اطـمی‌نان حـاصـل شـود کـه درخواسـت از یـک مـنبـع غـیـرمـجـاز ارسـال نـشـده اسـت، یـا بـا تـنـظـیـم هـدر Cross-Origin-Resource-Policy بـرای مـحـدود کـردن دسـتـرسـی مـرورگـر بـه مـحـتـوا.	3

V3.6 یکپارچگی منابع خارجی (External Resource Integrity)

این بخش، راهنمایی‌هایی برای می‌زبانی ایمن محتوا در سایت‌های شخص ثالث (Third-party) ارائه می‌دهد.

#	Description	Level
3.6.1	منابع سمت کلای‌نت مانند کتابخانه‌های جاوااسکریپت، فای‌ل‌های CSS، یا فونت‌های وب، تنها در صورتی به‌صورت خارجی (مثلاً از طریق شبکه تحویل محتوا - CDN) می‌زبانی شوند که آن منابع: استاتیک (تغییرناپذیر) باشند، نسخه‌گذاری‌شده (versioned) باشند و یکپارچگی آن‌ها با استفاده از Subresource Integrity (SRI) بررسی شود. اگر این موارد امکان‌پذیر نیست، باید برای هر منبع، یک تصمیم امنیتی مستند وجود داشته باشد که توجیه کند چرا این منبع به این شکل بارگذاری می‌شود.	3

V3.7 سایر ملاحظات امنیتی مرورگر (Other Browser Security Considerations)

این بخش شامل مجموعه‌ای از کنترل‌های امنیتی و ویژگی‌های امنیتی مدرن مرورگر است که برای افزایش امنیت در سمت کلاینت (مرورگر کاربر) توصیه می‌شوند.

#	Description	Level
3.7.1	برنامه صرفا باید از فناوری‌های سمت کلاینتی استفاده کند که همچنان پشتیبانی می‌شوند و از نظر امنیتی قابل اعتماد هستند.مثال‌هایی از فناوری‌هایی که این الزام را برآورده نمی‌کنند عبارتند از NSAPI NACL، Silverlight، ActiveX، Shockwave، Flash، Plugins و هایApplet جاوا در سمت کلاینت	2
3.7.2	برنامه فقط زمانی کاربر را به دامنه‌ها یا هایhostname دیگر(که تحت کنترل برنامه نیستند) به صورت خودکار هدایت می‌کند، که آن مقصد در یک فهرست مجاز (allowlist) قرار داشته باشد.	2
3.7.3	برنامه باید هنگام ریدایرکت کاربر به یک URL که خارج از کنترل برنامه است، یک پیام هشدار نشان می‌دهد و امکان لغو آن را به کاربر می‌دهد.	3
3.7.4	باید دامنه اصلی برنامه، مثلاً (site.tld) به فهرست preload عمومی HSTS اضافه شده باشد.	3
3.7.5	اگر مرورگر کاربر از ویژگی‌های امنیتی مورد انتظار برنامه پشتیبانی نمی‌کند، برنامه رفتار مناسبی داشته باشد، مثل هشدار دادن به کاربر یا مسدود کردن دسترسی.	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- Set-Cookie __Host- prefix details

- OWASP Content Security Policy Cheat Sheet
- OWASP Secure Headers Project
- OWASP Cross-Site Request Forgery Prevention Cheat Sheet
- HSTS Browser Preload List submission form
- OWASP DOM Clobbering Prevention Cheat Sheet

V4 API and Web Service

هدف

برخی ملاحظات امنیتی صرفا برای برنامه‌هایی که هایAPI را برای استفاده توسط مرورگره‌ای وب یا مصارف دیگر(معمولاً مبتنی بر ,XML, GraphQl JSON می باشند(ارائه می‌دهند، مطرح می‌شوند. این فصل شامل پیکربندی‌ها و مکانیزم‌های امنیتی مرتبطی است که باید به این نوع برنامه‌ها اعمال شوند.

توجه کنید نگرانی‌های عمومی امنیتی مانند احراز هویت، مدیریت نشست و اعتبارسنجی ورودی‌ها که در فصل‌های دیگر مطرح شده‌اند، در مورد هاAPI نیز صدق می‌کنند. بنابراین این فصل نباید به صورت جداگانه یا خارج از زمینه کلی استاندارد، بررسی یا آزموده شود.

V4.1 Generic Web Service Security

این بخش به ملاحظات کلی امنیت سرویس‌های وب می‌پردازد و در نتیجه بر رعایت استانداردهای اولیه و اصول به‌داشتی امنیتی در طراحی و پیاده‌سازی سرویس‌های وب تأکید دارد.

#	Description	Level
4.1.1	اطمینان حاصل شود که در هر پاسخ HTTP که شامل Body می باشد، هدر Content-Type وجود داشته باشد که نوع آن، با محتوای پاسخ دریافتی از سرور مطابقت داشته باشد. این هدر باید شامل پارامتر charset نیز باشد تا رمزگذاری کاراکتر ایمن (مانند UTF-8 یا ISO-8859-1) را مشخص کند. این مقادیر باید مطابق با استانداردIANA Media Types باشند؛ برای مثال، انواعی مانند ،/text" "/+xml" و "/xml" را در بر می‌گیرند.	1
4.1.2	اطمینان حاصل شود که تنها نقاط پایانی که برای استفاده کاربرانه‌ای از طریق مرورگر طراحی شده‌اند (User-Facing Endpoints) به‌صورت خودکار از HTTP به HTTPS هدایت شوند، سایر سرویس‌ها یا نقاط پایانی نباید (ریدایرکت شفاف، (Transparent Redirect داشته باشند. این اقدام برای جلوگیری از وضعیتی است که در آن یک کلاینت به اشتباه درخواست‌های رمزنگاری‌نشده (HTTP) ارسال می‌کند، اما چون این درخواست‌ها به‌طور خودکار به HTTPS ریدایرکت می‌شوند، نشت احتمالی داده‌ای حساس پنهان خواهد ماند.	2

#	Description	Level
4.1.3	اطمی‌نان حاصل شود که هدره‌ای HTTP که در برنامه استفاده می شود و توسط ی ک لایه واسطه، مانند ی کbalancer load ، ی ک پروکسی وب یا ی ک سرویس backend-for-frontend تنظیم شده است نمی‌تواند توسط کاربر نهایی بازنویسی شود. هدره‌ای نمونه ممکن است شامل X-Real-IP، X-Forwarded-* یا X-User-ID باشند.	2
4.1.4	اطمی‌نان حاصل شود که فقط متدهای HTTP که صریحاً توسط برنامه یا API آن پشتیبانی می‌شوند (از جمله OPTIONS در درخواست‌های preflight) می‌توانند مورد استفاده قرار گیرند و متدهای استفاده نشده مسدود شوند.	3
4.1.5	اطمی‌نان حاصل شود که برای درخواست‌ها یا تراکنش‌هایی که بسیار حساس هستند یا از چندین سامانه عبور می کنند، از امضاهای دیجیتال در سطح پیام استفاده شود تا اطمی‌نان امنیتی بیشتری علاوه بر محافظت‌های سطح انتقال (مانند TLS فراهم گردد.	3

V4.2 HTTP Message Structure Validation

این بخش به‌صورت دقیق مشخص می کند که ساختار و هدره‌ای درخواست HTTP باید چگونه اعتبارسنجی شوند تا از حملاتی مانند HTTP Request Smuggling، response splitting، header injection و denial of service از طریق ی ک پیام طولانی HTTP جلوگیری شود.

این الزامات برای پردازش و تولید کلی پیام‌های HTTP اهمیت دارند، اما به‌ویژه هنگام تبدیل پیام‌های HTTP بین نسخه‌های مختلف HTTP مهم هستند.

#	Description	Level
4.2.1	اطمی‌نان حاصل شود که تمامی اجزای برنامه)از جمله لود بالانسرها، فایروال‌ها و سرورهای اپلی کیشن(مرزهای پیام‌های HTTP ورودی را با استفاده از مکانیزم مناسب نسخه های HTTP تعیین می کنند تا از حملات HTTP request smuggling جلوگیری شود. در HTTP/1.x اگر هدر Transfer-Encoding وجود داشته باشد، هدر Content-Length باید مطابق RFC 2616 نادیده گرفته شود. در مقابل، هنگام استفاده از HTTP/2 یا HTTP/3، اگر هدر Content-Length وجود داشته باشد، گیرنده باید اطمی‌نان حاصل کند که این مقادیر با طول فریم‌های DATA سازگار می‌باشد.	2
4.2.2	اطمی‌نان حاصل شود که هنگام تولید پیام‌های HTTP، هدر Content-Length با طول محتوای ی که توسط چارچوب‌بندی پروتکل HTTP تعیین می‌شود، تداخل نداشته باشد تا از حملات request smuggling جلوگیری شود.	3
4.2.3	اطمی‌نان حاصل شود برنامه، پیام‌های HTTP/2 یا HTTP/3 با هدره‌ای connection-specific مانند Transfer-Encoding را به منظور جلوگیری از حملات response splitting یا header injection ارسال یا قبول نمی کند.	3

#	Description	Level
4.2.4	اطمی‌نان حاصل شود برنامه تنها درخواست های HTTP/2 and HTTP/3 که هدرها و مقادیر آن شامل هیچ توالی CR (\r), LF (\n), CRLF (\r\n) نباشد را می پذیرد تا از حملات header injection جلوگیری کند.	3
4.2.5	اطمی‌نان حاصل کنید که اگر برنامه (Frontend or Backend) اقدام به ساخت و ارسال درخواست‌های HTTP می کند از مکانیزم های validation ، sanitization یا سایر مکانیزم‌ها استفاده کند تا از ایجاد هاURI (مانند فراخوانیAPI (یا فیلدهای هدر درخواست HTTP (مانند Authorization یا(Cookie که بیش از حد مجاز، طولانی هستند، جلوگیری شود که منجر به عدم پذیرش درخواست توسط مولفه دریافت کننده(هر بخشی از زیرساخت یا سامانه که درخواست HTTP را دریافت و پردازش می کند) شود. عدم رعایت این موضوع می تواند باعث وقوع حمله Danial of Service شود، ارسال یک درخواست بیش از حد طولانی (مانند یک فیلد هدر کوکی طولانی) باعث می شود سرور همواره با وضعیت خطا پاسخ دهد.	3

V4.3 GraphQL

GraphQL به‌طور فزاینده‌ای در حال تبدیل شدن به روشی رایج برای توسعه‌ی کلاینت‌های داده‌محور است، که به‌صورت مستقیم به چندین سرویس مختلف در backend وابسته نیستند. این بخش به ملاحظات امنیتی ویژه مربوط به استفاده از GraphQL می‌پردازد.

#	Description	Level
4.3.1	اطمی‌نان حاصل شود یکی از مکانیزم ها برای جلوگیری از حملات انکار سرویس (DoS) درلایه GraphQL یا لایه داده استفاده شده است. این مکانیزم‌ها عبارتند از استفاده از فهرست مجاز کوئری‌ها (Query ،(Allowlist محدودسازی عمق کوئری‌ها Depth) محدودسازی،(Limiting تعداد یا حجم داده(Amount (Limiting و تحلیل هزینه کوئری .(Query Cost Analysis) این اقدامات، به منظور مقابله با درخواست های پیچیده، تو در تو یا کوئری‌های پرهزینه که می توانند موجب مصرف بیش از حد منابع و اختلال در سرویس دهی شوند ضروری است.	2
4.3.2	اطمی‌نان حاصل شود که قابلیت Introspection در GraphQL در محیط تولید (Production) غیرفعال شده‌است، مگرAPI مبتنی بر GraphQL به‌صورت صریح برای استفاده توسط اشخاص یا سامانه های ثالث (Third Parties) طراحی شده باشد.	2

V4.4 WebSocket

WebSocket یک پروتکل ارتباطی است که امکان ارتباط دوطرفه همزمان را از طریق یک اتصال TCP فراهم می کند. این پروتکل در سال ۲۰۱۱ توسط IETF در قالب RFC 6455 استانداردسازی شد.

اگ‌رچه WebSocket برای کار کردن از طریق پورت‌های رایج HTTP یعنی پورت 443 (HTTPS) و پورت 80 (HTTP) طراحی شده، اما پروتکل‌های مجزا از HTTP محسوب می‌شود.

این بخش به الزامات کلیدی امنیتی می‌پردازد که برای جلوگیری از حملاتی طراحی شده‌اند که از طریق این کانال ارتباطی بلادرنگ، به‌ویژه در حوزه‌های امنیت ارتباطات و مدیریت نشست انجام می‌گیرند.

#	Description	Level
4.4.1	اطمینان حاصل شود که تمامی ارتباطات WebSocket از طریق پروتکل TLS (WSS) برقرار می‌شوند.	1
4.4.2	اطمینان حاصل شود که در هنگام Handshake اولیه‌ی HTTP برای WebSocket، مقدار هدر Origin بررسی شود و در صورتی که با لیست از مبداهای مجاز (allowed origins) برای برنامه مطابقت داشته باشد، اتصال پذیرفته می‌شود.	2
4.4.3	اطمینان حاصل شود که در صورتی که امکان استفاده از مکانیزم استاندارد مدیریت نشست برنامه وجود نداشته باشد، از توکن‌های اختصاصی (Duplicated Tokens) برای این منظور استفاده شود که با الزامات امنیتی مدیریت نشست به طور کامل مطابقت داشته باشند.	2
4.4.4	اطمینان حاصل شود که توکن‌های اختصاصی مدیریت نشست WebSocket در هنگام انتقال از یک نشست HTTPS موجود به کانالWebSocket، از طریق نشست HTTPS احراز هویت‌شده قبلی، دریافت یا اعتبارسنجی می‌شوند.	2

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- OWASP REST Security Cheat Sheet
- Resources on GraphQL Authorization from graphql.org and Apollo.
- OWASP Web Security Testing Guide: GraphQL Testing
- OWASP Web Security Testing Guide: Testing WebSockets

V5 File Handling

هدف

استفاده از فایل‌ها می‌تواند انواع مختلفی از ریسک‌ها را برای برنامه ایجاد کند، از جمله حملات انکار سرویس (DOS)، دسترسی غیرمجاز به اطلاعات، پر شدن فضای ذخیره‌سازی. این فصل شامل مجموعه‌ای از الزامات امنیتی برای مقابله با این تهدیدات است.

V5.1 File Handling Documentation

این بخش شامل الزاماتی است که بیان می‌کند ویژگی‌های مورد انتظار برای فایل‌هایی که توسط برنامه پذیرفته می‌شوند باید مستندسازی شوند. این مستندسازی پیش‌نیازی ضروری برای طراحی و بررسی کنترل‌های امنیتی مرتبط با بارگذاری فایل‌ها است.

#	Description	Level
5.1.1	مستندات نوع فایل‌های مجاز، پسوندهای مورد انتظار، و حداکثر اندازه هر فایل (شامل اندازه پس از استخراج) را برای هر قابلیت بارگذاری مشخص کرده باشد. همچنین، مستندات توضیح می‌دهند چگونه فایل‌ها برای دانلود و پردازش توسط کاربران نهایی ایمن‌سازی می‌شوند؛ برای مثال، رفتار برنامه هنگام شناسایی یک فایل مخرب چگونه خواهد بود.	2

V5.2 File Upload and Content

قابلیت بارگذاری فایل یکی از منابع اصلی ورود فایل‌های غیرقابل اعتماد به سامانه می‌باشد. این بخش الزامات لازم را برای اطمینان از اینکه وجود، حجم یا محتوای این فایل‌ها نمی‌توانند به برنامه آسیب برسانند، تشریح می‌کند.

#	Description	Level
5.2.1	برنامه تنها فایل‌هایی با حجم مشخص، که قادر به پردازش آن است را می‌پذیرد تا از کاهش عملکرد یا حملات انکار سرویس (DoS) جلوگیری شود.	1
5.2.2	هنگام پذیرش فایل توسط برنامه، چه به صورت مستقیم و چه به صورت یک آرشیو مانند فایل zip پسوند فایل با پسوند مورد انتظار مطابقت داشته باشد و اعتبارسنجی محتوای فایل صورت گیرد تا اینکه شود محتوای فایل با نوع تعریف‌شده توسط پسوند همخوانی دارد. این فرآیند می‌تواند شامل مواردی مانند بررسی بایت‌های ابتدایی (magic bytes) فایل، انجام بازنویسی تصاویر برای حذف ساختارها و اطلاعات ناخواسته و استفاده از کتابخانه‌های تخصصی برای اعتبارسنجی کامل محتوای فایل باشد اما محدود به آن‌ها نیست. در سطح L1، کافی است این بررسی‌ها فقط بر روی فایل‌هایی که در فرآیندهای تجاری یا امنیتی خاص استفاده می‌شوند متمرکز باشد. در سطح L2 و بالاتر، این فرآیند باید برای تمامی فایل‌های ورودی برنامه اجرا شود.	1
5.2.3	برنامه باید پیش از استخراج محتوای فایل‌های فشرده (مانند zip، gz، docx، odt)، آن‌ها را از نظر حداکثر اندازه مجاز پس از استخراج و همچنین حداکثر تعداد فایل‌های درون آن بررسی می‌کند.	2

#	Description	Level
5.2.4	برنامه فضای مشخصی برای اندازه فایل و تعداد فایل‌های مجاز برای هر کاربر اعمال می‌کند تا از این طریق، یک کاربر نتواند با بارگذاری تعداد زیادی فایل یا فایل‌های بیش از حد بزرگ، فضای ذخیره‌سازی را پر کند.	3
5.2.5	برنامه اجازه بارگذاری فایل‌های فشرده که حاوی لینک های نمادین (symlinks) هستند را نمی‌دهد، مگر اینکه این رفتار به طور مشخص مورد نیاز باشد. در چنین شرایطی باید یک فهرست مجاز (allowlist) از مسیرها یا فایل‌هایی که می توانند برای آن‌ها لینک نمادین ایجاد شود، اعمال و اعتبارسنجی گردد.	3
5.2.6	برنامه تصاویر بارگذاری‌شده با اندازه پیکسلی بزرگتر از حد مجاز را رد کند تا از حملات pixel flood attacks جلوگیری شود.	3

V5.3 File Storage

این بخش شامل الزامات جلوگیری از اجرای غیرمجاز فایل‌ها پس از بارگذاری، شناسایی محتوای خطرناک و جلوگیری از استفاده از داده‌های غیرمعتبر برای کنترل مکان ذخیره‌سازی فایل‌ها است.

#	Description	Level
5.3.1	فایل هایی که از ورودی های غیرمعتبر بارگذاری یا تولید می شوند و در پوشه های عمومی قرار می گیرند، هنگام دستررسی مستقیم از طریق یک درخواست HTTP، به عنوان کدهای سمت سرور اجرا نشوند.	1
5.3.2	هنگام ایجاد مسیره‌ای فایل برای عملیات مختلف، به جای استفاده از نام فایل‌های ارسال شده توسط کاربر، از داده‌های تولید شده داخلی یا منابع مورد اعتماد استفاده شود. و در صورتی که استفاده از نام فایل‌ها یا متادیتای ارسالی توسط کاربر ضروری باشد، می بایست validation و Sanitization اعمال شود. این کار برای جلوگیری از حملاتی مانند Path Traversal، Local or Remote File Inclusion(LFI, RFI) و Server-Side Request Forgery(SSRF) می‌باشد.	1
5.3.3	پردازش فایل‌های سمت سرور، مانند استخراج فایل‌های فشرده، اطلاعات مسیر ارائه شده توسط کاربر را نادیده می گیرد تا از آسیب‌پذیری‌های مانند zip slip جلوگیری شود.	3

V5.4 File Download

این بخش شامل الزامات برای کاهش ریسک‌ها هنگام ارائه فایل‌ها برای دانلود است، از جمله حملاتPath Traversal و حملات Injection می باشد. همچنین اطمینان حاصل می‌شود که فایل‌ها حاوی محتوای خطرناک

نباشند.

#	Description	Level
5.4.1	برنامه نام فایل ارسال‌شده توسط کاربر را چه در قالب JSON، JSONP یا پارامتر URL، اعتبارسنجی می‌کند یا نادیده می‌گیرد، و در پاسخ HTTP، نام فایل را به‌طور صریح در فیلد Content-Disposition هدر مشخص می‌نماید.	2
5.4.2	نام‌های فایل‌های ارائه شده (مانند فیلدهای هدر پاسخ HTTP یا پیوست‌های ایمیل)، به‌درستی encoded یا sanitized شده‌اند (مطابق RFC 6266، تا ساختار سند حفظ شده و از حملات تزریق جلوگیری شود.	2
5.4.3	فایل‌های دریافت شده از منابع غیرقابل اعتماد توسط آنتی ویروس اسکن شود تا از ارائه محتوای مخرب شناخته‌شده جلوگیری شود.	2

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- OWASP File Upload Cheat Sheet
- Example of using symlinks for arbitrary file read
- Explanation of “Magic Bytes” from Wikipedia

V6 Authentication

Control Objective

احراز هویت فرآیند تأیید یا اثبات اصالت یک فرد یا دستگاه می‌باشد. این فرآیند شامل بررسی صحت ادعاهای ارائه شده توسط یک شخص یا یک دستگاه بوده به گونه‌ای که در برابر جعل هویت مقاوم باشد و از بازیابی یا رهگیری گذرواژه‌ها جلوگیری کند.

استاندارد NIST SP 800-63 یک استاندارد مدرن و مبتنی بر شواهد است که برای سازمان‌های سراسر جهان ارزشمند محسوب می‌شود و برای سازمان‌های دولتی ایالات متحده و نهادهایی که با این سازمان‌ها تعامل دارند، اهمیت ویژه‌ای دارد.

اگرچه بسیاری از الزامات مطرح شده در این فصل بر اساس بخش دوم این استاندارد (با عنوان NIST SP 800-63B تحت عنوان «راهنمای هویت دیجیتال – احراز هویت و مدیریت چرخه عمر») بنا شده اند، تمرکز این فصل بر تهدیدات رایج و ضعف‌های پرتکرار در احراز هویت می‌باشد. این فصل تلاشی برای پوشش جامع تمامی نکات استاندارد نمی‌کند. در مواردی که رعایت کامل استاندارد NIST SP 800-63 ضروری باشد، لطفاً به متن کامل این استاندارد مراجعه فرمایید.

همچنین، ممکن است اصطلاحات به‌کاررفته در NIST SP 800-63 با اصطلاحات عمومی‌تر تفاوت داشته باشد؛ به همین دلیل، این فصل عمدتاً از واژگان رایج‌تر و قابل فهم‌تر استفاده می‌کند تا وضوح مطلب افزایش یابد.

یکی از ویژگی‌های رایج در سامانه‌های پیشرفته‌تر، تطبیق‌پذیری مراحل احراز هویت با توجه به عوامل مختلف ریسک است. این ویژگی در فصل Authorization مورد بررسی قرار می‌گیرد، چرا که این مکانیزم‌ها باید در تصمیم‌گیری‌های مربوط به Authorization نیز لحاظ شوند.

V6.1 Authentication Documentation

این بخش شامل الزامات مربوط به مستندسازی احراز هویت است که باید برای یک برنامه کاربردی نگه‌داری شود. این مستندسازی برای پیاده‌سازی و ارزیابی نحوه پیکربندی کنترل‌های مرتبط با احراز هویت از اهمیت بالایی برخوردار است.

#	Description	Level
6.1.1	مستندات برنامه، نحوه استفاده از کنترل هایی مانند محدودیت نرخ (Rate Limit)، مقابله با خودکارسازی (Anti-Automation) و پاسخ تطبیقی (Adaptive Response) را برای دفاع در برابر حملاتی مانند Credential Stuffing و حملات Password Brute Force به وضوح تعریف کرده باشد. مستندات باید به‌وضوح نحوه پیکربندی این کنترل‌ها را تشریح کرده و نشان دهند که چگونه از قفل شدن مخرب حساب‌های کاربری جلوگیری می‌شود.	1
6.1.2	یک لیست از کلمات خاص مرتبط با زمینه سازمانی (Context-Specific) مستندسازی شود تا از بکارگیری آن‌ها در رمزهای عبور جلوگیری شد. این فهرست می‌تواند شامل نام سازمان، نام محصول، شناسه سامانه، نام کدپروژه، نام دپارتمان یا نقش های سازمانی و موارد مشابه باشد.	2
6.1.3	در صورت وجود مسیرهای متعدد احراز هویت در برنامه کاربردی، تمامی این مسیرها به همراه کنترل‌های امنیتی و سطح قدرت احراز هویت مورد نیاز، مستندسازی شده‌اند. لازم است این کنترل‌ها به‌صورت یکسان و سازگار در تمامی مسیرهای احراز هویت اجرا شوند.	2

V6.2 Password Security

کلمات عبور، که توسط NIST SP 800-63 به عنوان "رمزهای به خاطر سپرده شده" (Memorized Secrets) شناخته می‌شوند، شامل کلمات عبور، عبارت‌های عبور، کدهای PIN، الگوهای بازگشایی unlock) Patterns)، انتخاب تصویر صحیح (مثلاً انتخاب بچه گربه صحیح از میان چند تصویر) هستند. این‌ها معمولاً به عنوان "چیزی که شما می‌دانید" در نظر گرفته می‌شوند و اغلب به عنوان یک مکانیزم احراز هویت تک‌عاملی استفاده می‌شوند.

بر این اساس، این بخش شامل الزامات و توصیه‌های برای ایجاد و مدیریت امن کلمات عبور می‌باشد. بی‌شتر این الزامات در سطح L1 مطرح شده‌اند، چرا که در این سطح نقش کلیدی دارند. از سطح L2 به بعد، استفاده از احراز هویت چندعاملی الزامی می‌باشد، که در آن گذرواژه می‌تواند یکی از عوامل تشکیل‌دهنده باشد.

الزامات این بخش عمدتا به بند 5.1.1.2 § از راهنمای‌های NIST's Guidance می‌باشد.

#	Description	Level
6.2.1	کلمات عبور تنظیم شده توسط کاربر حداقل ۸ کاراکتر طول داشته باشد. هرچند حداقل ۱۵ کاراکتر به شدت توصیه می‌شود.	1
6.2.2	کاربران قادر به تغییر گذرواژه خود باشند.	1
6.2.3	هنگام تغییر کلمه عبور، گذرواژه فعلی و گذرواژه جدید توسط کاربر تکمیل شود.	1
6.2.4	کلمات عبوری که در حین ثبت‌نام حساب کاربری یا تغییر کلمه عبور ارسال می شوند با یک فهرست در دسترس از حداقل ۳۰۰۰ کلمه عبور رایج که) با سیاست‌های کلمه عبور برنامه (مانند حداقل طول) سازگاری دارند) بررسی شوند.	1
6.2.5	کلمات عبور با هر ترکیبی قابل استفاده باشند، بدون اینکه قوانینی برای محدودکردن نوع کاراکترهای مجاز وجود داشته باشد. نبایدهیچ گونه الزامی برای تعداد حداقل کاراکترهای حروف بزرگ یا کوچک، اعداد یا کاراکترهای خاص وجود داشته باشد.	1
6.2.6	فیلدهای ورودی کلمه عبور از نوع "type=password" استفاده می کنند تا ورودی‌ها پنهان شوند. برنامه‌ها می‌توانند به کاربر این امکان را بدهند که موقتاً کلمه عبور کامل یا آخرین کاراکتر تایپ شده را مشاهده کند.	1
6.2.7	عمل کرد "چسباندن"، (paste) ابزارهای کمکی گذرواژه مرورگر و مدیریت کننده‌های گذرواژه خارجی مجاز باشند.	1
6.2.8	برنامه کلمه عبور کاربر را دقیقاً همانطورکه از کاربر دریافت شده است، بدون هیچ گونه تغییراتی مانند کوتاه‌سازی(truncation) یا تبدیل حروف بزرگ و کوچک (case transformation)، بررسی کند.	1
6.2.9	استفاده از گذرواژه‌های با حداقل طول ۶۴ کاراکتر مجاز باشد.	2
6.2.10	کلمه عبور کاربر تا زمانی که مشخص شود که به خطر افتاده است یا کاربر آن را تغییر دهد، معتبر باقی بماند. برنامه نبایدالزام به تغییر دوره‌ای گذرواژه‌ها را داشته باشد.	2
6.2.11	فهرست مسدود شده ای از کلمات خاص مرتبط با زمینه سازمانی برای جلوگیری از ایجاد کلمات عبور آسان قابل حدس، مورد استفاده قرار گیرد.	2
6.2.12	کلمات عبور وارد شده در حین ثبت‌نام حساب کاربری یا تغییر کلمه عبور با مجموعه‌ای از کلمات عبور افشا شده بررسی شوند.	2

V6.3 General Authentication Security

این بخش شامل الزامات عمومی برای امنیت مکانیزم‌های احراز هویت و همچنین تعیین انتظارات مربوط به سطوح مختلف امنیتی می‌باشد. در سطح L2، سامانه‌ها بایدالزاماً از احراز هویت چندعاملی (MFA) استفاده کنند. در سطح L3، استفاده از احراز هویت مبتنی بر سخت‌افزار الزامی می‌باشد

که بای‌د درون ی‌ک مح‌ی‌ط اج‌رای م‌ورد اع‌ت‌م‌اد و ق‌اب‌ل گ‌واه‌ی (Trusted Execution Environment – TEE) انج‌ام ش‌ود. ای‌ن م‌ی‌ت‌وان‌د ش‌ام‌ل ک‌ل‌ی‌ده‌ای ع‌ب‌ور واب‌س‌ت‌ه ب‌ه دس‌ت‌گ‌اه (Device-Bound Passkeys)، اح‌رازک‌ن‌ن‌ده‌ای ب‌ا س‌طح اط‌م‌ی‌ن‌ان ب‌الا م‌طاب‌ق ب‌ا اس‌ت‌ان‌دارد eIDAS (س‌طح LoA High)، اح‌رازک‌ن‌ن‌ده‌ای دارای س‌طح اط‌م‌ی‌ن‌ان AAL3 ط‌ب‌ق اس‌ت‌ان‌دارد NIST، ی‌ا م‌ک‌ان‌ی‌زم‌ه‌ای م‌ع‌ادل دی‌گ‌ر ب‌اش‌د.

اگ‌رچ‌ه ای‌ن الزام‌ات در م‌ق‌ای‌س‌ه ب‌ا ب‌س‌ی‌اری از چ‌ارچ‌وب‌ه‌ای دی‌گ‌ر، روی‌ک‌ردی س‌خ‌ت‌گ‌ی‌ران‌ه ن‌س‌ب‌ت ب‌ه MFA دارن‌د، ام‌ا ای‌ن س‌خ‌ت‌گ‌ی‌ری ب‌رای ارت‌ق‌ای س‌طح ام‌ن‌ی‌ت ک‌ارب‌ران ح‌ی‌ات‌ی م‌ی‌ب‌اش‌د. ه‌ر گ‌ون‌ه ت‌لاش ب‌رای ک‌اه‌ش ای‌ن الزام‌ات ب‌ای‌د ب‌ا ی‌ک ب‌رن‌ام‌ه ش‌ف‌اف و دق‌ی‌ق ب‌رای ک‌اه‌ش ری‌س‌ک‌ه‌ای م‌رت‌ب‌ط ب‌ا اح‌راز ه‌وی‌ت م‌راه ب‌اش‌د، ب‌ه‌ط‌وری‌ک‌ه راه‌ن‌م‌ای‌ه‌ا و پ‌ژوه‌ش‌ه‌ای ارائ‌ه‌ش‌ده ت‌وس‌ط NIST ن‌ی‌ز در آن ل‌ح‌اظ ش‌ده ب‌اش‌د.

ت‌وج‌ه داش‌ت‌ه ب‌اش‌ی‌د ک‌ه در زم‌ان ان‌ت‌ش‌ار ای‌ن س‌ن‌د، اس‌ت‌ان‌دارد NIST SP 800-63 اس‌ت‌ف‌اده از ای‌م‌ی‌ل را ب‌ه ع‌ن‌وان ی‌ک م‌ک‌ان‌ی‌زم اح‌راز ه‌وی‌ت ق‌اب‌ل ق‌ب‌ول ن‌م‌ی‌دان‌د)..(archived copy

الزام‌ات ای‌ن ب‌خ‌ش ب‌ه ب‌خ‌ش‌ه‌ای م‌خ‌ت‌ل‌ف‌ی از راه‌ن‌م‌ای NIST's Guidance م‌رب‌وط م‌ی‌ش‌ون‌د، از ج‌م‌ل‌ه ب‌ن‌ده‌ای 4.2.1 §، 4.3.1 §، 5.2.2 § و 6.1.2. §

#	Description	Level
6.3.1	ک‌ن‌ت‌رل‌ه‌ای ل‌ازم ب‌رای ج‌ل‌وگ‌ی‌ری از ح‌م‌لات‌ی م‌ان‌ن‌د Credential Stuffing و password brute force ط‌ب‌ق م‌س‌ت‌ن‌دات ام‌ن‌ی‌ت‌ی ب‌رن‌ام‌ه پ‌ی‌اده‌س‌ازی ش‌ون‌د.	1
6.3.2	ح‌س‌اب‌ه‌ای ک‌ارب‌ری پ‌ی‌ش‌ف‌رض (م‌ان‌ن‌د "admin"، "root" ی‌ا "sa") در ب‌رن‌ام‌ه وج‌ود ن‌داش‌ت‌ه ب‌اش‌ن‌د ی‌ا غ‌ی‌رف‌ع‌ال ش‌ده ب‌اش‌ن‌د.	1
6.3.3	ب‌رای دس‌ت‌رس‌ی ب‌ه ب‌رن‌ام‌ه، از ی‌ک م‌ک‌ان‌ی‌زم اح‌راز ه‌وی‌ت چ‌ن‌دع‌ام‌ل‌ی (MFA) ی‌ا ت‌رک‌ی‌ب‌ی از م‌ک‌ان‌ی‌زم‌ه‌ای اح‌راز ه‌وی‌ت ت‌ک‌ع‌ام‌ل‌ی اس‌ت‌ف‌اده ش‌ود. ب‌رای س‌طح L3، ح‌داق‌ل ی‌ک‌ی از ع‌وام‌ل اح‌راز ه‌وی‌ت ب‌ای‌د م‌ب‌ت‌ن‌ی ب‌ر س‌خ‌ت‌اف‌زار ب‌اش‌د در ب‌راب‌ر ح‌م‌لات ف‌ی‌ش‌ی‌ن‌گ، م‌ق‌اوم‌ت در ب‌راب‌ر ن‌ف‌وذ و ج‌ع‌ل ه‌وی‌ت را ف‌راه‌م ک‌ن‌د، در ح‌ال‌ی‌ک‌ه ب‌ا اق‌دام آگ‌اه‌ان‌ه ک‌ارب‌ر (م‌ان‌ن‌د ف‌ش‌ردن د‌ک‌م‌ه روی ک‌ل‌ی‌د س‌خ‌ت‌اف‌زاری FIDO ی‌ا ت‌ل‌ف‌ن م‌راه) ق‌ص‌د ک‌ارب‌ر ب‌رای اح‌راز ه‌وی‌ت را ن‌ی‌ز ت‌أی‌ی‌د ن‌م‌ای‌د. ه‌ر گ‌ون‌ه ت‌س‌ه‌ی‌ل ی‌ا ت‌ع‌دی‌ل در ای‌ن الزام، م‌س‌ت‌ل‌زم ارائ‌ه دل‌ای‌ل م‌س‌ت‌ن‌د و م‌ج‌م‌وع‌ه‌ای ج‌ام‌ع از ک‌ن‌ت‌رل‌ه‌ای ج‌ب‌ران‌ی اس‌ت.	2
6.3.4	ت‌أی‌ی‌د ک‌ن‌ی‌د ک‌ه در ص‌ورت وج‌ود م‌س‌ی‌ره‌ای م‌ت‌ع‌دد اح‌راز ه‌وی‌ت در ب‌رن‌ام‌ه، ه‌ی‌چ م‌س‌ی‌ر اح‌راز ه‌وی‌تِ م‌س‌ت‌ن‌دس‌ازی‌ن‌ش‌ده‌ای وج‌ود ن‌داش‌ت‌ه ب‌اش‌د و ک‌ن‌ت‌رل‌ه‌ای ام‌ن‌ی‌ت‌ی و س‌طح ق‌درت اح‌راز ه‌وی‌ت در ت‌م‌ام‌ی م‌س‌ی‌ره‌ا ب‌ه‌ص‌ورت ی‌ک‌س‌ان و س‌ازگ‌ار اع‌م‌ال ش‌ون‌د.	2
6.3.5	ک‌ارب‌ران از ت‌لاش‌ه‌ای م‌ش‌ک‌وک ب‌رای اح‌راز ه‌وی‌ت (م‌وف‌ق ی‌ا ن‌ام‌وف‌ق) م‌ط‌ل‌ع ش‌ون‌د. ای‌ن اط‌لاع‌رس‌ان‌ی م‌ی‌ت‌وان‌د ش‌ام‌ل ت‌لاش ب‌رای اح‌راز ه‌وی‌ت از م‌وق‌ع‌ی‌ت م‌ک‌ان‌ی ی‌ا ک‌ل‌ای‌ن‌ت غ‌ی‌رم‌ع‌م‌ول، اح‌راز ه‌وی‌ت ن‌اق‌ص (ت‌ن‌ه‌ا م‌وف‌ق‌ی‌ت ی‌ک‌ی از ع‌وام‌ل)، ت‌لاش ب‌رای اح‌راز ه‌وی‌ت پ‌س از ی‌ک دوره ط‌ول‌ان‌ی ع‌دم ف‌ع‌ال‌ی‌ت، ی‌ا اح‌راز ه‌وی‌ت م‌وف‌ق پ‌س از چ‌ن‌دی‌ن ت‌لاش ن‌ام‌وف‌ق ب‌اش‌د.	3
6.3.6	ای‌م‌ی‌ل ب‌ه‌ع‌ن‌وان م‌ک‌ان‌ی‌زم اح‌راز ه‌وی‌ت، چ‌ه ب‌ه‌ص‌ورت ت‌ک‌ع‌ام‌ل‌ی و چ‌ه ب‌ه‌ع‌ن‌وان ی‌ک‌ی از ع‌وام‌ل اح‌راز ه‌وی‌ت چ‌ن‌دع‌ام‌ل‌ی، اس‌ت‌ف‌اده ن‌م‌ی‌ش‌ود.	3

#	Description	Level
6.3.7	کاربران پس از به‌روزرسانی اطلاعات احراز هویت خود، مانند بازنشانی گذرواژه یا تغییر نام کاربری یا آدرس ایمیل، مطلع شوند.	3
6.3.8	با توجه به پیامهای خطا، کدهای پاسخ HTTP یا تفاوت در زمان پاسخدهی، نتوان کاربران مجتبر را از طریق تلاشهای ناموفق احراز هویت شناسایی کرد. همچنین، عملکردهای ثبتنام و بازیابی گذرواژه نیز باید از این نوع افشای اطلاعات محافظت شده باشند.	3

V6.4 Authentication Factor Lifecycle and Recovery

عوامل احراز هویت میتوانند شامل کلمه عبور، توکنهای نرمافزاری، توکنهای سختافزاری و دستگاههای بیومتریک باشند. مدیریت امن چرخه عمر این مکانیزمها برای حفظ امنیت برنامه اهمیت بسیاری دارد و این بخش شامل الزامات مرتبط با این موضوع میباشد.

الزامات این بخش عمدتاً مربوط به بندهای 5.1.1.2 § و 6.1.2.3 § از راهنمای NIST's Guidance میباشد.

#	Description	Level
6.4.1	کلمه عبور اولیه یا کدهای فعالسازی تولید شده توسط سیستم، بهصورت تصادفی و ایمن تولید شوند، با سیاست فعلی رمزعبور مطابقت دارند، و پس از مدت زمان کوتاهی یا پس از اولین استفاده منقضی میشوند. این رمزهای اولیه نباید به عنوان رمزعبور بلندمدت مورد استفاده قرار گیرند.	1
6.4.2	راهنمای کلمه عبور یا احراز هویت مبتنی بر دانش (که به عنوان "سؤالات امنیتی" شناخته میشوند) در سیستم وجود ندارد.	1
6.4.3	فرآیند امنی برای بازنشانی کلمه عبور فراموششده پیادهسازی شده است که هیچیک از مکانیزمهای احراز هویت چندعاملی فعال را دور نزند.	2
6.4.4	در صورت از دست رفتن یکی از عوامل احراز هویت چندعاملی، فرآیند اثبات هویت با همان سطحی که در زمان ثبتنام انجام شده است، اجرا شود.	2
6.4.5	دستورالعملهای تمدید مکانیزمهای احراز هویت دارای تاریخ انقضا برای کاربران به موقع ارسال میشود تا پیش از انقضای مکانیزم فعلی فرصت کافی برای تمدید وجود داشته باشد، و در صورت لزوم، یادآورهای خودکار نیز تنظیم و ارسال شده باشند.	3
6.4.6	کاربران ادمین میتوانند فرآیند بازنشانی کلمه عبور را برای کاربران عادی آغاز کنند، اما اجازه تغییر مستقیم یا انتخاب رمزعبور برای کاربر را ندارند. این کنترل برای جلوگیری از دسترسی ادمین به رمزعبور کاربران طراحی شده است.	3

V6.5 General Multi-factor authentication requirements

ای‌ن بخ‌ش راه‌نم‌ای‌ی‌ه‌ای ک‌ل‌ی ارائ‌ه م‌ی‌دهد ک‌ه ب‌رای ان‌واع م‌خ‌ت‌ل‌ف م‌ک‌ان‌ی‌زم‌ه‌ای اح‌راز ه‌وی‌ت چ‌ن‌دع‌ام‌ل‌ی (MFA) ق‌اب‌ل اس‌ت‌ف‌اده اس‌ت.

م‌ک‌ان‌ی‌زم‌ه‌ای ب‌ررس‌ی‌ش‌ده ش‌ام‌ل م‌وارد زی‌ر ه‌س‌ت‌ن‌د:

- Lookup Secrets (ف‌رای‌ن‌د ی‌اف‌ت‌ن ی‌ا ب‌ازی‌اب‌ی اط‌لاع‌ات م‌ح‌رمان‌ه)
- رم‌زه‌ای ع‌ب‌ور ی‌ک‌ب‌ار م‌ص‌رف م‌ب‌ت‌ن‌ی ب‌ر زم‌ان (TOTPs)
- م‌ک‌ان‌ی‌زم‌ه‌ای خ‌ارج از ب‌ان‌د (Out-of-Band Mechanisms)

Lookup Secret ک‌ه‌ای‌ی ه‌س‌ت‌ن‌د ک‌ه ب‌ه‌ص‌ورت از پ‌ی‌ش ت‌ول‌ی‌دش‌ده ارائ‌ه م‌ی‌ش‌ون‌د و م‌ش‌اب‌ه ب‌ا ک‌ه‌ای م‌ج‌وز ت‌راک‌ن‌ش ،(TAN) ک‌ه‌ای ب‌ازی‌اب‌ی ح‌س‌اب‌ه‌ای ش‌ب‌ک‌ه‌ه‌ای اج‌ت‌م‌اع‌ی ی‌ا ش‌ب‌ک‌ه‌ای از م‌ق‌ادی‌ر ت‌ص‌ادف‌ی م‌ی‌-ب‌اش‌ن‌د. ای‌ن ن‌وع م‌ک‌ان‌ی‌زم ب‌ه‌ع‌ن‌وان "چ‌ی‌زی ک‌ه در اخ‌ت‌ی‌ار داری‌د" در ن‌ظر گ‌رف‌ت‌ه م‌ی‌ش‌ود، زی‌را ک‌ه‌ا ع‌م‌داً غ‌ی‌رق‌اب‌ل ح‌ف‌ظ ک‌ردن ه‌س‌ت‌ن‌د و ن‌ی‌از ب‌ه ذخ‌ی‌ره‌س‌ازی دارن‌د.

رم‌زه‌ای ی‌ک‌ب‌ار م‌ص‌رف چ‌ن‌دع‌ام‌ل‌ی (Multi-factor TOTPs) م‌ش‌اب‌ه رم‌زه‌ای ی‌ک‌ب‌ار م‌ص‌رف ت‌ک‌ع‌ام‌ل‌ی ه‌س‌ت‌ن‌د، ام‌ا ب‌رای ای‌ج‌اد رم‌ز ن‌ه‌ای‌ی ،(OTP) ن‌ی‌از ب‌ه وارد ک‌ردن ک‌د PIN م‌ع‌ت‌ب‌ر، ب‌ازک‌ردن ب‌ا ب‌ی‌وم‌ت‌ری ک، وارد ک‌ردن USB ی‌ا ج‌ف‌ت‌س‌ازی ،NFC ی‌ا م‌ق‌دار اض‌اف‌ی دی‌گ‌ری (م‌ان‌ند م‌اش‌ی‌ن‌ح‌س‌اب‌ه‌ای ام‌ض‌ای ت‌راک‌ن‌ش) دارن‌د.

ج‌زئ‌ی‌ات م‌رب‌وط ب‌ه م‌ک‌ان‌ی‌زم‌ه‌ای خ‌ارج از ب‌ان‌د (Out-of-Band) در ب‌خ‌ش ب‌ع‌دی ارائ‌ه خ‌واه‌د ش‌د.

ال‌زام‌ات م‌وج‌ود در ای‌ن ب‌خ‌ش‌ه‌ا ع‌م‌دت‌اً م‌رب‌وط ب‌ه ب‌ن‌ده‌ای 5.1.2 §، 5.1.3 §، 5.1.4.2 §، 5.1.5.2 §، 5.2.1 § و § 5.2.3 از راه‌نم‌ای NIST's Guidance م‌ی‌ب‌اش‌ن‌د.

#	Description	Level
6.5.1	Lookup Secrets درخ‌واس‌ت‌ه‌ا ی‌ا ک‌ه‌ای اح‌راز ه‌وی‌ت خ‌ارج از ب‌ان‌د (Out-of-Band) و رم‌زه‌ای ی‌ک‌ب‌ار م‌ص‌رف م‌ب‌ت‌ن‌ی ب‌ر زم‌ان (TOTP) ت‌ن‌ه‌ا ی‌ک‌ب‌ار ق‌اب‌ل اس‌ت‌ف‌اده ب‌اش‌ن‌د و پ‌س از اس‌ت‌ف‌اده م‌وف‌ق، م‌ج‌دداً ق‌اب‌ل اس‌ت‌ف‌اده ن‌ب‌اش‌ن‌د.	2
6.5.2	در ص‌ورت ذخ‌ی‌ره‌س‌ازی Lookup Secrets در س‌م‌ت Backend ب‌رن‌ام‌ه، اگ‌ر م‌ق‌دار آنت‌روپ‌ی ک‌م‌ت‌ر از ۱۱۲ ب‌ی‌ت ب‌اش‌ن‌د (م‌ع‌ادل ۱۹ ک‌اراک‌ت‌ر ت‌ص‌ادف‌ی الف‌ب‌ای‌ی-ع‌ددی ی‌ا ۳۴ رق‌م ت‌ص‌ادف‌ی)، ب‌ای‌د ب‌ا ی‌ک ال‌گ‌وری‌ت‌م ت‌ای‌ی‌د ش‌ده ب‌رای ذخ‌ی‌ره رم‌زع‌ب‌ور و ب‌ا اس‌ت‌ف‌اده از ی‌ک Salt ت‌ص‌ادف‌ی ۳۲ ب‌ی‌ت‌ی ه‌ش ش‌ون‌د. در ص‌ورت‌ی ک‌ه آنت‌روپ‌ی ک‌د ۱۱۲ ب‌ی‌ت ی‌ا ب‌ی‌ش‌ت‌ر ب‌اش‌د، م‌ی‌ت‌وان از ی‌ک ت‌اب‌ع ه‌ش اس‌ت‌ان‌دارد ب‌رای ذخ‌ی‌ره س‌ازی اس‌ت‌ف‌اده ک‌رد.	2
6.5.3	Lookup Secrets ک‌ه‌ای اح‌راز ه‌وی‌ت خ‌ارج از ب‌ان‌د (out of band) و ت‌ول‌ی‌د رم‌زه‌ای ی‌ک‌ب‌ار م‌ص‌رف م‌ب‌ت‌ن‌ی ب‌ر زم‌ان(TOTP) ه‌م‌گ‌ی ب‌ا اس‌ت‌ف‌اده از ی‌ک ت‌ول‌ی‌د ک‌ن‌ن‌ده ع‌دد ش‌ب‌ه ت‌ص‌ادف‌ی رم‌زن‌گ‌اری‌ش‌ده (CSPRNG) ت‌ول‌ی‌د ش‌ده ب‌اش‌ن‌د ت‌ا از ت‌ول‌ی‌د م‌ق‌ادی‌ر ق‌اب‌ل پ‌ی‌ش‌ب‌ی‌ن‌ی ج‌ل‌وگ‌ی‌ری ش‌ود.	2

#	Description	Level
6.5.4	Lookup Secrets و کدهای احراز هویت خارج از باند (out of band) حداقل دارای ۲۰ بیت آنتروپی باشند (معمولاً ۴ کاراکتر تصادفی الفبایی-عددی یا ۶ رقم تصادفی برای رسیدن به این میزان آنتروپی کافی است).	2
6.5.5	درخواستها، کدها یا توکنهای احراز هویت خارج از باند (Out-of-Band) و همچنین رمزهای عبور یکبار مصرف مبتنی بر زمان (TOTP) دارای طول عمر مشخص باشند. حداکثر طول عمر برای درخواستهای خارج از باند باید ۱۰ دقیقه و برای TOTP حداکثر ۳۰ ثانیه باشد.	2
6.5.6	امکان لغو (Revoke) هر عامل احراز هویت، از جمله دستگاههای فیزیکی، در صورت سرقت یا مفقود شدن وجود داشته باشد.	3
6.5.7	مکانیزمهای احراز هویت بیومتریک فقط به‌عنوان عامل دوم و در کنار یکی از عوامل «چیزی که در اختیار دارید» یا «چیزی که کاربر می داند» مورد استفاده قرار می گیرند.	3
6.5.8	اعتبارسنجی رمزهای یکبار مصرف مبتنی بر زمان (TOTP) بر اساس منبع زمانی از یک سرویس قابل اعتماد انجام می‌شود و نه بر اساس زمان ارائه‌شده توسط کلاینت یا منابع غیرقابل اعتماد.	3

V6.6 Out-of-Band authentication mechanisms

این نوع احراز هویت معمولاً شامل ارتباط سرور احراز هویت با یک دستگاه فیزیکی از طریق یک کانال ثانویه امن است. به‌عنوان مثال، ارسال Push Notifications به دستگاه‌های تلفن همراه. این نوع از مکانیزم احراز هویت به‌عنوان "چیزی که در اختیار دارید" در نظر گرفته می‌شود.

مکانیزمهای ناامن احراز هویت خارج از باند نظیر ایمیل و VOIP مجاز نیستند. همچنین، احراز هویت از طریق خطوط تلفن عمومی (PSTN) و پیامک (SMS) در حال حاضر توسط NIST به‌عنوان مکانیزم-های"محدودشده")"restricted" authentication mechanisms) دسته‌بندی شده‌اند و باید به تدریج با مکانیزمهایی مانند رمز یکبار مصرف مبتنی بر زمان) مکانیزمهای رمزنگاری‌شده یا روش‌های مشابه امنتر جایگزین شوند. استانداردNIST SP 800-63B در بند 5.1.3.3 § توصیه می کند که در صورت ضرورت پشتیبانی از احراز هویت خارج از باند تلفنی یا پیامکی، باید ریسک‌های مانند تعویض دستگاه، تغییر سیم کارت، انتقال شماره یا رفتارهای غیرعادی دیگر مدنظر قرار گیرد. هرچند این بخش از چارچوب ASVS الزام صریحی برای رعایت این موارد ندارد، اما عدم توجه به این ملاحظات امنیتی در یک اپلیکیشن حساس با سطح امنیتی L2 یا L3 باید به‌عنوان یک علامت هشدار جدی تلقی شود.

توجه داشته باشید که NIST اخیراً راهنماییهای ارائه کرده است که استفاده از Push Notifications را نامطلوب می‌داند. هرچند این بخش از ASVS به این موضوع اشاره نکرده است، آگاهی از خطرات ناشی از حمله Push Bombing اهمیت بالایی دارد.

#	Description	Level
6.6.1	مکانیزم‌های احراز هویت مبتنی بر شبکه تلفن عمومی (PSTN) برای ارسال رمزهای یک‌بار مصرف (OTP) از طریق تماس تلفنی یا پیام‌ک تنها در صورتی که شماره تلفن قبلا اعتبارسنجی شده باشد، یا روش‌های جایگزین قوی‌تر (رمزهای عبور یک‌بار مصرف مبتنی بر زمان) در دسترس باشند ارائه می‌شوند و سرویس‌دهنده باید کاربران را از ریسک‌های امنیتی این روش‌ها مطلع سازد. برای اپلیکیشن‌های سطح L3، استفاده از تماس تلفنی و پیام‌ک به‌عنوان گزینه، مجاز نیست.	2
6.6.2	درخواست‌ها، کدها یا توکن‌های احراز هویت خارج از باند (Out-of-Band) به درخواست احراز هویت اصلی که برای آن تولید شده‌اند متصل باشند و قابل استفاده برای درخواست‌های قبلی یا بعدی نباشند.	2
6.6.3	مکانیزم‌های احراز هویت خارج از باند (Out-of-Band) مبتنی بر کد در برابر حملات brute force از طریق اعمال کنترل محدودیت نرخ (Rate Limiting) محافظت شوند. همچنین، استفاده از کدی با حداقل ۶۴ بیت آنتروپی مدنظر(Entropy) قرار گیرد.	2
6.6.4	در صورت استفاده از Push Notifications جهت احراز هویت چند عاملی، محدودیت نرخ (Rate Limiting) به منظور جلوگیری از حملات Push Bombing اعمال شده باشد. استفاده از تطبیق عدد (Matching Number) نیز می‌تواند به کاهش این ریسک کمک کند.	3

V6.7 Cryptographic authentication mechanism

مکانیزم‌های احراز هویت رمزنگاری‌شده شامل کارت‌های هوشمند یا کلیدهای FIDO هستند، که در آن‌ها کاربر باید دستگاه رمزنگاری را به رایانه متصل کرده یا با آن جفت کند تا فرآیند احراز هویت تکمیل شود. در این روش، سرور احراز هویت یک عدد تصادفی (Nonce) چالشی به دستگاه یا نرم‌افزار رمزنگاری ارسال می‌کند و دستگاه یا نرم‌افزار بر اساس کلید رمزنگاری securely ذخیره‌شده، پاسخی را محاسبه می‌کند. الزامات این بخش راهنمایی‌های خاص پیاده‌سازی این مکانیزم‌ها را ارائه می‌دهد و راهنمایی‌های مربوط به الگوریتم‌های رمزنگاری در فصل «رمزنگاری» آورده شده است.

در مواردی که کلیدهای مشترک یا کلیدهای محرمانه برای احراز هویت رمزنگاری‌شده مورد استفاده قرار می‌گیرند، لازم است این کلیدها به همان روشی که سایر اسرار سیستمی مدیریت می‌شوند، نگه‌داری شوند. همانطور که در بخش «Secret Management» از فصل «Configuration» مستندسازی شده است.

الزامات این بخش عمدتاً مربوط به بند 5.1.7.2 § از راهنمای NIST's Guidance می‌باشد.

#	Description	Level
6.7.1	گواهی‌نامه‌های مورد استفاده برای تأیید ادعاهای احراز هویت رمزنگاری‌شده به گونه‌ای ذخیره شوند که از تغییر یا دستکاری آن‌ها محافظت شود.	3
6.7.2	مقدار چالش (Challenge Nonce) حداقل ۶۴ بیت طول داشته باشد و از نظر آماری یکتا باشد یا حداقل در طول عمر دستگاه رمزنگاری منحصر به فرد باقی بماند.	3

V6.8 Authentication with an Identity Provider

ارائه‌دهندگان هویت (Identity Providers یا IdPs) هویت فدرال شده برای کاربران فراهم می‌کنند. کاربران اغلب بیش از یک هویت با چندین ارائه‌دهنده هویت دارند، مانند هویت سازمانی که از طریق Azure، AD، Okta Ping Identity یا Google مدیریت می‌شود، یا هویت مصرف‌کننده که از طریق شبکه‌های اجتماعی مانند Facebook، Twitter Google یا WeChat ایجاد می‌شود. این فهرست تأیید یا احراز هویت از این شرکت‌ها یا خدمات نیست، بلکه صرفاً تشویقی برای توسعه‌دهندگان است تا واقعیت وجود چندین هویت برای بسیاری از کاربران را در نظر بگیرند. سازمان‌ها باید با توجه به پروفایل ریسک و قدرت اثبات هویت ارائه‌دهنده هویت، به ادغام با هویت‌های موجود کاربران کمک کنند. به عنوان مثال، سازمان‌های دولتی به‌احتمال زیاد هویت شبکه‌های اجتماعی را به‌عنوان روش ورود به سیستم‌های حساس قبول نمی‌کنند، زیرا ایجاد هویت‌های جعلی یا موقت در این فضا ساده است، در حالی که شرکت‌های فعال در حوزه بازی‌های موبایل ممکن است برای افزایش تعداد بازیکنان فعال خود نیاز به ادغام با پلتفرم‌های اصلی شبکه‌های اجتماعی داشته باشند.

استفاده از ارائه‌دهندگان هویت خارجی نیازمند پیکربندی و تأیید دقیق است تا از جعل هویت یا ادعاهای ساخت‌گی جلوگیری شود. این بخش شامل الزامات لازم برای مقابله با این ریسک‌ها می‌باشد.

#	Description	Level
6.8.1	اگر برنامه از چند ارائه‌دهنده هویت (IdP) پشتیبانی می‌کند، مکانیزم جعل هویت کاربر از طریق یک ارائه‌دهنده هویت دیگر وجود نداشته باشد (برای مثال، با استفاده از شناسه کاربری یکسان). مکانیسم استاندارد برای جلوگیری از این تهدید، این است که برنامه، کاربران را با استفاده از ترکیب شناسه IdP (به‌عنوان فضای نام یا namespace) و شناسه کاربر در آن، IdP ثبت و شناسایی کند.	2
6.8.2	وجود و یکپارچگی امضاهای دیجیتال بر روی تصدیق‌های (تقاضاها یا ادعاهای) احراز هویت برای مثال بر روی (توکن‌های JWT یا SAML assertions) همواره اعتبارسنجی می‌شود و هرگونه تصدیقی که فاقد امضا یا دارای امضای نامعتبر باشد، رد می‌گردد.	2

#	Description	Level
6.8.3	تأیید می‌کند که تأکیدهای (یا ادعاهای) SAML به‌صورت منحصربه‌فرد پردازش شده و تنها یک‌بار در بازه‌ی اعتبار خود استفاده می‌شوند تا از حملات بازپخش (Replay Attacks) جلوگیری گردد.	2
6.8.4	اگر برنامه‌ای از یک ارائه‌دهنده هویت (IdP) جداگانه استفاده می‌کند و انتظار قدرت، روش‌ها یا تازگی احراز هویت خاصی را برای عملکرد خاص دارد، برنامه این موضوع را با استفاده از اطلاعات برگردانده شده توسط IdP تأیید می‌کند. به عنوان مثال، اگر از OIDC استفاده شود، این کار می‌تواند با اعتبارسنجی ادعاهای (Claims) موجود در توکن شناسایی (ID Token) مانند «amr»، «acr» و «auth_time» (در صورت وجود) انجام شود. اگر ارائه‌دهنده هویت این اطلاعات را ارائه نکند، برنامه باید یک روش جایگزین مستندسازی‌شده داشته باشد که فرض می‌کند حداقل سطح احراز هویت استفاده شده است (مثلاً احراز هویت تک‌عاملی با نام کاربری و گذرواژه).	2

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- NIST SP 800-63 - Digital Identity Guidelines
- NIST SP 800-63B - Authentication and Lifecycle Management
- NIST SP 800-63 FAQ
- OWASP Web Security Testing Guide: Testing for Authentication
- OWASP Password Storage Cheat Sheet
- OWASP Forgot Password Cheat Sheet
- OWASP Choosing and Using Security Questions Cheat Sheet
- CISA Guidance on "Number Matching"
- Details on the FIDO Alliance

V7 Session Management

هدف

مکانیزم‌های مدیریت نشست (Session Management Mechanisms) به برنامه‌ها این امکان را می‌دهند که تعاملات کاربر و دستگاه را در طول زمان، حتی هنگام استفاده از پروتکل‌های stateless (مانند HTTP)، با یک‌دیگر مرتبط کنند. برنامه‌های مدرن معمولاً از چندین توکن نشست (Session Token) با ویژگی‌ها و اهداف متفاوت استفاده می‌کنند. یک سیستم مدیریت نشست ایمن، سیستمی است که به‌درستی آوردن، استفاده یا سوءاستفاده مهاجم از نشست قربانی جلوگیری می‌کند. برنامه‌هایی که نشست‌ها را نگه‌داری می‌کنند باید اطمینان حاصل کنند که الزامات سطح-بالای زیر در مدیریت نشست رعایت شده‌اند:

- نشست‌ها برای هر کاربر(شخص) منحصر به فرد هستند و نمی‌توان آن‌ها را حدس زد یا به اشتراک گذاشت.
- نشست‌ها زمانی که دیگر مورد نیاز نیستند، باطل می‌شوند و در دوره‌های عدم فعالیت، منقضی می‌گردند.

بسیاری از الزامات مطرح‌شده در این فصل به کنترل‌های انتخاب‌شده از دست‌ورالعمل‌های هویت دیجیتال NIST SP 800-63 مربوط می‌شوند و بر ته‌دیدات رایج و ضعف‌های متداول در احراز هویت که معمولاً مورد سوءاستفاده قرار می‌گیرند، تمرکز دارند.

توجه داشته باشید که الزامات مربوط به جزئیات پیاده‌سازی خاصِ برخی از مکانیزم‌های مدیریت نشست را می‌توان در منابع دیگری یافت:

- Cookie HTTPها یک مکانیزم مشترک برای امن سازی توکن‌های نشست هستند. الزامات امنیتی خاصی برای کوکی‌ها در بخش "Web Frontend Security" قرار داده شده است.
- توکن های Self-contained به عنوان روشی برای نگه‌داری نشست‌ها بطور مکرر مورد استفاده قرار می‌گیرند. الزامات امنیتی خاصی در بخش "Self-contained Tokens" قرار داده شده است.

V7.1 Session Management Documentation

هیچ الگوی واحدی که برای تمامی برنامه‌های کاربردی مناسب باشد، وجود ندارد. بنابراین، تعریف مرزها و محدودیت‌های عمومی که برای تمامی موارد مناسب باشد غیر ممکن است. یک تحلیل ریسک با تصمیمات امنیتی مستند شده مرتبط با مدیریت نشست باید به عنوان یک پیش نیاز برای پیاده سازی و ارزیابی انجام شود. این امر تضمین می کند که سیستم مدیریت نشست با الزامات خاص آن برنامه کاربردی طراحی شده است.

فارغ از این‌که از مکانیزم نشست stateful یا stateless استفاده شود، تحلیل باید به‌صورت کامل و مستند انجام شود تا نشان دهد راه‌حل انتخاب‌شده توانایی برآورده کردن تمامی الزامات امنیتی مرتبط را دارد. همچنین باید تعامل با هرگونه مکانیزم ورود یک‌پارچه (Single Sign-On) یا SSO که در حال استفاده است، مورد توجه قرار گیرد.

#	Description	Level
7.1.1	بازه عدم فعالیت نشست کاربر و بی‌شینه طول عمر نشست قبل از احراز هویت مجدد، در ترکیب با سایر کنترل‌ها مستند و متناسب سازی شود. مستند سازی انجام شده باید شامل توجیه برای هر انحراف(نقض) از الزامات احراز هویت مجدد NIST SP 800-63B باشد.	2
7.1.2	در مستند باید تعداد نشست همزمان مجاز برای یک کاربر و همچنین رفتار و اعمال مناسب برای زمانی که این تعداد به حداکثر رسید مشخص شود.	2

#	Description	Level
7.1.3	تمام سیستم‌هایی که نشست‌های کاربر را به عنوان بخشی از یک اکوسیستم مدیریت هویت فدرال(مانند سیستم‌های SSO) ایجاد و مدیریت می‌کنند، باید در کنار سایر کنترل‌های مربوط به طول عمر نشست‌ها، خاتمه آن‌ها و هرگونه شرایط دیگری که نیاز به احراز هویت مجدد دارد، مستند شوند.	2

V7.2 Fundamental Session Management Security

این بخش الزامات موردنیاز از نشست‌های امن را از طریق بررسی امنیت تولید و اعتبارسنجی توکن‌ها احصا می‌نماید.

#	Description	Level
7.2.1	برنامه کاربردی باید تمام بررسی‌های توکن نشست را با استفاده از یک سرویس backend مطمئن انجام دهد.	1
7.2.2	برنامه کاربردی که از هر یک از توکن‌های Self-Contained یا reference که به صورت پویا برای مدیریت نشست تولید شده‌اند استفاده کند اما از کلیدهای API ثابت و کلیدها استفاده نکند.	1
7.2.3	در صورتی که برای نمایش نشست کاربر از توکن‌های reference استفاده می شود، این توکن‌ها منحصر به فرد بوده و با استفاده از یک تولید کننده عدد شبه تصادفی امن رمزنگاری(CSPRNG) که دارای انتروپی حداقل128 بیتی است تولید شوند.	1
7.2.4	برنامه کاربردی در هنگام احراز هویت کاربر یک توکن نشست جدید تولید نمایدکه شامل احراز هویت مجدد و ابطال توکن نشست کنونی است.	1

V7.3 Session Timeout

مکانیزم های انقضای نشست به منظور به حداقل رساندن فرصت برای session hijacking و سایر روش‌های سواستفاده از نشست‌ها بکار می‌روند.

#	Description	Level
7.3.1	بر اساس تحلیل ریسک و تصمیمات امنیتی مستند شده، پس از گذشت مهلت زمانی عدم فعالیت، کاربر مجبور به احراز هویت مجدد گردد.	2

#	Description	Level
7.3.2	بر اساس تحلیل ریسک و تصمیمات امنیتی مستند شده، پس از گذشت بیشینه زمان مطلق طول عمر، کاربر مجبور به احراز هویت مجدد گردد.	2

V7.4 Session Termination

ابطال نشست ممکن است توسط خود برنامه کاربردی و یا در صورتی که مکانیزم SSO برای مدیریت نشست مورد استفاده قرار گرفته است، توسط SSO به جای برنامه صورت پذیرد. ممکن است لازم باشد هنگام بررسی الزامات این بخش، تصمیم گرفته شود که آیا ارائه‌دهنده‌ی SSO در محدوده‌ی این الزامات قرار گیرد یا خیر، زیرا برخی از این هاSSO ممکن است توسط ارائه‌دهنده کنترل شوند.

ابطال نشست بایدمنجر به الزام کاربر به احراز هویت مجدد گردد و در سراسر برنامه، ورود یک‌پارچه (Federated Login) و سایر بخش ها اثربخش باشد.

برای مکانیزم های نشست ،stateful ابطال عموما شامل نامعتبرسازی نشست در .استbackend در مواردی که توکن‌ها self-contained هستند، معیاره‌ای بیشتری به منظور ابطال یا بلاک کردن این توکن-ها مورد نیاز هستند؛ در غیر این صورت این توکن‌ها تا زمان منقضی شدن بصورت معتبر باقی می‌مانند.

#	Description	Level
7.4.1	در مواقعی که فرآیند خاتمه نشست کاربر فعال می‌شود (مانند خروج کاربر یا انقضای نشست)، برنامه اجازه هیچ فعالیتی را به آن نشست نمی‌دهد. برای Reference tokens یا stateful ،session این به معنی نامعتبرسازی اطلاعات نشست در برنامه Backend می‌باشد. برنامه‌هایی که از توکن‌های Self-contained استفاده می‌کنند نیاز به راه حل‌هایی نظیر نگه‌داری فهرستی از توکن‌های خاتمه‌یافته، عدم پذیرش توکن‌هایی که پیش از یک تاریخ و زمان مشخص برای هر کاربر تولید شده‌اند، یا چرخش کلید امضای مخصوص هر کاربر دارند.	1
7.4.2	برنامه کاربردی تمامی نشست‌های فعال را در زمانی که حساب کاربری غیرفعال یا حذف می‌شود را ابطال نماید. (مانند کارمندی که شرکت را ترک می‌کند)	1
7.4.3	برنامه کاربردی قابلیت ارائه گزینه ابطال تمامی نشست های فعال دیگر را پس از یک تغییر موفقیت آمیز یا حذف هریک از عوامل احراز هویت (نظیر تغییر پسورد از طریق ریست یا بازنشانی و در صورت وجود به روزرسانی تنظیمات (MFA را داشته باشد.	2
7.4.4	تمامی صفحاتی که نیازمند احراز هویت هستند دارای دسترسی آسان و مشخص به فرآیند خروج باشند.	2

#	Description	Level
7.4.5	مدیران برنامه کاربردی قادر به ابطال نشست‌های فعال برای یک شخص یا تمامی کاربران باشند.	2

V7.5 Defenses Against Session Abuse

این بخش الزاماتی را برای کاهش ریسک‌های ناشی از نشست‌های فعال که می‌توانند از طریق وکت‌ورهایی که مت‌کی بر وجود نشست‌های کاربر فعال و قابلیت‌های آن‌ها هست‌ند مورد سرقت و یا سوء استفاده قرار گیرند را ارایه می‌دهد. بطور مثال، با استفاده از اجرای محتواهای آلوده مرورگر احراز هویت شده قربانی را مجبور کنیم که یک عملی را با استفاده از نشست کاربر قربانی انجام دهد.

توجه داشته باشید که راهنمای سطح‌محور موجود در فصل «احراز هویت» بایدهنگام در نظر گرفتن الزامات این بخش مدنظر قرار گیرد.

#	Description	Level
7.5.1	برنامه کاربردی قبل از اعطای مجوز تغییر ویژگی‌های حساس حساب کاربری که بر روی احراز هویت تاثیر دارد نظیر آدرس ایمیل، شماره تلفن، تنظیمات MFA یا اطلاعات دیگرکه در بازگردانی حساب کاربری مورد استفاده قرار می‌گیرند، به احراز هویت مجدد کامل نیاز دارد.	2
7.5.2	حساب‌های کاربری قادر به دیدن(پس از احراز هویت مجدد با حداقل یک عامل) و ابطال یک یا همه نشست‌های فعال باشد.	2
7.5.3	برنامه کاربردی قبل از اجرای تراکنش‌ها یا عملیات‌های حساس نیازمند احراز هویت مجدد با حداقل یک عامل یا صحت سنجی ثانویه باشد.	3

V7.6 Federated Re-authentication

این بخش مربوط به کسانی است که کدطرف اتکا (RP) یا ارائه دهنده هویت (IdP) را می‌نویسند. این الزامات از NIST SP 800-63C برای هویت اشتراکی و ادعاها گرفته شده است.

#	Description	Level
7.6.1	طول عمر و ابطال نشست بین RP و idp طبق سند- که نیازمند احرازهویت مجدد در زمان مورد نیاز مانند وقتی که بی‌شینه زمان بین رخداد احراز هویت idp رسیده باشد- انجام شود.	2

#	Description	Level
7.6.2	ساخت نشست‌های جدید برنامه بدون تعامل کاربر جلوگیری می‌شود.	2

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- OWASP Web Security Testing Guide: Session Management Testing
- OWASP Session Management Cheat Sheet

V8 Authorization

هدف

مجوز دسترسی، تضمین می‌کند که دسترسی‌ها فقط در اختیار مصرف کنندگان مجاز (کاربران،سرورها و سایر کلاینت‌ها) قرار گیرد. به منظور اعمال اصل حداقل دسترسی(POLP)، برنامه‌های کاربردی مورد تایید باید این الزامات سطح بالا را احصاء نمایند:

- قوانین صدور مجوزها که شامل عوامل دخیل در تصمیم گیری و عوامل محیطی است را مستند سازی نمایید.
- دسترسی مصرف کنندگان به منابع باید بر اساس حقوق تعریف شده آن‌ها مجاز شمرده شود.

V8.1 Authorization Documentation

مستند سازی جامع مجوزها به منظور اطمینان از اعمال پیوسته، قابلیت ممیزی(Auditable) و همسو بودن تصمیمات امنیتی با سیاست‌های سازمانی ضروری است و همچنین با شفاف سازی و اجرایی کردن الزامات امنیتی برای توسعه دهندگان، مدیران سیستم و آزمون گره‌ای ریسک دسترسی غیر مجاز را کاهش می‌دهد.

#	Description	Level
8.1.1	مستند اعطای مجوز باید شامل قوانینی به منظور محدودسازی دسترسی در سطح عملکرد و دسترسی مبتنی بر داده بر اساس مجوزهای کاربران و ویژگی‌های منابع باشد.	1

#	Description	Level
8.1.2	مس‌تن‌د اع‌طای مج‌وز ب‌ای‌د ق‌وان‌ی‌ن‌ی را ب‌رای مح‌دودی‌ت‌ه‌ای دس‌ت‌رس‌ی در س‌طح Field ب‌ر اس‌اس مج‌وزه‌ای ک‌ارب‌ران و وی‌ژگ‌ی‌ه‌ای م‌ن‌اب‌ع ت‌ع‌ری‌ف ن‌م‌ای‌د(ه‌ر دو دس‌ت‌رس‌ی خ‌وان‌دن و ن‌وش‌ت‌ن). ت‌وج‌ه داش‌ت‌ه ب‌اش‌ی‌د ک‌ه ای‌ن ق‌وان‌ی‌ن م‌م ک‌ن اس‌ت ب‌ه م‌ق‌ادی‌ر دی‌گ‌ر وی‌ژگ‌ی‌ه‌ای ش‌یء داده ی م‌رب‌وط‌ه (Relevant Data Object) ب‌س‌ت‌گ‌ی داش‌ت‌ه ب‌اش‌ن‌د، م‌ان‌ن‌د ح‌ال‌ت (state) ی‌ا وض‌ع‌ی‌ت .(status)	2
8.1.3	مس‌تن‌د ب‌رن‌ام‌ه ک‌ارب‌ردی ب‌ای‌د وی‌ژگ‌ی‌ه‌ای مح‌ی‌ط‌ی و زم‌ی‌ن‌ه‌ای (ش‌ام‌ل و ن‌ه مح‌دود ب‌ه زم‌ان روز، م‌ک‌ان، آدرس IP ی‌ا دس‌ت‌گ‌اه) ک‌ه در ب‌رن‌ام‌ه ک‌ارب‌ردی ب‌ه م‌ن‌ظ‌ور ات‌خ‌اذ ت‌ص‌م‌ی‌م‌ات ام‌ن‌ی‌ت‌ی (ن‌ظ‌ی‌ر آن دس‌ت‌ه از ت‌ص‌م‌ی‌م‌ات‌ی ک‌ه م‌رب‌وط ب‌ه اح‌راز ه‌وی‌ت و اع‌طای مج‌وز م‌ی ب‌اش‌د) را م‌ش‌خ‌ص ک‌ن‌د.	3
8.1.4	مس‌تن‌دات اح‌راز ه‌وی‌ت و اع‌طای مج‌وز، ع‌لاوه ب‌ر اع‌طای مج‌وز در س‌طح ،Field س‌طح ع‌م‌ل‌ک‌رد و داده‌ای خ‌اص، چ‌گ‌ون‌گ‌ی اس‌ت‌ف‌اده از ع‌وام‌ل مح‌ی‌ط‌ی و زم‌ی‌ن‌ه‌ای در ات‌خ‌اذ ت‌ص‌م‌ی‌م‌ات را م‌ش‌خ‌ص ن‌م‌ای‌د.	3

V8.2 General Authorization Design

پ‌ی‌اده س‌ازی ک‌ن‌ت‌رل‌ه‌ای اع‌طای مج‌وز ج‌زئ‌ی‌در س‌طح ع‌م‌ل‌ک‌رد، داده و ف‌ی‌ل‌ده‌ا، ت‌ض‌م‌ی‌ن م‌ی‌ک‌ن‌د ک‌ه م‌ص‌رف ک‌ن‌ن‌د‌گ‌ان ف‌ق‌ط ق‌ادر ب‌ه دس‌ت‌رس‌ی ب‌ه آن چ‌ی‌زی ه‌س‌ت‌ن‌د ک‌ه ص‌ری‌ح‌ا ب‌ه آن‌ه‌ا دس‌ت‌رس‌ی داده ش‌ده اس‌ت.

#	Description	Level
8.2.1	ب‌رن‌ام‌ه ک‌ارب‌ردی ب‌ای‌د از مح‌دود ب‌ودن دس‌ت‌رس‌ی‌ه‌ای در س‌طح ع‌م‌ل‌ک‌رد ب‌ه م‌ش‌ت‌ری‌ان‌ی ک‌ه دارای دس‌ت‌رس‌ی ص‌ری‌ح ه‌س‌ت‌ن‌د اط‌م‌ی‌ن‌ان ح‌اص‌ل ن‌م‌ای‌د.	1
8.2.2	ب‌رن‌ام‌ه ک‌ارب‌ردی ب‌ای‌د از مح‌دود ب‌ودن دس‌ت‌رس‌ی‌ه‌ای در س‌طح داده ب‌ه م‌ش‌ت‌ری‌ان‌ی ک‌ه دارای دس‌ت‌رس‌ی ص‌ری‌ح ب‌ه اق‌ل‌ام داده‌ای خ‌اص ه‌س‌ت‌ن‌د اط‌م‌ی‌ن‌ان ح‌اص‌ل ن‌م‌ای‌د ت‌ا از ای‌ن ط‌ری‌ق خ‌ط‌ر آس‌ی‌ب پ‌ذی‌ری IDOR و broken object level authorization را ک‌اه‌ش ده‌د .	1
8.2.3	ب‌رن‌ام‌ه ک‌ارب‌ردی دس‌ت‌رس‌ی در س‌طح Field را ب‌ا مج‌وزه‌ای ص‌ری‌ح ب‌ه Field ه‌ا ب‌رای م‌ص‌رف ک‌ن‌ن‌د‌گ‌ان ج‌ه‌ت ک‌اه‌ش خ‌ط‌ر broken object property level authorization (BOPLA) مح‌دود ک‌رده ب‌اش‌د.	2
8.2.4	ک‌ن‌ت‌رل‌ه‌ای ام‌ن‌ی‌ت‌ی ت‌ط‌ب‌ی‌ق‌ی ک‌ه ب‌ر اس‌اس وی‌ژگ‌ی‌ه‌ای مح‌ی‌ط‌ی و زم‌ی‌ن‌ه‌ای ک‌ارب‌ر (م‌ان‌ن‌د زم‌ان روز، م‌ک‌ان، آدرس IP ی‌ا دس‌ت‌گ‌اه) ه‌س‌ت‌ن‌د، ب‌رای ت‌ص‌م‌ی‌م‌گ‌ی‌ری‌ه‌ای اح‌راز ه‌وی‌ت (Authentication) و مج‌وزده‌ی (Authorization) ط‌ب‌ق م‌س‌تن‌دات ب‌رن‌ام‌ه پ‌ی‌اده‌س‌ازی ش‌ده‌ان‌د. ای‌ن ک‌ن‌ت‌رل‌ه‌ا ب‌ای‌د ه‌م در زم‌ان ت‌ل‌اش ک‌ارب‌ر ب‌رای ش‌روع ی‌ک ن‌ش‌س‌ت (Session) ج‌دی‌د و ه‌م در ط‌ول ی‌ک ن‌ش‌س‌ت ف‌ع‌ال، اع‌م‌ال ش‌ون‌د.	3

V8.3 Operation Level Authorization

اجرای فوری تغییرات در مجوزهای دسترسی در لایه مناسب از مهاری برنامه کاربردی به منظور جلوگیری از اقدامات غیرمجاز بویژه در محیط های پویا ضروری است.

#	Description	Level
8.3.1	برنامه کاربردی قوانین مجوزدهی را در یک لایه سرویس مجتبر اعمال نموده و بر آن دستهای از کنترلهایی که مصرف کنندگان نامعتبر قادر به دستکاری آنها هستند مکی نباشد. (نظیر جاوا اسکریپت سمت کاربر)	1
8.3.2	در مواقعی که امکان اعمال فوری تغییرات وجود ندارد (مثلاً وقتی که سیستم برای تصمیمگیری در مجوزدهی از توکنهای Self-Contained مثل JWT استفاده میکند که اطلاعات را داخل خود ذخیره کرده و تا زمان انقضا تغییر نمیکنند)، باید کنترلهای جبرانی وجود داشته باشد. این کنترلها باید بتوانند زمانی که کاربر عملی را انجام میدهد که دیگر مجاز به آن نیست، هشدار بدهند و تغییر را برگردانند. با این حال، توجه کنید که این راهکار جایگزین نمیتواند جلوی نشت اطلاعات را بگیرد.	3
8.3.3	دسترسی به یک object بر اساس مجوزهای کاربری شخص درخواست کننده(مثال: مصرف کننده) باشد نه بر اساس مجوزهای یک واسطه یا سرویسی که به نمایندگی از او عمل میکند. بطور مثال، در صورتی که یک مصرف کننده یک وب سرویس را با استفاده از یک توکن self-contained برای احراز هویت فراخوانی میکند و سپس آن سرویس اطلاعات را از یک سرویس دیگر درخواست میکند، سرویس دوم باید همان توکن مصرف کننده را استفاده کند، نه یک توکن سرویس-به-سرویس (Machine-to-Machine token) که متعلق به سرویس اول است، تا تصمیمات مجوزدهی دقیقاً بر اساس مجوزهای کاربر گرفته شوند.	3

V8.4 Other Authorization Considerations

ملاحظات اضافهتری برای مجوزدهی ، بویژه برای درگاههای مدیریتی و محیطهای چند کاربره(multi tenants) وجود دارد که برای جلوگیری از دسترسی غیر مجاز مفید هستند.

#	Description	Level
8.4.1	برنامه های کاربردی چند کاربره(multi tenants) به cross-tenant های از کنترل به منظور تضمین این امر که عملیاتهای مصرف کنندگان هرگز کاربرانی (tenants) را که مجوز تعامل با آنها را ندارند را تحت تأثیر قرار نمیدهد.	2

#	Description	Level
8.4.2	دس‌ت‌رس‌ی ب‌ه درگ‌اه‌ه‌ای پ‌ی ک‌ربن‌دی ش‌امل چ‌ن‌دی‌ن ل‌ای‌ه ام‌ن‌ی‌ت‌ی از ق‌ب‌ی‌ل ت‌أی‌ی‌د م‌داوم ه‌وی‌ت م‌ص‌رف ک‌ن‌ن‌ده، ارزی‌اب‌ی وض‌ع‌ی‌ت ام‌ن‌ی‌ت‌ی دس‌ت‌گ‌اه و ت‌ح‌ل‌ی‌ل ری‌س‌ک زم‌ی‌ن‌ه‌ای ب‌اش‌د ت‌ا از ای‌ن طری‌ق اط‌م‌ی‌ن‌ان ح‌اص‌ل ش‌ود ک‌ه م‌وق‌ع‌ی‌ت ش‌ب‌ک‌ه‌ای ی‌ا ن‌ق‌اط پ‌ای‌ان‌ی م‌ع‌ت‌ب‌ر ت‌ن‌ه‌ا ع‌وام‌ل ب‌رای م‌ج‌وزه‌ی ن‌ی‌س‌ت‌ند اگ‌ر چ‌ه ک‌ه ای‌ن ع‌وام‌ل اح‌ت‌م‌ال دس‌ت‌رس‌ی غ‌ی‌رم‌ج‌از را ک‌اه‌ش م‌ی‌ده‌ند.	3

References

ب‌رای اط‌لاع‌ات ب‌ی‌ش‌ت‌ر، ه‌م‌چ‌ن‌ی‌ن ب‌ه م‌ن‌اب‌ع زی‌ر م‌راج‌ع‌ه ک‌ن‌ی‌د:

- OWASP Web Security Testing Guide: Authorization
- OWASP Authorization Cheat Sheet

V9 Self-contained Tokens

ه‌دف

م‌ف‌ه‌وم ت‌وک‌ن self-contained در RFC 6749 Oauth2 اص‌ل‌ی م‌رب‌وط ب‌ه س‌ال 2012 ذک‌ر ش‌ده اس‌ت. ای‌ن م‌ف‌ه‌وم ب‌ه ت‌وک‌ن‌ی اش‌اره دارد ک‌ه ش‌ام‌ل داده ی‌ا Claim اس‌ت ک‌ه ی‌ک س‌روی‌س گ‌ی‌رن‌ده ب‌رای ت‌ص‌م‌ی‌م‌گ‌ی‌ری‌-ه‌ای ام‌ن‌ی‌ت‌ی ب‌ه آن‌ه‌ا م‌ت‌ک‌ی خ‌واه‌د ب‌ود. از ای‌ن رو ب‌ای‌د آن‌ه‌ا را از ت‌وک‌ن‌ه‌ای س‌اده‌ای ک‌ه ف‌ق‌ط ح‌اوی ی‌ک ش‌ن‌اس‌ه ه‌س‌ت‌ند ک‌ه س‌روی‌س گ‌ی‌رن‌ده از آن‌ه‌ا ب‌رای ج‌س‌ت‌ج‌وی داده ب‌ص‌ورت م‌ح‌ل‌ی اس‌ت‌ف‌اده م‌ی‌-ک‌ند م‌ت‌م‌ای‌ز دان‌س‌ت. رای‌ج ت‌ری‌ن م‌ث‌ال‌ه‌ا از ت‌وک‌ن‌ه‌ای self-contained ه‌ا، JSON Web Tokens (JWTs) وSAML assertion ه‌س‌ت‌ند.

ک‌ارب‌رد ت‌وک‌ن‌ه‌ای self-contained ح‌ت‌ی خ‌ارج از ح‌وزه OAuth و OIDC ب‌س‌ی‌ار ف‌راگ‌ی‌ر ش‌ده اس‌ت. در ه‌م‌ی‌ن زم‌ان ام‌ن‌ی‌ت ای‌ن م‌ک‌ان‌ی‌زم ب‌ر ق‌اب‌ل‌ی‌ت اع‌ت‌ب‌ارس‌نج‌ی ی‌ک‌پ‌ارچ‌گ‌ی ت‌وک‌ن و ت‌ض‌م‌ی‌ن ای‌ن م‌س‌ال‌ه ک‌ه ای‌ن ت‌وک‌ن ب‌رای ی‌ک زم‌ی‌ن‌ه خ‌اص م‌ع‌ت‌ب‌ر اس‌ت اس‌ت‌وار اس‌ت. م‌وان‌ع زی‌ادی(چ‌ال‌ش‌ه‌ا) در ای‌ن ف‌رآی‌ن‌د وج‌ود دارد و ای‌ن ب‌خ‌ش ج‌زئ‌ی‌ات وی‌ژه‌ای از ای‌ن م‌ک‌ان‌ی‌زم‌ه‌ا را ارائ‌ه م‌ی‌ده‌د ک‌ه ب‌رن‌ام‌ه‌ه‌ا ب‌ای‌د ب‌ه م‌ن‌ظ‌ور ج‌ل‌وگ‌ی‌ری از ای‌ن چ‌ال‌ش‌ه‌ا، ارائ‌ه ده‌ند.

V9.1 Token source and integrity

ای‌ن ب‌خ‌ش ش‌ام‌ل ال‌زام‌ات‌ی اس‌ت ک‌ه ت‌ض‌م‌ی‌ن م‌ی‌ک‌ند ک‌ه ت‌وک‌ن ت‌وس‌ط ب‌خ‌ش م‌ط‌م‌ئ‌ن(م‌ورد اع‌ت‌م‌اد) ت‌ول‌ی‌د ش‌ده اس‌ت و م‌ورد دس‌ت‌ک‌اری ق‌رار ن‌گ‌رف‌ت‌ه اس‌ت.

#	Description	Level
9.1.1	قبل از پذیرش محتوای توکن‌ها، آن‌ها را با استفاده از امضای دیجیتال یا MAC به منظور حفاظت در مقابل دستکاری‌ها مورد اعتبارسنجی قرار داده شود.	1
9.1.2	فقط الگوریتم‌های موجود در لیست مجاز قادر به استفاده در ساخت و صحت سنجی توکن‌های self-contained در یک زمینه مشخص باشند. این لیست مجاز باید در حالت ایده آل شامل هم الگوریتم‌های متقارن و هم نامتقارن‌ها باشد و نباید شامل الگوریتم‌های NONE باشد. در صورتی که هر دو الگوریتم‌های متقارن و نامتقارن مورد نیاز باشند کنترل‌های بیشتری باید به منظور جلوگیری از سردرگمی کلیدها(Confusion key) بکار گرفته شود.	1
9.1.3	به منظور جلوگیری از مشخص نمودن منابع غیر مطمئن و کلیدها توسط مهاجمین، موارد کلیدی که در اعتبارسنجی توکن‌های self-contained مورد استفاده قرار می گیرند، از منابع از پیش پیکربندی شده مطمئنی که برای صادر کننده توکن است تعیین شود. برای JWTs و سایر ساختارهای JWS ، هدرهای نظیر 'x5u', 'jku' و 'jwk' باید با یک لیست مجاز از منابع مطمئن مورد اعتبارسنجی قرار گیرند.	1

V9.2 Token content

قبل از اتخاذ تصمیمات امنیتی براساس محتوای توکن ،self-contained ضروری است ابتدا بررسی شود که آیا توکن در دوره مهلت‌برخود و برای هدفی که توسط سرویس گیرنده درجهت کاربرد آن در نظر گرفته و ارائه شده است. این موضوع به منظور جلوگیری از استفاده متقابل نا امن بین سرویس‌های مختلف یا با انواع توکن‌های مختلف از صادرکننده مشابه است.

الزامات خاص برای OAuth و OIDC در این بخش مورد پوشش قرار گرفته شده‌اند.

#	Description	Level
9.2.1	اگر یک بازه زمانی مهلت‌بر در داده‌های توکن وجود داشته باشد، توکن و محتوای آن تنها در صورتی پذیرفته می‌شوند که زمان بررسی در این بازه زمانی مهلت‌بر باشد. به عنوان مثال، برای ،هاJWT ادعاهای 'nbf' و 'exp' باید تایید شوند.	1
9.2.2	سرویس‌هایی که یک توکن دریافت می کنند قبل از پذیرش محتوای آن، از لحاظ درستی نوع بررسی نموده و برای هدف مشخص شده در نظر گیرد. بطور مثال، فقط توکن‌های access می‌توانند به منظور تصمیمات مجوزی و فقط توکن های ID می‌توانند به منظور اثبات هویت کاربر مورد پذیرش قرار گیرند.	2

#	Description	Level
9.2.3	ی ک س رویس ف ق ط ت وک ن ه ای ی را ب پ ذی رد ک ه ب ه م ن ظ ور اس ت ف اده ت وس ط آن س رویس ه س ت ن د(audience).ب رای JWT ه ا،ای ن ام ر م ی ت وان د از ط ری ق اع ت ب ارس ن ج ی ادع ای 'aud' ب ا ی ک ل ی س ت م ج از ت ع ری ف ش ده در س رویس ص ورت پ ذی رد.	2
9.2.4	اگ ر ص ادر ک ن ن ده ت وک ن از ک ل ی د خ ص وص ی م ش اب ه ی ب رای ه ای audience م ت ف اوت اس ت ف اده م ی ک ن د، ت وک ن ه ای ت ول ی د ش ده دارای م ح دودی ت م خ اط ب ی ب اش ن د ک ه ب ط ور م ن ح ص ر ب ه ف ردی م خ اط ب ی ن م ورد ن ظ ر را م ش خ ص م ی ک ن د. ای ن ام ر از اس ت ف اده م ج دد ت وک ن ت وس ط م خ اط ب ن اخ واس ت ه ج لوگ ی ری م ی ک ن د. در ص ورت ی ک ه ش ن اس ه م خ اط ب ه ص ورت پ وی ا م ش خ ص ش ده ب اش د، ص ادرک ن ن ده ت وک ن ب ای د ای ن م خ اط ب ی ن را ب ه م ن ظ ور اط م ی ن ان از ع دم ج ع ل ه وی ت م خ اط ب م ورد اع ت ب ارس ن ج ی ق رار ده د.	2

References

ب رای اط لاع ات ب ی ش ت ر، ه م چ ن ی ن ب ه م ن اب ع زی ر م راج ع ه ک ن ی د:

- OWASP JSON Web Token Cheat Sheet

V10 OAuth and OIDC

ه دف

OAuth2 (ک ه در ای ن ف ص ل ب ا ع ن وان OAuth از آن ی اد م ی ش ود) ی ک چ ارچ وب اس ت ان دارد در ص ن ع ت ب رای واگ ذاری م ج وز دس ت رس ی(delegated authorization) اس ت. ب رای م ث ال، ب ا اس ت ف اده از OAuth ی ک ب رن ام ه ی ک ارب ری (Client Application) م ی ت وان د ب ه ن م ای ن د گ ی از ک ارب ر، ب ه ه اAPI (م ن اب ع س رور) دس ت رس ی پ ی دا ک ن د، م ش روط ب ر ای ن ک ه ک ارب ر، دس ت رس ی م ورد ن ی از را ب ه آن ب رن ام ه اع ط ا ک رده ب اش د.

ب ه ت ن ه ای ی، OAuth ب رای اح راز ه وی ت ک ارب ر ط راح ی ن ش ده اس ت. چ ارچ وب OpenID Connect (OIDC) ب ا اف زودن ی ک ل ای ه ی ه وی ت ک ارب ر ب ر روی OAuth، آن را گ س ت رش م ی ده د. ام ک ان ات ی OIDC م ان ن د اط لاع ات اس ت ان دارد ه وی ت ک ارب ر، ورود ی ک پ ارچ ه (Single Sign-On - SSO) و م دی ری ت ن ش س ت (Session Management) را پ ش ت ی ب ان ی م ی ک ن د.از آن ج ا ک ه OIDC ی ک اف زون ه ی OAuth م ح س وب م ی ش ود، ال زام ات م رب وط ب ه OAuth در ای ن ف ص ل، ش ام ل OIDC ن ی ز م ی گ ردد.

ن ق ش ه ای زی ر در OAuth ت ع ری ف ش ده ان د:

- م ش ت ری (OAuth Client) ب رن ام ه ای اس ت ک ه ت ل اش م ی ک ن د ب ه م ن اب ع س رور دس ت رس ی پ ی دا ک ن د (ب رای م ث ال، ب ا ف راخ وان ی ی ک API ب ه ک م ک ت وک ن دس ت رس ی ص ادرش ده) م ش ت ری OAuth اغ ل ب ی ک ب رن ام ه ی س م ت س رور (Server-side Application) اس ت.
  - ی ک Confidential Client ن وع ی OAuth Client اس ت ک ه ت وان ای ی ح ف ظ م ح رم ان گ ی اع ت ب ارن ام ه-ه ای ی (Credentials) را دارد ک ه ب رای اح راز ه وی ت خ ود در ب راب ر س رور م ج وز (Authorization Server) اس ت ف اده م ی ک ن د.

- ی‌ک Public Client توانایی‌ی حفظ محرمانگی‌ی اعتبارنامه‌ها (Credentials) برای احراز هویت در برابر Authorization Server را ندارد. بنابراین، به جای احراز هویت خود (برای مثال با استفاده از پارامترهای client_id و client_secret)، صرفاً خودش را شناسایی می‌کند (با استفاده از پارامتر client_id)

- OAuth Resource Server (RS) سروری است که از طریق API، منابع را در اختیار OAuth Clients قرار می‌دهد.
- OAuth Authorization Server (AS) یک برنامه‌ی سرور است که به OAuth Clients توکن‌های دسترسی (Access Tokens) صادر می‌کند. این توکن‌ها به OAuth Clients امکان می‌دهند تا به منابع موجود در RS دسترسی پیدا کنند، یا به نمایندگی از کاربر نهایی، (End-User) یا به نمایندگی از خود OAuth Client. AS معمولاً یک برنامه‌ی مجزا است، اما در صورت لزوم می‌توانند در یک RS مناسب نیز ادغام شود.
- Resource Owner (RO) همان کاربر نهایی (End-User) است که به OAuth Clients اجازه می‌دهد تا به نمایندگی از او، دسترسی محدودی به منابع موجود روی Resource Server داشته باشند. Resource Owner با تعامل با Authorization Server، رضایت خود را برای این واگذاری مجوز دسترسی (Delegated Authorization) اعلام می‌کند.

نقش‌های زیر در OIDC تعریف شده‌اند:

- Relying Party (RP) برنامه‌ی کاربری (Client Application) است که از طریق OpenID Provider درخواست احراز هویت کاربر نهایی (End-User) را می‌دهد. این نقش در واقع همان نقش یک OAuth Client را بر عهده دارد.
- OpenID Provider (OP) یک OAuth AS است که توانایی‌ی احراز هویت کاربر نهایی (End-User) را دارد و اطلاعات هویتی (OIDC Claims) را در اختیار RP قرار می‌دهد.OP می‌تواند همان Identity Provider (IdP) باشد، اما در سناریوهای فدراسیونی (Federated OP Scenarios) و Identity Provider (جایی که کاربر نهایی احراز هویت می‌شود) ممکن است دو برنامه‌ی سرور جداگانه باشند.

OAuth و OIDC در ابتدا برای برنامه‌های سوم‌شخص (Third-Party Applications) طراحی شده بودند. امروزه، این پروتکل‌ها به‌طور گسترده در برنامه‌های اول‌شخص (First-Party Applications) نیز مورد استفاده قرار می‌گیرند. با این حال، هنگام استفاده در سناریوهای اول‌شخص مانند احراز هویت (Authentication) و مدیریت نشست (Session Management) این پروتکل‌ها پیچیدگی‌های را به همراه می‌آورند که ممکن است چالش‌های امنیتی جدی‌ی ایجاد کنند.

OAuth وOIDC را می‌توان در انواع مختلف برنامه‌ها به کار برد، اما تمرکز در ASVS و همچنین الزامات مطرح‌شده در این فصل، بر روی برنامه‌های وب (Web Applications) و رابط‌های برنامه‌نویسی کاربردی (APIs) است.

از آنجاکه OAuth وOIDC را می‌توان به‌عنوان لایه‌ای منطقی بر بستر فناوری‌های وب در نظر گرفت، الزامات عمومی مطرح‌شده در فصل‌های دیگر همواره در اینجا نیز کاربرد دارند. بنابراین، این فصل را نمی‌توان به‌صورت جدا از بقیه‌ی محتوا در نظر گرفت.

این فصل به بهترین شیوه‌های جاری (Best Current Practices) برای OAuth2 و OIDC می‌پردازد که با مشخصات منتشرشده در نشانی‌های https://oauth.net/2/ و https://openid.net/developers/specs/ همسو هستند. حتی اگرچه RFCها به بلوغ رسیده محسوب می‌شوند، اما به‌طور مداوم به‌روزرسانی می‌شوند.

بنابراین، هنگام به کارگیری الزامات این فصل، همسویی با آخرین نسخه‌ها بسیار اهمیت دارد. برای جزئیات بیشتر به بخش مراجع (References) مراجعه کنید.

با توجه به پیچیدگی این حوزه، برای داشتن یک راه کار امن مبتنی بر OAuth یا OIDC بسیار ضروری است که از Authorization های Server شناخته‌شده و استاندارد در صنعت استفاده شود و پیکربندی‌های امنیتی توصیه‌شده به کار گرفته شوند.

اصطلاحات به کاررفته در این فصل با هایOAuth RFC و مشخصات OIDC همسو هستند. با این حال، توجه داشته باشید که اصطلاحات OIDC تنها در مواردی استفاده می‌شوند که مربوط به الزامات خاص OIDC باشند؛ درغیر این صورت، از اصطلاحات OAuth استفاده خواهد شد.

در زمینه‌ی OAuth و ،OIDC اصطلاح توکن(Token) در این فصل اشاره دارد به:

- Access ،Tokens که فقط برای توسط RS مصرف شوند و می‌توانند یاTokens Reference باشند که با استفاده از Introspection اعتبارسنجی می‌شوند، یا Self-contained Tokens باشند که با استفاده از کلید رمزنگاری (Key Material) اعتبارسنجی می‌شوند.
- Refresh ،Tokens که فقط برای توسط Authorization ایServer مصرف شوند که توکن را صادر کرده است.
- OIDC ID ،Tokens که فقط برای توسط ایClient مصرف شوند که جریان احراز مجوز (Authorization Flow) را آغاز کرده است.

سطوح ریسک برای برخی از الزامات این فصل به این بستگی دارد که آیا Client یک Confidential Client است یا به‌عنوان یک Public Client در نظر گرفته می‌شود. از آنجا که استفاده از احراز هویت قوی Client بسیاری از برداره‌ای حمله را کاهش می‌دهد، برخی از الزامات ممکن است در هنگام استفاده از یک Confidential Client برای برنامه‌های L1 آسان‌تر در نظر گرفته شوند.

V10.1 Generic OAuth and OIDC Security

این بخش الزامات معماری عمومی را پوشش می‌دهد که برای همه‌ی برنامه‌هایی که ازOAuth یا OIDC استفاده می‌کنند، اعمال می‌شوند.

#	Description	Level
10.1.1	توکن‌ها باید فقط به مؤلفه‌های ارسال شوند که به‌طور دقیق به آن‌ها نیاز دارند. برای مثال، هنگام استفاده از الگوی Backend-for-Frontend برای برنامه‌های JavaScript مبتنی بر مرورگر، Access Tokens و Refresh Tokens باید فقط برای Backend قابل دسترس باشند.	2

#	Description	Level
10.1.2	Client باید فقط مقادیری را از Authorization Server بپذیرد (مانند Authorization Code یا ID Token) که این مقادیر حاصل از یک Authorization Flow باشند که توسط همان نشست (Session) و تراکنش User Agent آغاز شده است. این موضوع مستلزم آن است که هایSecret تولیدشده توسط کلاینت مانند code_verifier در PKCE، یا مقادیر state و nonce در OIDC قابل حدس زدن نباشند، مخصوص همان تراکنش باشند، و به‌صورت امن هم به کلاینت و هم به نشست (Session) مرورگری که تراکنش در آن شروع شده، متصل شده باشند.	2

V10.2 OAuth Client

این الزامات مسئولیت‌هایی را که بر عهده برنامه‌های کلاینتِ OAuth است مشخص می‌کنند. کلاینت می‌تواند مثلاً یک بک‌اندِ وب‌سرور باشد (که اغلب به‌عنوان Backend For Frontend یا BFF عمل می‌کند) یک سرویس بک‌اندِ یک‌پارچه‌سازی، یا یک برنامه فرانت‌اند تک‌صفحه‌ای (SPA) یا همان برنامه مبتنی بر مرورگر باشد.

به‌طور کلی، Backend Clients به‌عنوان Clients Confidential و Clients Frontend به‌عنوان Clients Public در نظر گرفته می‌شوند. با این حال، برنامه‌های Native که روی دستگاه کاربر نهایی اجرا می‌شوند می‌توانند در صورتی که از Dynamic Client Registration OAuth استفاده کنند، به‌عنوان Confidential در نظر گرفته شوند.

#	Description	Level
10.2.1	در Code، Flow باید OAuth Client در برابر حملات Request Forgery Browser-based که عموماً با عنوان Cross-Site Request Forgery (CSRF) شناخته می‌شوند و درخواست‌های توکن را ایجاد می‌کنند، محافظت داشته باشد. این محافظت باید با استفاده از قابلیت PKCE یا با بررسی پارامتر state که در درخواست احراز مجوز ارسال شده است.	2
10.2.2	اگر OAuth Client بتواند با بیش از یک Authorization Server تعامل داشته باشد، باید دارای سازوکاری برای دفاع در برابر Mix-up Attacks باشد. برای مثال، می‌تواند الزام کند که Authorization Server مقدار پارامتر 'iss' را برگرداند و این مقدار هم در Response Authorization و هم در Response Token اعتبارسنجی شود.	2
10.2.3	OAuth Client باید فقط Scopes (یا سایر پارامترهای مجوزدهی) مورد نیاز را در درخواست‌ها به Authorization Server درخواست کند.	3

V10.3 OAuth Resource Server

در زمی‌نه‌ی ASVS و ای‌ن ف‌ص‌ل، Resource Server ی‌ک API اس‌ت. ب‌رای ف‌راه‌م ک‌ردن دس‌ت‌رس‌ی ای‌م‌ن، Resource Server ب‌ای‌د:

- Access Token را م‌طاب‌ق ب‌ا ف‌رم‌ت ت‌وک‌ن و م‌ش‌خ‌ص‌ات پ‌روت‌ک‌ل م‌رب‌وط‌ه اع‌ت‌ب‌ارس‌نج‌ی ک‌ن‌د، ب‌رای م‌ث‌ال ب‌ا اس‌ت‌ف‌اده از JWT-validation ی‌ا OAuth Token Introspection.
- اگ‌ر Access Token م‌ع‌ت‌ب‌ر ب‌اش‌د، ت‌ص‌م‌ی‌م‌ات م‌ج‌وزه‌ی (Authorization Decisions) را ب‌ر اس‌اس اط‌لاع‌ات م‌وج‌ود در ت‌وک‌ن و دس‌ت‌رس‌ی‌ه‌ای اع‌ط‌ا ش‌ده اع‌م‌ال ن‌م‌ای‌د. ب‌رای م‌ث‌ال، Resource Server ب‌ای‌د ب‌ررس‌ی ک‌ن‌د ک‌ه آی‌ا Client (ک‌ه ب‌ه ن‌م‌ای‌ن‌د‌گ‌ی از RO ع‌م‌ل م‌ی‌ک‌ن‌د) اج‌ازه‌ی دس‌ت‌رس‌ی ب‌ه Resource درخ‌واس‌ت‌ی را دارد ی‌ا خ‌ی‌ر.

ب‌ن‌اب‌رای‌ن، ال‌زام‌ات ذک‌رش‌ده در ای‌ن‌ج‌ا م‌خ‌ت‌ص OAuth ی‌ا OIDC ه‌س‌ت‌ن‌د و ب‌ای‌د پ‌س از اع‌ت‌ب‌ارس‌نج‌ی ت‌وک‌ن و پ‌ی‌ش از اع‌م‌ال ت‌ص‌م‌ی‌م‌ات م‌ج‌وزه‌ی ب‌ر اس‌اس اط‌لاع‌ات م‌وج‌ود در ت‌وک‌ن ان‌ج‌ام ش‌ون‌د.

#	Description	Level
10.3.1	Resource Server ب‌ای‌د ف‌ق‌ط Access ه‌ای‌یToken را ب‌پ‌ذی‌رد ک‌ه ب‌رای اس‌ت‌ف‌اده ب‌ا ه‌م‌ان س‌روی‌س در ن‌ظ‌ر گ‌رف‌ت‌ه ش‌ده‌ان‌د(Audience). Audience م‌ی‌ت‌وان‌د در ی‌ک Access Token س‌اخ‌ت‌اری‌اف‌ت‌ه (م‌ان‌ن‌د م‌ق‌دار aud در (JWT درج ش‌ده ب‌اش‌د، ی‌ا از ط‌ری‌ق Token Introspection Endpoint ب‌ررس‌ی ش‌ود.	2
10.3.2	Resource Server ب‌ای‌د ت‌ص‌م‌ی‌م‌ات م‌ج‌وزه‌ی (Authorization Decisions) را ب‌ر اس‌اس Claims م‌وج‌ود در Access Token ک‌ه واگ‌ذاری م‌ج‌وز (Delegated Authorization) را ت‌ع‌ری‌ف م‌ی‌ک‌ن‌ن‌د، اع‌م‌ال ک‌ن‌د. اگ‌ر ه‌ای‌یClaims م‌ان‌ن‌د ،scope sub و authorization_details م‌وج‌ود ب‌اش‌ن‌د، ب‌ای‌د ج‌زو ف‌رآی‌ن‌د ت‌ص‌م‌ی‌م‌گ‌ی‌ری ق‌رار گ‌ی‌رن‌د.	2
10.3.3	اگ‌ر ب‌رای گ‌رف‌ت‌ن ی‌ک ت‌ص‌م‌ی‌م ک‌نت‌رل دس‌ت‌رس‌ی، ل‌ازم اس‌ت ی‌ک ک‌ارب‌رِ ی‌ک‌ت‌ا از داخ‌ل Access Token (چ‌ه JWT و چ‌ه پ‌اس‌خ (Token Introspection ش‌ن‌اس‌ای‌ی ش‌ود، س‌رور م‌ن‌ب‌ع (Resource Server) ک‌ارب‌ر را از روی ه‌ای‌یClaim ش‌ن‌اس‌ای‌ی ک‌ن‌د ک‌ه ق‌اب‌ل اخ‌ت‌ص‌اص‌دادن ب‌ه ک‌ارب‌ران دی‌گ‌ر ن‌ی‌س‌ت‌ن‌د. م‌ع‌م‌ولاً ای‌ن ی‌ع‌ن‌ی اس‌ت‌ف‌اده از ت‌رک‌ی‌ب دو Claim ی‌ع‌ن‌ی ’iss‘ (ص‌ادرک‌ن‌ن‌ده ت‌وک‌ن) و ’sub‘ (ش‌ن‌اس‌ه ی‌ک‌ت‌ای ک‌ارب‌ر)	2
10.3.4	اگ‌ر Resource Server ن‌ی‌ازم‌ن‌د ق‌درت، روش ی‌ا ت‌ازگ‌ی خ‌اص اح‌راز ه‌وی‌ت ب‌اش‌د، ب‌ررس‌ی ک‌ن‌د ک‌ه Access Token ارائ‌ه‌ش‌ده ای‌ن م‌ح‌دودی‌ت‌ه‌ا را ب‌رآورده م‌ی‌ک‌ن‌د. ب‌رای م‌ث‌ال، در ص‌ورت وج‌ود، از ه‌ای‌یOIDC Claims م‌ان‌ن‌د ،amr acr و auth_time ب‌ه ت‌رت‌ی‌ب اس‌ت‌ف‌اده ش‌ود.	2
10.3.5	ت‌أی‌ی‌د ک‌ن‌ی‌د ک‌ه Resource Server از اس‌ت‌ف‌اده از Access ه‌ای‌یToken س‌رق‌ت‌ش‌ده ی‌ا Replay ت‌وک‌ن‌ه‌ا (ت‌وس‌ط اف‌راد غ‌ی‌رم‌ج‌از) ج‌ل‌وگ‌ی‌ری ک‌ن‌د، ب‌ا ال‌زام اس‌ت‌ف‌اده از Sender-Constrained Access Tokens، م‌ان‌ن‌د Mutual TLS ب‌رای OAuth 2 ی‌ا OAuth2 Demonstration of Proof of Possession (DPoP).	3

V10.4 OAuth Authorization Server

ای‌ن الزامات، م‌س‌ئول‌ی‌ت‌ه‌ای م‌رب‌وط ب‌ه OAuth Authorization Servers، ش‌ام‌ل OpenID Providers را ت‌ش‌ری‌ح م‌ی ک‌ن‌ن‌د.

ب‌رای Client Authentication، روش 'self_signed_tls_client_auth' م‌ج‌از اس‌ت، ه‌م‌راه ب‌ا پ‌ی‌ش‌ن‌ی‌ازه‌ای‌ی ک‌ه در ب‌خ‌ش 2.2 از RFC 8705 م‌ش‌خ‌ص ش‌ده‌ان‌د.

#	Description	Level
10.4.1	س‌رور اح‌راز ه‌وی‌ت ب‌ای‌د آدرس‌ه‌ای ب‌ازگ‌ش‌ت‌ی (Redirect URI) را ب‌ر اس‌اس ف‌ه‌رس‌ت م‌ج‌از (Allowlist) از ه‌ایURI از پ‌ی‌ش‌ث‌ب‌ت‌ش‌ده و م‌خ‌ص‌وص ه‌ر ک‌ل‌ای‌ن‌ت، ب‌ا اس‌ت‌ف‌اده از م‌ق‌ای‌س‌ه دق‌ی‌قِ رش‌ت‌ه‌ای (Exact String Comparison) اع‌ت‌ب‌ارس‌نج‌ی م‌ی ک‌ن‌د.	1
10.4.2	اگ‌ر Authorization Server ک‌د اح‌راز م‌ج‌وز (Authorization Code) را در Authorization Response ب‌رم‌ی گ‌ردان‌د، ای‌ن ک‌د ف‌ق‌ط ی ک‌ب‌ار ب‌رای درخ‌واس‌ت ت‌وک‌ن ب‌ای‌د ق‌اب‌ل اس‌ت‌ف‌اده ب‌اش‌د. ب‌رای دوم‌ی‌ن درخ‌واس‌ت م‌ع‌ت‌ب‌ر ب‌ا ی ک Authorization Code ک‌ه ق‌ب‌لاً ب‌رای ص‌دور Access Token اس‌ت‌ف‌اده ش‌ده اس‌ت، Authorization Server ب‌ای‌د درخ‌واس‌ت ت‌وک‌ن را رد ک‌ن‌د و ه‌ر ت‌وک‌ن ص‌ادرش‌ده‌ی م‌رت‌ب‌ط ب‌ا آن Authorization Code را ب‌اط‌ل ن‌م‌ای‌د.	1
10.4.3	ت‌أ‌ی‌ی‌د ک‌ن‌ی‌د ک‌ه Authorization Code دارای م‌دت زم‌ان م‌ع‌ت‌ب‌ر ب‌ودن م‌ح‌دود ب‌اش‌د. ح‌داک‌ث‌ر ط‌ول ع‌م‌ر م‌ی‌ت‌وان‌د ب‌رای ب‌رن‌ام‌ه‌ه‌ای ک‌ارب‌ردی در س‌ط‌وح L1 و L2 ت‌ا ۱۰ دق‌ی‌ق‌ه و ب‌رای ب‌رن‌ام‌ه‌ه‌ای ک‌ارب‌ردی در س‌ط‌ح L3 ت‌ا ۱ دق‌ی‌ق‌ه ب‌اش‌د.	1
10.4.4	ب‌رای ی ک Client م‌ش‌خ‌ص، Authorization Server ف‌ق‌ط اس‌ت‌ف‌اده از ه‌ای‌Grant را م‌ج‌از ب‌دان‌د ک‌ه آن Client ب‌ه اس‌ت‌ف‌اده از آن‌ه‌ا ن‌ی‌از دارد. ت‌وج‌ه داش‌ت‌ه ب‌اش‌ی‌د ک‌ه ه‌ای‌Grant 'token' (Implicit Flow) و 'password' (Resource Owner Password Credentials Flow) دی گ‌ر ن‌ب‌ای‌د اس‌ت‌ف‌اده ش‌ون‌د.	1
10.4.5	Authorization Server ب‌ای‌د ح‌م‌لات Refresh Token Replay را ب‌رای Clientاه Public ک‌اه‌ش ده‌د، ت‌رج‌ی‌حاً ب‌ا اس‌ت‌ف‌اده از Sender-Constrained Refresh Tokens، ی‌ع‌نی Certificate-Bound Access Tokens ی‌ا Demonstrating Proof of Possession (DPoP) ب‌ا اس‌ت‌ف‌اده از Mutual TLS (mTLS). ب‌رای ب‌رن‌ام‌ه‌ه‌ای L1 و L2، م‌ی‌ت‌وان از Refresh Token Rotation اس‌ت‌ف‌اده ک‌رد. اگ‌ر Refresh Token Rotation ب‌ه ک‌ار گ‌رف‌ت‌ه ش‌ود، Authorization Server ب‌ای‌د پ‌س از اس‌ت‌ف‌اده، Refresh Token را ب‌اط‌ل ک‌ن‌د و اگ‌ر ی ک Refresh Token ک‌ه ق‌ب‌لاً اس‌ت‌ف‌اده و ب‌اط‌ل ش‌ده ارائ‌ه ش‌ود، ت‌م‌ام Refresh ه‌ای‌Token م‌رب‌وط ب‌ه آن م‌ج‌وز را اب‌ط‌ال ن‌م‌ای‌د.	1

#	Description	Level
10.4.6	اگر از Code Grant استفاده شود، Authorization Server باید حملات Authorization Code Interception را با الزام به استفاده از Proof Key for Code Exchange (PKCE) کاهش دهد. برای Authorization Server Requestها، Authorization باید یک مقدار معتبر برای 'code_challenge' الزام کند و نباید مقدار 'code_challenge_method' از نوع 'plain' را بپذیرد. برای Token Request، باید اعتبارسنجی پارامتر 'code_verifier' الزامی باشد.	2
10.4.7	اگر Authorization Server از Unauthenticated Dynamic Client Registration پشتیبانی می‌کند، ریسک مربوط به Client هایApplication مخرب را باید کاهش دهد. این کار باید شامل اعتبارسنجی Client Metadata (مانند هر URI)، Registered اطمینان از رضایت کاربر، و هشدار به کاربر پیش از پردازش یک Authorization Request با یک Client Application غیرقابل اعتماد باشد.	2
10.4.8	Refresh Tokenها باید دارای یک انقضای مطلق (Absolute Expiration) باشند، حتی اگر از Sliding Refresh Token Expiration استفاده شود.	2
10.4.9	Refresh Tokenها و Reference Access Tokenها باید توسط کاربر مجاز از طریق User Interface Authorization Server قابل ابطال باشند، تا ریسک هایClient مخرب یا توکن‌های سرقت‌شده کاهش یابد.	2
10.4.10	Confidential Client باید برای درخواست‌های Backchannel بین Client و Authorization Server، مانند Token Requests، Pushed Authorization Requests (PAR) و Token Revocation Requests، احراز هویت شود.	2
10.4.11	Authorization Server باید فقط Scopes موردنیاز را به OAuth Client اختصاص دهد.	2
10.4.12	برای یک Client مشخص، Authorization Server فقط مقدار 'response_mode' را که این Client نیاز دارد، مجاز بداند. برای مثال، با اعتبارسنجی این مقدار توسط Authorization Server در برابر مقادیر مورد انتظار یا با استفاده از Pushed Authorization Request (PAR) یا JWT-secured Authorization Request (JAR).	3
10.4.13	Grant Type 'code' همواره همراه با Pushed Authorization Requests (PAR) باید استفاده شود.	3
10.4.14	Authorization Server باید فقط توکن‌های دسترسی از نوع وابسته به ارسال‌کننده (Proof-of-Possession) صادر می‌کند؛ چه به‌صورت توکن‌های دسترسی مبتنی بر گواهی از طریق TLS دوسویه (mTLS) و چه به‌صورت توکن‌های دسترسی وابسته به DPoP (Demonstration of Proof of Possession).	3

#	Description	Level
10.4.15	برای ی ک کل‌ای‌نت س‌مت‌س‌رور (که روی دست‌گاه کاربر نه‌ای‌ی اجرا نم‌ی‌ش‌ود)، س‌رور احراز هویت اطم‌ی‌نان حاصل م‌ی ک‌ند که مقدار پارامت‌رِ authorization_details از ب ک‌اند کل‌ای‌نت ارس‌ال ش‌ده و کاربر در آن دست‌کاری ن‌کرده است. به‌ع‌نوان م‌ثال، با الزام به اس‌ت‌فاده از درخواست احراز هویت پوش‌ش‌ده (PAR) ی‌ا درخواست احراز هویت ای‌م‌ن‌ش‌ده با JWT (JAR).	3
10.4.16	کل‌ای‌نت باید از نوع Confidential Client باشد و س‌رور احراز هویت اس‌ت‌فاده از روش‌های قدرت‌م‌ند احراز هویت کل‌ای‌نت ـ مبت‌نی بر رمزنگاری کل‌ید ع‌موم‌ی و مقاوم در برابر ح‌مل‌ات replay attacks ـ را الزام‌ی ک‌ند؛ مانند TLS دوس‌ویه (tls_client_auth ، self_signed_tls_client_auth ی‌ا JWT مبت‌نی بر کل‌ید خ‌صوص‌ی .(private_key_jwt)	3

V10.5 OIDC Client

از آن‌جا ک‌ه Relying Party در OIDC نقش ی ک کل‌ای‌نت OAuth را هم ای‌فا م‌ی ک‌ند، الزامات بخ‌ش «OAuth Client» نی‌ز در ای‌نج‌ا قابل اع‌مال هس‌ت‌ند.

هم‌چن‌ی‌ن ت‌وج‌ه داش‌ته باش‌ید که بخ‌ش «Authentication with an Identity Provider» در ف‌ص‌ل «Authentication» نی‌ز ش‌امل الزامات ع‌موم‌ی مرتبط و هم‌ی است.

#	Description	Level
10.5.1	Client به‌ع‌نوان Relying پ‌ای‌دParty در برابر ح‌مل‌ات بازپخ‌ش (Replay) ت‌وک‌ن ID مح‌اف‌ظت ش‌ده است. برای م‌ثال، با بررس‌ی ای‌نک‌ه مقدار claim مربوط به nonce در ID Token دقی‌قاً با مقدار **nonce**ای ک‌ه در درخواست احراز هویت به OpenID Provider ارس‌ال ش‌ده، ی ک‌س‌ان باش‌د(که در OAuth 2 به آن authorization request ارس‌ال‌ی به authorization server گف‌ته م‌ی‌ش‌ود).	2
10.5.2	Client باید کاربر را به‌ص‌ورت ی ک‌تا از ID Token Claims ش‌ناس‌ای‌ی ک‌ند، مع‌مولاً از ‘sub’ Claim ، که قابل انت‌س‌اب مج‌دد به کاربران دی گ‌رن‌ی‌س‌ت (در مح‌دوده‌ی هم‌ان Identity Provider).	2
10.5.3	Client باید ت‌لاش‌ه‌ای ی ک Authorization Server مخرب برای جا زدن خود به‌ج‌ای ی ک Authorization Server دی گ‌راز طریق مت‌ادی‌ت‌ای Authorization Server را رد م‌ی ک‌ند. کل‌ای‌نت باید مت‌ادی‌ت‌ای Authorization Server را رد ک‌ند اگ‌ر مقدار issuer URL موجود در مت‌ادی‌ت‌ا دقی‌قاً با issuer URL از پ‌ی‌ش پ‌ی ک‌ربن‌دی‌ش‌ده و مورد انت‌ظار کل‌ای‌نت ی ک‌س‌ان نباش‌د.	2
10.5.4	Client باید اع‌ت‌بارس‌نج‌ی ک‌ند ID Token برای اس‌ت‌فاده ت‌وس‌ط هم‌ان Client در نظر گ‌رف‌ته ش‌ده است ،(Audience) با بررس‌ی ای‌نک‌ه Claim ‘aud’ در ت‌وک‌ن برابر با مقدار ‘client_id’ مربوط به Client باش‌د.	2

#	Description	Level
10.5.5	هنگام استفاده از Back-Channel OIDC، Denial of Service از Relying Party Logout از طریق Forced Logout و Cross-JWT Confusion در جریان Logout باید جلوگیری کند. Client باید بررسی کند که Logout Token به درستی با مقدار typ آن برابر با logout+jwt باشد ، شامل Claim ‘event’ با نام عضو صحیح باشد و شامل Claim ‘nonce’ نباشد. همچنین توصیه می‌شود Expiration کوتاه باشد (مثلاً ۲ دقیقه).	2

V10.6 OpenID Provider

از آنجا که OpenID Providerها نقش Authorization Server در OAuth را نیز ایفا می‌کنند، الزامات بخش «OAuth Authorization Server» در اینجا نیز قابل اعمال هستند.

توجه شود در صورتی که از ID Token Flow و نه Code Flow استفاده شود، Access ایTokenها صادر نمی‌شود و در نتیجه بسیاری از الزامات مربوط به OAuth Authorization Server قابل اعمال نخواهند بود.

#	Description	Level
10.6.1	OpenID Provider باید فقط مقادیر ‘code’، ‘ciba’، ‘id_token’ یا ‘id_token code’ را برای Response Mode مجاز بداند. توجه داشته باشید که ‘code’ نسبت به ‘id_token code’ (که OIDC Hybrid Flow است) ترجیح داده می‌شود و ‘token’ (در Implicit Flow) نباید استفاده شود.	2
10.6.2	OpenID Provider از Denial of Service از طریق Forced Logout جلوگیری کند. این کار با دریافت تأیید صریح از کاربر نهایی یا در صورت وجود، اعتبارسنجی پارامترها در Logout Request (که توسط Relying Party آغاز شده است)، مانند ‘id_token_hint’، انجام می‌شود.	2

V10.7 Consent Management

این الزامات، فرآیند تأیید رضایت (Consent) کاربر توسط Authorization Server را پوشش می‌دهند.در صورت نبودِ تأیید صحیح رضایت کاربر، یک مهاجم می‌تواند از طریق جعل (Spoofing) یا مهندسی اجتماعی (Social Engineering) به‌جای کاربر مجوزها (Permissions) را به دست آورد.

#	Description	Level
10.7.1	Authorization Server باید تضمین کند کاربر برای هر درخواست مجوز (Authorization Request) رضایت خود را اعلام کرده است. در صورتی که هویت کلاینت قابل اطمینان نباشد، Authorization Server باید در همه موارد کاربر را به‌صورت صریح برای اعلام رضایت (Consent) راهنمایی و از او تأیید بگیرد.	2
10.7.2	زمانی که Authorization Server از کاربر درخواست رضایت (Consent) می‌کند باید اطلاعات کافی و شفافی درباره آنچه کاربر در حال تأیید آن است ارائه دهد. در صورت لزوم، این اطلاعات باید شامل این موارد باشد: ماهیت مجوزهای درخواستی (معمولاً بر اساس resource server scope)، جزئیات مجوز در Rich Authorization Requests – (RAR)، هویت برنامه یا اپلیکیشن دریافت‌کننده مجوز و مدت اعتبار این مجوزها.	2
10.7.3	کاربر باید بتواند رضایت‌های (Consents) را که از طریق Authorization Server صادر کرده است، مشاهده، ویرایش و لغو (Revoke) کند.	2

References

برای اطلاعات بیشتر دربارهٔ OAuth، لطفاً مشاهده کنید:

- oauth.net
- OWASP OAuth 2.0 Protocol Cheat Sheet

برای الزامات مرتبط با OAuth در ASVS، از RFCهای منتشر شده و در وضعیت پیشنویس زیر استفاده می‌شود:

- RFC6749 The OAuth 2.0 Authorization Framework
- RFC6750 The OAuth 2.0 Authorization Framework: Bearer Token Usage
- RFC6819 OAuth 2.0 Threat Model and Security Considerations
- RFC7636 Proof Key for Code Exchange by OAuth Public Clients
- RFC7591 OAuth 2.0 Dynamic Client Registration Protocol
- RFC8628 OAuth 2.0 Device Authorization Grant
- RFC8707 Resource Indicators for OAuth 2.0
- RFC9068 JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens
- RFC9126 OAuth 2.0 Pushed Authorization Requests
- RFC9207 OAuth 2.0 Authorization Server Issuer Identification
- RFC9396 OAuth 2.0 Rich Authorization Requests
- RFC9449 OAuth 2.0 Demonstrating Proof of Possession (DPoP)
- RFC9700 Best Current Practice for OAuth 2.0 Security
- draft OAuth 2.0 for Browser-Based Applications
- draft The OAuth 2.1 Authorization Framework

برای اطلاعات بیشتر در مورد OpenID Connect، لطفاً به بخش زیر مراجعه کنید:

- OpenID Connect Core 1.0
- FAPI 2.0 Security Profile

V11 Cryptography

هدف

هدف این فصل تعریف بهترین رویه‌ها برای استفاده عمومی از رمزنگاری است. همچنین به دنبال آن است که درک بنیادی از اصول رمزنگاری ایجاد کرده و تغییری به سمت روی کرده‌ای مقاوم‌تر و مدرن‌تر را ترغیب کند. این فصل این موارد را تشویق(پیشنهاد) می کند:

- پیاده‌سازی سیستم‌های رمزنگاری مقاوم که به‌طور ایمن دچار خطا شوند، با تهدیدات در حال تکامل سازگار شوند و آینده‌نگر باشند.
- به کارگیری سازوکارهای رمزنگاری که هم ایمن باشند و هم با بهترین رویه‌های صنعت همسو باشند.
- حفظ یک سیستم مدیریت کلیدهای رمزنگاری امن، همراه با کنترل‌های دسترسی مناسب و قابلیت ممیزی.
- ارزیابی منظم چشم‌انداز رمزنگاری به‌منظور شناسایی خطرات جدید و تطبیق الگوریتم‌ها بر اساس آن‌ها.
- کشف و مدیریت موارد استفاده‌ی رمزنگاری در طول چرخه‌ی عمر برنامه، به‌منظور اطمینان از ثبت و حفاظت تمامی دارایی‌های رمزنگاری.

علاوه بر تشریح اصول عمومی و بهترین شیوه‌ها، این سند اطلاعات فنی عمیق‌تری درباره الزامات در پیوست C استاندارده‌ای رمزنگاری ارائه می‌دهد. شامل الگوریتم‌ها و حالت‌هایی که برای اهداف الزامات این فصل «تأییدشده» محسوب می‌شوند.

الزامات استفاده کننده از رمزنگاری برای حل مسائل دیگر، مانند مدیریت secrets یا امنیت ارتباطات ، در بخش‌های دیگر استاندارد بررسی خواهند شد.

V11.1 Cryptographic Inventory and Documentation

برنامه‌ها باید با معماری رمزنگاری قدرتمند طراحی شوند تا از دارایی‌های داده مطابق با طبقه‌بندی آن‌ها محافظت شود. رمزگذاری همچیز کاری پرهزینه و بی‌هوده است؛ درحالی‌که رمزگذاری نکردن هیچ-چیز، از نظر قانونی سهلانگاری محسوب می‌شود. باید تعادلی برقرار شود، معمولاً در مرحله معماری یا طراحی سطح بالا، اسپرینت‌های طراحی یا آزمایش‌های معماری .(architectural spikes) طراحی رمزنگاری به صورت لحظه‌ای یا اضافه کردن آن به سیستم پس از طراحی، به مراتب پرهزینه‌تر و دشوارتر از لحاظ امنیتی خواهد بود نسبت به حالتی که از ابتدا در طراحی لحاظ شده باشد.

مهم است که اطمینان حاصل شود تمامی دارایی‌های رمزنگاری به‌طور منظم شناسایی، فهرست‌بندی و ارزیابی شوند. لطفاً برای کسب اطلاعات بیشتر درباره نحوه انجام این کار به پیوست مراجعه کنید.

نی‌از به آی‌نده‌نگ‌ری در براب‌ر ظه‌ور اج‌ت‌ناب‌ناپ‌ذی‌ر رای‌انش ک‌وانت‌ومی برای مقاوم‌سازی س‌ی‌س‌ت‌م-های رمزنگ‌اری نی‌ز ح‌ی‌ات‌ی است. رمزنگ‌اری پ‌س‌اک‌وانت‌ومی (Post-Quantum Cryptography) ی‌ا (PQC به ال‌گ‌وری‌ت‌م‌های رمزنگ‌اری‌ای اش‌اره دارد ک‌ه برای مقاومت در براب‌ر ح‌م‌لات رای‌انه‌ای ک‌وانت‌ومی طراح‌ی ش‌ده‌اند؛ رای‌انه‌ای‌ی ک‌ه انت‌ظار م‌ی‌رود ال‌گ‌وری‌ت‌م‌های پرک‌اربردی ه‌مچ‌ون RSA و رمزنگ‌اری منح‌نی ب‌ی‌ض‌وی (ECC) را ب‌ش‌ک‌نند.

ل‌ط‌فاً برای دری‌اف‌ت راه‌نم‌ای‌ی‌های ک‌نون‌ی درباره اب‌ت‌دای‌ی‌ات و اس‌ت‌ان‌دارده‌ای ت‌أی‌ی‌دش‌ده در زم‌ی‌نه رمزنگ‌اری پ‌س‌اک‌وانت‌ومی ،(PQC) به ض‌م‌ی‌مه م‌راجع‌ه ک‌ن‌ی‌د.

#	Description	Level
11.1.1	اط‌م‌ی‌نان ح‌اص‌ل ک‌ن‌ی‌د ک‌ه ی‌ک س‌ی‌اس‌ت م‌س‌ت‌ند برای م‌دی‌ری‌ت ک‌ل‌ی‌ده‌ای رمزنگ‌اری و چ‌رخ‌ه‌ع‌م‌ر ک‌ل‌ی‌ده‌ای رمزنگ‌اری وج‌ود دارد ک‌ه از ی‌ک اس‌ت‌ان‌دارد م‌دی‌ری‌ت ک‌ل‌ی‌د مانند NIST SP 800-57 پ‌ی‌روی م‌ی‌ک‌ند. ای‌ن ب‌ای‌د ش‌ام‌ل اط‌م‌ی‌نان از ای‌ن م‌وض‌وع ب‌اش‌د ک‌ه ک‌ل‌ی‌ده‌ا ب‌ی‌ش از ح‌د به اش‌ت‌راک گ‌ذاش‌ته ن‌ش‌وند (برای م‌ث‌ال، در م‌ورد Secrets م‌ش‌ت‌رک با ب‌ی‌ش از دو م‌وج‌ودی‌ت و در م‌ورد ک‌ل‌ی‌ده‌ای خ‌ص‌وص‌ی با ب‌ی‌ش از ی‌ک م‌وج‌ودی‌ت).	2
11.1.2	ف‌ه‌رس‌ت اج‌زاء رمزنگ‌اری ب‌ای‌د ت‌ه‌ی‌ه و ن‌گ‌ه‌داری ش‌ود و ب‌ه‌ص‌ورت من‌ظ‌م ب‌ه‌روزرس‌انی گ‌ردد و ش‌ام‌ل ت‌م‌ام ک‌ل‌ی‌ده‌ای رمزنگ‌اری، ال‌گ‌وری‌ت‌م‌ها و گ‌واه‌ی‌های دی‌ج‌ی‌ت‌ال م‌ورد اس‌ت‌ف‌اده در ب‌رنام‌ه ب‌اش‌د. ای‌ن ف‌ه‌رس‌ت ه‌مچ‌ن‌ی‌ن ب‌ای‌د م‌ش‌خ‌ص ک‌ن‌د ک‌ه ک‌ل‌ی‌ده‌ا در ک‌دام ب‌خ‌ش‌ه‌ای س‌ی‌س‌ت‌م م‌جاز ی‌ا غ‌ی‌رم‌جاز ب‌ه اس‌ت‌ف‌اده ه‌س‌ت‌ند و چ‌ه ن‌وع داده‌ه‌ای‌ی م‌ی‌ت‌وانند ی‌ا ن‌م‌ی‌ت‌وانند با ای‌ن ک‌ل‌ی‌ده‌ا م‌ح‌اف‌ظ‌ت ش‌ون‌د.	2
11.1.3	م‌ک‌ان‌ی‌زم‌های ک‌ش‌ف رمزنگ‌اری برای ش‌ن‌اس‌ای‌ی ت‌م‌ام م‌وارد اس‌ت‌ف‌اده از رمزنگ‌اری در س‌ی‌س‌ت‌م ب‌ای‌د اس‌ت‌ف‌اده ش‌ود؛ از ج‌م‌له رمزنگ‌اری ،(encryption) ه‌ش‌ک‌ردن (hashing) و ع‌م‌ل‌ی‌ات امض‌ای دی‌ج‌ی‌ت‌ال .(signing)	3
11.1.4	ف‌ه‌رس‌ت اج‌زاء رمزنگ‌اری ب‌ای‌د ت‌ه‌ی‌ه ش‌ود. ای‌ن ف‌ه‌رس‌ت ب‌ای‌د ش‌ام‌ل ی‌ک ب‌رنام‌ه م‌س‌ت‌ند ب‌اش‌د ک‌ه م‌س‌ی‌ر م‌ه‌اج‌رت ب‌ه اس‌ت‌ان‌دارده‌ای ج‌دی‌د رمزنگ‌اری (مانند رمزنگ‌اری پ‌س‌اک‌وانت‌ومی) را م‌ش‌خ‌ص م‌ی‌ک‌ند ت‌ا ام‌ک‌ان واک‌ن‌ش من‌اس‌ب ب‌ه ت‌ه‌دی‌دات آی‌نده ف‌راه‌م ش‌ود.	3

V11.2 Secure Cryptography Implementation

ای‌ن ب‌خ‌ش ال‌زام‌ات م‌رب‌وط ب‌ه انت‌خ‌اب، پ‌ی‌اده‌س‌ازی و م‌دی‌ری‌ت م‌س‌ت‌م‌ر ال‌گ‌وری‌ت‌م‌های اص‌ل‌ی رمزنگ‌اری برای ی‌ک ب‌رنام‌ه را ت‌ع‌ری‌ف م‌ی‌ک‌ند. ه‌دف ای‌ن اس‌ت ک‌ه ت‌ن‌ه‌ا اب‌ت‌دای‌ی‌ات رمزنگ‌اری (primitives) ق‌درت‌م‌ند و م‌ورد پ‌ذی‌رش ص‌ن‌ع‌ت، م‌طاب‌ق با اس‌ت‌ان‌دارده‌ای ف‌ع‌ل‌ی (مانند NIST و ISO/IEC و ب‌ه‌ت‌ری‌ن روی‌ه‌ا ب‌ه ک‌ار گ‌رف‌ته ش‌ون‌د. س‌ازمان‌ه‌ا ب‌ای‌د اط‌م‌ی‌نان ح‌اص‌ل ک‌ن‌ند ک‌ه ه‌ر ج‌زء رمزنگ‌اری ب‌ر اس‌اس ش‌واه‌د داوری‌ش‌ده ت‌وس‌ط م‌تخ‌ص‌ص‌ان (peer-reviewed) و آزم‌ای‌ش‌های ع‌م‌ل‌ی ام‌ن‌ی‌ت‌ی انت‌خ‌اب ش‌ده اس‌ت.

#	Description	Level
11.2.1	برای عملیات رمزنگاری بایـد از پیادهسازیهای مورد تأیید (شامل کتابخانهها و پیادهسازیهای سختافزاری) استفاده شود.	2
11.2.2	برنامه با انعطافپذیری رمزنگاری (crypto agility) بایـد طراحی شده باشد، بهطوری که الگوریتمهای تولید اعداد تصادفی، رمزنگاری احراز هویت شده، MAC یا هش، طول کلیدها، تعداد دورها، رمزها و حالتها بتوانند در هر زمان مجدداً پیکربندی، ارتقاء یا تعویض شوند تا در برابر شکستهای رمزنگاری محافظت شوند. به همین ترتیب، بایـد امکان جایگزینی کلیدها و رمزهای عبور و رمزگذاری مجدد دادهها نیز وجود داشته باشد. این امکان ارتقاء بدون وقفه به رمزنگاری پساکوانتومی (PQC) را فراهم میکند، زمانی که پیادهسازیهای با اطمینان بالا از طرحها یا استانداردهای تأییدشده PQC بهطور گستردە در دسترس قرار گیرند.	2
11.2.3	تمام بدویات رمزنگاری (cryptographic primitives) بر اساس الگوریتم، اندازه کلید و پیکربندی، حداقل ۱۲۸ بیت سطح امنیتی را فراهم میکنند(2128)، برای مثال، یک کلید ECC با طول ۲۵۶ بیت تقریباً ۱۲۸ بیت امنیت ارائه میدهد، در حالی که برای رسیدن به همین سطح امنیت، الگوریتم RSA به کلید ۳۰۷۲ بیتی نیاز دارد.	2
11.2.4	تأیید کنید که تمامی عملیات رمزنگاری زمان ثابت (constant-time) باشند و هیچ عملیات «کوتاهشده» (short-circuit) در مقایسهها، محاسبات یا بازگشتها وجود نداشته باشد تا از نشت اطلاعات جلوگیری شود.(مهاجم میتواند از همین تفاوتهای زمانی، از طریق Timing Side-Channel Attack، اطلاعات حساس را استخراج کند.)	3
11.2.5	تأیید کنید که تمامی ماژولهای رمزنگاری بهصورت امن دچار خطا (fail securely) شوند و خطاها به گونهای مدیریت شوند که ایجاد آسیبپذیریهایی مانند حملات Padding Oracle را ممکن نکنند.	3

V11.3 Encryption Algorithms

الگوریتمهای رمزنگاری احراز هویتشده (Authenticated Encryption) مبتنی بر AES و CHACHA20، زیربنای عملکردهای رمزنگاری مدرن را تشکیل میدهند.

#	Description	Level
11.3.1	از مدهای بلاک ناامن (مانند ECB) و روشهای Padding ضعیف (مانند PKCS#1 v1.5) نباید استفاده شود.	1
11.3.2	تنها رمزها و مدهای تأییدشده، مانند AES با GCM بایـد استفاده شوند.	1

#	Description	Level
11.3.3	داده‌های رمزنگاری‌شده باید با استفاده از ترجیحاً یک روش رمزنگاری احراز هویت‌شده تأییدشده یا با ترکیب یک روش رمزنگاری تأییدشده با یک الگوریتم MAC تأییدشده در برابر تغییرات غیرمجاز محافظت شوند.	2
11.3.4	تأییدکنندهٔ Initialization Vector و سایر اعداد یکبار مصرف برای بیش از یک جفت کلید رمزنگاری و عنصر داده استفاده نشوند. روش تولید آن‌ها باید متناسب با الگوریتم مورد استفاده باشد.	3
11.3.5	تأییدکنندهٔ هر ترکیبی از یک الگوریتم رمزنگاری و یک الگوریتم MAC در حالت Encrypt-then-MAC عمل کند.	3

V11.4 Hashing and Hash-based Functions

توابع هش در انواع گسترده‌ای از پروتکل‌های رمزنگاری استفاده می‌شوند، مانند امضای دیجیتال، HMAC، توابع مشتق‌سازی کلید (KDF)، تولید بیت تصادفی و ذخیره‌سازی پسورد. امنیت سیستم رمزنگاری تنها به اندازه‌ی امنیت توابع هش پایه‌ای است که استفاده می‌شوند. این بخش الزامات استفاده از توابع هش امن در عملیات رمزنگاری را بیان می‌کند.

برای ذخیره‌سازی رمزعبور، علاوه بر پیوست رمزنگاری، OWASP Password Storage Cheat Sheet نیز زمینه و راهنمایی مفیدی ارائه می‌دهد.

#	Description	Level
11.4.1	تأییدکنندهٔ تنها توابع هش تأییدشده برای موارد استفاده عمومی رمزنگاری، از جمله امضاهای دیجیتال، HMAC، KDF و تولید بیت تصادفی استفاده شوند. توابع هش غیرمجاز، مانند MD5، نباید برای هیچ هدف رمزنگاری به‌کار گرفته شوند.	1
11.4.2	رمزعبورها با استفاده از یک تابع مشتق‌سازی کلید تأییدشده و پره‌زینه محاسباتی (که به آن «تابع هش رمزعبور» نیز گفته می‌شود) ذخیره شوند، و تنظیمات پارامترها بر اساس راهنمایی‌های کنونی پیکربندی شده باشد. این تنظیمات باید تعادل بین امنیت و عملکرد برقرار کنند تا حملات Brute-force برای سطح امنیت مورد نیاز به‌طور کافی دشوار باشد.	2
11.4.3	توابع هش استفاده‌شده در امضاهای دیجیتال، باید به‌عنوان بخشی از احراز هویت داده یا یکپارچگی داده، در برابر collision مقاوم بوده و طول بیت مناسب داشته باشند. اگر مقاومت در برابر collision لازم باشد، طول خروجی باید حداقل 256 بیت باشد. اگر تنها مقاومت در برابر حملات Second Pre-image لازم باشد، طول خروجی باید حداقل 128 بیت باشد.	2

#	Description	Level
11.4.4	برنامه‌ای که از توابع مشتق‌سازی کلید (KDF) تأییدشده با پارامترهای Key Stretching استفاده می‌کند. پارامترهای مورد استفاده باید تعادلی بین امنیت و عملکرد برقرار کنند تا از نفوذ حملات brute-force و به خطر افتادن کلید رمزنگاری تولیدشده جلوگیری شود.	2

V11.5 Random Values

تولید اعداد شبه‌تصادفی امن از نظر رمزنگاری (CSPRNG) بسیار دشوار است. به‌طور کلی، منابع خوب انتروپی در یک سیستم اگر ربیش از حد استفاده شوند، سریعاً تخلیه می‌شوند، اما منابع با تصادفی‌بودن کمتر می‌توانند منجر به ایجاد کلیدها و secrets قابل پیش‌بینی شوند.

#	Description	Level
11.5.1	اعداد و رشته‌های تصادفی که قرار است قابل حدس نباشند، باید با استفاده از یک تولیدکننده اعداد شبه‌تصادفی امن رمزنگاری‌شده (CSPRNG) تولید شوند و حداقل 128 بیت انتروپی داشته باشند. توجه داشته باشید که UUIDها این شرط را رعایت نمی‌کنند.	2
11.5.2	مکانیزم تولید اعداد تصادفی مورد استفاده به‌گونه‌ای طراحی شده باشد که حتی تحت بار سنگین نیز به‌طور امن عمل کند.	3

V11.6 Public Key Cryptography

رمزنگاری کلید عمومی (Public Key Cryptography) زمانی استفاده می‌شود که امکان یا تمایل به اشتراک‌گذاری کلید مخفی (secret key) بین چندین طرف وجود نداشته باشد.

به‌عنوان بخشی از این موضوع، نیاز به مکانیزم‌های تبادل کلید تأییدشده وجود دارد، مانند Diffie-Hellman و Elliptic Curve Diffie-Hellman (ECDH)، تا اطمینان حاصل شود که سیستم رمزنگاری در برابر ته‌دیدات مدرن امن باقی می‌مانند. فصل "ارتباط امن" الزامات مربوط به TLS را ارائه می‌دهد، بنابراین الزامات این بخش برای موقعیتی در نظر گرفته شده است که رمزنگاری کلید عمومی در موارد استفاده‌ای به غیر از TLS به کار گرفته شود.

#	Description	Level
11.6.1	فقط الگوریتم‌های رمزنگاری و مدهای عملیاتی تأیید شده برای تولید کلید و مقدارده‌ی اولیه، و همچنین برای تولید و تأیید امضاهای دیجیتال باید استفاده می‌شوند. الگوریتم‌های تولید کلید نباید کلیدهای ناامن تولید کنند که در برابر حملات شناخته شده آسیب‌پذیر باشند؛ به‌عنوان مثال، کلیدهای RSA که در برابر Fermat factorization آسیب‌پذیر هستند.	2
11.6.2	برای تبادل کلید از الگوریتم‌های رمزنگاری تأیید شده (مانند Diffie-Hellman) استفاده می‌شود و تمرکز بر این است که مکانیزم‌های تبادل کلید از پارامترهای امن به‌ره ببرند. این کار از حملات بر فرآیند برقراری کلید جلوگیری می‌کند و مانع حملات مرد میانی (MITM) یا شکست‌های رمزنگاری می‌شود.	3

V11.7 In-Use Data Cryptography

حفاظت از داده‌ها در حین پردازش اهمیت بالایی دارد. توصیه می‌شود از تکنیک‌هایی مانند رمزنگاری کامل حافظه، رمزنگاری داده‌ها در حال انتقال و اطمینان از رمزنگاری سریع داده‌ها پس از استفاده به‌ره گرفته شود.

#	Description	Level
11.7.1	رمزنگاری کامل حافظه (Full Memory Encryption) باید فعال باشد تا داده‌های حساس هنگام استفاده محافظت شوند و دسترسی کاربران یا فرآیندهای غیرمجاز به آن‌ها جلوگیری شود.	3
11.7.2	کاهش داده‌ها (data minimization) تضمین می‌کند که حداقل مقدار داده در حین پردازش افشا شود، و اطمینان حاصل شود که داده‌ها بلافاصله پس از استفاده یا در اولین فرصت ممکن رمزنگاری شوند.	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- OWASP Web Security Testing Guide: Testing for Weak Cryptography
- OWASP Cryptographic Storage Cheat Sheet
- FIPS 140-3
- NIST SP 800-57

V12 Secure Communication

هدف

این فصل شامل الزامات مربوط به مکانیزم‌های خاصی است که باید برای محافظت از داده‌ها در حین انتقال برقرار باشند، هم بین کلاینت کاربر نهایی و سرویس باکاند و هم بین سرویس‌های داخلی و باکاند.

مفاهیم کلیدی که در این فصل تروریج می‌شوند عبارتند از:

- اطمینان از اینکه ارتباطات به صورت خارجی رمزگذاری شده‌اند و در ایده‌آل حالت، به صورت داخلی نیز رمزگذاری شوند.
- پیکربندی مکانیزم‌های رمزگذاری با استفاده از آخرین دستورالعمل‌ها، شامل الگوریتم‌ها و رمزهای ترجیحی.
- استفاده از گواهی‌نامه‌های امضا شده برای اطمینان از اینکه ارتباطات توسط افراد غیرمجاز رهگیری نمی‌شوند.

علاوه بر ارائه اصول کلی و بهترین شیوه‌ها، ASVS اطلاعات فنی عمیق‌تری درباره قدرت رمزنگاری نیز در ضمیمه C استانداردهای رمزنگاری ارائه می‌دهد.

V12.1 General TLS Security Guidance

این بخش راهنمایی اولیه در مورد نحوه‌ی ایمن‌سازی ارتباطات TLS را ارائه می‌دهد. ابزارهای به‌روز باید برای بازبینی پیکربندی TLS به‌صورت مستمر استفاده شوند.

اگرچه استفاده از wildcard TLS ذاتاً ناامن نیست، اما در صورتی که یک گواهی که در تمامی محیط‌های test، development staging و production به‌کار رفته است به خطر بیفتد، ممکن است وضعیت امنیتی تمامی برنامه‌های استفاده‌کننده از آن گواهی نیز دچار آسیب شود. در صورت امکان، باید از حفاظت مناسب، مدیریت صحیح و همچنین استفاده از گواهی‌های TLS مجزا در محیط‌های مختلف بهره گرفته شود.

#	Description	Level
12.1.1	از نسخه‌های پیش‌نهادی پروتکل TLS فعال باشند، مانند TLS 1.2 و TLS 1.3 نسخه‌ی جدیدتر پروتکل TLS باید گزینه‌ی ترجیحی باشد.	1
12.1.2	تنها suiteهای cipher پیش‌نهادی فعال باشند و قوی‌ترین suiteهای cipher به‌عنوان گزینه‌ی ترجیحی تنظیم شده باشند. برنامه‌های L3 باید تنها از suiteهای cipher پشتیبانی کنند که Forward Secrecy را فراهم می‌کنند.	2
12.1.3	برنامه باید پیش از استفاده از هویت گواهی برای احراز هویت یا مجوزدهی، اعتبار و مورد اعتماد بودن گواهی‌های کلاینت در mTLS را بررسی و تأیید کند.	2

#	Description	Level
12.1.4	لغو صحی گواهی‌ها، مانند OCSP ، باید فعال و به‌درستی پیکربندی شده باشد.	3
12.1.5	Encrypted Client Hello (ECH) باید در تنظیمات TLS برنامه فعال باشد تا از افشای متادیتای حساس، مانند Server Name Indication (SNI)، در طول فرآیند Handshake TLS جلوگیری شود.	3

V12.2 HTTPS Communication with External Facing Services

تمام ترافیک HTTP به سمت سرویس‌های برون‌سازمانی (external-facing) که برنامه ارائه می‌دهد، به‌صورت رمزنگاری‌شده و با استفاده از گواهی‌های عمومیِ مورد اعتماد ارسال می‌شود.

#	Description	Level
12.2.1	برای تمام ارتباطات بین کلاینت و سرویس‌های مبتنی بر HTTP که در معرض دسترسی خارجی هستند، از TLS باید استفاده شود و هیچ گونه بازگشت (fallback) به ارتباطات ناامن یا بدون رمزنگاری انجام نگیرد.	1
12.2.2	سرویس‌هایی که در معرض دسترسی خارجی قرار دارند از گواهی‌های TLS عمومی و مورد اعتماد استفاده می‌کنند.	1

V12.3 General Service to Service Communication Security

ارتباطات سرور (چه داخلی و چه خارجی) تنها شامل HTTP نمی‌باشد. ارتباطات به و از سایر سیستم‌ها نیز باید امن باشند، ترجیحاً با استفاده از TLS.

#	Description	Level
12.3.1	یک پروتکل رمزنگاری‌شده مانند TLS باید برای تمامی ارتباطات ورودی و خروجی به و از برنامه استفاده شود، از جمله سیستم‌های مانیتورینگ، ابزارهای مدیریت، دسترسی از راه دور و SSH، پایگاه‌های داده، mainframes، سیستم‌های partner یا APIهای خارجی. سرور نباید به پروتکل‌های ناامن یا بدون رمزگذاری بازگردد.	2
12.3.2	کلاینت‌های TLS باید قبل از برقراری ارتباط با سرور، TLS گواهی‌های دریافتی را اعتبارسنجی کنند.	2
12.3.3	TLS یا دیگر مکانیزم‌های مناسب رمزگذاری انتقال باید برای تمامی ارتباطات بین سرویس‌های داخلی مبتنی بر HTTP درون برنامه استفاده شود و ارتباط به حالت‌های ناامن یا بدون رمزگذاری بازنگردد.	2

#	Description	Level
12.3.4	ارتباطات TLS بین سرویس‌های داخلی باید از گواهی‌های معتبر استفاده کنند. در صورتی که از گواهی‌های داخلی تولیدشده یا Self-Signed استفاده شود، سرویس مصرف‌کننده باید به گونه‌ای پیکربندی شود که تنها به هایCA داخلی مشخص و گواهی‌های Self-Signed خاص اعتماد کند.	2
12.3.5	سرویس‌هایی که درون یک سیستم با یکدیگر ارتباط دارند (ارتباطات درون‌سرویسی)، باید از احراز هویت قوی استفاده کنند تا هر نقطه انتهایی (endpoint) تأیید شود. روش‌های احراز هویت قوی، مانند احراز هویت کلاینت TLS، باید به‌کار گرفته شوند تا هویت تضمین شود (با استفاده از زیرساخت کلید عمومی (PKI) و مکانیزم‌هایی که در برابر حملات تکرار (replay attacks) مقاوم هستند). برای معماری‌های میکروسرویس، استفاده از سرویس مش برای ساده‌سازی مدیریت گواهی‌ها و افزایش امنیت در نظر گرفته شود	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- OWASP - Transport Layer Security Cheat Sheet
- Mozilla's Server Side TLS configuration guide
- Mozilla's tool to generate known good TLS configurations.
- O-Saft - OWASP Project to validate TLS configuration

V13 Configuration

هدف

پیکربندی پیش‌فرض برنامه برای استفاده در اینترنت باید امن باشد.

این فصل راهنمایی‌های در مورد پیکربندی‌های مختلف لازم برای دستیابی به این هدف را ارائه می‌دهد، از جمله پیکربندی‌های که در طول توسعه، ساخت و استقرار اعمال می‌شوند.

موضوعات پوشش داده‌شده شامل جلوگیری از نشت داده‌ها، مدیریت امن ارتباط بین اجزا و حفاظت از secrets است.

V13.1 Configuration Documentation

این بخش الزامات مستندسازی را برای نحوه‌ی ارتباط برنامه با سرویس‌های داخلی و خارجی، و همچنین تکنیک‌های برای جلوگیری از کاهش دسترسی به دلیل عدم دسترسی به سرویس ارائه می‌دهد. همچنین به مستندسازی مرتبط با secrets نیز پرداخته می‌شود.

#	Description	Level
13.1.1	تمامی نیازهای ارتباطی برنامه باید به‌طور کامل مستندسازی شده باشند. این مستندسازی شامل سرویس‌های خارجی که برنامه به آن‌ها وابسته است و همچنین مواردی که کاربر نهایی جهت اتصال به سرویس خارجی مشخص می‌کند باید باشد.	2
13.1.2	برای هر سرویسی که برنامه استفاده می‌کند، مستندسازی باید شامل حداکثر تعداد ارتباطات همزمان (مثلاً محدودیت‌های Connection Pool و نحوه‌ی رفتار برنامه هنگام رسیدن به این حد باشد، از جمله هر مکانیزم بازیابی یا جایگزینی، تا از بروز شرایط Denial of Service جلوگیری شود.	3
13.1.3	مستندات برنامه باید استراتژی‌های مدیریت منابع را برای هر سیستم یا سرویس خارجی که استفاده می‌کند مشخص کرده باشد (مثلاً پایگاه‌های داده، هندل‌های فایل، رشته‌ها، اتصالات HTTP). این مستندسازی باید شامل این موارد باشد: رویه‌های آزادسازی منابع، تنظیمات تایم‌اوت، مدیریت خطاها، جایی که منطق Retry اعمال می‌شود، شامل محدودیت تعداد Retry، تأخیرها و الگوریتم‌های Back-off. برای عملیات همزمان درخواست-پاسخ HTTP (Synchronous)، مستندات باید تایم‌اوت کوتاه را الزام کند و یا Retry را غیرفعال کند یا به‌طور سخت‌گیرانه محدود کند تا از تاخیرهای زنجیره‌ای و هدررفت منابع جلوگیری شود.	3
13.1.4	مستندسازی برنامه باید شامل هایsecret حیاتی برای امنیت برنامه و همچنین برنامه‌ای برای چرخش آن‌ها باشد(بر اساس مدل تهدید سازمان و نیازهای کسب‌وکار).	3

V13.2 Backend Communication Configuration

برنامه‌ها با چندین سرویس از جمله APIه، پایگاه‌های داده یا سایر اجزا تعامل دارند. این سرویس-ها ممکن است نسبت به برنامه اجزای داخلی محسوب شوند اما در مکانیزم‌های کنترل دسترسی استانداردبرنامه گنجانده نشده باشند، یا کاملاً خارجی باشند. در هر صورت، لازم است برنامه برای تعامل امن با این اجزا پیکربندی شود و در صورت نیاز، این پیکربندی نیز محافظت گردد.

توجه: فصل "Secure Communication" راهنمایی‌های برای رمزگذاری در حین انتقال را ارائه می‌دهد.

#	Description	Level
13.2.1	رت‌باطات بی‌ن اجزای ب ک‌اند برنامه ک‌ه از م ک‌انی‌زم اس‌ت‌اندارد نش‌س‌ت ک‌اربران برنامه پ‌ش‌ت‌ی‌بانی نم‌ی ک‌ن‌ند، از ج‌م‌له ،ه‌APIا م‌ی‌انافزاره‌ا و ل‌ای‌ه‌ه‌ای داده، ب‌ای‌د اح‌راز ه‌وی‌ت ش‌ده ب‌اش‌ند. اح‌راز ه‌وی‌ت ب‌ای‌د ب‌ا اس‌ت‌ف‌اده از ح‌س‌اب‌ه‌ای س‌روی‌س ج‌داگ‌انه، ت‌وک‌ن‌ه‌ای ک‌وت‌اه‌م‌دت ی‌ا اح‌راز ه‌وی‌ت م‌ب‌ت‌نی ب‌ر گ‌واه‌ی انج‌ام ش‌ود و ن‌ب‌ای‌د از اع‌ت‌ب‌ارنام‌ه‌ه‌ای ث‌اب‌ت م‌انند رم‌ز ع‌ب‌ور، ک‌ل‌ی‌ده‌ای API ی‌ا ح‌س‌اب‌ه‌ای م‌ش‌ت‌رک ب‌ا دس‌ت‌رس‌ی‌ه‌ای س‌طح ب‌الا اس‌ت‌ف‌اده ش‌ود.	2
13.2.2	ارت‌باطات بی‌ن اجزای ب ک‌اند، از ج‌م‌له س‌روی‌س‌ه‌ای م‌ح‌لی ی‌ا س‌ی‌س‌ت‌م‌ع‌ام‌ل، ،ه‌APIا م‌ی‌انافزاره‌ا و ل‌ای‌ه‌ه‌ای داده، ب‌ا ح‌س‌اب‌ه‌ای‌ی ب‌ای‌د انج‌ام ش‌ود ک‌ه ک‌م‌ت‌ری‌ن س‌طح دس‌ت‌رس‌ی لازم را دارند.	2
13.2.3	اگ‌ر اع‌ت‌ب‌ارنام‌ه‌ای ب‌رای اح‌راز ه‌وی‌ت س‌روی‌س اس‌ت‌ف‌اده م‌ی‌ش‌ود، اع‌ت‌ب‌ارنام‌ه‌ای ک‌ه ت‌وس‌ط م‌ص‌رف ک‌ن‌نده اس‌ت‌ف‌اده م‌ی‌ش‌ود، اع‌ت‌ب‌ارنام‌ه پ‌ی‌ش‌ف‌رض ن‌ب‌اش‌د (م‌ث‌لاً root/root ی‌ا admin/admin).	2
13.2.4	ی‌ک allowlist ب‌رای ت‌ع‌ی‌ی‌ن م‌ن‌اب‌ع ی‌ا س‌ی‌س‌ت‌م‌ه‌ای خ‌ارج‌ی ک‌ه ب‌رنام‌ه م‌ج‌از ب‌ه ارت‌ب‌اط ب‌ا آن‌ه‌ا اس‌ت ب‌ای‌د وج‌ود داش‌ت‌ه ب‌اش‌د (م‌ث‌لاً ب‌رای درخ‌واس‌ت‌ه‌ای خ‌روج‌ی، ب‌ارگ‌ذاری داده ی‌ا دس‌ت‌رس‌ی ب‌ه ف‌ای‌ل). ای‌ن allowlist م‌ی‌ت‌وان‌د در ل‌ای‌ه ب‌رنام‌ه، وب‌س‌رور، ف‌ای‌روال ی‌ا ت‌رک‌ی‌ب‌ی از ل‌ای‌ه‌ه‌ای م‌خ‌ت‌ل‌ف پ‌ی‌اده‌س‌ازی ش‌ود.	2
13.2.5	وب‌س‌رور ی‌ا س‌رور ب‌رنام‌ه ب‌ا ی‌ک allowlist از م‌ن‌اب‌ع ی‌ا س‌ی‌س‌ت‌م‌ه‌ای‌ی ک‌ه س‌رور م‌ی‌ت‌وان‌د ب‌ه آن‌ه‌ا درخ‌واس‌ت ارس‌ال ک‌ن‌د ی‌ا داده‌ه‌ا و ف‌ای‌ل‌ه‌ا را ب‌ارگ‌ذاری ک‌ن‌د پ‌ی‌ک‌ربن‌دی ش‌ده ب‌اش‌د.	2
13.2.6	در م‌واق‌ع‌ی ک‌ه ب‌رنام‌ه ب‌ه س‌روی‌س‌ه‌ای ج‌داگ‌انه م‌ت‌ص‌ل م‌ی‌ش‌ود، پ‌ی‌ک‌ربن‌دی م‌س‌ت‌ن‌دس‌ازی‌ش‌ده ب‌رای ه‌ر ات‌ص‌ال رع‌ای‌ت ش‌ود، از ج‌م‌له ح‌داک‌ث‌ر ارت‌ب‌اطات ه‌م‌زمان، رف‌ت‌ار ه‌ن‌گ‌ام رس‌ی‌دن ب‌ه ح‌داک‌ث‌ر ارت‌ب‌اط م‌ج‌از، زمان‌ه‌ای Timeout ات‌ص‌ال و اس‌ت‌رات‌ژی‌ه‌ای Retry.	3

V13.3 Secret Management

م‌دی‌ری‌ت secret ی‌ک وظی‌ف‌ه پ‌ی‌ک‌ربن‌دی ح‌ی‌ات‌ی اس‌ت ت‌ا از ح‌ف‌اظت داده‌ه‌ای م‌ورد اس‌ت‌ف‌اده در ب‌رنام‌ه اط‌م‌ی‌نان ح‌اص‌ل ش‌ود. ال‌زام‌ات خ‌اص م‌رب‌وط ب‌ه رم‌زن‌گ‌اری در ف‌ص‌ل "Cryptography" ق‌اب‌ل دس‌ت‌رس‌ی اس‌ت، ام‌ا ای‌ن ب‌خ‌ش ب‌ر م‌دی‌ری‌ت و ن‌ح‌وه‌ی ب‌رخ‌ورد ب‌ا secret ت‌م‌رک‌ز دارد.

#	Description	Level
13.3.1	ی ک راه ک ار م دی ری ت ،secret م انن د Vault ،Key ب رای ای ج اد ام ن، ذخ ی ره، ک ن ت رل دس ت رس ی و از ب ی ن ب ردن secret ب ک ان د اس ت ف اده ش ود. ای ن secret م ی ت وان د ش ام ل رمزعب ورها، داده ای رمز ن گ اری، اط لاع ات ح س اس م رب وط ب ه ات ص ال ب ه پ ای گ اه داده و س ی س ت م ه ای ش خ ص ث الث، ک ل ی ده ا و ه ایseed ت وک ن ه ای زمان م ح ور، س ای ر ه ایsecret داخ ل ی و ک ل ی ده ای API ب اش د. Secret ن ب ای د در ک د م ن ب ع ب رنام ه ی ا در build artifacts ق رار گ ی رد. ب رای ب رنام ه ه ای ،L3 ای ن ب ای د ش ام ل راه ک ار ی م ب ت ن ی ب ر س خ ت اف زار م انن د HSM ب اش د.	2
13.3.2	دس ت رس ی ب ه دارای ی ه ای secret ب ر اس اس اص ل ح داق ل دس ت رس ی (Least Privilege) رع ای ت ش ود.	2
13.3.3	ت م ام ع م ل ی ات رمزن گ اری ب ای د ب ا اس ت ف اده از ی ک م اژول ام ن ی ت ی ای زوله (م انن د Vault ی ا Hardware Security Module) انج ام ش ود ت ا م واد ک ل ی دی ب ه ط ور ام ن م دی ری ت و م ح اف ظ ت ش ده و از اف ش ای خ ارج از م اژول ام ن ی ت ی ج لوگ ی ری ش ود.	3
13.3.4	ه اsecret ب ر اس اس م س ت ن دس ازی ب رنام ه ط وری پ ی ک ربن دی ش ون د ک ه م ن ق ض ی ش ده و چ رخ ش داده ش ون د.	3

V13.4 Unintended Information Leakage

پ ی ک ربن دی ه ای production ب ای د ب ه گ ون ه ای ام ن ش ون د ک ه از اف ش ای داده ه ای غ ی رض رور ی ج لوگ ی ری ش ود. ب س ی اری از ای ن م س ائ ل ب ه ن درت ب ه ع ن وان ری س ک ه ای م ه م ارزی اب ی م ی ش ون د، ام ا اغ ل ب ب ا س ای ر آس ی ب پ ذی ری ه ا زنج ی ره ای م ی ش ون د. اگ ر ای ن م س ائ ل ب ه ص ورت پ ی ش ف رض وج ود ن داش ت ه ب اش ن د، س ط ح دش واری ح م ل ه ب ه ب رنام ه اف زای ش م ی اب د.

ب ه ع ن وان م ث ال، م خ ف ی ک ردن ن س خ ه ی اج زای س رور ب اع ث ح ذف ن ی از ب ه ب ه روزرس ان ی ت م ام اج زا ن م ی ش ود، و غ ی رف ع ال ک ردن ن م ای ش ف ه رس ت پ وش ه ا ن ی از ب ه اس ت ف اده از ک ن ت رل ه ای دس ت رس ی ی ا ن گ ه داری ف ای ل ه ا دور از پ وش ه ع م وم ی را از ب ی ن ن م ی ب رد، ام ا س ط ح دش واری ح م ل ه را اف زای ش م ی ده د.

#	Description	Level
13.4.1	ب رنام ه ب ای د ب ه گ ون ه ای م س ت ق ر ش ود ک ه ه ی چ گ ون ه م ت ادی ت ای ک ن ت رل ن س خ ه، از ج م ل ه پ وش ه ای git. ی ا ،svn. در دس ت رس ن ب اش د ی ا ب ه ش ک ل ی ب اش د ک ه ای ن پ وش ه ا ن ه از ب ی رون و ن ه ت وس ط خ ود ب رنام ه ق اب ل دس ت رس ی ن ب اش ن د.	1
13.4.2	ح ال ت ه ای Debug ب رای ت م ام اج زا در م ح ی ط ه ای production ب ای د غ ی رف ع ال ش ده ب اش ن د ت ا از اف ش ای ق اب ل ی ت ه ای دی ب اگ و ن ش ت اط لاع ات ج لوگ ی ری ش ود.	2
13.4.3	وب س رورها ف ه رس ت ب ن دی دای رک ت وری ه ا را ب ه ک ارب ر ن ه ای ی ن ب ای د ن م ای ش ده ن د م گ ر ای ن ک ه ب ه ط ور ص ری ح م ورد ن ظ ر ب اش د.	2
13.4.4	از م ت د HTTP TRACE در م ح ی ط ه ای production ن ب ای د پ ش ت ی ب ان ی ش ود ت ا از اح ت م ال ن ش ت اط لاع ات ج لوگ ی ری ش ود.	2

#	Description	Level
13.4.5	مس‌تندس‌ازی‌ها (مانند مس‌تندات های‌API داخ‌لی) و نق‌اط پ‌ای‌ش (Monitoring Endpoints) در دس‌ترس نب‌اش‌ند م‌گ‌ر‌ای‌نک‌ه به‌طور ص‌ری‌ح م‌ورد نظر ب‌اش‌د.	2
13.4.6	ت‌أی‌ی‌د ک‌نی‌د ک‌ه برنام‌ه اط‌لاع‌ات نس‌خه دق‌ی‌ق اج‌زای backend را اف‌ش‌ا ن‌ک‌ن‌د.	3
13.4.7	ل‌ای‌ه وب ب‌ای‌د به‌گ‌ون‌ه‌ای پ‌ی‌ک‌ربن‌دی ش‌ده ب‌اش‌د ک‌ه ف‌ق‌ط ف‌ای‌ل‌ه‌ای‌ی ب‌ا پ‌س‌ون‌ده‌ای م‌ش‌خ‌ص را س‌رو ک‌ن‌د ت‌ا از اف‌ش‌ای ت‌ص‌ادف‌ی اط‌لاع‌ات، پ‌ی‌ک‌ربن‌دی و ک‌د م‌نبع ج‌لوگ‌ی‌ری ش‌ود.	3

References

ب‌رای اط‌لاع‌ات ب‌ی‌ش‌تر، ه‌مچ‌ن‌ی‌ن به م‌نابع زی‌ر م‌راجع‌ه ک‌نی‌د:

- OWASP Web Security Testing Guide: Configuration and Deployment Management Testing

V14 Data Protection

ه‌دف

برنام‌ه‌ها نم‌ی‌ت‌وانند ت‌م‌ام ال‌گ‌وه‌ای اس‌ت‌فاده و رفت‌اره‌ای ک‌اربران را پ‌ی‌ش‌ب‌ی‌نی ک‌نند، بن‌اب‌ران‌ین ب‌ای‌د ک‌نت‌رل‌ه‌ای‌ی را پ‌ی‌اده‌س‌ازی ک‌نند ت‌ا از دس‌ت‌رس‌ی غ‌ی‌رم‌جاز به داده‌ه‌ای ح‌س‌اس روی دس‌ت‌گ‌اه‌ه‌ای ک‌اربر ج‌لوگ‌ی‌ری ش‌ود.

ای‌ن ف‌ص‌ل ش‌ام‌ل ال‌زام‌ات‌ی اس‌ت ک‌ه چ‌ه داده‌ه‌ای‌ی ب‌ای‌د م‌ح‌اف‌ظت ش‌ون‌د، چ‌گ‌ون‌ه ب‌ای‌د م‌ح‌اف‌ظت ش‌ون‌د، و ه‌مچ‌ن‌ی‌ن م‌ک‌ان‌ی‌زم‌ه‌ای خ‌اص‌ی ک‌ه ب‌ای‌د پ‌ی‌اده‌س‌ازی ش‌ون‌د ی‌ا اش‌ت‌ب‌اه‌ات‌ی ک‌ه ب‌ای‌د از آن‌ه‌ا اج‌ت‌ناب ش‌ود.

ی‌ک‌ی دی‌گ‌راز م‌واردی ک‌ه ب‌ای‌د در ح‌ف‌اظت از داده‌ه‌ا در ن‌ظر گ‌رفت، اس‌ت‌خ‌راج ع‌م‌ده، ت‌غ‌ی‌ی‌ر ی‌ا اس‌ت‌فاده ب‌ی‌ش از ح‌د از داده‌ه‌ا اس‌ت. ال‌زام‌ات ه‌ر س‌ی‌س‌ت‌م اح‌ت‌م‌الاً ب‌س‌ی‌ار م‌ت‌ف‌اوت خ‌واه‌ند ب‌ود، بن‌اب‌ران‌ین ت‌ع‌ی‌ی‌ن ای‌نک‌ه چ‌ه چ‌ی‌زی «غ‌ی‌رع‌ادی» م‌ح‌س‌وب م‌ی‌ش‌ود ب‌ای‌د ب‌ر اس‌اس م‌دل ت‌ه‌دی‌د و ری‌س‌ک ک‌س‌ب‌وک‌ار انج‌ام گ‌ی‌رد. از دی‌د گ‌اه ASVS، ش‌ن‌اس‌ای‌ی ای‌ن م‌س‌ائ‌ل در ف‌ص‌ل «Security Logging and Error Handling» ب‌ررس‌ی م‌ی‌ش‌ود و ت‌ع‌ی‌ی‌ن م‌ح‌دودی‌ت‌ه‌ا در ف‌ص‌ل «Validation and Business Logic» م‌طرح ش‌ده اس‌ت.

V14.1 Data Protection Documentation

ی‌ک‌ی از پ‌ی‌ش‌ن‌ی‌ازه‌ای اص‌ل‌ی ب‌رای ح‌ف‌اظت از داده‌ه‌ا، دس‌ت‌ه‌ب‌ن‌دی داده‌ه‌ای‌ی اس‌ت ک‌ه ب‌ای‌د به‌ع‌ن‌وان داده ح‌س‌اس در ن‌ظر گ‌رفت‌ه ش‌ون‌د. به‌اح‌ت‌م‌ال زی‌اد چ‌ن‌د س‌طح م‌ت‌ف‌اوت از ح‌س‌اس‌ی‌ت وج‌ود خ‌واه‌د داش‌ت و ب‌رای ه‌ر س‌طح، ک‌نت‌رل‌ه‌ای لازم ج‌ه‌ت ح‌ف‌اظت از داده‌ه‌ا م‌ت‌ف‌اوت خ‌واه‌د ب‌ود.

م‌ق‌ررات و ق‌وان‌ی‌ن م‌خ‌ت‌ل‌ف‌ی در ح‌وزه ح‌ری‌م خ‌ص‌وص‌ی وج‌ود دارند ک‌ه ب‌ر ن‌ح‌وه ذخ‌ی‌ره‌س‌ازی، اس‌ت‌فاده و ان‌ت‌ق‌ال داده‌ه‌ای ش‌خ‌ص‌ی ح‌س‌اس ت‌وس‌ط برنام‌ه‌ها ت‌أث‌ی‌ر م‌ی‌گ‌ذارند. ای‌ن ب‌خ‌ش دی‌گ‌ر ت‌ل‌اش‌ی ب‌رای

تکرارپذیر قوانین یا مقررات در زمینه حفاظت از داده یا حریم خصوصی نمی‌کند، بلکه تمرکز آن بر ملاحظات فنی کلیدی برای حفاظت از داده‌های حساس است. لطفاً قوانین و مقررات محلی را بررسی کرده و در صورت نیاز با یک متخصص حریم خصوصی یا وکیل واجد شرایط مشورت کنید.

#	Description	Level
14.1.1	تمامی داده‌های حساس ایجادشده و پردازش‌شده توسط برنامه باید شناسایی شده و بر اساس سطوح حفاظتی طبقه‌بندی شده‌اند. این موضوع شامل داده‌هایی می‌شود که صرفاً کُدگذاری شده‌اند و به‌راحتی قابل بازیابی هستند، مانند رشته‌های Base64 یا محتوای متنی (plaintext) داخل یک JWT. سطوح حفاظتی باید الزامات قوانین و استانداردهای حفاظت از داده و حریم خصوصی که برنامه ملزم به رعایت آن‌هاست را در نظر بگیرند.	2
14.1.2	برای تمامی سطوح حفاظت از داده‌های حساس، مجموعه‌ای مستند از الزامات حفاظتی تعریف شده باشد. این الزامات باید شامل (اما محدود به آن نباشد) موارد زیر باشند: الزامات مرتبط با رمزنگاری کلی، بررسی صحت و یکپارچگی، دوره‌های نگه‌داری داده، نحوه ثبت داده‌ها در لاگ، کنترل‌های دسترسی به داده‌های حساس در لاگ‌ها، رمزنگاری در سطح پایگاه داده، الزامات مربوط به حریم خصوصی و فناوری‌های تقویت‌کننده حریم خصوصی مورد استفاده، و سایر الزامات مرتبط با محرمانگی.	2

V14.2 General Data Protection

این بخش شامل مجموعه‌ای از الزامات عملی برای حفاظت از داده‌ها می‌باشد. بی‌شتر این الزامات به مسائل خاصی مانند نشت ناخواسته داده‌ها مربوط می‌شوند، اما یک الزام کلی نیز وجود دارد که بر پیاده‌سازی کنترل‌های حفاظتی بر اساس سطح حفاظت موردنیاز برای هر داده تأکید دارد.

#	Description	Level
14.2.1	داده‌های حساس باید فقط در بدنه پیام HTTP یا فیلدهای هدر به سرور ارسال شوند، و اینکه URL و query string شامل اطلاعات حساس (مانند API key یا session token) نباشند.	1
14.2.2	برنامه باید از cache شدن داده‌های حساس در اجزای سرور مانند Load Balancer و Application Cache جلوگیری کند، یا اطمینان حاصل شود که داده‌ها پس از استفاده به‌طور ایمن پاک‌سازی می‌شوند.	2
14.2.3	داده‌های حساس تعریف‌شده به طرف‌های غیرقابل اعتماد (مانند User Trackers) نباید ارسال شوند تا از جمع‌آوری ناخواسته داده‌ها خارج از کنترل برنامه جلوگیری شود.	2

#	Description	Level
14.2.4	کنترل‌های مرتبط با داده‌های حساس شامل رمزنگاری، بررسی صحت و یکپارچگی، نگه‌داری، نحوه ثبت داده‌ها در لاگ، کنترل‌های دسترسی به داده‌های حساس در لاگ‌ها، حریم خصوصی و فناوری‌های مربوط به حریم خصوصی، مطابق با آنچه در مستندات برای سطح حفاظت مشخص هر داده تعریف شده است، پیاده‌سازی شوند.	2
14.2.5	مکانیزم‌های cache باید طوری پیکربندی شده باشند که تنها پاسخ‌های با نوع محتوای مورد انتظار برای آن منبع cache شوند و شامل محتوای حساس یا پویا نباشند. وب‌سرور باید هنگام دسترسی به فایل ناموجود، پاسخ 404 یا 302 برگرداند و نه فایل معتبر دیگر، تا از حملات Web Cache Deception جلوگیری شود.	3
14.2.6	برنامه باید تنها حداقل داده‌های حساس مورد نیاز برای عملکرد برنامه را بازگرداند. برای مثال، تنها برخی از ارقام شماره کارت اعتباری بازگردانده شود و نه شماره کامل. در صورتی که داده کامل لازم باشد، باید در رابط کاربری masked شود مگر اینکه کاربر به‌طور مشخص آن را مشاهده کند.	3
14.2.7	اطلاعات حساس باید تحت طبقه‌بندی دوره‌ای نگه‌داری داده قرار گرفته باشند، به‌گونه‌ای که داده‌های قدیمی یا غیرضروری به‌طور خودکار، در یک برنامه زمان‌بندی‌شده مشخص، یا براساس نیاز حذف شوند.	3
14.2.8	اطلاعات حساس باید از متادیتای فایل‌های ارسال‌شده توسط کاربر حذف شود، مگر اینکه ذخیره‌سازی آن با رضایت کاربر انجام شود.	3

V14.3 Client-side Data Protection

این بخش شامل الزامات جلوگیری از نشت داده‌ها به روش‌های خاص در سمت کلاینت یا سمت User Agent برنامه است.

#	Description	Level
14.3.1	داده‌های احراز هویت‌شده از حافظه کلاینت، مانند DOM مرورگر، پس از خروج کلاینت یا Session پاک شوند. فیلد هدر Clear-Site-Data در پاسخ سرور می‌تواند در این زمینه مفید باشد، اما کلاینت باید بتواند داده‌ها را حتی زمانی که اتصال به سرور در دسترس نیست در زمان پایان Session (خروج از آن) پاک کند.	1
14.3.2	برنامه باید هدرهای پاسخ anti-cach HTTP، مناسب (مانند :Cache-Control no-store) را تنظیم کند تا داده‌های حساس در مرورگرها cache نشوند.	2
14.3.3	برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:	2

References

For more information, see also:

- Consider using the Security Headers website to check security and anti-caching header fields
- Documentation about anti-caching headers by Mozilla
- OWASP Secure Headers project
- OWASP Privacy Risks Project
- OWASP User Privacy Protection Cheat Sheet
- Australian Privacy Principle 11 - Security of personal information
- European Union General Data Protection Regulation (GDPR) overview
- European Union Data Protection Supervisor - Internet Privacy Engineering Network
- Information on the "Clear-Site-Data" header
- White paper on Web Cache Deception

V15 Secure Coding and Architecture

Control Objective

ب‌سی‌اری از الزامات ASVS ی‌ا به ی‌ک حوزه خاص امنی‌تی مانند احراز هویت ی‌ا مجوزده‌ی و ی‌ا به ی‌ک نوع خاص از کارک‌رده‌ای نرم‌افزار مانند ثبت رخدادها ی‌ا مدی‌ریت فای‌ل مرتبط می‌باشند.

ای‌ن ف‌ص‌ل الزامات ک‌لی امنی‌تی‌ک‌ه بای‌د هن‌گام طراحی و توسعه نرم‌افزارها در نظر گرفته شوند را ارائه می‌دهد. ای‌ن الزامات نه تن‌ها بر مع‌ماری تمی‌ز و کی‌فی‌ت کدتم‌رک‌زدارند، بل‌ک‌ه شامل شی‌وه‌ای خاص مع‌ماری و ک‌دنوی‌سی نی‌ز می‌شوند ک‌ه برای امنی‌ت نرم‌افزار ضروری هستند.

V15.1 Secure Coding and Architecture Documentation

ب‌سی‌اری از الزامات مربوط به ای‌جاد ی‌ک مع‌ماری امن و قابل دفاع، به مستندسازی شفاف تصم‌می‌مات‌ی بس‌ت‌گ‌ی دارند ک‌ه در مورد پی‌اده‌سازی ک‌نترل‌ه‌ای امنی‌تی مشخص و مؤلف‌ه‌ه‌ای اس‌تف‌اده‌شده در نرم‌افزار اتخاذ می‌شوند.

ای‌ن بخش، الزامات مس‌تندسازی را شرح می‌دهد، از جمل‌ه شناسای‌ی مؤلف‌ه‌ه‌ای‌ی ک‌ه دارای «ع‌م‌ل‌ک‌رد خطرناک» در نظر گرفته می‌شوند ی‌ا به عنوان «مؤلف‌ه‌ه‌ای پرری‌س‌ک» شناخته می‌شوند.

ی‌ک مؤلف‌ه با «ع‌م‌ل‌ک‌رد خطرناک» مم‌کن اس‌ت ی‌ک مؤلف‌ه توسعه‌ی‌اف‌ته داخل‌ی ی‌ا شخص ثالث باشد ک‌ه ع‌م‌ل‌ی‌ات‌ه‌ای‌ی مانند deserialization داده‌ای غی‌رقابل‌اع‌تماد، پردازش داده‌ای خام ی‌ا بای‌نری، اجرای ک‌دبه‌ص‌ورت پویا، ی‌ا دست‌ک‌اری مس‌تق‌یم حافظه را انجام می‌دهد. آسی‌ب‌پذی‌ری‌ها در ای‌ن نوع ع‌م‌ل‌ی‌ات‌ها ری‌س‌ک بالای‌ی برای به خطر انداختن برنامه و احتمالاً افشای زی‌رساخت زی‌ربنای‌ی آن ای‌جاد می‌کنند.

ی‌ک «مؤلف‌ه پرری‌س‌ک» ی‌ک ک‌تاب‌خانه شخص ثالث است (ی‌عنی توسعه‌ی‌اف‌ته داخل‌ی نی‌ست) ک‌ه ک‌نترل‌ه‌ای امنی‌تی در فرآین‌د توسعه ی‌ا ع‌م‌ل‌ک‌رد آن وجود ندارد ی‌ا ضع‌ی‌ف پی‌اده‌سازی شده‌اند.

نمونه‌ها شامل مؤلفه‌هایی هستند که به‌خوبی نگه‌داری نمی‌شوند، پشتیبانی نمی‌شوند، در مرحله end-of-life قرار دارند، یا سابقه‌ای از آسیب‌پذیری‌های جدی دارند.

این بخش همچنین بر اهمیت تعیین بازه‌های زمانی مناسب برای رسیدگی به آسیب‌پذیری‌ها در مؤلفه‌های شخص ثالث تأکید می‌کند.

#	Description	Level
15.1.1	مستندات برنامه باید بازه‌های زمانی جهت برطرف سازی آسیب‌پذیری نسخه‌های مؤلفه شخص ثالث (بر اساس ریسک) و همچنین به‌روزرسانی کتابخانه‌ها را به طور کلی تعریف کرده باشد تا ریسک ناشی از این مؤلفه‌ها به حداقل برسد.	1
15.1.2	یک کاتالوگ دارایی مانند Software Bill of Materials (SBOM) باید برای تمامی کتابخانه‌های شخص ثالث مورد استفاده نگه‌داری شود. شامل تاییدیه این که مؤلفه‌ها از مخازن از پیش تعیین‌شده، قابل اعتماد و به‌طور مستمر نگه‌داری شده تهیه شده‌اند.	2
15.1.3	مستندات برنامه باید شامل عملکردهایی که زمان‌بر یا پرمصرف(منابع سیستمی) هستند نیز باشد. این مورد باید شامل روش‌هایی برای جلوگیری از دست‌رفتن دسترس‌پذیری به دلیل استفاده بیش از حد از این عملکردها و همچنین جلوگیری از وضعیتی باشد که پردازش یک پاسخ طولانی‌تر از زمان انتظار مصرف‌کننده شود. تدابیر احتمالی می‌توانند شامل پردازش غیرهمزمان، استفاده از صف‌ها و محدود کردن فرآیندهای موازی برای هر کاربر و هر برنامه باشد.	2
15.1.4	مستندات برنامه باید کتابخانه‌های شخص ثالثی را که به‌عنوان «کامپوننت‌های پرخطر» شناخته می‌شوند، مشخص کرده باشد.	3
15.1.5	مستندات برنامه باید بخش‌هایی از برنامه را که به‌عنوان «عملکرد خطرناک» استفاده می‌شود، مشخص کرده باشد.	3

V15.2 Security Architecture and Dependencies

این بخش شامل الزامات مربوط به مدیریت وابستگی‌ها و کامپوننت‌ها می‌باشد تا اجزای پرخطر، قدیمی یا ناامن به‌طور مؤثر شناسایی، کنترل و اداره شوند.

همچنین شامل استفاده از تکنیک‌های سطح عمارای مانند containerization encapsulation، sandboxing و ایزوله‌سازی شبکه می‌باشد تا تأثیر استفاده از «عملیات خطرناک» (Dangerous operations) یا «کامپوننت‌های پرخطر» (dangerous operations) (همانطورکه در بخش قبلی تعریف شد) کاهش یابد و از کاهش دسترسی به دلیل استفاده بیش از حد از عملکردهای پرمصرف منابع جلوگیری شود.

#	Description	Level
15.2.1	برنامه بایدتنها شامل کامپونتهایی باشد که چارچوبهای زمانی مستندشده برای بهروزرسانی و رفع آسیبپذیریها را نقض نکردهباشند.	1
15.2.2	برنامه باید بر اساس تصمیمات و استراتژیهای امنیتی مستندسازیشده، تدابیری برای جلوگیری از کاهش دسترسپذیری ناشی از عملکردهایی که زمانبر یا پرمصرف منابع هستند، پیادهسازی کرده باشد.	2
15.2.3	محیط production تنها شامل عملکردهایی میباشد که برای اجرای برنامه ضروری هستند و هیچ قابلیت اضافی مانند کدهای تست، نمونه کدها و عملکردهای توسعهای در دسترس نمیباشد.	2
15.2.4	کامپونتهای شخص ثالث و تمام وابستگیهای انتقالی آنها از مخزن مورد انتظار بایدریافت شوند، چه داخلی و چه منبع خارجی، و ریسک حملهی dependency confusion مدیریت شود.	3
15.2.5	برنامه بایدمحافظتهای اضافی را در بخشهایی از برنامه که بهعنوان «عملکرد خطرناک» مستند شدهاند یا از کتابخانههای شخص ثالثی که بهعنوان «کامپونتهای پرریسک» شناختهمیشوند، پیادهسازی میکنند. این میتواند شامل تکنیکهایی مانند sandboxing، encapsulation، containerization یا جداسازی در سطح شبکه باشد تا حملهکنندگان پس از نفوذ به یک بخش از برنامه، نتوانند به بخشهای دیگر برنامه منتقل شوند و زمان لازم برای پیشروی آنها افزایش یابد.	3

V15.3 Defensive Coding

این بخش انواع آسیبپذیریها را پوشش میدهد، از جمله prototype pollution juggling، type و سایر مواردی که ناشی استفاده از الگوهای ناامن برنامهنویسی در یک زبان خاص هستند. برخی از این آسیبپذیریها ممکن است برای همه زبانها مرتبط نباشند، در حالی که برخی دیگر راهحلهای خاص زبان دارند یا به نحوه مدیریت ویژگیهایی مانند پارامترهای HTTP توسط یک زبان یا فریمورک خاص مربوط میشوند. همچنین، این بخش به خطرعدم اعتبارسنجی)رمزنگاری شده(بهروزرسانیهای برنامه نیز میپردازد.

این بخش همچنین خطرات مرتبط با استفاده از Objects برای نمایش دادهها و پذیرش و بازگرداندن آنها از طریق هایAPI خارجی را مورد بررسی قرار میدهد. در این حالت، برنامه بایداطمینان حاصل کند که فیلدهای دادهای که نبایدقابل ویرایش باشند توسط ورودی کاربر تغییر نمیکنند mass) (assignment و API بهصورت انتخابی مشخص کند که کدام فیلدهای داده بازگردانده شوند. هر جایی که دسترسی به فیلدها به مجوزهای کاربر وابسته باشد، این موضوع بایدد در چارچوب الزامات کنترل دسترسی در سطح فیلدها در فصل Authorization در نظر گرفته شود.

#	Description	Level
15.3.1	برنامه بایـد تنها زیرمجموعه‌ای از فیلدهای مورد نیاز از یک data object را بازگرداند. به‌عنوان مثال، نبایـد کل data object بازگردانده شود، زیرا برخی فیلدهای منفرد نبایـد برای کاربران قابل دسترسی باشند.	1
15.3.2	درجایی که بکاند برنامه به هایURL خارجی فراخوانی می‌فرستد، طوری بایـد پیکربندی شده باشد که تنها در صورت نیاز و منظور مشخص، از تغییر مسیرها (redirects) پیروی کند.	2
15.3.3	برنامه بایـد دارای تدابیر مقابله‌ای برای محافظت در برابر حملات mass assignment باشد، به‌طوری که فیلدهای مجاز برای هر کنترلر و اکشن محدود شده باشند؛ به عنوان مثال، نبایـد بتوان مقداری برای فیلدی وارد یا به‌روزرسانی کرد که قرار نبوده بخشی از آن اکشن باشد.	2
15.3.4	تمام اجزای proxying و middleware بایـد آدرس IP اصلی کاربر را به‌درستی منتقل کنند، با استفاده از فیلدهای داده معتبر که توسط کاربر نهایی قابل دستکاری نباشند، و برنامه و وبسرور از این مقدار صحیح برای ثبت لاگ و تصمیم‌گیری‌های امنیتی مانند rate limiting استفاده کنند، با در نظر گرفتن اینکه حتی آدرس IP اصلی ممکن است به دلیل هایIP پویا، هVPN یا فایروال‌های سازمانی قابل اعتماد نباشد.	2
15.3.5	برنامه بایـد به‌طور صریح اطمینان حاصل کند که variables از نوع صحیح هستند و عملیات‌های مقایسه و comparator به‌صورت دقیق (strict) انجام می‌شوند. این اقدام برای جلوگیری از آسیب‌پذیری‌های type juggling یا type confusion است که ممکن است به دلیل فرضیات نادرست برنامه درباره نوع یک متغیر ایجاد شود.	2
15.3.6	کدJavaScript بایـد به‌گونه‌ای نوشته شده باشد که از prototype pollution جلوگیری کند، برای مثال با استفاده از Set() یا Map() به جای object literals.	2
15.3.7	برنامه بایـد دارای تدابیر محافظتی در برابر HTTP parameter pollution باشد، به‌ویژه اگر چارچوب برنامه تفاوتی بین منابع پارامتره‌ای درخواست (رشته‌های query، پارامتره‌ای body، کوکی‌ها یا فیلدهای هدر) قائل نشود.	2

V15.4 Safe Concurrency

مسائل همزمانی مانند race condition، آسیب‌پذیری‌های time-of-check to time-of-use (TOCTOU)، deadlock، livelock، thread starvation و همگام‌سازی نادرست می‌توانند منجر به رفتار غیرقابل پیش‌بینی و خطرات امنیتی شوند. این بخش شامل تکنیک‌ها و استراتژی‌های مختلفی است که به کاهش این ریسک‌ها کمک می‌کنند.

#	Description	Level
15.4.1	shared objects در ک دmulti-threaded (م انن د ،هcache ف ای ل ه ا، ی ا اش ی ائ ی ک ه ت وس ط multiple threads دس ت رس ی پ ی دا م ی ک ن ن د) ب ای د ب ه ص ورت ای م ن م ورد اس ت ف اده ق رار ب گ ی رن د، ب ا اس ت ف اده از ن وع ه ای thread-safe و م ک ان ی زم ه ای ه م گ ام س ازی م انن د lock ی ا semaphore ت ا از race condition و ت خ ری ب داده ج ل وگ ی ری ش ود.	3
15.4.2	ب ررس ی وض ع ی ت ی ک ،resource م انن د وج ود آن ی ا دس ت رس ی ه ا،و اق دام ات ی ک ه ب ه ای ن وض ع ی ت واب س ت ه ان د، ب ه ص ورت ی ک ع م ل ی ات واح د انج ام ش ون د ت ا از race condition ن وع (TOCTOU) time-of-check to time-of-use ج ل وگ ی ری ش ود. ب رای م ث ال، ب ررس ی ای ن ک ه ی ک ف ای ل وج ود دارد ق ب ل از ب از ک ردن آن، ی ا ت أ ی ی د دس ت رس ی ی ک ک ارب ر ق ب ل از اع ط ای آن.	3
15.4.3	ب ای د از ق ف ل ه ا (Locks) ب ه ط ور ی ک ن واخ ت اس ت ف اده ش ود ت ا از گ ی ر ک ردن (Threads) ج ل وگ ی ری ش ود، چ ه ب ا ان ت ظ ار ب رای ی ک ی گ ر و چ ه ب ا ت ل اش ب ی پ ای ان، و ه م چ ن ی ن م ن ط ق ق ف ل گ ذاری ب ای د در ه م ان ک ی ب اق ی ب م ان د ک ه م س ئ ول م دی ری ت م ن ب ع اس ت؛ ای ن ک ار ت ض م ی ن م ی ک ن د ک ه ق ف ل ه ا ن م ی ت وان ن د ب ه ص ورت ن اخ واس ت ه ی ا خ راب ک ارانه ت وس ط ک ل اس ه ا ی ا ک ه ای ب ی رون ی ت غ ی ی ر داده ش ون د.	3
15.4.4	ت أ ی ی د ک ن ی د ک ه س ی اس ت ه ای ت خ ص ی ص resource از thread starvation ج ل وگ ی ری م ی ک ن ن د، ب ا اط م ی ن ان از دس ت رس ی ع ادلان ه ب ه م ن اب ع، م انن د اس ت ف اده از thread pools و اج ازه دادن ب ه threads ب ا اول وی ت پ ای ی ن ت ر ت ا در ب ازه زم ان ی م ن ط ق ی پ ی ش رف ت ک ن ن د.	3

References

ب رای اط لاع ات ب ی ش ت ر، ه م چ ن ی ن ب ه م ن اب ع زی ر م راج ع ه ک ن ی د:

- OWASP Prototype Pollution Prevention Cheat Sheet
- OWASP Mass Assignment Prevention Cheat Sheet
- OWASP CycloneDX Bill of Materials Specification
- OWASP Web Security Testing Guide: Testing for HTTP Parameter Pollution

V16 Security Logging and Error Handling

ه دف

ل اگ ه ای ام ن ی ت ی ب ا ل اگ ه ای خ ط ا ی ا ل اگ ه ای ع م ل ک رد م ت ف اوت ه س ت ن د و ب رای ث ب ت روی داده ای م رت ب ط ب ا ام ن ی ت اس ت ف اده م ی ش ون د؛ م انن د ت ص م ی م ه ای م رب وط ب ه اح راز ه وی ت، ک ن ت رل دس ت رس ی، و ت ل اش ه ا ب رای دور زدن ک ن ت رل ه ای ام ن ی ت ی (م انن د اع ت ب ارس ن ج ی ورودی ی ا اع ت ب ارس ن ج ی م ن ط ق ت ج اری).ه دف

آن‌ها پشتیبانی از تشخیص، پاسخ و بررسی است، از طریق ارائه داده‌های ساختاریافته و با کیفیت بالا برای ابزارهای تحلیلی مانند SIEM.

لاگ‌ها نباید شامل داده‌های شخصی حساس باشند مگر در مواردی که قانون الزام کند. هر داده‌ای که لاگ می‌شود باید به‌عنوان یک دارایی بسیار باارزش محافظت شود. لاگ‌برداری نباید باعث به خطر افتادن حریم خصوصی یا امنیت سیستم شود. همچنین برنامه باید در شرایط خطا به‌صورت ایمن عمل کند و از افشای غیرضروری اطلاعات یا ایجاد اختلال جلوگیری کند.

برای راهنمایی دقیق در مورد پیاده‌سازی، به OWASP Cheat Sheets در بخش منابع مراجعه کنید.

V16.1 Security Logging Documentation

این بخش تضمین می‌کند که یک دارایی (Inventory) شفاف و کامل از لاگ‌گیری در سراسر stack برنامه وجود داشته باشد. این کار برای نظارت امنیتی مؤثر، پاسخ به حوادث و انطباق (Compliance) ضروری است.

#	Description	Level
16.1.1	فهرستی باید وجود داشته باشد که در آن ثبت وقایع (Logging) انجام‌شده در هر لایه از stack فناوری برنامه، رویدادهای ثبت‌شده، فرمت‌های لاگ، محل ذخیره‌سازی، نحوه استفاده، نحوه کنترل دسترسی به آن‌ها و مدت زمان نگه‌داری لاگ‌ها مستند شده است.	2

V16.2 General Logging

این بخش الزامات لازم را ارائه می‌دهد تا اطمینان حاصل شود که لاگ‌های امنیتی به‌صورت یک‌پارچه ساختاردهی شده‌اند و متادیتا مورد انتظار را درخود دارند. هدف این است که لاگ‌ها قابل‌خواندن توسط ماشین باشند و بتوان آن‌ها را در سیستم‌ها و ابزارهای توزیع‌شده تحلیل کرد.

به‌طور طبیعی، رویدادهای امنیتی اغلب شامل داده‌های حساس هستند. اگر چنین داده‌ای بدون ملاحظه در لاگ‌ها ثبت شود، خود لاگ‌ها به‌عنوان داده‌ای طبقه‌بندی‌شده محسوب می‌شوند و بنابراین مشمول الزامات رمزنگاری، سیاست‌های سخت‌گیرانه‌تر نگه‌داری و همچنین احتمال افشا در طول ممیزی‌ها خواهند بود.

بنابراین، بسیار مهم است که فقط موارد ضروری در لاگ‌ها ثبت شوند و داده‌ای لاگ با همان دقت و مراقبتی که برای سایر دارایی‌های حساس به کار می‌رود، مورد توجه قرار گیرند.

الزامات زیر، نیازمندی‌های پایه‌ای مربوط به متادیتا در لاگ‌ها، مگاسازی، فرمت و کنترل را مشخص می‌کنند.

#	Description	Level
16.2.1	هر ورودی لاگ بایـد شامل متادیتای لازم (مانند زمان، مکان، چه کسی و چه چیزی) باشد تا امکان بررسی دقیق خط زمانی هنگام وقوع یک رویداد فراهم شود.	2
16.2.2	منابع زمانی برای تمام مؤلفه‌های لاگ با هم هماهنگ‌سازی شده‌اند و هر زمان (timestamp) در متادیتای رویدادهای امنیتی از UTC استفاده کند یا شامل یک اختلاف‌زمان (time zone offset) صریح باشد. استفاده از UTC توصیه می‌شود تا یک‌پارچگی در سراسر سیستم‌های توزیع‌شده تضمین شود و از بروز سردرگمی در زمان تغییر ساعت تابستانی جلوگیری گردد.	2
16.2.3	برنامه تنها لاگ‌ها را در فایل‌ها و سرویس‌های ذخیره یا ارسال می‌کند که در فهرست مستند لاگ‌ها ثبت شده‌اند.	2
16.2.4	لاگ‌ها بایـد توسط پردازشگر لاگی که در حال استفاده است خوانده و هم‌بسته شوند، و ترجیحاً با استفاده از یک فرمت لاگ مشترک این کار انجام گیرد.	2
16.2.5	هنگام لاگ‌گیری از داده‌های حساس، برنامه بایـد لاگ گیری را بر اساس سطح حفاظت داده اعمال کند. برای مثال، ممکن است لاگ‌کردن برخی داده‌ها مانند اطلاعات ورود یا جزئیات پرداخت مجاز نباشد. داده‌های دیگر، مانند session token، تنها در صورتی بایـد لاگ شوند که به‌صورت hash یا masked، چه به‌طور کامل یا جزئی، ثبت شوند.	2

V16.3 Security Events

این بخش الزامات مربوط به لاگ‌برداری از رویدادهای مرتبط با امنیت در داخل برنامه را تعریف می‌کند. ثبت این رویدادها برای شناسایی رفتارهای مشکوک، پشتیبانی از تحقیقات، و انجام تعهدات مربوط به انطباق است.

این بخش انواع رویدادهایی که بایـد لاگ شوند را مشخص می‌کند، اما سعی ندارد جزئیات کامل و جامعی ارائه دهد. هر برنامه دارای عوامل ریسک و زمینه عملیاتی منحصربه‌فرد خود است.

توجه داشته باشید ASVS در حالی شامل لاگ‌گیری رویدادهای امنیتی در محدوده خود می‌شود که Alerting و Correlation (مثلاً قوانین SIEM یا زیرساخت‌های مانیتورینگ) خارج از محدوده در نظر گرفته شده و توسط سیستم‌های عملیاتی و نظارتی مدیریت می‌شوند.

#	Description	Level
16.3.1	تمام عملیات احراز هویت بایـد لاگ‌گیری شوند، از جمله تلاش‌های موفق و ناموفق. همچنین بایـد متادیتای اضافی، مانند نوع احراز هویت یا عوامل استفاده‌شده، جمع‌آوری شود.	2

#	Description	Level
16.3.2	تلاش‌های ناموفق برای مجوزدهی (authorization) باید لاگ‌گیری شوند. برای برنامه‌های L3، این باید شامل لاگ‌گیری تمام تصمیمات مجوزدهی باشد، از جمله ثبت زمانی که داده‌های حساس دسترسی پیدا می‌کنند، بدون آنکه خود داده‌های حساس ثبت شوند.	2
16.3.3	برنامه باید رویدادهای امنیتی تعریف‌شده در مستندات (ابتدای بخش) را لاگ نماید و همچنین تلاش‌ها برای دور زدن کنترل‌های امنیتی، مانند اعتبارسنجی ورودی، منطق کسب‌وکار و ضدخودکارسازی، را نیز ثبت کند.	2
16.3.4	برنامه باید خطاهای غیرمنتظره و شکست‌های کنترل‌های امنیتی، مانند شکست‌های TLS در بک اند را لاگ کند.	2

V16.4 Log Protection

لاگ‌ها به عنوان آثار ارزشمند forensic محسوب می‌شوند و باید محافظت شوند. اگرلاگ‌ها به‌سادگی قابل تغییر یا حذف باشند، یکپارچگی خود را از دست می‌دهند و برای تحقیقات حادثه یا پرونده‌های حقوقی غیرقابل اعتماد می‌شوند. همچنین، لاگ‌ها ممکن است رفتار داخلی برنامه یا متادیتای حساس را فاش کنند و این موضوع آن‌ها را به هدف جذابی برای مهاجمان تبدیل می‌کند.

این بخش الزامات را برای اطمینان از حفاظت لاگ‌ها در برابر دسترسی غیرمجاز، دستکاری و افشا تعریف می‌کند و همچنین تضمین می‌کند که لاگ‌ها به‌صورت ایمن مدیریت و در سیستم‌های امن و ایزوله ذخیره شوند.

#	Description	Level
16.4.1	تمام مؤلفه‌های لاگ‌گیری باید داده‌ها را به‌طور مناسب رمزگذاری کنند تا از حملات log injection جلوگیری شود.	2
16.4.2	لاگ‌ها باید در برابر دسترسی غیرمجاز محافظت شده و امکان تغییر آن‌ها وجود نداشته باشد.	2
16.4.3	لاگ‌ها به‌صورت امن باید به یک سیستم مدیریت جداگانه برای تحلیل، تشخیص، هشدار و ارجاع مدیریت شوند. هدف این است که در صورت نفوذ به برنامه، لاگ‌ها به خطر نیفتند.	2

V16.5 Error Handling

این بخش الزامات لازم را برای اطمینان از این‌که برنامه‌ها به‌صورت امن و بدون افشای جزئیات حساس داخلی دچار خطا می‌شوند، تعریف می‌کند.

#	Description	Level
16.5.1	هنگام بروز خطاهای غیرمنتظره یا حساس به امنیت، پیامی عمومی به کاربر باید بازگردانده شود، به گونه‌ای که هیچ گونه اطلاعات حساس داخلی سیستم مانند stack trace، کوئری‌ها، کلیدها و توکن‌های محرمانه افشا نشود.	2
16.5.2	برنامه باید هنگام بروز خطا در دسترسی به منابع خارجی همچنان به‌صورت امن عمل نماید، برای مثال با استفاده از الگوهایی مانند circuit breaker یا کاهش تدریجی عملکرد (graceful degradation).	2
16.5.3	برنامه باید به‌صورت امن و کنترل‌شده دچار خطا شود، حتی هنگام وقوع استثنا، و از شرایط fail-open جلوگیری نماید؛ برای مثال، از پردازش تراکنش در صورت بروز خطا در منطق اعتبارسنجی جلوگیری شود.	2
16.5.4	یک «last resort» باید تعریف شده باشد که تمامی unhandled exception را مدیریت کند. این کار هم برای جلوگیری از از دست رفتن جزئیات خطا که باید در فایل‌های لاگ ثبت شوند و هم برای اطمینان از این است که خطا باعث از کار افتادن کل فرآیند برنامه و در نتیجه از دست رفتن دسترسی نشود، ضروری است.	3

توجه: برخی زبان‌ها (از جمله Swift و Go و همچنین بسیاری از زبان‌های تابعی بر اساس رویه‌های متداول طراحی) از exceptions یا last resort پشتیبانی نمی‌کنند. در این موارد، معماران و توسعه‌دهندگان باید از یک الگو، زبان، یا روش سازگار با چارچوب کاری استفاده کنند تا اطمینان حاصل شود که برنامه‌ها می‌توانند رویدادهای استثنائی، غیرمنتظره یا مرتبط با امنیت را به‌صورت امن مدیریت کنند.

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- OWASP Web Security Testing Guide: Testing for Error Handling
- OWASP Authentication Cheat Sheet section about error messages
- OWASP Logging Cheat Sheet
- OWASP Application Logging Vocabulary Cheat Sheet

V17 WebRTC

هدف

Web Real-Time Communication (WebRTC) امکان تبادل صوت، تصویر و داده به‌صورت لحظه‌ای در برنامه‌های مدرن را فراهم می‌کند. با افزایش پذیرش این فناوری، تأمین امنیت زیرساخت WebRTC

اه‌ی‌ت ح‌ی‌ات‌ی پ‌ی‌دا م‌ی ک‌ن‌د. ای‌ن ب‌خ‌ش ال‌زام‌ات ام‌ن‌ی‌ت‌ی را ب‌رای ذی‌ن‌ف‌ع‌ان‌ی ارائ‌ه م‌ی‌ده‌د ک‌ه س‌ی‌س‌ت‌م-ه‌ای WebRTC را ت‌وس‌ع‌ه، م‌ی‌زب‌ان‌ی ی‌ا ی ک‌پ‌ارچ‌ه م‌ی ک‌ن‌ن‌د

ص‌ن‌ع‌ت WebRTC را م‌ی‌ت‌وان ب‌ه‌ط‌ور ک‌ل‌ی ب‌ه س‌ه ب‌خ‌ش ت‌ق‌س‌ی‌م ک‌رد:

1. ت‌وس‌ع‌ه‌ده‌ن‌د‌گ‌ان م‌ح‌ص‌ول: ف‌روش‌ن‌د‌گ‌ان اخ‌ت‌ص‌اص‌ی و م‌ت‌ن‌ب‌از ک‌ه م‌ح‌ص‌ول‌ات و راه‌ح‌ل‌ه‌ای WebRTC را ای‌ج‌اد و ع‌رض‌ه م‌ی ک‌ن‌ن‌د. ت‌م‌رک‌ز آن‌ه‌ا ب‌ر ت‌وس‌ع‌ه ف‌ن‌اوری‌ه‌ای ق‌وی و ام‌ن WebRTC اس‌ت ک‌ه ت‌وس‌ط دی‌گ‌ران ق‌اب‌ل اس‌ت‌ف‌اده ب‌اش‌ن‌د.
2. ارائ‌ه‌ده‌ن‌د‌گ‌ان خ‌دم‌ات: س‌ازم‌ان‌ه‌ای‌ی ک‌ه از م‌ح‌ص‌ول‌ات ت‌وس‌ع‌ه‌ده‌ن‌د‌گ‌ان ی‌ا ارائ‌ه‌ده‌ن‌د‌گ‌ان CPaaS اس‌ت‌ف‌اده م‌ی ک‌ن‌ن‌د ی‌ا راه‌ک‌اره‌ای WebRTC خ‌ود را ت‌وس‌ع‌ه م‌ی‌ده‌ن‌د. آن‌ه‌ا ب‌رن‌ام‌ه‌ه‌ای‌ی را ب‌رای ک‌ن‌ف‌ران‌س آن‌ل‌ای‌ن، س‌ل‌ام‌ت، آم‌وزش ال‌ک‌ت‌رون‌ی‌ک‌ی و س‌ای‌ر ح‌وزه‌ه‌ای‌ی ک‌ه در آن‌ه‌ا ارت‌ب‌اط ب‌ل‌ادرن‌گ ح‌ی‌ات‌ی اس‌ت، ای‌ج‌اد و پ‌ی‌اده‌س‌ازی م‌ی ک‌ن‌ن‌د.
3. ارائ‌ه‌ده‌ن‌د‌گ‌ان خ‌دم‌ات: س‌ازم‌ان‌ه‌ای‌ی ک‌ه از م‌ح‌ص‌ول‌ات ت‌وس‌ع‌ه‌ده‌ن‌د‌گ‌ان ی‌ا ارائ‌ه‌ده‌ن‌د‌گ‌ان CPaaS اس‌ت‌ف‌اده م‌ی ک‌ن‌ن‌د ی‌ا خ‌ود راه‌ک‌اره‌ای WebRTC خ‌ود را ت‌وس‌ع‌ه م‌ی‌ده‌ن‌د. آن‌ه‌ا ب‌رن‌ام‌ه‌ه‌ای‌ی ب‌رای ک‌ن‌ف‌ران‌س‌ه‌ای آن‌ل‌ای‌ن، ح‌وزه س‌ل‌ام‌ت، آم‌وزش ال‌ک‌ت‌رون‌ی‌ک‌ی و س‌ای‌ر زم‌ی‌ن‌ه‌ه‌ای‌ی ک‌ه ارت‌ب‌اط ب‌ل‌ادرن‌گ ح‌ی‌ات‌ی اس‌ت، ای‌ج‌اد و پ‌ی‌اده‌س‌ازی م‌ی ک‌ن‌ن‌د.

ن‌ی‌ازم‌ن‌دی‌ه‌ای ام‌ن‌ی‌ت‌ی م‌ط‌رح ش‌ده در ای‌ن‌ج‌ا ع‌م‌دت‌اً ب‌ر ت‌وس‌ع‌ه‌ده‌ن‌د‌گ‌ان م‌ح‌ص‌ول، ارائ‌ه‌ده‌ن‌د‌گ‌ان CPaaS و ارائ‌ه‌ده‌ن‌د‌گ‌ان خ‌دم‌ات م‌ت‌م‌رک‌ز اس‌ت ک‌ه:

- از راه‌ح‌ل‌ه‌ای م‌ت‌ن‌ب‌از ب‌رای س‌اخ‌ت ب‌رن‌ام‌ه‌ه‌ای WebRTC خ‌ود اس‌ت‌ف‌اده م‌ی ک‌ن‌ن‌د.
- از م‌ح‌ص‌ول‌ات ت‌ج‌اری WebRTC ب‌ه‌ع‌ن‌وان ب‌خ‌ش‌ی از زی‌رس‌اخ‌ت خ‌ود اس‌ت‌ف‌اده م‌ی ک‌ن‌ن‌د.
- از راه‌ح‌ل‌ه‌ای WebRTC ک‌ه ب‌ه‌ص‌ورت داخ‌ل‌ی ت‌وس‌ع‌ه داده‌ان‌د اس‌ت‌ف‌اده م‌ی ک‌ن‌ن‌د ی‌ا اج‌زای م‌خ‌ت‌ل‌ف را در ی ک س‌روی‌س ی ک‌پ‌ارچ‌ه ت‌رک‌ی‌ب م‌ی ک‌ن‌ن‌د.

ق‌اب‌ل ت‌وج‌ه اس‌ت ک‌ه ای‌ن ال‌زام‌ات ام‌ن‌ی‌ت‌ی ش‌ام‌ل ت‌وس‌ع‌ه‌ده‌ن‌د‌گ‌ان‌ی ن‌م‌ی‌ش‌ود ک‌ه ص‌رف‌اً از ه‌ایSDK و ه‌ایAPI ارائ‌ه‌ش‌ده ت‌وس‌ط ارائ‌ه‌ده‌ن‌د‌گ‌ان CPaaS اس‌ت‌ف‌اده م‌ی ک‌ن‌ن‌د. ب‌رای چ‌ن‌ی‌ن ت‌وس‌ع‌ه‌ده‌ن‌د‌گ‌ان‌ی، م‌س‌ئ‌ول‌ی‌ت ب‌ی‌ش‌ت‌ر م‌س‌ائ‌ل ام‌ن‌ی‌ت‌ی زی‌رس‌اخ‌ت‌ی م‌ع‌م‌ولاً ب‌ر ع‌ه‌ده ارائ‌ه‌ده‌ن‌د‌گ‌ان CPaaS اس‌ت و اس‌ت‌ان‌دارد ام‌ن‌ی‌ت‌ی ع‌م‌وم‌ی م‌ان‌ن‌د ASVS م‌م‌ک‌ن اس‌ت ب‌ه‌ط‌ور ک‌ام‌ل ن‌ی‌ازه‌ای آن‌ه‌ا را پ‌وش‌ش ن‌ده‌د.

V17.1 TURN Server

ای‌ن ب‌خ‌ش ال‌زام‌ات ام‌ن‌ی‌ت‌ی را ب‌رای س‌ی‌س‌ت‌م‌ه‌ای‌ی ک‌ه س‌رور‌ه‌ای TURN (Traversal Using Relays around NAT) خ‌ود را راه‌ان‌دازی م‌ی ک‌ن‌ن‌د، ت‌ع‌ری‌ف م‌ی ک‌ن‌د. س‌رور‌ه‌ای TURN در ه‌دای‌ت Media در م‌ح‌ی‌ط‌ه‌ای ش‌ب‌ک‌ه-ای م‌ح‌دود ک‌م‌ک م‌ی ک‌ن‌ن‌د، ام‌ا در ص‌ورت پ‌ی‌ک‌ر‌ب‌ن‌دی ن‌ادرس‌ت م‌ی‌ت‌وان‌ن‌د خ‌ط‌رات‌ی ای‌ج‌اد ک‌ن‌ن‌د. ای‌ن ک‌ن‌ت‌رل‌ه‌ا ب‌ر ف‌ی‌ل‌ت‌ر ک‌ردن ام‌ن آدرس‌ه‌ا و ج‌ل‌وگ‌ی‌ری از ت‌م‌ام ش‌دن م‌ن‌اب‌ع س‌ی‌س‌ت‌م (resource exhaustion) ت‌م‌رک‌ز دارن‌د.

#	Description	Level
17.1.1	بای‌د تن‌ها به آدرس‌های IP دست‌رسی ده‌د که برای اه‌داف ویژه رزرو نش‌ده‌اند (م‌ثل‌اً ش‌بک‌ه‌های داخ‌لی، broadcast، loopback). توجه داشته باش‌ی‌د که این مورد هم برای آدرس‌های IPv4 و هم برای IPv6 صدق می‌ک‌ند.	2
17.1.2	سرویس Traversal Using Relays around NAT (TURN)، نبای‌د به resource exhaustion آسی‌ب‌پذیر باشد، حتی زمانی‌که کاربران قانونی تلاش کنند تع‌داد زیادی پورت روی سرور TURN باز کنند.	3

V17.2 Media

این الزامات فقط به سی‌ستم‌های‌ی اعمال می‌شود که سروره‌ای رسانه‌ای WebRTC خود را می‌زبانی می‌کنند، مانند Selective Forwarding Units (SFU)، Multipoint Control Units (MCU)، سروره‌ای Recording یا سروره‌ای Gateway. سروره‌ای رسانه(Media) وظی‌فه پردازش و توزیع جری‌انه‌ای رسانه‌ای را برع‌ه‌ده دارند و امنی‌ت آن‌ها برای محافظت از ارتباطات بی‌ن همتاها حیاتی است. حفاظت از جری‌انه‌ای رسانه‌ای در برنامه‌های WebRTC از اهمیت بالای‌ی برخوردار است تا از شنود، دست‌کاری و حملات محروم‌سازی از سرویس(denial-of-service) جلوگی‌ری شود که می‌توانند حریم خصوصی کاربران و کی‌فی‌ت ارتباط را به خطر بی‌ندازد.

به‌ویژه، لازم است تدابی‌ری برای محافظت در برابر حملات flood مانند rate limiting، اعتبارسنجی timestamps، استفاده از ساعت‌های همگام برای تطبی‌ق بازه‌های زمانی واقع‌ی و مدی‌ریت بافرها به‌منظور جلوگی‌ری از overflow و حفظ زمان‌بندی مناسب، اعمال شود. اگر بس‌ته‌های مربوط به یک جلسه رسانه‌ای خاص خی‌لی سریع برسند، بس‌ته‌های اضافی بای‌د حذف شوند. همچنی‌ن محافظت از سی‌ستم در برابر بس‌ته‌های خراب یا مخرب اهمی‌ت دارد که با اعمال اعتبارسنجی ورودی، مدی‌ریت ای‌من سرری‌ز عدد صحی‌ح، جلوگی‌ری از buffer overflows و استفاده از سای‌ر تک‌نی‌ک‌های مقاوم در برابر خطا، حاصل می‌شود.

سی‌ستم‌های‌ی که صرفاً بر ارتباط رسانه‌ای همتاب‌ه‌همتا بی‌ن مرورگره‌ای وب متکی هستند و سروره‌ای می‌انی رسانه‌ای در آن‌ها دخالتی ندارند، از این الزامات امنی‌تی خاص مرتبط با رسانه مستثنی هستند.

این بخش به استفاده از Datagram Transport Layer Security (DTLS) در زمی‌نه WebRTC اشاره دارد. الزامی مربوط به داشتن سی‌است مستند برای مدی‌ریت کلی‌دهای رمزنگاری در فصل «Cryptography» قابل یافت است. اطلاعات مربوط به روش‌های رمزنگاری تأی‌ی‌دشده را می‌توان یا در پی‌وست Cryptography استاندارد ASVS یا در اسنادی مانند NIST SP 800-52 Rev. 2 و BSI TR-02102-2 (نسخه 2025-01) پی‌دا کرد.

#	Description	Level
17.2.1	کل‌ید مربوط به گواهی Datagram Transport Layer Security (DTLS) بر اساس سیاست مستند مدیریت کلیدهای رمزنگاری مدیریت و محافظت می‌شود.	2
17.2.2	سرور Media طوری پیکربندی شده باشد که از مجموعه‌های رمزنگاری تاییدشده Datagram Transport Layer Security (DTLS) استفاده کند و از یک پروفایل محافظتی امن برای گسترش DTLS به منظور ایجاد کلیدها برای پروتکل Secure Real-time Transport Protocol (DTLS-SRTP) پشتیبانی نماید.	2
17.2.3	احراز هویت Secure Real-time Transport Protocol (SRTP) در سرور Media بررسی شود تا از حملات تزریق Real-time Transport Protocol (RTP) جلوگیری شده و این حملات باعث بروز شرایط Denial of Service یا وارد شدن محتوای صوتی یا تصویری ناخواسته به جریان‌های رسانه‌ای نشوند.	2
17.2.4	سرور Media باید قادر باشد پردازش ترافیک رسانه‌ای ورودی را حتی هنگام مواجهه با بسته‌های Secure Real-time Transport Protocol (SRTP) ناقص یا خراب ادامه دهد.	2
17.2.5	سرور Media باید قادر باشد پردازش ترافیک رسانه‌ای ورودی را حتی در هنگام دریافت سیل(flood) از بسته‌های Secure Real-time Transport Protocol (SRTP) از کاربران قانونی ادامه دهد.	3
17.2.6	سرور Media نسبت به آسیب‌پذیری "ClientHello Race Condition" در Datagram Transport Layer Security (DTLS) نباید آسیب‌پذیر نباشد، این کار می‌تواند با بررسی اینکه آیا سرور رسانه به‌صورت عمومی به عنوان آسیب‌پذیر شناخته شده است یا با انجام آزمون Race Condition انجام شود.	3
17.2.7	مکانیزم‌های ضبط صدا یا تصویر مرتبط با سرور رسانه در هنگام طوفان بسته‌ای (Flood) از پروتکل انتقال بلادرنگ امن (SRTP) توسط کاربران مجاز، باید قادر به ادامه پردازش ترافیک رسانه ورودی است.	3
17.2.8	گواهی‌نامه امنیت لایه انتقال دیتاگرام (DTLS) باید با ویژگی اثر انگشت (Fingerprint) پروتکل توصیف نشست (SDP) بررسی می‌شود و در صورت شکست در بررسی، جریان رسانه قطع می‌شود تا اصالت جریان رسانه تضمین گردد.	3

V17.3 Signaling

این بخش الزامات سیستم‌هایی را تعریف می‌کند که سروره‌ای سیگنالینگ WebRTC خود را اداره می‌کنند. سیگنالینگ وظیفه مهاجرت ارتباط peer-to-peer را دارد و باید در برابر حملاتی که می‌توانند بر ایجاد یا کنترل جلسات تأثیر بگذارند مقاوم باشد.

برای اطمینان از سیگنالینگ امن، سیستم‌ها باید ورودی‌های ناقص یا مخدوش را به‌صورت امن مدیریت کنند و حتی تحت بار زیاد نیز در دسترس باقی بمانند.

#	Description	Level
17.3.1	س‌رور س‌ی‌گ‌نال‌ی‌ن‌گ ب‌ای‌د ق‌ادر ب‌اش‌د ح‌ت‌ی در ج‌ری‌ان flood، attack پ‌ی‌ام‌ه‌ای س‌ی‌گ‌نال‌ی‌ن‌گ ورودی م‌ش‌روع را پ‌ردازش ک‌ن‌د. ای‌ن ب‌ای‌د ب‌ا اع‌م‌ال م‌ح‌دودی‌ت ن‌رخ (rate limiting) در س‌طح س‌ی‌گ‌نال‌ی‌ن‌گ م‌ح‌ق‌ق ش‌ود.	2
17.3.2	س‌رور س‌ی‌گ‌نال‌ی‌ن‌گ ب‌ای‌د ق‌ادر ب‌اش‌د پ‌ی‌ام‌ه‌ای س‌ی‌گ‌نال‌ی‌ن‌گ م‌ش‌روع را ح‌ت‌ی در م‌واج‌ه‌ه ب‌ا پ‌ی‌ام‌ه‌ای س‌ی‌گ‌نال‌ی‌ن‌گ ن‌اق‌ص ی‌ا م‌خ‌رب ک‌ه م‌ی‌ت‌وان‌ن‌د ب‌اع‌ث ای‌ج‌اد ش‌رای‌ط ع‌دم دس‌ت‌رس‌ی (Denial of Service) ش‌ون‌د، پ‌ردازش ک‌ن‌د. ای‌ن م‌ی‌ت‌وان‌د ش‌ام‌ل اع‌م‌ال اع‌ت‌ب‌ارس‌نج‌ی ورودی، م‌دی‌ری‌ت ای‌م‌ن س‌رری‌ز ع‌دد ص‌ح‌ی‌ح (integer overflow)، ج‌ل‌وگ‌ی‌ری از س‌رری‌ز ب‌اف‌ر (buffer overflow) و اس‌ت‌ف‌اده از دی‌گ‌ر ت‌ک‌ن‌ی‌ک‌ه‌ای م‌ق‌اوم م‌دی‌ری‌ت خ‌ط‌ا ب‌اش‌د.	2

References

ب‌رای اط‌لاع‌ات ب‌ی‌ش‌ت‌ر، ه‌م‌چ‌ن‌ی‌ن ب‌ه م‌ن‌اب‌ع زی‌ر م‌راج‌ع‌ه ک‌ن‌ی‌د:

- The WebRTC DTLS ClientHello DoS is best documented at Enable Security's blog post aimed at security professionals and the associated white paper aimed at WebRTC developers
- RFC 3550 - RTP: A Transport Protocol for Real-Time Applications
- RFC 3711 - The Secure Real-time Transport Protocol (SRTP)
- RFC 5764 - Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP))
- RFC 8825 - Overview: Real-Time Protocols for Browser-Based Applications
- RFC 8826 - Security Considerations for WebRTC
- RFC 8827 - WebRTC Security Architecture
- DTLS-SRTP Protection Profiles

Appendix A: Glossary

- **Absolute Maximum Session Lifetime** – ک‌ه ت‌وس‌ط NIST ب‌ا ع‌ن‌وان «م‌ه‌ل‌ت زم‌ان‌ی ک‌ل‌ی (Overall Timeout)» ن‌ی‌ز ش‌ن‌اخ‌ت‌ه م‌ی‌ش‌ود، ح‌داک‌ث‌ر م‌دت زم‌ان‌ی اس‌ت ک‌ه ی‌ک ن‌ش‌س‌ت پ‌س از اح‌راز ه‌وی‌ت م‌ی‌ت‌وان‌د ف‌ع‌ال ب‌اق‌ی ب‌م‌ان‌د، ص‌رف‌ن‌ظر از ه‌ر گ‌ون‌ه ت‌ع‌ام‌ل ک‌ارب‌ر. ای‌ن م‌ف‌ه‌وم ی‌ک‌ی از م‌ؤل‌ف‌ه‌ه‌ای ان‌ق‌ض‌ای ن‌ش‌س‌ت (Session Expiration) م‌ح‌س‌وب م‌ی‌ش‌ود.
- **Allowlist** ف‌ه‌رس‌ت‌ی از داده‌ه‌ا ی‌ا ع‌م‌ل‌ی‌ات م‌ج‌از؛ ب‌رای م‌ث‌ال، ف‌ه‌رس‌ت‌ی از ک‌ارا ک‌ت‌ره‌ای‌ی ک‌ه انج‌ام اع‌ت‌ب‌ارس‌نج‌ی ورودی (Input Validation) ب‌ر اس‌اس آن‌ه‌ا م‌ج‌از اس‌ت.
- **Anti-forgery token** م‌ک‌ان‌ی‌زم‌ی ک‌ه در آن ی‌ک ی‌ا چ‌ن‌د ت‌وک‌ن ه‌م‌راه ب‌ا درخ‌واس‌ت ارس‌ال ش‌ده و ت‌وس‌ط س‌رور ب‌رن‌ام‌ه اع‌ت‌ب‌ارس‌نج‌ی م‌ی‌ش‌ون‌د ت‌ا اط‌م‌ی‌ن‌ان ح‌اص‌ل ش‌ود درخ‌واس‌ت از ی‌ک ن‌ق‌ط‌ه ان‌ت‌ه‌ای‌ی (Endpoint) م‌ورد ان‌ت‌ظ‌ار ارس‌ال ش‌ده اس‌ت.
- **Application Security** ام‌ن‌ی‌ت در س‌طح ب‌رن‌ام‌ه ب‌ر ت‌ح‌ل‌ی‌ل م‌ؤل‌ف‌ه‌ه‌ای‌ی ت‌م‌رک‌ز دارد ک‌ه ل‌ای‌ه ک‌ارب‌ردی در م‌دل م‌رج‌ع ات‌ص‌ال س‌ی‌س‌ت‌م‌ه‌ای ب‌از (OSI) را ت‌ش‌ک‌ی‌ل م‌ی‌ده‌ن‌د، ن‌ه ب‌ر اج‌زای زی‌رب‌ن‌ای‌ی

مانند سیستم‌عامل یا شبکه‌ای متصل.

- **Application Security Verification** – ارزیابی فنی یک برنامه کاربردی بر اساس استانداردOWASP ASVS.
- **Application Security Verification Report** – گزارشی که نتایج کلی و تحلیل‌های پشتیبان تهیه‌شده توسط ارزیاب (Verifier) را برای یک برنامه کاربردی مشخص مستند می‌کند.
- **Authentication** – فرایند تأیید هویت ادعاشده کاربر یک برنامه کاربردی.
- **Automated Verification** – استفاده از ابزاره‌ای خودکار (شامل ابزاره‌ای تحلیل پویا، ابزاره‌ای تحلیل ایستا، یا هر دو) که با تکیه بر امضاهای آسیب‌پذیری برای شناسایی مشکلات به کار می‌روند.
- **Black box testing** – روشی از آزمون نرم‌افزار که عملکرد یک برنامه را بررسی می‌کند بدون آنکه به ساختارها یا عملکرد داخلی آن نگاه کند.
- **Common Weakness Enumeration** (CWE) – فهرستی که توسط جامعه توسعه‌دهندگان ایجاد شده و شامل نقاط ضعف رایج امنیت نرم‌افزار است. این فهرست به‌عنوان زبان مشترک، هم‌ای‌اری برای ابزاره‌ای امنیت نرم‌افزار، و پایه‌ای برای شناسایی، کاهش و پیشگیری از نقاط ضعف عمل می‌کند.
- **Component** – یک واحد مستقل از کد است که دارای رابطه‌ای مرتبط با دیسک و شبکه می‌باشد و با سایر مولفه‌ها ارتباط برقرار می‌کند.
- **Credential Service Provider** (CSP) – که به آن ارائه‌دهنده هویت (Identity Provider - IdP) نیز گفته می‌شود، منبعی از داده‌های کاربری است که می‌تواند به‌عنوان منبع احراز هویت توسط سایر برنامه‌ها استفاده شود.
- **Cross-Site Script Inclusion** (XSSI) - گونه‌ای از حمله اسکریپت‌نویسی بین‌سایتی (XSS) که در آن یک برنامه وب کدمخرب را از یک منبع خارجی دریافت کرده و آن کدرا به‌عنوان بخشی از محتوای خود وارد می‌کند.
- **Cross-Site Scripting** (XSS) – یک آسیب‌پذیری امنیتی معمول در برنامه‌های وب که امکان تزریق اسکریپت‌های سمت کاربر (Client-Side Scripts) به محتوا را فراهم می‌کند.
- **Cryptographic module** – سخت‌افزار، نرم‌افزار و/یا فریم‌ورکه الگوریتم‌های رمزنگاری را اجرا کرده و/یا کلیدهای رمزنگاری تولید می‌کند.
- **Cryptographically secure pseudo-random number generator** (CSPRNG) - یک تولیدکننده اعداد شبه‌تصادفی با ویژگی‌هایی که آن را مناسب برای استفاده در رمزنگاری می‌کند؛ که گاهی به آن تولیدکننده اعداد تصادفی رمزنگاری (Cryptographic Random Number Generator - CRNG) نیز گفته می‌شود.
- **Datagram Transport Layer Security** (DTLS) – یک پروتکل رمزنگاری که امنیت ارتباطی را بر روی اتصال شبکه فراهم می‌کند. این پروتکل بر اساس TLS ساخته شده اما برای محافظت از پروتکل‌های مبتنی بر دیتاگرام (معمولاً روی UDP تطبیق داده شده است. نسخه DTLS 1.3 در RFC 9147 تعریف شده است.
- **Datagram Transport Layer Security Extension to Establish Keys for the Secure Real-time Transport Protocol** (DTLS-SRTP) – مکانیزمی که از DTLS Handshake برای ایجاد مواد کلیدی (Key Material) جهت یک جلسه SRTP استفاده می‌کند. این افزونه در RFC 5764 تعریف شده است.
- **Design Verification** – ارزیابی فنی معماری امنیتی یک برنامه کاربردی.
- **Dynamic Application Security Testing** (DAST) – فناوری‌هایی که برای شناسایی شرایطی که نشان‌دهنده آسیب‌پذیری امنیتی در یک برنامه در حال اجرا هستند طراحی شده‌اند.

- **Dynamic Verification** – اس‌ت‌فاده از اب‌زاره‌ای خ‌ود ک‌ار ک‌ه ب‌ا ت ک‌ی‌ه ب‌ر ام‌ض‌اه‌ای آس‌ی‌ب‌پ‌ذی‌ری ب‌ه ش‌ن‌اس‌ای‌ی م‌ش ک‌ل‌ات در ح‌ی‌ن اج‌رای ب‌رن‌ام‌ه م‌ی‌پ‌ردازن‌د.
- **Fast IDentity Online** (FIDO) – م‌ج‌م‌وع‌ه‌ای از اس‌ت‌ان‌دارده‌ای اح‌راز ه‌وی‌ت ک‌ه ام ک‌ان اس‌ت‌فاده از روش-ه‌ای م‌خ‌ت‌ل‌ف اح‌راز ه‌وی‌ت از ج‌م‌ل‌ه ب‌ی‌وم‌ت‌ری ک، م‌اژول‌ه‌ای پ‌ل‌ت‌ف‌رم م‌ورد اع‌ت‌م‌اد ،(TPM) ت‌وک‌ن‌ه‌ای ام‌ن‌ی‌ت‌ی USB و غ‌ی‌ره را ف‌راه‌م م‌ی ک‌ن‌د.
- **Hardware Security Module** (HSM) ی ک ج‌زء س‌خ‌ت‌اف‌زاری ک‌ه ک‌ل‌ی‌ده‌ای رم‌زن‌گ‌اری و س‌ای‌ر اس‌رار را ب‌ه‌ص‌ورت م‌ح‌اف‌ظ‌ت‌ش‌ده ذخ‌ی‌ره م‌ی ک‌ن‌د.
- **Hibernate Query Language** (HQL) ی ک زب‌ان پ‌رس‌وج‌و ک‌ه از ن‌ظر ظاه‌ری ش‌ب‌ی‌ه SQL اس‌ت و ت‌وس‌ط ک‌ت‌اب‌خ‌ان‌ه Hibernate ORM اس‌ت‌فاده م‌ی‌ش‌ود.
- **HTTP Strict Transport Security** (HSTS) س‌ی‌اس‌ت‌ی ک‌ه م‌رورگ‌ر را م‌ج‌ب‌ور م‌ی ک‌ن‌د ف‌ق‌ط از ط‌ری‌ق TLS ب‌ه دام‌ن‌ه‌ای ک‌ه ه‌در HSTS را ارس‌ال م‌ی ک‌ن‌د م‌ت‌ص‌ل ش‌ود و ت‌ن‌ه‌ا زم‌ان‌ی ک‌ه گ‌واه‌ی‌ن‌ام‌ه م‌ع‌ت‌ب‌ر ارائ‌ه ش‌ده ب‌اش‌د. ای‌ن س‌ی‌اس‌ت ب‌ا ف‌ی‌ل‌د ه‌در پ‌اس‌خ Strict-Transport-Security ف‌ع‌ال م‌ی‌ش‌ود.
- **HyperText Transfer Protocol** (HTTP) ی ک پ‌روت ک‌ل ب‌رن‌ام‌ه‌ای ب‌رای س‌ی‌س‌ت‌م‌ه‌ای اط‌لاع‌ات‌ی ت‌وزی‌ع‌ش‌ده، م‌ش‌ارک‌ت‌ی و اب‌ررس‌ان‌ه‌ای اس‌ت. HTTP پ‌ای‌ه و اس‌اس ارت‌ب‌اط داده‌ه‌ا در ش‌ب‌ک‌ه ج‌ه‌ان‌ی وب م‌ح‌س‌وب م‌ی‌ش‌ود.
- **HyperText Transfer Protocol over SSL/TLS** (HTTPS) روش‌ی ب‌رای ام‌ن‌س‌ازی ارت‌ب‌اط HTTP از ط‌ری‌ق رم‌زن‌گ‌اری آن ب‌ا اس‌ت‌فاده از .(Transport Layer Security) TLS
- **Identity Provider** (IdP) ک‌ه در م‌ن‌اب‌ع NIST گ‌اه‌ی ب‌ه آن ارائ‌ه‌ده‌ن‌ده خ‌دم‌ات اع‌ت‌ب‌ارن‌ام‌ه (Credential Service Provider - CSP) ن‌ی‌ز گ‌ف‌ت‌ه م‌ی‌ش‌ود، م‌وج‌ودی‌ت‌ی اس‌ت ک‌ه م‌ن‌ب‌ع اح‌راز ه‌وی‌ت را ب‌رای س‌ای‌ر ب‌رن‌ام‌ه‌ا ف‌راه‌م م‌ی ک‌ن‌د.
- **Inactivity Timeout** – م‌دت‌ی اس‌ت ک‌ه ی ک ن‌ش‌س‌ت م‌ی‌ت‌وان‌د در ص‌ورت ع‌دم ت‌ع‌ام‌ل ک‌ارب‌ر ب‌ا ب‌رن‌ام‌ه ف‌ع‌ال ب‌اق‌ی ب‌م‌ان‌د. ای‌ن م‌ع‌ی‌ار ی ک‌ی از م‌ؤل‌ف‌ه‌ای ان‌ق‌ض‌ای ن‌ش‌س‌ت (Session Expiration) اس‌ت.
- **Input Validation** – اس‌ت‌ان‌دارد‌س‌ازی (Canonicalization) و اع‌ت‌ب‌ارس‌ن‌ج‌ی ورودی‌ه‌ای ک‌ارب‌ر غ‌ی‌رق‌اب‌ل اع‌ت‌م‌اد.
- **JSON Web Token** (JWT) – RFC 7519 ی ک اس‌ت‌ان‌دارد ب‌رای ی ک ش‌یء داده JSON ت‌ع‌ری‌ف م‌ی ک‌ن‌د ک‌ه از ی ک ب‌خ‌ش ه‌در ت‌ش ک‌ی‌ل ش‌ده ک‌ه ن‌ح‌وه اع‌ت‌ب‌ارس‌ن‌ج‌ی ش‌یء را ت‌وض‌ی‌ح م‌ی‌ده‌د، ی ک ب‌خ‌ش ب‌دن‌ه ک‌ه ش‌ام‌ل م‌ج‌م‌وع‌ه‌ای از ادع‌اه‌ا اس‌ت، و ی ک ب‌خ‌ش ام‌ض‌ا ک‌ه ش‌ام‌ل ام‌ض‌ای دی‌ج‌ی‌ت‌ال اس‌ت و م‌ی-ت‌وان از آن ب‌رای اع‌ت‌ب‌ارس‌ن‌ج‌ی م‌ح‌ت‌وای ب‌خ‌ش ب‌دن‌ه اس‌ت‌فاده ک‌رد. ای‌ن ن‌وع‌ی ت‌وک‌ن خ‌ود ک‌فا (self-contained token) اس‌ت.
- **Local File Inclusion** (LFI) - ح‌م‌ل‌ه‌ای ک‌ه از روی‌ه‌ه‌ای آس‌ی‌ب‌پ‌ذی‌ر درج ف‌ای‌ل در ی ک ب‌رن‌ام‌ه س‌وءاس‌ت‌فاده م‌ی ک‌ن‌د و م‌ن‌ج‌ر ب‌ه درج ف‌ای‌ل‌ه‌ای م‌ح‌ل‌یِ م‌وج‌ود روی س‌رور م‌ی‌ش‌ود.
- **Malicious Code** – ک‌دی ک‌ه در ط‌ول ت‌وس‌ع‌ه ی ک ب‌رن‌ام‌ه ب‌دون اط‌لاع م‌ال ک ب‌رن‌ام‌ه وارد ش‌ده و س‌ی‌اس‌ت ام‌ن‌ی‌ت‌ی م‌ورد ن‌ظر ب‌رن‌ام‌ه را دور م‌ی‌زن‌د. ای‌ن ب‌ا ب‌داف‌زاره‌ای‌ی م‌ان‌ند وی‌روس ی‌ا ک‌رم ی ک‌س‌ان ن‌ی‌س‌ت!
- **Malware** – ک‌داج‌رای‌ی ک‌ه در ح‌ی‌ن اج‌رای ب‌رن‌ام‌ه و ب‌دون اط‌لاع ک‌ارب‌ر ی‌ا م‌دی‌ر ب‌رن‌ام‌ه وارد م‌ی‌ش‌ود.
- **Message authentication code** (MAC) - ی ک چ ک‌س‌ام رم‌زن‌گ‌اری‌ش‌ده ب‌ر داده‌ه‌ا اس‌ت ک‌ه ت‌وس‌ط ال‌گ‌وری‌ت‌م ت‌ول‌ی‌د MAC م‌ح‌اس‌به م‌ی‌ش‌ود و ب‌رای اط‌م‌ی‌ن‌ان از ص‌ح‌ت و اص‌الت داده‌ه‌ا اس‌ت‌فاده م‌ی‌ش‌ود.
- **Multi-factor authentication** (MFA) – اح‌راز ه‌وی‌ت‌ی ک‌ه ش‌ام‌ل دو ی‌ا چ‌ن‌د ع‌ام‌ل از ع‌وام‌ل اح‌راز ه‌وی‌ت ت ک‌ی اس‌ت.

- **Mutual TLS** (mTLS) – مراجعه کنید به احراز هویت کلاینت TLS (TLS Client Authentication).
- **Object-relational Mapping** (ORM) – سیستمی که امکان ارجاع و پرسوجوی پایگاه دادهای رابطهای/جدولی را در یک برنامه کاربردی با استفاده از مدل شیءمحور سازگار با برنامه فراهم میکند.
- **One-time Password** (OTP) – رمزی که بهصورت منحصر به فرد تولید شده و فقط برای یک بار استفاده کاربرد دارد.
- **Open Worldwide Application Security Project** (OWASP) – یک جامعه جهانی، رایگان و باز است که بر بهبود امنیت نرمافزارهای کاربردی تمرکز دارد. مأموریت آن این است که امنیت برنامههای کاربردی را «قابل مشاهده» کند تا افراد و سازمانها بتوانند تصمیمات آگاهانهای در مورد ریسکهای امنیتی برنامههای کاربردی بگیرند. برای اطلاعات بیشتر مشاهده شود : https://www.owasp.org/.
- **Password-Based Key Derivation Function 2** (PBKDF2) – یک الگوریتم یکطرفه ویژه است که برای ایجاد یک کلید رمزنگاری قوی از یک متن ورودی (مانند گذرواژه) و یک مقدار salt تصادفی اضافی استفاده میشود و بنابراین میتوان از آن برای سختتر کردن شکستن گذرواژه بهصورت آفلاین استفاده کرد، اگر مقدار حاصل بهجای گذرواژه اصلی ذخیره شود.
- **Public Key Infrastructure** (PKI) – ساختاری که کلیدهای عمومی را با هویتهای مربوط به موجودیتها پیوند میدهد. این پیوند از طریق فرآیند ثبت و صدور گواهینامه توسط یک مرجع صدور گواهی (Certificate Authority - CA) برقرار میشود.
- **Public Switched Telephone Network** (PSTN) – شبکه تلفن سنتی که شامل تلفنهای خط ثابت و تلفنهای همراه میشود.
- **Real-time Transport Protocol** (RTP) and **Real-time Transport Control Protocol** (RTCP) – دو پروتکل که بهصورت ترکیبی برای انتقال جریانهای چندرسانهای استفاده میشوند. این پروتکلها توسط پشته WebRTC استفاده میشوند و در RFC 3550 تعریف شدهاند.
- **Reference Token** – نوعی توکن که بهعنوان اشارهگر یا شناسهای برای وضعیت یا متادادهٔ ذخیرهشده روی سرورعمل میکند و گاهی به آن توکنهای تصادفی (Random Tokens) یا توکنهای مات (Opaque Tokens) نیز گفته میشود. برخلاف توکنهای خودکفا (Self-Contained Tokens) که بخشی از دادههای مرتبط خود را درون توکن جای میدهند، توکنهای مرجع هیچ اطلاعات ذاتیای ندارند و برای دریافت زمینه (Context) به سرور وابسته هستند. توکن مرجع یا یک شناسه نشست (Session Identifier) است یا شامل آن میباشد.
- **Relying Party** (RP) – معمولاً یک برنامه کاربردی است که بر احراز هویت کاربر توسط یک ارائهدهنده احراز هویت جداگانه متکی است. این برنامه بر نوعی توکن یا مجموعهای از ادعاهای امضا شده ارائهشده توسط آن ارائهدهنده اعتماد میکند تا مطمئن شود کاربر همان کسی است که ادعا میکند.
- **Remote File Inclusion** (RFI) - حملهای که از رویههای آسیبپذیر درج فایل در برنامه سوءاستفاده میکند و منجر به درج فایلهای راهدور (Remote) میشود.
- **Scalable Vector Graphics** (SVG) – یک زبان نشانهگذاری مبتنی بر XML برای توصیف گرافیکهای برداری دوبعدی.
- **Secure Real-time Transport Protocol** (SRTP) and **Secure Real-time Transport Control Protocol** (SRTCP) – یک پروفایل از پروتکلهای RTP و RTCP که پشتیبانی از رمزنگاری پیام، احراز هویت و محافظت از صحت دادهها را فراهم میکند. این پروتکلها در RFC 3711 تعریف شدهاند.

- **Security Architecture** – یک انتزاع از طراحی برنامه کاربردی است که محل و نحوه استفاده از کنترل‌های امنیتی را شناسایی و توصیف می‌کند و همچنین مکان و حساسیت داده‌های کاربر و برنامه را مشخص می‌نماید.
- **Security Assertion Markup Language** (SAML) – یک استاندارد باز برای احراز هویت Single Sign-On است که بر اساس انتقال ادعاهای امضا شده (معمولاً اشیاء XML بین ارائه‌دهنده هویت (Identity Provider) و Relying Party عمل می‌کند.
- **Security Configuration** – پیکربندی زمان اجرا یک برنامه که نحوه استفاده از کنترل‌های امنیتی را تحت تأثیر قرار می‌دهد.
- **Security Control** – یک عمل کرد یا مؤلفه که یک بررسی امنیتی انجام می‌دهد (مثلاً بررسی دسترسی) یا هنگام فراخوانی منجر به اثر امنیتی می‌شود (مثلاً تولید یک رکورد حسابرسی).
- **Security information and event management** (SIEM) - سیستمی برای شناسایی تهدیدات، انطباق با مقررات و مدیریت رخدادهای امنیتی از طریق جمع‌آوری و تحلیل داده‌های مرتبط با امنیت از منابع مختلف در زیرساخت فناوری اطلاعات یک سازمان.
- **Self-Contained Token** – توکنی که یک یا چند ویژگی را در خود جای می‌دهد و به وضعیت سمت سرور یا سایر ذخیره‌سازی‌های خارجی متکی نیست. این توکن‌ها اصالت و صحت ویژگی‌های درون خود را تضمین می‌کنند و امکان تبادل امن و بدون حالت (Stateless) اطلاعات بین سیستم‌ها را فراهم می‌سازند. توکن‌های خودکفا معمولاً با تکنیک‌های رمزنگاری مانند امضاهای دیجیتال یا کدهای تأیید پیام (MAC) ایمن می‌شوند تا اصالت، صحت و در برخی موارد محرمانگی داده‌ها تضمین گردد. نمونه‌های رایج شامل ادعاهای (SAML Assertions) SAML و JWTها هستند.
- **Server-side Request Forgery** (SSRF) – حمله‌ای که از قابلیت‌های سرور برای خواندن یا به‌روزرسانی منابع داخلی سوءاستفاده می‌کند. مهاجم یک URL را ارائه یا تغییر می‌دهد که کدی که روی سرور اجرا می‌شود آن را خوانده یا داده‌ای به آن ارسال می‌کند.
- **Session Description Protocol** (SDP) – یک قالب پیام برای راه‌اندازی نشست‌های چندرسانه‌ای (به‌عنوان مثال در WebRTC استفاده می‌شود). این پروتکل در RFC 4566 تعریف شده است.
- **Session Identifier** or **Session ID** – یک کلید که یک نشست stateful ذخیره‌شده در سمت سرور را شناسایی می‌کند. این شناسه به‌صورت مستقیم یا درون یک «توکن مرجع» (Reference Token) بین سرور و کلاینت منتقل می‌شود.
- **Session Token** – اصطلاحی کلی که در این استاندارد برای اشاره به توکن یا مقدار مورد استفاده در مکانیزم‌های نشست stateless (که از توکن استفاده‌شده self-contained می‌کنند) یا مکانیزم‌های نشست stateful (که از توکن مرجع استفاده می‌کنند) به کار می‌رود.
- **Session Traversal Utilities for NAT** (STUN) – یک پروتکل که برای کمک به عبور از NAT به منظور برقراری ارتباط‌های همتا به همتا (Peer-to-Peer) استفاده می‌شود. این پروتکل در RFC 3489 تعریف شده است.
- **Single-factor authenticator** – مکانیزمی برای بررسی احراز هویت کاربر که بایدیکی از موارد زیر باشد: چیزی که می‌دانید (رمزهای عبور، عبارات عبور، PIN)، چیزی که هستید (بیومتریک، اثر انگشت، اسکن چهره) یا چیزی که در اختیار دارید (توکن‌های OTP، دست‌گاه رمزنگاری مانند کارت هوشمند).
- **Single Sign-on Authentication** (SSO) – زمانی رخ می‌دهد که کاربر وارد یک برنامه می‌شود و به-

صورت خودکار به برنامه‌های دیگر نیز وارد می‌شود بدون نیاز به احراز هویت مجدد. به‌عنوان مثال، وقتی کاربر وارد Google می‌شود، به‌طور خودکار وارد سایر سرویس‌های Google مانند YouTube، Google Docs و Gmail نیز می‌گردد.

- **Software bill of materials** (SBOM) یک فهرست ساخت‌اری‌افته و جامع از تمام مؤلفه‌ها، ماژول‌ها، کتابخانه‌ها، فریم‌ورک‌ها و سایر منابع مورد نیاز برای ساخت یا مونتاژ یک برنامه نرم‌افزاری است.
- **Software Composition Analysis** (SCA) – مجموعه‌ای از فناوری‌ها که برای تحلیل ترکیب برنامه، وابستگی‌ها، کتابخانه‌ها و بسته‌ها به‌منظور شناسایی آسیب‌پذیری‌های امنیتی نسخه‌های خاص مؤلفه‌های مورد استفاده طراحی شده است. این مفهوم با تحلیل کد منبع (SAST) اشتباه گرفته نشود.
- **Software development lifecycle** (SDLC) – فرایند گام‌به‌گام توسعه نرم‌افزار که از نیازمندی‌های اولیه تا استقرار و نگه‌داری پیش می‌رود.
- **SQL Injection** (SQLi) یک تکنیک تزریق کدکه برای حمله به برنامه‌های مبتنی بر داده استفاده می‌شود و در آن دستورات SQL مخرب به یک نقطه ورودی تزریق می‌گردند.
- **Stateful Session Mechanism** – در این مکانیزم، برنامه وضعیت نشست را در سمت سرور نگه می‌دارد که معمولاً با توکن نشست مطابقت دارد و این توکن با استفاده از تولیدکننده اعداد شبه‌تصادفی امن رمزنگاری (CSPRNG) ایجاد و به کاربر نهایی ارائه می‌شود.
- **Stateless Session Mechanism** – یک مکانیزم نشست stateless از یک self-contained token استفاده می‌کند که به کلاینت‌ها منتقل می‌شود و شامل اطلاعات نشست است که لزوماً در سرویس دریافت‌کننده ذخیره نمی‌شود و سپس آن سرویس توکن را اعتبارسنجی می‌کند. در عمل، یک سرویس نیاز دارد به دسترسی به برخی اطلاعات نشست (مانند فهرست لغو JWT) تا بتواند کنترل‌های امنیتی مورد نیاز را اعمال کند.
- **Static application security testing** (SAST) مجموعه‌ای از فناوری‌ها که برای تحلیل کد منبع، بایت‌کد و بایناری‌های برنامه به‌منظور شناسایی شرایط برنامه‌نویسی و طراحی که نشان‌دهنده آسیب‌پذیری‌های امنیتی هستند طراحی شده‌اند. راه‌کاره‌ای SAST برنامه را از داخل به خارج و در حالت غیر اجرایی تحلیل می‌کنند.
- **Threat Modeling** یک تکنیک که شامل توسعه معماری‌های امنیتی به‌طور پیوسته و دقیق‌تر برای شناسایی عاملان تهدید، مناطق امنیتی، کنترل‌های امنیتی و دارایی‌های فنی و تجاری مهم است.
- **Time-of-check to time-of-use** (TOCTOU) وضعیتی که در آن یک برنامه وضعیت یک منبع را قبل از استفاده از آن بررسی می‌کند، اما وضعیت منبع می‌تواند بین زمان بررسی و استفاده تغییر کند. این می‌تواند نتایج بررسی را نامعتبر کند و باعث شود برنامه به دلیل عدم تطابق وضعیت اقدامات نامعتبر انجام دهد.
- **Time based One-time Passwords** (TOTPs) روشی برای تولید OTP که در آن زمان فعلی بخشی از الگوریتم تولید رمز عبور محسوب می‌شود.
- **TLS client authentication**, also called **Mutual TLS** (mTLS) – در یک اتصال استاندارد TLS، کلاینت می‌تواند از گواهی ارائه‌شده توسط سرور برای اعتبارسنجی هویت سرور استفاده کند. زمانی که احراز هویت کلاینت TLS استفاده می‌شود، کلاینت نیز از کلید خصوصی و گواهی خود استفاده می‌کند تا سرور بتواند هویت کلاینت را اعتبارسنجی کند.
- **Transport Layer Security** (TLS) پروتکل‌های رمزنگاری که امنیت ارتباطات را بر روی اتصال شبکه فراهم می‌کنند.

- **Traversal Using Relays around NAT** (TURN) – ی ک اف زونه از پ روت ک ل STUN ک ه از ی ک س رور TURN ب ه ع نوان رل ه اس ت ف اده م ی ک ند زم انی ک ه ات ص ال م س ت ق ی م ه م ت ا ب ه ه م ت ا ق ابل ب رق راری ن ی س ت. ای ن پ روت ک ل در RFC 8656 ت ع ری ف ش ده اس ت.
- **Trusted execution environment** (TEE) - ی ک م ح ی ط پ ردازش ی ای زول ه ک ه در آن ب رن ام ه ه ا م ی ت وان ند ب ه ط ور ام ن اج را ش ون د، ب دون ت وج ه ب ه ب اق ی س ی س ت م.
- **Trusted Platform Module** (TPM) – نوع ی HSM ک ه م ع م ولاً ب ه ی ک ج زء س خ ت اف زاری ب زرگ ت ر م انند م ادرب ورد م ت ص ل اس ت و ب ه ع نوان ری ش ه اع ت م اد (Root of Trust) ب رای آن س ی س ت م ع م ل م ی ک ند.
- **Trusted Service Layer** – ه ر ن ق ط ه اج رای ک ن ت رل م ورد اع ت م اد، م انند ی ک م ی ک روس روی س، API ب دون س رور (Serverless API)، س م ت س رور، ی ک API م ورد اع ت م اد روی دس ت گ اه ک ل ای ن ت ک ه ب وت ام ن دارد، ه ای API ش رک اء ی ا خ ارج ی و غ ی ره. م ورد اع ت م اد ب ه ای ن م ع ن اس ت ک ه ن گ ران ی وج ود ن دارد ک ه ی ک ک ارب ر غ ی رم ورد اع ت م اد ب ت وان د ل ای ه ی ا ک ن ت رل ه ای پ ی اده س ازی ش ده در آن ل ای ه را دور ب زن د ی ا ن ادی ده ب گ ی رد.
- **Uniform Resource Identifier** (URI)- ی ک رش ت ه ی ی ک ت ا از ک اراک ت ره ا ک ه ی ک م نب ع را ش ن اس ای ی م ی ک ند، م انند ی ک ص ف ح ه وب، آدرس ای م ی ل ی ا م ک ان ه ا.
- **Uniform Resource Locator** (URL) – رش ت ه ای ک ه م ک ان ی ک م نب ع در ای ن ت رن ت را م ش خ ص م ی ک ند.
- **Universally Unique Identifier** (UUID) – ی ک ش م اره م رج ع ی ک ت ا ک ه ب ه ع نوان ش ن اس ه در ن رم اف زار اس ت ف اده م ی ش ود.
- **Verifier** – ف رد ی ا ت ی م ی ک ه ی ک ب رن ام ه را ب ر اس اس ال زام ات OWASP ASVS ب ررس ی م ی ک ند.
- **Web Real-Time Communication** (WebRTC) – ی ک پ ش ت ه پ روت ک ل و API وب م رت ب ط ک ه ب رای ان ت ق ال ج ری ان ه ای چ ن درس ان ه ای در ب رن ام ه ه ای وب اس ت ف اده م ی ش ود، م ع م ولاً در زم ی ن ه ک ن ف ران س ه ای وی دی وی ی. ای ن پ ش ت ه ب ر اس اس SRTP، SRTCP، SDP DTLS و STUN/TURN س اخ ت ه ش ده اس ت.
- **WebSocket over TLS** (WSS) – روش ی ب رای ام ن س ازی ارت ب اط وب س وک ت از ط ری ق ق رار دادن وب-س وک ت ب ر روی پ روت ک ل TLS.
- **What You See Is What You Get** (WYSIWYG) – نوع ی وی رای ش گ رم ح ت وای غ ن ی ک ه ن ش ان م ی ده د م ح ت وا ه ن گ ام ن م ای ش چ گ ون ه ب ه ن ظ ر م ی رس د، ب ه ج ای آن ک ه ک ه ای ی ک ه ب رای ن م ای ش م ح ت وا اس ت ف اده م ی ش ون د را ن ش ان ده د.
- **X.509 Certificate** – ی ک گ واه ی دی ج ی ت ال اس ت ک ه از اس ت ان دارد ب ی ن ال م ل ل ی پ ذی رف ت ه ش ده X.509 در زی رس اخ ت ک ل ی د ع م وم ی (PKI) اس ت ف اده م ی ک ند ت ا ت أ ی ی د ک ند ک ل ی د ع م وم ی م ت ع ل ق ب ه ه وی ت ک ارب ر، ک ام پ ی وت ر ی ا س روی س اس ت ک ه در گ واه ی ذ ک ر ش ده اس ت.
- **XML eXternal Entity** (XXE) – نوع ی م وج ودی ت XML ک ه م ی ت وان د از ط ری ق ش ن اس ه س ی س ت م اع ل ام-ش ده ب ه م ح ت وای م ح ل ی ی ا راه دور دس ت رس ی پ ی دا ک ند. ای ن م ی ت وان د م ن ج ر ب ه ان واع ح م ل ات ت زری ق ی ش ود.

Appendix B: References

پ روژه ه ای زی ر از OWASP ب ه اح ت م ال زی اد ب رای ک ارب ران و اس ت ف اده ک ن ن د گ ان ای ن اس ت ان دارد م ف ی د خ واه ند ب ود:

OWASP Core Projects

1. OWASP Top 10 Project: https://owasp.org/www-project-top-ten/
2. OWASP Web Security Testing Guide: https://owasp.org/www-project-web-security-testing-guide/
3. OWASP Proactive Controls: https://owasp.org/www-project-proactive-controls/
4. OWASP Software Assurance Maturity Model (SAMM): https://owasp.org/www-project-samm/
5. OWASP Secure Headers Project: https://owasp.org/www-project-secure-headers/

OWASP Cheat Sheet Series project

این پروژه دارای چندین برگه راهنما (Cheat Sheet) می‌باشد که برای موضوعات مختلف در ASVS مرتبط و کاربردی خواهند بود.

نگاشتی به ASVS وجود دارد که می‌توان آن را در اینجا پیدا کرد: https://cheatsheetseries.owasp.org/IndexASVS.html

Mobile Security Related Projects

1. OWASP Mobile Security Project: https://owasp.org/www-project-mobile-security/
2. OWASP Mobile Top 10 Risks: https://owasp.org/www-project-mobile-top-10/
3. OWASP Mobile Security Testing Guide and Mobile Application Security Verification Standard: https://owasp.org/www-project-mobile-security-testing-guide/

OWASP Internet of Things related projects

1. OWASP Internet of Things Project: https://owasp.org/www-project-internet-of-things/

OWASP Serverless projects

1. OWASP Serverless Project: https://owasp.org/www-project-serverless-top-10/

Others

به‌طور مشابه، وب‌سایت‌های زیر به‌احتمال زیاد برای کاربران و استفاده‌کنندگان این استاندارد مفید خواهند بود.

1. SecLists Github: https://github.com/danielmiessler/SecLists
2. MITRE Common Weakness Enumeration: https://cwe.mitre.org/
3. PCI Security Standards: https://www.pcisecuritystandards.org/standards/

Appendix C: Cryptography Standards

فصل «رمزنگاری» فراتر از ارائه‌ی ساده‌ی بهترین روی‌ه‌ا عمل می‌کند. این فصل با هدف افزایش درک اصول رمزنگاری و تشویق به به‌کارگیری روش‌های امنیتی مدرن‌تر و مقاوم‌تر تدوین شده است. این پیوست، اطلاعات فنی تفصیلی مربوط به هر یک از الزامات را ارائه می‌دهد و استانداردهای کلی مطرح‌شده در فصل «رمزنگاری» را تکمیل می‌کند.

این پیوست سطح تأیید را برای مکانیزم‌های مختلف رمزنگاری تعریف می‌کند:

- مکانیزم‌های تأییدشده (A) می‌توانند در برنامه‌ها استفاده شوند.
- مکانیزم‌های قدیمی (L) نباید در برنامه‌ها استفاده شوند، اما ممکن است فقط برای سازگاری با برنامه‌ها یا کدهای قدیمی موجود به کار روند. اگرچه استفاده از این مکانیزم‌ها در حال حاضر به خودی خود آسیب‌پذیری محسوب نمی‌شود، اما باید هرچه سریع‌تر با مکانیزم‌های امن‌تر و آینده‌نگرجایگزین شوند.
- مکانیزم‌های غیرمجاز (D) نباید استفاده شوند زیرا در حال حاضر شکسته محسوب می‌شوند یا امنیت کافی ارائه نمی‌کنند.

این فهرست ممکن است در زمینه یک برنامه خاص به دلایل مختلف لغو شود، از جمله:

- تحولات جدید در حوزه رمزنگاری؛
- رعایت مقررات و قوانین.

Cryptographic Inventory and Documentation

این بخش اطلاعات تکمیلی را برای V11.1 موجودی و مستندسازی رمزنگاری ارائه می‌دهد.

مهم است که تمام دارایی‌های رمزنگاری، مانند الگوریتم‌ها، کلیدها و گواهی‌نامه‌ها، به‌طور منظم شناسایی، فهرست‌بندی و ارزیابی شوند. برای سطح 3، این باید شامل استفاده از اسکن ایستا و پویا برای شناسایی استفاده از رمزنگاری در یک برنامه باشد. ابزارهایی مانند SAST و DAST می‌توانند در این زمینه کمک کنند، اما ممکن است ابزارهای اختصاصی برای پوشش جامع‌تر مورد نیاز باشند. نمونه‌های رایگان از این ابزارها عبارتند از:

- CryptoMon - Network Cryptography Monitor - using eBPF, written in python
- Cryptobom Forge Tool: Generating Comprehensive CBOMs from CodeQL Outputs

Equivalent Strengths of Cryptographic Parameters

قدرت‌های امنیتی نسبی برای سیستم‌های مختلف رمزنگاری در این جدول آمده است (از NIST SP 800-57 Part 1, صفحه71):

Security Strength	Symmetric Key Algorithms	Finite Field	Integer Factorization	Elliptic Curve
<= 80	2TDEA	L = 1024 N = 160	k = 1024	f = 160-223

Security Strength	Symmetric Key Algorithms	Finite Field	Integer Factorization	Elliptic Curve
112	3TDEA	L = 2048 N = 224	k = 2048	f = 224-255
128	AES-128	L = 3072 N = 256	k = 3072	f = 256-383
192	AES-192	L = 7680 N = 384	k = 7680	f = 384-511
256	AES-256	L = 15360 N = 512	k = 15360	f = 512+

نمونه‌ای از برنامه‌ها:

- رمزنگاری میدان محدود (Finite Field Cryptography): DSA، MQV FFDH
- رمزنگاری تجزیه عدد صحیح (Integer Factorization Cryptography): RSA
- رمزنگاری منحنی بیضوی (Elliptic Curve Cryptography): ECDSA، EdDSA، MQV ECDH

توجه: این بخش فرض می‌کند که هیچ کامپیوتر کوانتومی وجود ندارد؛ اگر چنین کامپیوتری وجود داشته باشد، برآورده‌ای سه ستون آخر دیگر معتبر نخواهند بود.

Random Values

این بخش اطلاعات تکمیلی برای Random Values 5 را ارائه می‌دهد.

Name	Version/Reference	Notes	Status
`/dev/random`	Linux 4.8+ (Oct 2016), also found in iOS, Android, and other Linux-based POSIX operating systems. Based on RFC7539	Utilizing ChaCha20 stream. Found in iOS `SecRandomCopyBytes` and Android `SecureRandom` with the correct settings provided to each.	A
`/dev/urandom`	Linux kernel's special file for providing random data	Provides high-quality, entropy sources from hardware randomness	A
`AES-CTR-DRBG`	NIST SP800-90A	As used in common implementations, such as Windows CNG API `BCryptGenRandom` set by `BCRYPT_RNG_ALGORITHM`.	A
`HMAC-DRBG`	NIST SP800-90A		A

Name	Version/Reference	Notes	Status
`Hash-DRBG`	NIST SP800-90A		A
`getentropy()`	OpenBSD, available in Linux glibc 2.25+ and macOS 10.12+	Provides secure random bytes directly from the kernel's entropy source with a straightforward and minimal API. It's more modern and avoids pitfalls associated with older APIs.	A

توابع هش زیربنایی که در HMAC-DRBG یا Hash-DRBG استفاده می‌شود، باید برای این کاربرد تأیید شده باشد.

Cipher Algorithms

این بخش اطلاعات تکمیلی برای الگوریتم‌های رمزنگاری را ارائه می‌دهد.

الگوریتم‌های رمزنگاری تأیید شده به ترتیب اولویت فهرست شده‌اند.

Symmetric Key Algorithms	Reference	Status
AES-256	FIPS 197	A
Salsa20	Salsa 20 specification	A
XChaCha20	XChaCha20 Draft	A
XSalsa20	Extending the Salsa20 nonce	A
ChaCha20	RFC 8439	A
AES-192	FIPS 197	A
AES-128	FIPS 197	L
2TDEA		D
TDEA (3DES/3DEA)		D
IDEA		D
RC4		D
Blowfish		D
ARC4		D

Symmetric Key Algorithms	Reference	Status
DES		D

AES Cipher Modes

رمزنگاری‌های بلوکی، مانند AES، می‌توانند با حالت‌های مختلف عملیاتی استفاده شوند. بسیاری از حالت‌های عملیاتی، مانند Electronic Codebook (ECB)، نامن هستند و نباید استفاده شوند. حالت‌های عملیاتی Galois/Counter Mode (GCM) و Counter with Cipher Block Chaining Message Authentication Code (CCM) رمزنگاری تأیید شده (Authenticated Encryption) ارائه می‌دهند و باید در برنامه‌های مدرن استفاده شوند.

حالت‌های تأیید شده به ترتیب اولویت فهرست شده‌اند.

Mode	Authenticated	Reference	Status	Restriction
GCM	Yes	NIST SP 800-38D	A	
CCM	Yes	NIST SP 800-38C	A	
CBC	No	NIST SP 800-38A	L	
CCM-8	Yes		D	
ECB	No		D	
CFB	No		D	
OFB	No		D	
CTR	No		D	

یادداشت‌ها:

- تمام پیام‌های رمزنگاری‌شده باید احراز هویت شوند. برای هر استفاده‌ای از حالت CBC، حتماً باید یک الگوریتم هش MAC مرتبط برای اعتبارسنجی پیام وجود داشته باشد. به طور کلی، این باید با روش Encrypt-Then-Hash اعمال شود (اما TLS 1.2 به جای آن از روش Hash-Then-Encrypt استفاده می‌کند). اگر این تضمین امکان‌پذیر نباشد، نباید از CBC استفاده شود. تنها کاربردی که در آن رمزنگاری بدون الگوریتم MAC مجاز است، رمزنگاری دیسک است.
- اگر از حالت CBC استفاده شود، باید تضمین شود که بررسی پدینگ (Padding) در زمان ثابت انجام می‌شود.
- هنگام استفاده از CCM-8، برچسب MAC تنها ۶۴ بیت امنیت ارائه می‌دهد. این با الزامات بند ۶.۲.۹ که حداقل ۱۲۸ بیت امنیت را نیاز دارد، مطابقت ندارد.
- رمزنگاری دیسک خارج از محدوده ASVS در نظر گرفته شده است. بنابراین، این ضمیمه هیچ روش تأیید شده‌ای برای رمزنگاری دیسک فهرست نمی‌کند. برای این کاربرد، معمولاً

رمزنگاری بدون احراز هویت پذیرفته می‌شود و حالت‌های XEX XTS، LRW و معمولاً استفاده می‌شوند.

Key Wrapping

بسته‌بندی کلید رمزنگاری (و باز کردن بسته‌ی مربوطه) روشی برای محافظت از یک کلید موجود است که با محصور کردن (یعنی بسته‌بندی) آن و استفاده از یک مکانیزم رمزنگاری اضافی انجام می‌شود تا کلید اصلی به‌طور آشکار در معرض دید قرار نگیرد، مثلاً هنگام انتقال. کلید اضافی که برای محافظت از کلید اصلی استفاده می‌شود، کلید بسته‌بندی (wrap key) نامیده می‌شود.

این عملیات ممکن است زمانی انجام شود که بخواهیم کلیدها را در مکانه‌ای که قابل اعتماد نیستند محافظت کنیم، یا کلیدهای حساس را از طریق شبکه‌های غیرقابل اعتماد یا درون برنامه‌ها ارسال کنیم. با این حال، باید با دقت به ماهیت کلید اصلی (مانند هویت و هدف آن) قبل از انجام فرآیند بسته‌بندی/بازکردن بسته توجه شود، زیرا این می‌تواند برای سیستم‌ها یا برنامه‌های مبدأ و مقصد پیام‌های زمینه امنیت و به ویژه تطابق قانونی داشته باشد، که ممکن است شامل مسیریابی حساب‌رسی عمل کرد کلید (مانند امضا) و همچنین ذخیره‌سازی مناسب کلید باشد.

به‌طور مشخص، برای بسته‌بندی کلید باید از AES-256 استفاده شود، مطابق با NIST SP 800-38F و با در نظر گرفتن تمهیدات آینده‌نگر در برابر تهدید کوانتومی. حالت‌های رمزنگاری استفاده‌شده با AES به ترتیب اولویت به شرح زیر هستند:

Key Wrapping	Reference	Status
KW	NIST SP 800-38F	A
KWP	NIST SP 800-38F	A

در صورتی که مورد استفاده نیاز داشته باشد، می‌توان از AES-192 و AES-128 نیز استفاده کرد، اما دلیل استفاده از آن‌ها باید در فهرست رمزنگاری موجود در سازمان مستندسازی شود.

Authenticated Encryption

به استثنای رمزنگاری دیسک، داده‌های رمزنگاری‌شده باید در برابر تغییرات غیرمجاز با استفاده از نوعی از طرح رمزنگاری احراز هویت‌شده (AE) محافظت شوند، که معمولاً از طرح رمزنگاری احراز هویت‌شده با داده‌های مرتبط (AEAD) استفاده می‌شود.

بهتر است برنامه از یک طرح AEAD تأیید شده استفاده کند. به طور جایگزین، می‌تواند یک طرح رمزنگاری تأیید شده و یک الگوریتم MAC تأیید شده را با ساختار Encrypt-then-MAC ترکیب کند.

روش MAC-then-encrypt همچنان برای سازگاری با برنامه‌های قدیمی مجاز است. این روش در TLS نسخه ۱.۲ با مجموعه‌های رمزنگاری قدیمی استفاده می‌شود.

AEAD mechanism	Reference	Status
AES-GCM	SP 800-38D	A
AES-CCM	SP 800-38C	A
ChaCha-Poly1305	RFC 7539	A
AEGIS-256	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
AEGIS-128	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
AEGIS-128L	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
Encrypt-then-MAC		A
MAC-then-encrypt		L

Hash Functions

ای ن ب خ ش اط لاع ات ت ک م ی ل ی در م ورد ه ش ک ردن و ت وابع م بت ن ی ب ر ه ش را ارائ ه م ی ده د.

Hash Functions for General Use Cases

ج دول زی ر ت وابع ه ش ت أی ی د ش ده ب رای ک ارب رده ای ع م ومی رمزن گ اری، م انن د امض اه ای دی ج ی ت ال، را ف ه رس ت م ی ک ن د:

- ت وابع ه ش ت أی ی د ش ده م قاوم ت ب ال ای ی در ب راب ر ب رخ ورد (collision) دارن د و ب رای ب رن ام ه ای ب ا ام ن ی ت ب ال ا م ناس ب ه س ت ن د.
- ب رخ ی از ای ن ال گ وری ت م ه ا وق ت ی ه م راه ب ا م دی ری ت ص ح ی ح ک ل ی ده ای رمزن گ اری اس ت ف اده ش ون د، م قاوم ت ب ال ای ی در ب راب ر ح م لات ارائ ه م ی ده ن د و ب ن اب رای ن ب رای ت وابع ،KDF HMAC و RBG ن ی ز ت أی ی د ش ده ان د.
- ت وابع ه ش ب ا خ روج ی ک م ت ر از ۲۵۴ ب ی ت م قاوم ت ک اف ی در ب راب ر ب رخ ورد (collision) ن دارن د و ن ب ای د ب رای ام ض اه ای دی ج ی ت ال ی ا س ای ر ک ارب رده ای ی ک ه ن ی از ب ه م قاوم ت در ب راب ر ب رخ ورد دارن د، اس ت ف اده ش ون د. ب رای س ای ر ک ارب رده ا، م م ک ن اس ت ت ن ه ا ب رای س ازگ اری و اع ت ب ارس ن ج ی ب ا س ی س ت م ه ای ق دی م ی اس ت ف اده ش ون د، ام ا ن ب ای د در ط راح ی ه ای ج دی د ب ه ک ار رون د.

Hash function	Reference	Status	Restrictions
SHA3-512	FIPS 202	A	
SHA-512	FIPS 180-4	A	
SHA3-384	FIPS 202	A	

Hash function	Reference	Status	Restrictions
SHA-384	FIPS 180-4	A	
SHA3-256	FIPS 202	A	
SHA-512/256	FIPS 180-4	A	
SHA-256	FIPS 180-4	A	
SHAKE256	FIPS 202	A	
BLAKE2s	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE2b	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE3	BLAKE3 one function, fast everywhere	A	
SHA-224	FIPS 180-4	L	Not suitable for HMAC, KDF, RBG, digital signatures
SHA-512/224	FIPS 180-4	L	Not suitable for HMAC, KDF, RBG, digital signatures
SHA3-224	FIPS 202	L	Not suitable for HMAC, KDF, RBG, digital signatures
SHA-1	RFC 3174 & RFC 6194	L	Not suitable for HMAC, KDF, RBG, digital signatures
CRC (any length)		D	
MD4	RFC 1320	D	
MD5	RFC 1321	D	

Hash Functions for Password Storage

برای هش کردن امن گذرواژه‌ها، باید از توابع هش مخصوص استفاده شود. این الگوریتم‌های هش کند، با افزایش سختی محاسباتی شکستن گذرواژه، حملات جست‌وجوی فراگیر (brute-force) و دیکشنری را کاهش می‌دهند.

KDF	Reference	Required Parameters	Status
argon2id	RFC 9106	t = 1: m ≥ 47104 (46 MiB), p = 1	A
		t = 2: m ≥ 19456 (19 MiB), p = 1	A
		t ≥ 3: m ≥ 12288 (12 MiB), p = 1	A
scrypt	RFC 7914	p = 1: N ≥ 2^17 (128 MiB), r = 8	A
		p = 2: N ≥ 2^16 (64 MiB), r = 8	A
		p ≥ 3: N ≥ 2^15 (32 MiB), r = 8	A
bcrypt	A Future-Adaptable Password Scheme	cost ≥ 10	A
PBKDF2-HMAC-SHA-512	NIST SP 800-132, FIPS 180-4	iterations ≥ 210, 000	A
PBKDF2-HMAC-SHA-256	NIST SP 800-132, FIPS 180-4	iterations ≥ 600, 000	A
PBKDF2-HMAC-SHA-1	NIST SP 800-132, FIPS 180-4	iterations ≥ 1, 300, 000	L

تواب ع اس تخ راج ک لی د م ب ت ن ی ب ر گ ن ز واژه (Password-Based Key Derivation Functions) ت أی ی د ش ده م ی ت وان ن د ب رای ذخ ی ره س ازی گ ن ز واژه اس ت ف اده ش ون د.

Key Derivation Functions (KDFs)

General Key Derivation Functions

KDF	Reference	Status
HKDF	RFC 5869	A
TLS 1.2 PRF	RFC 5248	L
MD5-based KDFs	RFC 1321	D
SHA-1-based KDFs	RFC 3174 & RFC 6194	D

Password-based Key Derivation Functions

KDF	Reference	Required Parameters	Status
argon2id	RFC 9106	t = 1: m ≥ 47104 (46 MiB), p = 1	A
		t = 2: m ≥ 19456 (19 MiB), p = 1	A
scrypt	RFC 7914	p = 1: N ≥ 2^17 (128 MiB), r = 8	A
		p = 2: N ≥ 2^16 (64 MiB), r = 8	A
		p ≥ 3: N ≥ 2^15 (32 MiB), r = 8	A
PBKDF2-HMAC-SHA-512	NIST SP 800-132, FIPS 180-4	iterations ≥ 210, 000	A
PBKDF2-HMAC-SHA-256	NIST SP 800-132, FIPS 180-4	iterations ≥ 600, 000	A
PBKDF2-HMAC-SHA-1	NIST SP 800-132, FIPS 180-4	iterations ≥ 1, 300, 000	L

Key Exchange Mechanisms

این بخش اطلاعات تکمیلی رمزنگاری کلید عمومی را ارائه می‌دهد.

KEX Schemes

برای همه طرح‌های تبادل کلید بایدقدرت امنیتی ۱۱۲ بیت یا بیشتر تضمین شود و پیاده-سازی آن‌ها باید از انتخاب پارامترهای موجود در جدول زیر پیروی کنند.

Scheme	Domain Parameters	Forward Secrecy	Status
Finite Field Diffie-Hellman (FFDH)	L >= 3072 & N >= 256	Yes	A
Elliptic Curve Diffie-Hellman (ECDH)	f >= 256-383	Yes	A

Scheme	Domain Parameters	Forward Secrecy	Status
Encrypted key transport with RSA-PKCS#1 v1.5		No	D

که پارامترهای زیر عبارتند از:

- k اندازه کلید برای کلیدهای RSA است.
- L اندازه کلید عمومی و N اندازه کلید خصوصی برای رمزنگاری بر منحنای میدان محدود است.
- f محدوده اندازههای کلید برای ECC است.

هیچ پیادهسازی جدیدی نباید از طرحی استفاده کند که با استانداردهای NIST SP 800-56A & B و NIST SP 800-77 مطابقت نداشته باشد. به طور مشخص، IKEv1 نباید در محیطهای تولیدی استفاده شود.

Diffie-Hellman groups

گروههای زیر برای پیادهسازی Diffie-Hellman تأیید شدهاند. قدرتهای امنیتی در ضمیمه D استاندارد NIST SP 800-56A و بخش 1 نسخه 5 استاندارد NIST SP 800-57 مستند شدهاند.

Group	Status
P-224, secp224r1	A
P-256, secp256r1	A
P-384, secp384r1	A
P-521, secp521r1	A
K-233, sect233k1	A
K-283, sect283k1	A
K-409, sect409k1	A
K-571, sect571k1	A
B-233, sect233r1	A
B-283, sect283r1	A
B-409, sect409r1	A
B-571, sect571r1	A
Curve448	A
Curve25519	A
MODP-2048	A

Group	Status
MODP-3072	A
MODP-4096	A
MODP-6144	A
MODP-8192	A
ffdhe2048	A
ffdhe3072	A
ffdhe4096	A
ffdhe6144	A
ffdhe8192	A

Message Authentication Codes (MAC)

کدهای احراز هویت پیام (MAC) سازه‌ای رمزنگاری هستند که برای بررسی صحت و اصالت یک پیام استفاده می‌شوند. یک MAC یک پیام و یک کلید مخفی را به عنوان ورودی می‌گیرد و یک برچسب با اندازه ثابت (مقدار MAC تولید می‌کند. هاMAC به طور گسترده در پروتکل‌های ارتباطی امن (مانند TLS/SSL استفاده می‌شوند تا اطمینان حاصل شود که پیام‌های تبادل‌شده بین طرفین معتبر و دستنخورده هستند.

MAC Algorithm	Reference	Status
HMAC-SHA-256	RFC 2104 & FIPS 198-1	A
HMAC-SHA-384	RFC 2104 & FIPS 198-1	A
HMAC-SHA-512	RFC 2104 & FIPS 198-1	A
KMAC128	NIST SP 800-185	A
KMAC256	NIST SP 800-185	A
BLAKE3 (keyed_hash mode)	BLAKE3 one function, fast everywhere	A
AES-CMAC	RFC 4493 & NIST SP 800-38B	A
AES-GMAC	NIST SP 800-38D	A
Poly1305-AES	The Poly1305-AES message-authentication code	A
HMAC-SHA-1	RFC 2104 & FIPS 198-1	L

MAC Algorithm	Reference	Status
HMAC-MD5	RFC 1321	D

Digital Signatures

طرح‌های امضا با‌ید از اندازه‌ها و پارامترهای کلید تأیید شده مطابق با NIST SP 800-57 بخش 1 استفاده کنند.

Signature Algorithm	Reference	Status
EdDSA (Ed25519, Ed448)	RFC 8032	A
XEdDSA (Curve25519, Curve448)	XEdDSA	A
ECDSA (P-256, P-384, P-521)	FIPS 186-4	A
RSA-RSSA-PSS	RFC 8017	A
RSA-SSA-PKCS#1 v1.5	RFC 8017	D
DSA (any key size)	FIPS 186-4	D

Post-Quantum Encryption Standards

پی‌اده‌سازی‌های رمزنگاری پساکوانتومی (PQC) با‌ید از استانداردهای FIPS-203، FIPS-204 و FIPS-205 پی‌روی کنند. در حال حاضر، نمونه‌های کدمقاوم یا پی‌اده‌سازی‌های مرجع زی‌ادی برای این استانداردها در دسترس نی‌ست. برای جزئی‌ات بی‌شتر، به اعلامیه NIST درباره سه استاندارد نه‌ای‌شده رمزنگاری پساکوانتومی (اوت ۲۰۲۴) مراجعه کنی‌د.

روش پی‌شنه‌ادی mlkem768x25519 برای توافق کلید هی‌بری‌دی TLS پساکوانتومی توسط مرورگره‌ای اصلی مانند Firefox نسخه ۱۳۲ و Chrome نسخه ۱۳۱ پشتی‌بانی می‌شود. این روش می‌توانند در محی‌طه‌ای آزمای‌شی رمزنگاری یا زمانی‌که در کتابخانه‌ه‌ای تأیی‌د شده توسط صنعت یا دولت در دسترس است، استفاده شود.

Appendix D: Recommendations

Introduction

در این آماده‌سازی نسخه ۵.۰ استاندارد اعتبارسنجی امنی‌ت برنامه‌ها، (ASVS) مشخص شدکه تعدادی از موارد موجود و همچنی‌ن موارد پی‌شنه‌ادی جدی‌د وجود دارندکه نبای‌د به‌عنوان الزام در نسخه ۵.۰ گنجانده شوند. این موضوع یا به این دلی‌ل بود که مطابق تعری‌ف نسخه ۵.۰ در محدوده ASVS قرار نمی‌گرفتند، یا به این خاطرکه هرچند ای‌ده‌ای خوبی بودند، اما امکان الزام‌آور کردن آن‌ها وجود نداشت.

برای این‌که همه‌ی این موارد به‌طور کامل از دست نروند، برخی از آن‌ها در این پی‌وست ثبت شده‌اند.

Recommended, in-scope mechanisms

موارد زیر در محدوده ASVS قرار دارند. این موارد نباید الزام‌آور شوند، اما قویاً توصیه می‌شود که به‌عنوان بخشی از یک برنامه‌ی امن مورد توجه قرار گیرند.

- باید ابزاری برای نمایش میزان قوی یا ضعیف بودن گذرواژه در اختیار کاربران قرار داده شود تا به آن‌ها در انتخاب یک گذرواژه امن‌تر کمک کند.
- یک فایل security.txt عمومی در ریشه سایت یا پوشه well-known. برنامه ایجاد کنید که به‌وضوح یک لینک یا آدرس ایمیل برای تماس افراد با مالکین در مورد مسائل امنیتی را مشخص کند.
- اعتبارسنجی ورودی‌ها در سمت کاربر باید علاوه بر اعتبارسنجی در لایه سرویس مورد اعتماد اعمال شود، زیرا این کار فرصت خوبی برای شناسایی زمانی فراهم می‌کند که کسی تلاش کرده است با عبور از کنترل‌های سمت کاربر به برنامه حمله کند.
- جلوگیری از دسترسی تصادفی و نمایش صفحات حساس در موتورهای جست‌جو با استفاده از فایل robots.txt، در پاسخ X-Robots-Tag یا تگ متای HTML robots.
- هنگام استفاده از GraphQL، منطق احراز هویت را در لایه منطق کسب‌وکار پیاده‌سازی کنید، نه در لایه GraphQL یا resolver، تا نیازی به مدیریت احراز هویت در هر رابط جداگانه نباشد.

مرجع:

- More information on security.txt including a link to the RFC

Software Security principles

موارد زیر قبلاً در ASVS ذکر شده بودند، اما در واقع الزام نیستند. درعوض، این‌ها اصولی هستند که هنگام پیاده‌سازی کنترل‌های امنیتی باید در نظر گرفته شوند و رعایت آن‌ها منجر به کنترل‌های مقاوم‌تر می‌شود. این موارد شامل:

- کنترل‌های امنیتی باید متمرکز، ساده (اقتصادی در طراحی)، قابل‌اعتبارسنجی و قابل استفاده مجدد باشند. این روی‌کرد باید از تکراری، ناقص یا ناکارآمد بودن کنترل‌ها جلوگیری کند.
- هرجا ممکن است، به‌جای پیاده‌سازی کنترل‌ها از ابتدا، از پیاده‌سازی‌های قبلاً نوشته شده و به‌خوبی بررسی‌شده کنترل‌های امنیتی استفاده کنید.
- به‌طور ایده‌آل، باید از یک مکانیزم کنترل دسترسی واحد برای دسترسی به داده‌ها و منابع محافظت‌شده استفاده شود. تمام درخواست‌ها باید از طریق همین مکانیزم واحد عبور کنند تا از مسیره‌های جایگزین ناامن یا کپی و چسباندن جلوگیری شود.
- کنترل دسترسی مبتنی بر ویژگی یا صفت، یک الگوی پیشنهادی است که در آن کد مجوز کاربر را برای یک ویژگی یا مورد داده بررسی می‌کند، نه صرفاً نقش او. با این حال، تخصیص دسترسی‌ها همچنان باید از طریق نقش‌ها انجام شود.

Software Security processes

تعدادی از فرآیندهای امنیتی وجود دارند که از ASVS نسخه ۵.۰ حذف شده‌اند اما همچنان ایده خوبی هستند. پروژه OWASP SAMM می‌تواند منبع مناسبی برای نحوه پیاده‌سازی مؤثر این فرآیندها باشد. مواردی که قبلاً در ASVS گنجانده شده بودند، شامل:

- استفاده از چرخه عمر توسعه نرم‌افزار امن را که امنیت را در تمامی مراحل توسعه در نظر می‌گیرد، تأیید کنید.
- استفاده از مدل‌سازی تهدید برای هر تغییر طراحی یا برنامه‌ریزی اسپرینت را تأیید کنید تا تهدیدها شناسایی شوند، برنامه‌ای برای مقابله با آن‌ها تدوین شود، پاسخ‌های مناسب به ریسک تست‌های گردد و تست‌های امنیتی هدایت شوند.
- تأیید کنید که تمام داستان‌های کاربری و ویژگی‌ها شامل محدودیت‌های عملکردی امنیتی باشند، مانند: «به‌عنوان یک کاربر، باید قادر باشم پروفایل خود را مشاهده و ویرایش کنم. نباید قادر باشم پروفایل سایر کاربران را مشاهده یا ویرایش کنم.
- اطمینان حاصل کنید که یک چک‌لیست برنامه‌نویسی امن، الزامات امنیتی، راهنما یا سیاست امنیتی در دسترس تمام توسعه‌دهندگان و تست‌کنندگان قرار دارد.
- اطمینان حاصل کنید که یک فرآیند مستمر برای بررسی وجود دارد تا کد منبع برنامه عاری از درب‌های پشتی، کدهای مخرب (مثلاً حملات سالامی، بمب‌های منطقی، بمب‌های زمانی) و ویژگی‌های مستندسازی نشده یا مخفی (مثلاً Easter eggs، ابزارهای اشکال‌زدایی ناامن) باشد. رعایت این بخش بدون دسترسی کامل به کد منبع، شامل کتابخانه‌های شخص ثالث، امکان‌پذیر نیست و بنابراین احتمالاً فقط برای برنامه‌های مناسب است که به بالاترین سطوح امنیت نیاز دارند.
- اطمینان حاصل کنید که مکانیزم‌های برای شناسایی و پاسخ به تغییرات ناخواسته پیکربندی در محیط‌های مستقر شده وجود دارد. این ممکن است شامل استفاده از زیرساخت غیرقابل تغییر، بازاستقرار خودکار از یک پایه امن، یا ابزارهای شناسایی تغییر که وضعیت فعلی را با پیکربندی‌های تأیید شده مقایسه می‌کنند، باشد.
- اطمینان حاصل کنید که امنسازی پیکربندی روی تمام محصولات، کتابخانه‌ها، فریم‌ورک‌ها و سرویس‌های شخص ثالث طبق توصیه‌های هر یک انجام شده باشد.

مراجع:

- OWASP Threat Modeling Cheat Sheet
- OWASP Threat modeling
- OWASP Software Assurance Maturity Model Project
- Microsoft SDL

Appendix E - Contributors

ما با سپاسگزاری از مشارکت افراد زیر که از زمان انتشار نسخه ASVS ۴.۰.۰ نظر داده‌اند یا درخواست‌های ادغام (Pull Request) باز کرده‌اند، قدردانی می‌کنیم.

اگر از وجود هرگونه خطای آگاه هستید یا ترجیح می‌دهید نام شما به شکلی متفاوت نمایش داده شود، لطفاً به ما اطلاع دهید.

Johan Sydseter (sydseter)	luis servin (lfservin)	Oleksii Dovydkov (oleksiidov)	IZUKA Masahiro (maizuka)
James Sulinski (jsulinski)	Eli Saad (ThunderSon)	kkshitish9	Andrew van der Stock (vanderaj)
Rick M (kingthorin)	Bankde Eakasit (Bankde)	Michael Gargiullo (mgargiullo)	Raphael Dunant (Racater)
Cesar Kohl (cesarkohl)	inaz0	Joerg Bruenner (JoergBruenner)	David Deatherage (securitydave)
John Carroll (yosignals)	Jim Fenton (jimfenton)	Matteo Pace (M4tteoP)	Sebastien gioria (SPoint42)
Steven van der Baan (vdbaan)	Jeremy Bonghwan Choi (jeremychoi)	craig-shony	Riccardo Sirigu (ricsirigu)
Tomasz Wrobel (tw2as)	Alena Dubeshko (belalena)	Rafael Green (RafaelGreen1)	mjang-cobalt
clallier94	Kevin W. Wall (kwwall)	Jordan Sherman (jsherm-fwdsec / deleterepo)	Ingo Rauner (ingo-rauner)
Dirk Wetter (drwetter)	Moshe Zioni (moshe-apiiro)	Patrick Dwyer (coderpatros)	David Clarke (davidclarke-au)
Takaharu Ogasa (takaharuogasa)	Arkadii Yakovets (arkid15r)	Motoyasu Saburi (motoyasu-saburi)	leirn
wet-certitude	timhemel	RL Thornton (thornshadow99)	Thomas Bandt (aspnetde)
Roel Storms (roelstorms)	Jeroen Willemsen (commjoen)	anonymous-31	Kamran Saifullah (deFr0ggy)
Steve Springett (stevespringett)	Spyros (northdpole)	Hans Herrera (hansphp)	Marx314
CarlosAllendes	Yonah Russ (yruss972)	Sander Maijers (sanmai-NL)	Luboš Bretschneider (bretik)
Eva Sarafianou (esarafianou)	ataseren	Steve Thomas (Sc00bz)	Dominique RIGHETTO (righettod)
Steven van der Baan (svdb-ncc)	Michael Vacarella (Aif4thah)	Tonimir Kisasondi (tkisason)	Stefan Streichsbier (streichsbaer)

hi-unc1e	sb3k (starbuck3000)	mario-platt	Devdatta Akhawe (devd)
Michael Gissing (scolytus)	Jet Anderson (thatsjet)	Dave Wichers (davewichers)	Jonny Schnittger (JonnySchnittger)
Silvia Väli (silviavali)	jackgates73	1songb1rd	Timur - (timurozkul)
Gareth Heyes (hackvertor)	appills	suvikaartinen	chaals (chaals)
DanielPharos (AtlasHackert)	will Farrell (willfarrell)	Alina Vasiljeva (avasiljeva)	Paul McCann (ismisepaul)
Sage (SajjadPourali)	rbsec	Benedikt Bauer (mastacheata)	James Jardine (jamesjardine)
Mark Burnett (m8urnett)	dschwarz91	Cyber-AppSec (Cyber-AppSec)	Tib3rius
BitnessWise (bitnesswise)	damienbod (damienbod)	Jared Meit (jmeit-fwdsec)	Stefan Seelmann (sseelmann)
Brendan O'Connor (ussjoin)	Andrei Titov (andrettv)	Hans-Petter Fjeld (atluxity)	markehack
Neil Madden (NeilMadden)	Michael Geramb (mgeramb)	Osama Elnaggar (ossie-git)	mackowski
Ravi Balla (raviballa)	Hazana (hazanasec)	David Means (dmeans82)	Alexander Stein (tohch4)
BaeSenseii (baesenseii)	Vincent De Schutter (VincentDS)	S Bani (sbani)	Mitsuaki Akiyama (mak1yama)
Christopher Loessl (hashier)	victorxm	Michal Rada (michalradacz)	Veeresh Devireddy (drveresh)
MaknaSEO	darkzero2022	Liam (LiamDobbelaere)	Frank Denis (jedisct1)
Otto Sulin (ottosulin)	carllaw6885	Anders Johan Holmefjord (aholmis)	Richard Fritsch (rfricz)
mesutgungor	Scott Helme (ScottHelme)	Carlo Reggiani (carloreggiani)	Suyash Srivastava (suyash5053)
Mark Potter (markonweb)	Arjan Lamers (alamers)	Gøran Breivik (gobrtg)	flo-blg
Guillaume Déflache (guillaume-d)	Toufik Airane (toufik-airane)	Keith Hoodlet (securingdev)	Sinner (SoftwareSinner)
iloving	Jeroen Beckers (TheDauntless)	Joubin Jabbari (joubin)	yu fujioka (fujiokayu)

execjosh (execjosh)	Alicja Kario (tomato42)	Sidney Ribeiro (srjsoftware)	Gabriel Marquet (Gby56)
Drew Schulz (drschulz)	bedirhan	muralito	Ronnie Flathers (ropnop)
Philippe De Ryck (philippederyck)	Malte (mal33)	MazeOfThoughts	Andreas Falk (andifalk)
Javi (javixeneize)	Daniel Hahn (averell23)	borislav-c	Robin Wood (digininja)
miro2ns	Jan Dockx (jandockx)	vipinsaini434	priyanshukumar397
Nat Sakimura (sakimura)	Benjamin Häublein (BenjaminHae)	unknown-user-from	Ali Ramazan TAŞDELEN (alitasdln)
Pedro Escaleira (oEscal)	Josh (josh-hemphill)	Tim Würtele (SECtim)	AviD (avidouglen)
SheHacksPurple (shehackspurple)	fcerullo-cycubix	Hector Eryx Paredes Camacho (heryxpc)	Irene Michlin (irene221b)
Jonah Y-M (TG-Techie)	Dhiraj Bahroos (bahroos)	Jef Meijvis (jefmeijvis)	IzmaDoesItbeta
Abdessamad TEMMAR (TmmmmmR)	sectroyer	Soh Satoh (sohsatoh)	regoravalaz
james-t (james-bitherder)	Aram Hovsepyan (aramhovsepyan)	JaimeGomezGarciaSan	ValdiGit01
iwatachan (ishowta)	Vinod Anandan (VinodAnandan)	Kevin Kien (KevinKien)	paul-williamson-swoop
endergzr	Radhwan Alshamamri (Rado0z)	Grant Ongers (rewtd)	Cure53 (cure53)
AliR2Linux	Ads Dawson (Gang-GreenTemperTatum)	William Reyor (BillReyor)	gabe (gcrow)
mascotter	luissaiz	Suren Manukyan (vx-sec)	Piotr Gliźniewicz (pgliźniewicz)
Tadeusz Wachowski (tadeuszwachowski)	Nasir aka Nate (andesec)	settantasette	Lars Haulin (LarsH)
Terence Eden (edent)	JasmineScholz	Arun Sivadasan (teavanist)	Yusuf GÜR (yusuffgur)
Troy Marshall (troymarshall)	Tanner Prynn (tprynn)	Nick K. (nickific)	raoul361
Azeem Ilyas (TheAxZim)	Evo Stamatov (avioli)	Tim Potter (timpotter87)	Gavin Ray (GavinRay97)
monis (demideus)	Marcin Hoppe (MarcinHoppe)	Grambulf (ramshazar)	Jordan Pike (computersarebad)

Jason Rogers (jason-invision)	Ben Hall (benbhall)	JamesPoppyCock (jamesly123)	WhiteHackLabs (whitehacklabs)
Alex Gaynor (alex)	Filip van Laenen (filipvanlaenen)	jeurgen	GraoMelo
Andreas Kurtz (ay-kay)	Tom Tervoort (TomTervoort)	old man (deveras)	Marco Schnüriger (marcortw)
stiiin	infoseclearn (teaminfoseclearn)	hljupkij	Noe (nmarher)
Lyz (lyz-code)	Martin Riedel (mrtnrdl)	KIM Jaesuck (tcaesvk)	Barbara Schachner (bschach)
René Reuter (AresSec)	carhackpils	Tyler (tyler2cr)	Hugo (hasousa)
Wouter Bloeyaert (Someniak)	Mark de Rijk (markderijkinfosec)	Ramin (picohub)	Philip D. Turner (philipdturner)
Will Chatham (willc)			